

RadioTherapy - Source Code Documentation

Generated by Doxygen 1.13.2

Chapter 1

RadioTherapy Dose Prediction

A PyTorch/MONAI-based application for training and running latent diffusion models to predict radiation dose distributions from CT scans, with support for Gaussian quadrature over multiple energy levels.

1.1 Project Structure

```
RadioTherapy/  
  sourcecode/  
    gui.py                # PyQt5 GUI entry point  
    system_manager.py     # Orchestrates training & inference  
    data_management.py    # Dataset & DataLoader logic  
    inference_module.py   # Gaussian-quadrature inference pipeline  
    training_pipeline.py  # Trainer classes for AE & diffusion  
    generative/          # Prebuilt architectures & inferers  
  testdata/              # Sample input/output cubes by energy  
  Master_thesis_Hagedorn.pdf # Hagedorn master's thesis (Gaussian quadrature)  
  requirements.txt        # Python dependencies  
  README.md              # You are here
```

1.2 Quickstart

1. Clone repo

```
git clone https://github.com/<your-username>/RadioTherapy.git  
cd RadioTherapy
```

2. Install dependencies

```
pip install -r requirements.txt
```

3. Run GUI

```
python sourcecode/gui.py
```

4. Run Tests

```
cd sourcecode  
pytest -q
```

1.3 GUI Workflow

1. Select a model checkpoint (only for inference)

- Pretrained autoencoder + UNet .ckpt file.

2. Choose an energy folder

- Point to one of the `traindata/<energy_ev>/` directories.

3. Train or infer

- **Train** runs AE + diffusion training for that energy, logging “Expected input channels” and “UNet-Forward” debug info.
- **Calculate Dose** loads CT .npy, picks Gaussian quadrature points/weights from your GUI input, then runs inference over all energies and aggregates results.

1.4 Core Components

1.4.1 1. ParameterManager

- Loads/stores list of quadrature energies & weights (dynamically from GUI selection).
- Passes them to `SystemManager`.

1.4.2 2. SystemManager

- `train_training()`
 - Iterates each epoch for AE & diffusion, per selected energy.
- `run_inference(ct_path, checkpoint)`
 - Preprocesses CT $\rightarrow$ tensor $\rightarrow$ runs `InferenceModule`.

1.4.3 3. InferenceModule

- `preprocess_ct()`
 - Resizes CT to a fixed cube (default 64^3).
- `run_inference_over_energies()`
 - Loops Gaussian quadrature energies & weights.
 - Calls `run_inference_conditioned_on_energy()` per energy.
 - Stacks & weighted-sums dose outputs.
- `run_inference_conditioned_on_energy()`
 - Concatenates a normalized energy channel to CT.
 - Encodes latent via AE, then samples with `LatentDiffusionInferer`.

1.5 Gaussian Quadrature

Based on Hagedorn's Master's Thesis (Chapter 4.4.3), we choose 8 optimal energy/weight pairs for integrating over the dose spectrum.

- **Quadrature energies:** e.g. [11.5, 20.0, ...] keV
- **Weights:** corresponding integration weights

The GUI lets you pick an **energy range**; we then compute the 8 quadrature points & weights on the fly and feed them into `InferenceModule`.

1.6 Training & Debug

- **Autoencoder**
 - Logs “Expected input channels: 2, Actual input channels: 1” if your AE-UNet coupling is misconfigured.
 - **Diffusion**
 - Prints [DIFF DEBUG] UNet-Forward-Fehler: ... when dimensions mismatch in cross-attention.
 - Common errors often stem from mismatched channel counts or missing `cross_attention_dim`.
-

Chapter 2

Namespace Index

2.1 Package List

Here are the packages with brief descriptions (if available):

analyze_traincubes	??
corrected_inference	??
corrected_training_pipeline	??
data_management	??
debug_inference_problem	??
debug_inference_systematic	??
debug_training_data	??
debug_visualization	??
gaussian_quadrature_inference	??
gui	??
hotspot_patch_sampler	??
inference_module	??
inference_module_standalone	??
log_config	??
parameter_manager	??
simple_inference_test	??
standalone_training	??
storage	??
system_manager	??
systematic_dose_diagnosis	??
test_critical_fixes	??
test_error_logging	??
test_gui	??
test_gui_inference	??
test_gui_training	??
test_pipeline_fixes	??
test_system_manager_inference	??
test_system_manager_real_data	??
test_systemmanager	??
test_training	??
test_visualization	??
training_pipeline	??
validate_energy_conditioning	??
visualization	??
visualizeTEST	??

Chapter 3

Hierarchical Index

3.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

training_pipeline.AutoencoderTrainer	??
corrected_inference.CorrectedInferenceModule	??
corrected_training_pipeline.CorrectedTrainingPipeline	??
data_management.DataLoaderModule	??
Dataset	
corrected_training_pipeline.CorrectedDoseNpyDataset	??
data_management.DoseNpyDataset	??
data_management.HotspotPatchDataset	??
training_pipeline.DiffusionTrainer	??
systematic_dose_diagnosis.DoseNormalizationDiagnostic	??
training_pipeline.EarlyStopping	??
FigureCanvasQTAgg	
visualization.MplCanvas	??
inference_module.InferenceModule	??
parameter_manager.ParameterManager	??
QMainWindow	
gui.MainWindow	??
QObject	
gui.InferenceWorker	??
gui.TrainingWorker	??
storage.Storage	??
system_manager.SystemManager	??
visualization.Visualization	??

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

training_pipeline.AutoencoderTrainer	??
corrected_training_pipeline.CorrectedDoseNpyDataset	??
corrected_inference.CorrectedInferenceModule	??
corrected_training_pipeline.CorrectedTrainingPipeline	??
data_management.DataLoaderModule	??
training_pipeline.DiffusionTrainer	??
systematic_dose_diagnosis.DoseNormalizationDiagnostic	??
data_management.DoseNpyDataset	??
training_pipeline.EarlyStopping	??
data_management.HotspotPatchDataset	??
inference_module.InferenceModule	??
gui.InferenceWorker	??
gui.MainWindow	??
visualization.MplCanvas	??
parameter_manager.ParameterManager	??
storage.Storage	??
system_manager.SystemManager	??
gui.TrainingWorker	??
visualization.Visualization	??

Chapter 5

File Index

5.1 File List

Here is a list of all files with brief descriptions:

sourcecode/	corrected_inference.py	??
sourcecode/	corrected_training_pipeline.py	??
sourcecode/	data_management.py	??
sourcecode/	gaussian_quadrature_inference.py	??
sourcecode/	gui.py	??
sourcecode/	hotspot_patch_sampler.py	??
sourcecode/	inference_module.py	??
sourcecode/	inference_module_standalone.py	??
sourcecode/	log_config.py	??
sourcecode/	parameter_manager.py	??
sourcecode/	standalone_training.py	??
sourcecode/	storage.py	??
sourcecode/	system_manager.py	??
sourcecode/	training_pipeline.py	??
sourcecode/	visualization.py	??
tests/	analyze_traincubes.py	??
tests/	debug_inference_problem.py	??
tests/	debug_inference_systematic.py	??
tests/	debug_training_data.py	??
tests/	debug_visualization.py	??
tests/	simple_inference_test.py	??
tests/	systematic_dose_diagnosis.py	??
tests/	test_critical_fixes.py	??
tests/	test_error_logging.py	??
tests/	test_gui.py	??
tests/	test_gui_inference.py	??
tests/	test_gui_training.py	??
tests/	test_pipeline_fixes.py	??
tests/	test_system_manager_inference.py	??
tests/	test_system_manager_real_data.py	??
tests/	test_systemmanager.py	??
tests/	test_training.py	??
tests/	test_visualization.py	??
tests/	validate_energy_conditioning.py	??
tests/	visualizeTEST.py	??

Chapter 6

Namespace Documentation

6.1 `analyze_traincubes` Namespace Reference

6.2 `corrected_inference` Namespace Reference

Classes

- class [CorrectedInferenceModule](#)

Functions

- [test_corrected_inference](#) ()

Variables

- [logger](#) = `logging.getLogger(__name__)`

6.2.1 Detailed Description

Corrected inference pipeline that properly uses the energy-conditioned model.

6.2.2 Function Documentation

6.2.2.1 `test_corrected_inference()`

`corrected_inference.test_corrected_inference ()`

Test the corrected inference module.

Definition at line [384](#) of file [corrected_inference.py](#).

6.2.3 Variable Documentation

6.2.3.1 `logger`

`corrected_inference.logger = logging.getLogger(__name__)`

Definition at line [12](#) of file [corrected_inference.py](#).

6.3 `corrected_training_pipeline` Namespace Reference

Classes

- class [CorrectedDoseNpyDataset](#)
- class [CorrectedTrainingPipeline](#)

Functions

- [main](#) ()

Variables

- `logger` = `logging.getLogger(__name__)`

6.3.1 Detailed Description

CORRECTED TRAINING PIPELINE - NACHHALTIGE LÖSUNG

Fixes all three critical problems:

1. Transform Fix: Use Resized instead of Spacingd
2. Dose Normalization Fix: Calculate from original data
3. Unified Energy Model: Train one model for all energies

This is the sustainable solution that addresses root causes.

6.3.2 Function Documentation

6.3.2.1 `main()`

`corrected_training_pipeline.main ()`

Main function to run the corrected training pipeline.

Definition at line 397 of file `corrected_training_pipeline.py`.

6.3.3 Variable Documentation

6.3.3.1 `logger`

`corrected_training_pipeline.logger = logging.getLogger(__name__)`

Definition at line 34 of file `corrected_training_pipeline.py`.

6.4 `data_management` Namespace Reference

Classes

- class `DataLoaderModule`
- class `DoseNpyDataset`
- class `HotspotPatchDataset`

Variables

- `logger` = `logging.getLogger(__name__)`

6.4.1 Variable Documentation

6.4.1.1 `logger`

`data_management.logger = logging.getLogger(__name__)`

Definition at line 184 of file `data_management.py`.

6.5 `debug_inference_problem` Namespace Reference

Variables

- `level`
- `INFO`
- `format`
- str `input_file` = `'traindata/11_5/inputcube/235101017859661465075472232303048949736_0.npy'`
- str `output_file` = `'traindata/11_5/outputcube/235101017859661465075472232303048949736_0.npy'`
- `input_data` = `np.load(input_file)`
- `ground_truth` = `np.load(output_file)`

- `str model_path = '/Users/giannigagliardi/Documents/Git/RadioTherapy/unified_energy_conditioned↵_model_res16.0_energies2.ckpt'`
- `device = torch.device("cpu")`
- `checkpoint = torch.load(model_path, map_location=device)`
- `autoencoder`
- `float energy_value = 11.5`
- `float energy_normalized = energy_value / 40.0`
- `input_2channel = np.zeros((2, *input_data.shape))`
- `input_tensor = torch.tensor(input_2channel, dtype=torch.float32).unsqueeze(0).to(device)`
- `encoded = autoencoder.encode(input_tensor)`
- `latent = encoded[0]`
- `decoded = autoencoder.decode(latent)`
- `reconstruction = decoded[0]`
- `reconstruction_np = reconstruction.cpu().numpy().squeeze()`
- `mse = np.mean((reconstruction_np - ground_truth) ** 2)`
- `mae = np.mean(np.abs(reconstruction_np - ground_truth))`
- `correlation = np.corrcoef(reconstruction_np.flatten(), ground_truth.flatten())[0, 1]`

6.5.1 Detailed Description

Debug-Script um das Inference-Problem zu identifizieren

6.5.2 Variable Documentation

6.5.2.1 autoencoder

`debug_inference_problem.autoencoder`

Initial value:

```
00001 = AutoencoderKL(
00002     spatial_dims=3,
00003     in_channels=2,
00004     out_channels=1,
00005     num_channels=(32, 32, 32),
00006     latent_channels=2,
00007     num_res_blocks=1,
00008     norm_num_groups=8,
00009     attention_levels=(False, False, True),
00010 ).to(device)
```

Definition at line 56 of file `debug_inference_problem.py`.

6.5.2.2 checkpoint

`debug_inference_problem.checkpoint = torch.load(model_path, map_location=device)`

Definition at line 52 of file `debug_inference_problem.py`.

6.5.2.3 correlation

`debug_inference_problem.correlation = np.corrcoef(reconstruction_np.flatten(), ground_truth.flatten())[0, 1]`

Definition at line 122 of file `debug_inference_problem.py`.

6.5.2.4 decoded

`debug_inference_problem.decoded = autoencoder.decode(latent)`

Definition at line 106 of file `debug_inference_problem.py`.

6.5.2.5 device

`debug_inference_problem.device = torch.device("cpu")`

Definition at line 51 of file `debug_inference_problem.py`.

6.5.2.6 encoded

`debug_inference_problem.encoded = autoencoder.encode(input_tensor)`
Definition at line 96 of file [debug_inference_problem.py](#).

6.5.2.7 energy_normalized

`float debug_inference_problem.energy_normalized = energy_value / 40.0`
Definition at line 77 of file [debug_inference_problem.py](#).

6.5.2.8 energy_value

`float debug_inference_problem.energy_value = 11.5`
Definition at line 76 of file [debug_inference_problem.py](#).

6.5.2.9 format

`debug_inference_problem.format`
Definition at line 17 of file [debug_inference_problem.py](#).

6.5.2.10 ground_truth

`debug_inference_problem.ground_truth = np.load(output_file)`
Definition at line 33 of file [debug_inference_problem.py](#).

6.5.2.11 INFO

`debug_inference_problem.INFO`
Definition at line 17 of file [debug_inference_problem.py](#).

6.5.2.12 input_2channel

`debug_inference_problem.input_2channel = np.zeros((2, *input_data.shape))`
Definition at line 80 of file [debug_inference_problem.py](#).

6.5.2.13 input_data

`debug_inference_problem.input_data = np.load(input_file)`
Definition at line 32 of file [debug_inference_problem.py](#).

6.5.2.14 input_file

`str debug_inference_problem.input_file = 'traindata/11_5/inputcube/235101017859661465075472232303048949736↵_0.npy'`
Definition at line 23 of file [debug_inference_problem.py](#).

6.5.2.15 input_tensor

`debug_inference_problem.input_tensor = torch.tensor(input_2channel, dtype=torch.float32).unsqueeze(0).to(device)`
Definition at line 85 of file [debug_inference_problem.py](#).

6.5.2.16 latent

`debug_inference_problem.latent = encoded[0]`
Definition at line 98 of file [debug_inference_problem.py](#).

6.5.2.17 level

`debug_inference_problem.level`
Definition at line 17 of file [debug_inference_problem.py](#).

6.5.2.18 mae

`debug_inference_problem.mae = np.mean(np.abs(reconstruction_np - ground_truth))`
 Definition at line 121 of file `debug_inference_problem.py`.

6.5.2.19 model_path

`str debug_inference_problem.model_path = '/Users/giannigagliardi/Documents/Git/RadioTherapy/unified_↵
 energy_conditioned_model_res16.0_energies2.ckpt'`
 Definition at line 46 of file `debug_inference_problem.py`.

6.5.2.20 mse

`debug_inference_problem.mse = np.mean((reconstruction_np - ground_truth) ** 2)`
 Definition at line 120 of file `debug_inference_problem.py`.

6.5.2.21 output_file

`str debug_inference_problem.output_file = 'traindata/11_5/outputcube/235101017859661465075472232303048949736_↵
 _0.npy'`
 Definition at line 24 of file `debug_inference_problem.py`.

6.5.2.22 reconstruction

`debug_inference_problem.reconstruction = decoded[0]`
 Definition at line 108 of file `debug_inference_problem.py`.

6.5.2.23 reconstruction_np

`debug_inference_problem.reconstruction_np = reconstruction.cpu().numpy().squeeze()`
 Definition at line 112 of file `debug_inference_problem.py`.

6.6 debug_inference_systematic Namespace Reference

Variables

- `level`
- `INFO`
- `format`
- `str input_file = 'traindata/11_5/inputcube/235101017859661465075472232303048949736_0.npy'`
- `str output_file = 'traindata/11_5/outputcube/235101017859661465075472232303048949736_0.npy'`
- `input_data = np.load(input_file)`
- `ground_truth = np.load(output_file)`
- `str model_path = '/Users/giannigagliardi/Documents/Git/RadioTherapy/unified_energy_conditioned_↵
 _model_res16.0_energies2.ckpt'`
- `device = torch.device("cpu")`
- `checkpoint = torch.load(model_path, map_location=device)`
- `config = checkpoint['training_config']`
- `training_cube_size = config['cube_size']`
- `autoencoder`
- `input_tensor = torch.tensor(input_data, dtype=torch.float32).unsqueeze(0).unsqueeze(0).to(device)`
- `ground_truth_tensor = torch.tensor(ground_truth, dtype=torch.float32).unsqueeze(0).unsqueeze(0).to(device)`
- `input_resized = interpolate(input_tensor, size=training_cube_size, mode='trilinear', align_↵
 corners=True)`
- `ground_truth_resized = interpolate(ground_truth_tensor, size=training_cube_size, mode='trilinear',
 align_corners=True)`
- `float energy_value = 11.5`
- `list energy_normalizations`

- `int best_correlation = -1`
- `best_method = None`
- `best_reconstruction = None`
- `input_2channel = torch.zeros((1, 2, *training_cube_size), device=device)`
- `encoded = autoencoder.encode(input_2channel)`
- `latent = encoded[0]`
- `decoded = autoencoder.decode(latent)`
- `reconstruction = decoded[0]`
- `reconstruction_full = interpolate(reconstruction, size=(100, 100, 100), mode='trilinear', align_corners=True)`
- `reconstruction_np = reconstruction_full.cpu().numpy().squeeze()`
- `mse = np.mean((reconstruction_np - ground_truth) ** 2)`
- `mae = np.mean(np.abs(reconstruction_np - ground_truth))`
- `correlation = np.corrcoef(reconstruction_np.flatten(), ground_truth.flatten())[0, 1]`

6.6.1 Detailed Description

Systematische Analyse der Inference-Pipeline Probleme

6.6.2 Variable Documentation

6.6.2.1 autoencoder

`debug_inference_systematic.autoencoder`

Initial value:

```
00001 = AutoencoderKL(
00002     spatial_dims=3,
00003     in_channels=2,
00004     out_channels=1,
00005     num_channels=(32, 32, 32),
00006     latent_channels=2,
00007     num_res_blocks=1,
00008     norm_num_groups=8,
00009     attention_levels=(False, False, True),
00010 ).to(device)
```

Definition at line 59 of file `debug_inference_systematic.py`.

6.6.2.2 best_correlation

`debug_inference_systematic.best_correlation = -1`

Definition at line 99 of file `debug_inference_systematic.py`.

6.6.2.3 best_method

`debug_inference_systematic.best_method = None`

Definition at line 100 of file `debug_inference_systematic.py`.

6.6.2.4 best_reconstruction

`debug_inference_systematic.best_reconstruction = None`

Definition at line 101 of file `debug_inference_systematic.py`.

6.6.2.5 checkpoint

`debug_inference_systematic.checkpoint = torch.load(model_path, map_location=device)`

Definition at line 35 of file `debug_inference_systematic.py`.

6.6.2.6 config

`debug_inference_systematic.config = checkpoint['training_config']`

Definition at line 40 of file `debug_inference_systematic.py`.

6.6.2.7 correlation

```
debug_inference_systematic.correlation = np.corrcoef(reconstruction_np.flatten(), ground_truth.flatten())[0, 1]
```

Definition at line 134 of file [debug_inference_systematic.py](#).

6.6.2.8 decoded

```
debug_inference_systematic.decoded = autoencoder.decode(latent)
```

Definition at line 120 of file [debug_inference_systematic.py](#).

6.6.2.9 device

```
debug_inference_systematic.device = torch.device("cpu")
```

Definition at line 34 of file [debug_inference_systematic.py](#).

6.6.2.10 encoded

```
debug_inference_systematic.encoded = autoencoder.encode(input_2channel)
```

Definition at line 113 of file [debug_inference_systematic.py](#).

6.6.2.11 energy_normalizations

```
list debug_inference_systematic.energy_normalizations
```

Initial value:

```
00001 = [
00002     ("div_40", energy_value / 40.0),
00003     ("div_100", energy_value / 100.0),
00004     ("div_50", energy_value / 50.0),
00005     ("raw", energy_value),
00006     ("normalized_0_1", (energy_value - 11.5) / (34.25 - 11.5)) # Min-Max normalization
00007 ]
```

Definition at line 91 of file [debug_inference_systematic.py](#).

6.6.2.12 energy_value

```
float debug_inference_systematic.energy_value = 11.5
```

Definition at line 90 of file [debug_inference_systematic.py](#).

6.6.2.13 format

```
debug_inference_systematic.format
```

Definition at line 17 of file [debug_inference_systematic.py](#).

6.6.2.14 ground_truth

```
debug_inference_systematic.ground_truth = np.load(output_file)
```

Definition at line 28 of file [debug_inference_systematic.py](#).

6.6.2.15 ground_truth_resized

```
debug_inference_systematic.ground_truth_resized = interpolate(ground_truth_tensor, size=training_cube_size, mode='trilinear', align_corners=True)
```

Definition at line 84 of file [debug_inference_systematic.py](#).

6.6.2.16 ground_truth_tensor

```
debug_inference_systematic.ground_truth_tensor = torch.tensor(ground_truth, dtype=torch.float32).unsqueeze(0).unsqueeze(0).t
```

Definition at line 80 of file [debug_inference_systematic.py](#).

6.6.2.17 INFO

```
debug_inference_systematic.INFO
```

Definition at line 17 of file [debug_inference_systematic.py](#).

6.6.2.18 input_2channel

`debug_inference_systematic.input_2channel = torch.zeros((1, 2, *training_cube_size), device=device)`
 Definition at line 107 of file [debug_inference_systematic.py](#).

6.6.2.19 input_data

`debug_inference_systematic.input_data = np.load(input_file)`
 Definition at line 27 of file [debug_inference_systematic.py](#).

6.6.2.20 input_file

`str debug_inference_systematic.input_file = 'traindata/11_5/inputcube/235101017859661465075472232303048949736↵_0.npy'`
 Definition at line 23 of file [debug_inference_systematic.py](#).

6.6.2.21 input_resized

`debug_inference_systematic.input_resized = interpolate(input_tensor, size=training_cube_size, mode='trilinear', align_corners=True)`
 Definition at line 83 of file [debug_inference_systematic.py](#).

6.6.2.22 input_tensor

`debug_inference_systematic.input_tensor = torch.tensor(input_data, dtype=torch.float32).unsqueeze(0).unsqueeze(0).to(device)`
 Definition at line 79 of file [debug_inference_systematic.py](#).

6.6.2.23 latent

`debug_inference_systematic.latent = encoded[0]`
 Definition at line 115 of file [debug_inference_systematic.py](#).

6.6.2.24 level

`debug_inference_systematic.level`
 Definition at line 17 of file [debug_inference_systematic.py](#).

6.6.2.25 mae

`debug_inference_systematic.mae = np.mean(np.abs(reconstruction_np - ground_truth))`
 Definition at line 133 of file [debug_inference_systematic.py](#).

6.6.2.26 model_path

`str debug_inference_systematic.model_path = '/Users/giannigagliardi/Documents/Git/RadioTherapy/unified_↵energy_conditioned_model_res16.0_energies2.ckpt'`
 Definition at line 33 of file [debug_inference_systematic.py](#).

6.6.2.27 mse

`debug_inference_systematic.mse = np.mean((reconstruction_np - ground_truth) ** 2)`
 Definition at line 132 of file [debug_inference_systematic.py](#).

6.6.2.28 output_file

`str debug_inference_systematic.output_file = 'traindata/11_5/outputcube/235101017859661465075472232303048949736↵_0.npy'`
 Definition at line 24 of file [debug_inference_systematic.py](#).

6.6.2.29 reconstruction

`debug_inference_systematic.reconstruction = decoded[0]`

Definition at line 122 of file `debug_inference_systematic.py`.

6.6.2.30 reconstruction_full

`debug_inference_systematic.reconstruction_full = interpolate(reconstruction, size=(100, 100, 100), mode='trilinear', align_corners=True)`

Definition at line 127 of file `debug_inference_systematic.py`.

6.6.2.31 reconstruction_np

`debug_inference_systematic.reconstruction_np = reconstruction_full.cpu().numpy().squeeze()`

Definition at line 129 of file `debug_inference_systematic.py`.

6.6.2.32 training_cube_size

`tuple debug_inference_systematic.training_cube_size = config['cube_size']`

Definition at line 48 of file `debug_inference_systematic.py`.

6.7 debug_training_data Namespace Reference

Variables

- list `test_files`
- list `output_files`
- list `energies` = [11.5, 15.75, 34.25]
- `input_data` = `np.load(input_file)`
- `output_data` = `np.load(output_file)`
- `correlation` = `np.corrcoef(input_data.flatten(), output_data.flatten())[0, 1]`
- `data_loader`
- `train_loader`
- `val_loader`
- `_`
- `input_batch` = `batch['inputcube']`
- `output_batch` = `batch['outputcube']`
- `energy_batch` = `batch['energy']`
- `device` = `torch.device("cpu")`
- `test_input` = `torch.randn(1, 2, 32, 32, 32)`
- `test_output` = `test_input[:, 0:1, :, :, :]`
- str `model_path` = `'/Users/giannigagliardi/Documents/Git/RadioTherapy/unified_energy_conditioned↵_model_res16.0_energies2.ckpt'`
- `checkpoint` = `torch.load(model_path, map_location=device)`
- `autoencoder`
- `encoded` = `autoencoder.encode(test_input)`
- `latent` = `encoded[0]`
- `decoded` = `autoencoder.decode(latent)`
- `reconstruction` = `decoded[0]`

6.7.1 Detailed Description

Überprüfe die Trainings-Daten auf Konsistenz

6.7.2 Variable Documentation

6.7.2.1 _

`debug_training_data._` [protected]

Definition at line 87 of file `debug_training_data.py`.

6.7.2.2 autoencoder

`debug_training_data.autoencoder`

Initial value:

```
00001 = AutoencoderKL(
00002     spatial_dims=3,
00003     in_channels=2,
00004     out_channels=1,
00005     num_channels=(32, 32, 32),
00006     latent_channels=2,
00007     num_res_blocks=1,
00008     norm_num_groups=8,
00009     attention_levels=(False, False, True),
00010 ).to(device)
```

Definition at line 137 of file [debug_training_data.py](#).

6.7.2.3 checkpoint

`debug_training_data.checkpoint = torch.load(model_path, map_location=device)`

Definition at line 134 of file [debug_training_data.py](#).

6.7.2.4 correlation

`debug_training_data.correlation = np.corrcoef(input_data.flatten(), output_data.flatten())[0, 1]`

Definition at line 60 of file [debug_training_data.py](#).

6.7.2.5 data_loader

`debug_training_data.data_loader`

Initial value:

```
00001 = DataLoaderModule(
00002     root_dir='traindata',
00003     transforms=Compose([]),
00004     resolutions=[16.0],
00005     energies=[11.5, 15.75, 34.25],
00006     batch_size=1,
00007     cube_size=(32, 32, 32),
00008     seed=42
00009 )
```

Definition at line 74 of file [debug_training_data.py](#).

6.7.2.6 decoded

`debug_training_data.decoded = autoencoder.decode(latent)`

Definition at line 158 of file [debug_training_data.py](#).

6.7.2.7 device

`debug_training_data.device = torch.device("cpu")`

Definition at line 122 of file [debug_training_data.py](#).

6.7.2.8 encoded

`debug_training_data.encoded = autoencoder.encode(test_input)`

Definition at line 152 of file [debug_training_data.py](#).

6.7.2.9 energies

`list debug_training_data.energies = [11.5, 15.75, 34.25]`

Definition at line 32 of file [debug_training_data.py](#).

6.7.2.10 energy_batch

`debug_training_data.energy_batch = batch['energy']`

Definition at line 95 of file [debug_training_data.py](#).

6.7.2.11 input_batch

`debug_training_data.input_batch = batch['inputcube']`
Definition at line 93 of file [debug_training_data.py](#).

6.7.2.12 input_data

`debug_training_data.input_data = np.load(input_file)`
Definition at line 40 of file [debug_training_data.py](#).

6.7.2.13 latent

`debug_training_data.latent = encoded[0]`
Definition at line 154 of file [debug_training_data.py](#).

6.7.2.14 model_path

`str debug_training_data.model_path = '/Users/giannigagliardi/Documents/Git/RadioTherapy/unified_energy_↔
conditioned_model_res16.0_energies2.ckpt'`
Definition at line 132 of file [debug_training_data.py](#).

6.7.2.15 output_batch

`debug_training_data.output_batch = batch['outputcube']`
Definition at line 94 of file [debug_training_data.py](#).

6.7.2.16 output_data

`debug_training_data.output_data = np.load(output_file)`
Definition at line 41 of file [debug_training_data.py](#).

6.7.2.17 output_files

`list debug_training_data.output_files`

Initial value:

```
00001 = [  
00002     'traindata/11_5/outputcube/235101017859661465075472232303048949736_0.npy',  
00003     'traindata/15_75/outputcube/235101017859661465075472232303048949736_0.npy',  
00004     'traindata/34_25/outputcube/235101017859661465075472232303048949736_0.npy'  
00005 ]
```

Definition at line 26 of file [debug_training_data.py](#).

6.7.2.18 reconstruction

`debug_training_data.reconstruction = decoded[0]`
Definition at line 160 of file [debug_training_data.py](#).

6.7.2.19 test_files

`list debug_training_data.test_files`

Initial value:

```
00001 = [  
00002     'traindata/11_5/inputcube/235101017859661465075472232303048949736_0.npy',  
00003     'traindata/15_75/inputcube/235101017859661465075472232303048949736_0.npy',  
00004     'traindata/34_25/inputcube/235101017859661465075472232303048949736_0.npy'  
00005 ]
```

Definition at line 20 of file [debug_training_data.py](#).

6.7.2.20 test_input

`debug_training_data.test_input = torch.randn(1, 2, 32, 32, 32)`
Definition at line 123 of file [debug_training_data.py](#).

6.7.2.21 test_output

`debug_training_data.test_output = test_input[:, 0:1, :, :, :]`
 Definition at line 124 of file [debug_training_data.py](#).

6.7.2.22 train_loader

`debug_training_data.train_loader`
 Definition at line 87 of file [debug_training_data.py](#).

6.7.2.23 val_loader

`debug_training_data.val_loader`
 Definition at line 87 of file [debug_training_data.py](#).

6.8 debug_visualization Namespace Reference

Functions

- [visualize_dose_distribution](#) ([cube_path](#), `smooth_sigma=0.5`, `title="Dose Distribution"`)
- [create_slice_comparison](#) ([cube_path1](#), [cube_path2](#), `slice_axis=2`)

Variables

- `str cube_path = "/Users/giannigagliardi/Documents/Git/RadioTherapy/traindata/11_5/outputcube/235101017859661_10.npy"`
- `fig = visualize_dose_distribution(cube_path, smooth_sigma=0.5)`

6.8.1 Function Documentation

6.8.1.1 create_slice_comparison()

```
debug_visualization.create_slice_comparison (
    cube_path1,
    cube_path2,
    slice_axis = 2)
```

Vergleiche zwei Dosisverteilungen slice-weise.

Definition at line 73 of file [debug_visualization.py](#).

6.8.1.2 visualize_dose_distribution()

```
debug_visualization.visualize_dose_distribution (
    cube_path,
    smooth_sigma = 0.5,
    title = "Dose Distribution")
```

Verbesserte Visualisierung für Dosisverteilungen.

Args:

```
cube_path: Pfad zur .npy Datei
smooth_sigma: Sigma für Gaussian-Filter (0 = kein Smoothing)
title: Titel für die Visualisierung
```

Definition at line 6 of file [debug_visualization.py](#).

6.8.2 Variable Documentation

6.8.2.1 cube_path

`str debug_visualization.cube_path = "/Users/giannigagliardi/Documents/Git/RadioTherapy/traindata/11_5/outputcube/235101017859661465075472232303048949736_10.npy"`
 Definition at line 121 of file [debug_visualization.py](#).

6.8.2.2 fig

`debug_visualization.fig = visualize_dose_distribution(cube_path, smooth_sigma=0.5)`
 Definition at line 123 of file `debug_visualization.py`.

6.9 gaussian_quadrature_inference Namespace Reference

Functions

- `setup_logging()`
- `load_combined_checkpoint` (str checkpoint_path)
- `load_ct` (Path path)
- `extract_energy_models` (dict models_by_energy, list target_energies)
- `run_gaussian_quadrature_inference` (ct_array, checkpoint_path, quad_energies, quad_weights, device='cpu', cube_size=(64, 64, 64))
- `save_dose` (Path path, np.ndarray dose, affine=None)
- `create_visualization` (integrated_dose, energy_results, output_dir)
- `main()`

Variables

- bool `has_nibabel` = True

6.9.1 Detailed Description

Gaussian Quadrature Inference for Radiotherapie Dose Prediction.

6.9.2 Function Documentation

6.9.2.1 create_visualization()

```
gaussian_quadrature_inference.create_visualization (
    integrated_dose,
    energy_results,
    output_dir)
```

Create visualization of quadrature integration results.

Definition at line 258 of file `gaussian_quadrature_inference.py`.

6.9.2.2 extract_energy_models()

```
gaussian_quadrature_inference.extract_energy_models (
    dict models_by_energy,
    list target_energies)
```

Extract models for specific energies.

Definition at line 87 of file `gaussian_quadrature_inference.py`.

6.9.2.3 load_combined_checkpoint()

```
gaussian_quadrature_inference.load_combined_checkpoint (
    str checkpoint_path)
```

Load combined checkpoint with all energy models.

Definition at line 37 of file `gaussian_quadrature_inference.py`.

6.9.2.4 load_ct()

```
gaussian_quadrature_inference.load_ct (
    Path path)
```

Load CT data from .npy or .nii/.nii.gz file.

Definition at line 68 of file [gaussian_quadrature_inference.py](#).

6.9.2.5 main()

```
gaussian_quadrature_inference.main ()
```

Main function.

Definition at line 319 of file [gaussian_quadrature_inference.py](#).

6.9.2.6 run_gaussian_quadrature_inference()

```
gaussian_quadrature_inference.run_gaussian_quadrature_inference (
    ct_array,
    checkpoint_path,
    quad_energies,
    quad_weights,
    device = 'cpu',
    cube_size = (64, 64, 64))
```

Run 4-point Gaussian Quadrature Inference.

Args:

```
ct_array: Input CT array
checkpoint_path: Path to combined checkpoint
quad_energies: List of 4 quadrature energies [x1, x2, x3, x4]
quad_weights: List of 4 quadrature weights [w1, w2, w3, w4]
device: PyTorch device
cube_size: Target cube size
```

Returns:

```
integrated_dose: Final integrated dose distribution
```

Definition at line 125 of file [gaussian_quadrature_inference.py](#).

6.9.2.7 save_dose()

```
gaussian_quadrature_inference.save_dose (
    Path path,
    np.ndarray dose,
    affine = None)
```

Save dose array to file.

Definition at line 244 of file [gaussian_quadrature_inference.py](#).

6.9.2.8 setup_logging()

```
gaussian_quadrature_inference.setup_logging ()
```

Setup logging configuration.

Definition at line 28 of file [gaussian_quadrature_inference.py](#).

6.9.3 Variable Documentation

6.9.3.1 has_nibabel

```
bool gaussian_quadrature_inference.has_nibabel = True
```

Definition at line 23 of file [gaussian_quadrature_inference.py](#).

6.10 gui Namespace Reference

Classes

- class [InferenceWorker](#)
- class [MainWindow](#)
- class [TrainingWorker](#)

Functions

- [handle_exception](#) (exc_type, exc_value, exc_tb)

Variables

- [logger](#) = logging.getLogger(__name__)
- [root_logger](#) = logging.getLogger()
- [console_handler](#) = logging.StreamHandler()
- [formatter](#) = logging.Formatter('%(asctime)s %(levelname)s: %(name)s: %(message)s')
- [excepthook](#)
- [app](#) = QApplication(sys.argv)
- [window](#) = [MainWindow](#)()

6.10.1 Function Documentation

6.10.1.1 [handle_exception\(\)](#)

```
gui.handle_exception (  
    exc_type,  
    exc_value,  
    exc_tb)
```

Definition at line 54 of file [gui.py](#).

6.10.2 Variable Documentation

6.10.2.1 [app](#)

```
gui.app = QApplication(sys.argv)
```

Definition at line 926 of file [gui.py](#).

6.10.2.2 [console_handler](#)

```
gui.console_handler = logging.StreamHandler()
```

Definition at line 20 of file [gui.py](#).

6.10.2.3 [excepthook](#)

```
gui.excepthook
```

Definition at line 58 of file [gui.py](#).

6.10.2.4 [formatter](#)

```
gui.formatter = logging.Formatter('%(asctime)s %(levelname)s: %(name)s: %(message)s')
```

Definition at line 22 of file [gui.py](#).

6.10.2.5 [logger](#)

```
gui.logger = logging.getLogger(__name__)
```

Definition at line 12 of file [gui.py](#).

6.10.2.6 root_logger

`gui.root_logger = logging.getLogger()`
 Definition at line 16 of file [gui.py](#).

6.10.2.7 window

`gui.window = MainWindow()`
 Definition at line 927 of file [gui.py](#).

6.11 hotspot_patch_sampler Namespace Reference

Functions

- [extract_hotspot_patches](#) (volume, patch_size=(32, 32, 32), min_hotspot_voxels=10, dose_threshold=0.1, max_patches=16, random_patches=2)

6.11.1 Function Documentation

6.11.1.1 extract_hotspot_patches()

```
hotspot_patch_sampler.extract_hotspot_patches (
    volume,
    patch_size = (32,32,32),
    min_hotspot_voxels = 10,
    dose_threshold = 0.1,
    max_patches = 16,
    random_patches = 2)
```

Extracts patches from a 3D volume where the number of voxels above a certain threshold is sufficient.

Args:

volume: 3D numpy array representing the dose distribution
 patch_size: Size of the patches to extract (z, y, x)
 min_hotspot_voxels: Minimum number of voxels in a patch that must be above the dose threshold
 dose_threshold: Dose threshold to consider a voxel as part of a hotspot
 max_patches: Maximum number of patches to extract
 random_patches: Number of additional random patches to add

Returns:

List of Patch-Start-Indices (e.g. [(z,y,x), ...])

Definition at line 3 of file [hotspot_patch_sampler.py](#).

6.12 inference_module Namespace Reference

Classes

- class [InferenceModule](#)

Variables

- [logger](#) = logging.getLogger(__name__)

6.12.1 Variable Documentation

6.12.1.1 logger

`inference_module.logger = logging.getLogger(__name__)`
 Definition at line 8 of file [inference_module.py](#).

6.13 inference_module_standalone Namespace Reference

Functions

- [load_ct](#) (Path path)
- [create_dose_visualization](#) (dose_array, output_path, title="Dose Distribution", input_path=None)
- [save_dose](#) (Path path, np.ndarray dose, affine=None)
- [save_nifti_with_manifest](#) (dose_array, output_path, root_dir=".")
- [load_models_from_checkpoint](#) (checkpoint_path, energies, device='cpu')
- [get_gaussian_quadrature_4point](#) (energy_min=3.47, energy_max=46.53)
- [main](#) ()

Variables

- bool [has_nibabel](#) = True

6.13.1 Detailed Description

Standalone inference script for radiotherapy dose prediction using the original InferenceModule. This script uses the InferenceModule from sourcecode/ directory.

6.13.2 Function Documentation

6.13.2.1 create_dose_visualization()

```
inference_module_standalone.create_dose_visualization (
    dose_array,
    output_path,
    title = "Dose Distribution",
    input_path = None)
```

Create PNG visualization of dose distribution with optional ground truth comparison.

Definition at line 49 of file [inference_module_standalone.py](#).

6.13.2.2 get_gaussian_quadrature_4point()

```
inference_module_standalone.get_gaussian_quadrature_4point (
    energy_min = 3.47,
    energy_max = 46.53)
```

Get 4-point Gaussian Quadrature nodes and weights for energy integration.

Uses the available energies from the checkpoint with optimized weights for Gaussian dose distribution.

Args:

```
energy_min (float): Minimum energy value (keV) - not used, kept for compatibility
energy_max (float): Maximum energy value (keV) - not used, kept for compatibility
```

Returns:

```
tuple: (energies, weights) for 4-point Gaussian Quadrature
```

Definition at line 465 of file [inference_module_standalone.py](#).

6.13.2.3 load_ct()

```
inference_module_standalone.load_ct (
    Path path)
```

Load CT data from .npy or .nii/.nii.gz file and return array and affine.

Definition at line 36 of file [inference_module_standalone.py](#).

6.13.2.4 load_models_from_checkpoint()

```
inference_module_standalone.load_models_from_checkpoint (
    checkpoint_path,
    energies,
    device = 'cpu')
```

Load models from checkpoint and create models_by_energy dict for InferenceModule.

Args:

```
checkpoint_path (str): Path to model checkpoint
energies (list): List of energy values
device (str): Device to load models on
```

Returns:

```
dict: models_by_energy dictionary
```

Definition at line 298 of file [inference_module_standalone.py](#).

6.13.2.5 main()

```
inference_module_standalone.main ()
```

Definition at line 494 of file [inference_module_standalone.py](#).

6.13.2.6 save_dose()

```
inference_module_standalone.save_dose (
    Path path,
    np.ndarray dose,
    affine = None)
```

Save dose array to .npy or .nii/.nii.gz file using affine if available.

Definition at line 247 of file [inference_module_standalone.py](#).

6.13.2.7 save_nifti_with_manifest()

```
inference_module_standalone.save_nifti_with_manifest (
    dose_array,
    output_path,
    root_dir = ".")
```

Save dose array as NIfTI file with optional manifest attachment for 3D GUI viewing.

Definition at line 257 of file [inference_module_standalone.py](#).

6.13.3 Variable Documentation

6.13.3.1 has_nibabel

```
bool inference_module_standalone.has_nibabel = True
```

Definition at line 20 of file [inference_module_standalone.py](#).

6.14 log_config Namespace Reference

Functions

- [configure_logging \(\)](#)

Variables

- `logger` = logging.getLogger('radiotherapy')

6.14.1 Function Documentation

6.14.1.1 `configure_logging()`

`log_config.configure_logging ()`

Definition at line 5 of file [log_config.py](#).

6.14.2 Variable Documentation

6.14.2.1 `logger`

`log_config.logger = logging.getLogger('radiotherapy')`

Definition at line 33 of file [log_config.py](#).

6.15 parameter_manager Namespace Reference

Classes

- class [ParameterManager](#)

Variables

- `logger` = `logging.getLogger(__name__)`

6.15.1 Variable Documentation

6.15.1.1 `logger`

`parameter_manager.logger = logging.getLogger(__name__)`

Definition at line 5 of file [parameter_manager.py](#).

6.16 simple_inference_test Namespace Reference

Functions

- `main ()`

Variables

- `level`
- `logger` = `logging.getLogger(__name__)`

6.16.1 Detailed Description

Simple test to compare inference output with ground truth data.

6.16.2 Function Documentation

6.16.2.1 `main()`

`simple_inference_test.main ()`

Simple comparison between inference and ground truth.

Definition at line 17 of file [simple_inference_test.py](#).

6.16.3 Variable Documentation

6.16.3.1 `level`

`simple_inference_test.level`

Definition at line 14 of file [simple_inference_test.py](#).

6.16.3.2 logger

`simple_inference_test.logger = logging.getLogger(__name__)`
 Definition at line 15 of file [simple_inference_test.py](#).

6.17 standalone_training Namespace Reference

Functions

- [parse_arguments](#) ()
- [validate_input_folder](#) (input_folder)
- [determine_cube_size](#) (input_folder)
- [get_energy_list](#) (dataset_root)
- [setup_device](#) (device_arg)
- [main](#) ()

Variables

- [project_root](#) = `os.path.abspath(os.path.join(os.path.dirname(__file__), '..'))`
- [logger](#) = `logging.getLogger(__name__)`

6.17.1 Detailed Description

Standalone Training Script for RadioTherapy Project

This script allows you to run training without the GUI. It provides the same functionality as the training feature in the GUI but can be run from command line.

Usage:

```
python standalone_training.py --input_folder /path/to/energy/folder [options]
```

Example:

```
python standalone_training.py --input_folder /path/to/dataset/62_0 --epochs 10 --batch_size 4
```

6.17.2 Function Documentation

6.17.2.1 determine_cube_size()

```
standalone_training.determine_cube_size (
    input_folder)
```

Determine cube size from the first available cube file.

Args:

`input_folder` (str): Path to input cube folder

Returns:

tuple: Cube size dimensions

Definition at line 140 of file [standalone_training.py](#).

6.17.2.2 get_energy_list()

```
standalone_training.get_energy_list (
    dataset_root)
```

Get list of available energies from dataset root directory.

Args:

`dataset_root` (str): Path to dataset root containing energy folders

Returns:

list: List of energy values

Definition at line 178 of file [standalone_training.py](#).

6.17.2.3 main()

`standalone_training.main ()`

Main training function.

Definition at line 216 of file [standalone_training.py](#).

6.17.2.4 parse_arguments()

`standalone_training.parse_arguments ()`

Parse command line arguments for training configuration.

Definition at line 43 of file [standalone_training.py](#).

6.17.2.5 setup_device()

`standalone_training.setup_device (
 device_arg)`

Setup compute device based on argument.

Definition at line 205 of file [standalone_training.py](#).

6.17.2.6 validate_input_folder()

`standalone_training.validate_input_folder (
 input_folder)`

Validate that the input folder structure is correct.

Args:

(str): Path to the energy folder

Returns:

 tuple: (input_cube_path, output_cube_path, dataset_root)

Raises:

 ValueError: If folder structure is invalid

Definition at line 105 of file [standalone_training.py](#).

6.17.3 Variable Documentation

6.17.3.1 logger

`standalone_training.logger = logging.getLogger(__name__)`

Definition at line 40 of file [standalone_training.py](#).

6.17.3.2 project_root

`standalone_training.project_root = os.path.abspath(os.path.join(os.path.dirname(__file__), '..'))`

Definition at line 26 of file [standalone_training.py](#).

6.18 storage Namespace Reference

Classes

- class [Storage](#)

6.19 system_manager Namespace Reference

Classes

- class [SystemManager](#)

Variables

- `logger` = `logging.getLogger(__name__)`

6.19.1 Variable Documentation

6.19.1.1 `logger`

`system_manager.logger` = `logging.getLogger(__name__)`
Definition at line 11 of file `system_manager.py`.

6.20 `systematic_dose_diagnosis` Namespace Reference

Classes

- class `DoseNormalizationDiagnostic`

Functions

- `main` ()

Variables

- `level`
- `INFO`
- `format`
- `logger` = `logging.getLogger(__name__)`
- bool `has_nibabel` = `True`

6.20.1 Detailed Description

Systematic Dose Normalization Diagnosis Script
Phase 1: Root Cause Investigation

This script performs comprehensive analysis of the training data pipeline to identify the exact source of the 21x dose scaling discrepancy.

6.20.2 Function Documentation

6.20.2.1 `main()`

`systematic_dose_diagnosis.main` ()

Main function for running systematic diagnosis.

Definition at line 563 of file `systematic_dose_diagnosis.py`.

6.20.3 Variable Documentation

6.20.3.1 `format`

`systematic_dose_diagnosis.format`

Definition at line 20 of file `systematic_dose_diagnosis.py`.

6.20.3.2 `has_nibabel`

bool `systematic_dose_diagnosis.has_nibabel` = `True`

Definition at line 28 of file `systematic_dose_diagnosis.py`.

6.20.3.3 `INFO`

`systematic_dose_diagnosis.INFO`

Definition at line 20 of file `systematic_dose_diagnosis.py`.

6.20.3.4 level

`systematic_dose_diagnosis.level`

Definition at line 20 of file [systematic_dose_diagnosis.py](#).

6.20.3.5 logger

`systematic_dose_diagnosis.logger = logging.getLogger(__name__)`

Definition at line 21 of file [systematic_dose_diagnosis.py](#).

6.21 test_critical_fixes Namespace Reference

Functions

- [test_dose_focused_preprocessing](#) ()
- [test_extreme_loss_functions](#) ()
- [test_sample_filtering](#) ()
- [test_realistic_training_scenario](#) ()
- [main](#) ()

Variables

- [logger](#) = `logging.getLogger(__name__)`

6.21.1 Detailed Description

Quick test script to validate the critical fixes for ultra-sparse dose training.
Tests dose-focused preprocessing, extreme weighted loss, and sample filtering.

6.21.2 Function Documentation

6.21.2.1 main()

`test_critical_fixes.main` ()

Run all critical tests

Definition at line 196 of file [test_critical_fixes.py](#).

6.21.2.2 test_dose_focused_preprocessing()

`test_critical_fixes.test_dose_focused_preprocessing` ()

Test the dose-focused preprocessing functions

Definition at line 22 of file [test_critical_fixes.py](#).

6.21.2.3 test_extreme_loss_functions()

`test_critical_fixes.test_extreme_loss_functions` ()

Test the extreme loss functions on sparse data

Definition at line 56 of file [test_critical_fixes.py](#).

6.21.2.4 test_realistic_training_scenario()

`test_critical_fixes.test_realistic_training_scenario` ()

Simulate realistic training with ultra-sparse data

Definition at line 137 of file [test_critical_fixes.py](#).

6.21.2.5 test_sample_filtering()

`test_critical_fixes.test_sample_filtering ()`

Test the sample filtering in DataLoaderModule

Definition at line 110 of file [test_critical_fixes.py](#).

6.21.3 Variable Documentation

6.21.3.1 logger

`test_critical_fixes.logger = logging.getLogger(__name__)`

Definition at line 20 of file [test_critical_fixes.py](#).

6.22 test_error_logging Namespace Reference

Functions

- [test_error_logging \(\)](#)

Variables

- [console_handler](#) = `logging.StreamHandler()`
- [formatter](#) = `logging.Formatter('%(asctime)s %(levelname)s: %(name)s: %(message)s')`
- [root_logger](#) = `logging.getLogger()`
- [logger](#) = `logging.getLogger(__name__)`

6.22.1 Detailed Description

Test error logging in the corrected inference pipeline

6.22.2 Function Documentation

6.22.2.1 test_error_logging()

`test_error_logging.test_error_logging ()`

Test error logging by providing an invalid model checkpoint

Definition at line 31 of file [test_error_logging.py](#).

6.22.3 Variable Documentation

6.22.3.1 console_handler

`test_error_logging.console_handler = logging.StreamHandler()`

Definition at line 15 of file [test_error_logging.py](#).

6.22.3.2 formatter

`test_error_logging.formatter = logging.Formatter('%(asctime)s %(levelname)s: %(name)s: %(message)s')`

Definition at line 17 of file [test_error_logging.py](#).

6.22.3.3 logger

`test_error_logging.logger = logging.getLogger(__name__)`

Definition at line 25 of file [test_error_logging.py](#).

6.22.3.4 root_logger

`test_error_logging.root_logger = logging.getLogger()`

Definition at line 21 of file [test_error_logging.py](#).

6.23 test_gui Namespace Reference

Functions

- [qapp](#) ()
- [window](#) ([qapp](#), [qtbot](#), [tmp_path](#), [monkeypatch](#))
- [test_initial_button_states](#) ([window](#))
- [test_energy_range_validation](#) ([window](#), [monkeypatch](#))
- [test_select_input_folder_enables_train](#) ([window](#))

6.23.1 Function Documentation

6.23.1.1 [qapp\(\)](#)

`test_gui.qapp ()`

Provides a single QApplication instance for all tests.

Definition at line 9 of file [test_gui.py](#).

6.23.1.2 [test_energy_range_validation\(\)](#)

```
test_gui.test_energy_range_validation (  
    window,  
    monkeypatch)
```

`calculate_dose` raises an exception when `min > max`.

Definition at line 51 of file [test_gui.py](#).

6.23.1.3 [test_initial_button_states\(\)](#)

```
test_gui.test_initial_button_states (  
    window)
```

`Dose`, `Train`, and `Visualize` buttons are initially disabled.

Definition at line 45 of file [test_gui.py](#).

6.23.1.4 [test_select_input_folder_enables_train\(\)](#)

```
test_gui.test_select_input_folder_enables_train (  
    window)
```

`select_input_folder` enables the `train_button` when structure is correct.

Definition at line 69 of file [test_gui.py](#).

6.23.1.5 [window\(\)](#)

```
test_gui.window (  
    qapp,  
    qtbot,  
    tmp_path,  
    monkeypatch)
```

Creates a `MainWindow`, constructs a dummy dataset in `tmp_path`, patches `QFileDialog.select_input_folder`, and returns the window.

Definition at line 15 of file [test_gui.py](#).

6.24 test_gui_inference Namespace Reference

Functions

- [test_gui_inference](#) ()

6.24.1 Detailed Description

Test that the GUI would produce corrected inference output

6.24.2 Function Documentation

6.24.2.1 `test_gui_inference()`

`test_gui_inference.test_gui_inference ()`

Simulate what the GUI would do for inference

Definition at line 14 of file `test_gui_inference.py`.

6.25 `test_gui_training` Namespace Reference

Functions

- `test_gui_training ()`

Variables

- `logger` = `logging.getLogger(__name__)`
- `success` = `test_gui_training()`

6.25.1 Detailed Description

Test GUI Training (nicht das vollständige GUI, nur die Training-Logik)

6.25.2 Function Documentation

6.25.2.1 `test_gui_training()`

`test_gui_training.test_gui_training ()`

Test the GUI training logic without actually running the GUI

Definition at line 20 of file `test_gui_training.py`.

6.25.3 Variable Documentation

6.25.3.1 `logger`

`test_gui_training.logger` = `logging.getLogger(__name__)`

Definition at line 14 of file `test_gui_training.py`.

6.25.3.2 `success`

`test_gui_training.success` = `test_gui_training()`

Definition at line 66 of file `test_gui_training.py`.

6.26 `test_pipeline_fixes` Namespace Reference

Functions

- `test_transforms_consistency ()`
- `test_dose_range_detection ()`
- `test_learning_rate_adjustment ()`
- `run_comprehensive_test ()`

Variables

- `level`
- `INFO`
- `format`
- `logger = logging.getLogger(__name__)`
- `success = run_comprehensive_test()`

6.26.1 Detailed Description

Test script to validate pipeline fixes for RadioTherapy dose prediction.
This script tests the key fixes applied to resolve the 30-50x too low dose values.

6.26.2 Function Documentation

6.26.2.1 `run_comprehensive_test()`

`test_pipeline_fixes.run_comprehensive_test ()`

Run all tests and summarize results.

Definition at line 127 of file `test_pipeline_fixes.py`.

6.26.2.2 `test_dose_range_detection()`

`test_pipeline_fixes.test_dose_range_detection ()`

Test dose range detection logic.

Definition at line 71 of file `test_pipeline_fixes.py`.

6.26.2.3 `test_learning_rate_adjustment()`

`test_pipeline_fixes.test_learning_rate_adjustment ()`

Test learning rate is properly adjusted after fixes.

Definition at line 107 of file `test_pipeline_fixes.py`.

6.26.2.4 `test_transforms_consistency()`

`test_pipeline_fixes.test_transforms_consistency ()`

Test that transforms produce consistent 64x64x64 outputs.

Definition at line 16 of file `test_pipeline_fixes.py`.

6.26.3 Variable Documentation

6.26.3.1 `format`

`test_pipeline_fixes.format`

Definition at line 13 of file `test_pipeline_fixes.py`.

6.26.3.2 `INFO`

`test_pipeline_fixes.INFO`

Definition at line 13 of file `test_pipeline_fixes.py`.

6.26.3.3 `level`

`test_pipeline_fixes.level`

Definition at line 13 of file `test_pipeline_fixes.py`.

6.26.3.4 logger

`test_pipeline_fixes.logger = logging.getLogger(__name__)`
 Definition at line 14 of file [test_pipeline_fixes.py](#).

6.26.3.5 success

`test_pipeline_fixes.success = run_comprehensive_test()`
 Definition at line 170 of file [test_pipeline_fixes.py](#).

6.27 test_system_manager_inference Namespace Reference

Functions

- [test_system_manager_inference](#) ()

Variables

- [console_handler](#) = logging.StreamHandler()
- [formatter](#) = logging.Formatter('%(asctime)s %(levelname)s: %(name)s: %(message)s')
- [root_logger](#) = logging.getLogger()
- [logger](#) = logging.getLogger(__name__)

6.27.1 Detailed Description

Test the corrected inference through SystemManager

6.27.2 Function Documentation

6.27.2.1 test_system_manager_inference()

`test_system_manager_inference.test_system_manager_inference ()`

Test that the SystemManager uses the corrected inference

Definition at line 31 of file [test_system_manager_inference.py](#).

6.27.3 Variable Documentation

6.27.3.1 console_handler

`test_system_manager_inference.console_handler = logging.StreamHandler()`
 Definition at line 15 of file [test_system_manager_inference.py](#).

6.27.3.2 formatter

`test_system_manager_inference.formatter = logging.Formatter('%(asctime)s %(levelname)s: %(name)s: %(message)s')`
 Definition at line 17 of file [test_system_manager_inference.py](#).

6.27.3.3 logger

`test_system_manager_inference.logger = logging.getLogger(__name__)`
 Definition at line 25 of file [test_system_manager_inference.py](#).

6.27.3.4 root_logger

`test_system_manager_inference.root_logger = logging.getLogger()`
 Definition at line 21 of file [test_system_manager_inference.py](#).

6.28 test_system_manager_real_data Namespace Reference

Functions

- [test_system_manager_with_real_data](#) ()

6.28.1 Detailed Description

Test SystemManager with real training data

6.28.2 Function Documentation

6.28.2.1 test_system_manager_with_real_data()

`test_system_manager_real_data.test_system_manager_with_real_data` ()

Test SystemManager with actual training data

Definition at line 14 of file [test_system_manager_real_data.py](#).

6.29 test_systemmanager Namespace Reference

Functions

- [temp_root](#) (tmp_path)
- [manager](#) (temp_root)
- [test_run_inference_no_models_and_no_checkpoint](#) (manager)
- [test_run_inference_invalid_checkpoint_type](#) (manager)
- [test_run_inference_unsupported_ct_extension](#) (manager)

6.29.1 Function Documentation

6.29.1.1 manager()

`test_systemmanager.manager` (
 temp_root)

Initialize a SystemManager with minimal settings.

Definition at line 14 of file [test_systemmanager.py](#).

6.29.1.2 temp_root()

`test_systemmanager.temp_root` (
 tmp_path)

Create a temporary root directory.

Definition at line 9 of file [test_systemmanager.py](#).

6.29.1.3 test_run_inference_invalid_checkpoint_type()

`test_systemmanager.test_run_inference_invalid_checkpoint_type` (
 manager)

Providing a checkpoint of unsupported type should raise ValueError.

Definition at line 42 of file [test_systemmanager.py](#).

6.29.1.4 test_run_inference_no_models_and_no_checkpoint()

```
test_systemmanager.test_run_inference_no_models_and_no_checkpoint (
    manager)
```

If no `models_by_energy` are loaded and no checkpoint is provided, `run_inference` should raise `RuntimeError`.

Definition at line 32 of file [test_systemmanager.py](#).

6.29.1.5 test_run_inference_unsupported_ct_extension()

```
test_systemmanager.test_run_inference_unsupported_ct_extension (
    manager)
```

Providing an unsupported CT file extension should raise `ValueError`.

Definition at line 51 of file [test_systemmanager.py](#).

6.30 test_training Namespace Reference

Variables

- [level](#)
- `device` = `torch.device("cuda" if torch.cuda.is_available() else "cpu")`
- dict [config](#)
- [system_manager](#)
- `result` = `system_manager.run_energy_conditioned_training()`

6.30.1 Detailed Description

Simple test script for energy-conditioned training

6.30.2 Variable Documentation

6.30.2.1 config

```
dict test_training.config
```

Initial value:

```
00001 = {
00002     'root_dir': 'traindata',
00003     'resolutions': [16.0], # Just one resolution
00004     'energies': [11.5, 15.75], # Just two energies for faster testing
00005     'quad_energies': [11.5, 15.75],
00006     'quad_weights': [1.0, 1.0],
00007     'batch_size': 1, # Small batch
00008     'num_epochs': 2, # Just 2 epochs for testing
00009     'learning_rate': 1e-4,
00010     'patience': 2,
00011     'cube_size': (32, 32, 32), # Small cube for speed
00012     'seed': 42
00013 }
```

Definition at line 30 of file [test_training.py](#).

6.30.2.2 device

```
test_training.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

Definition at line 26 of file [test_training.py](#).

6.30.2.3 level

```
test_training.level
```

Definition at line 23 of file [test_training.py](#).

6.30.2.4 result

test_training.result = system_manager.run_energy_conditioned_training()
 Definition at line 71 of file [test_training.py](#).

6.30.2.5 system_manager

test_training.system_manager

Initial value:

```
00001 = SystemManager(
00002     root_dir=config['root_dir'],
00003     transforms=Compose([]), # Will be set in method
00004     resolutions=config['resolutions'],
00005     energies=config['energies'],
00006     quad_energies=config['quad_energies'],
00007     quad_weights=config['quad_weights'],
00008     batch_size=config['batch_size'],
00009     device=device,
00010     num_epochs=config['num_epochs'],
00011     learning_rate=config['learning_rate'],
00012     patience=config['patience'],
00013     cube_size=config['cube_size'],
00014     seed=config['seed']
00015 )
```

Definition at line 51 of file [test_training.py](#).

6.31 test_visualization Namespace Reference

Functions

- [test_interactive_slice_viewer_slicing](#) ()
- [test_invalid_volume_raises](#) ()

6.31.1 Function Documentation

6.31.1.1 test_interactive_slice_viewer_slicing()

test_visualization.test_interactive_slice_viewer_slicing ()

Test that interactive_slice_viewer correctly updates the image data when the slider value changes.

Definition at line 5 of file [test_visualization.py](#).

6.31.1.2 test_invalid_volume_raises()

test_visualization.test_invalid_volume_raises ()

interactive_slice_viewer should reject non-3D arrays.

Definition at line 33 of file [test_visualization.py](#).

6.32 training_pipeline Namespace Reference

Classes

- class [AutoencoderTrainer](#)
- class [DiffusionTrainer](#)
- class [EarlyStopping](#)

Variables

- int [DOSE_EPS](#) = 1e-8
- int [TV_WEIGHT](#) = 5e-3
- [CLIP_MIN](#) = None
- [CLIP_MAX](#) = None
- [logger](#) = logging.getLogger(__name__)

6.32.1 Variable Documentation

6.32.1.1 CLIP_MAX

`training_pipeline.CLIP_MAX = None`

Definition at line 25 of file [training_pipeline.py](#).

6.32.1.2 CLIP_MIN

`training_pipeline.CLIP_MIN = None`

Definition at line 24 of file [training_pipeline.py](#).

6.32.1.3 DOSE_EPS

`int training_pipeline.DOSE_EPS = 1e-8`

Definition at line 20 of file [training_pipeline.py](#).

6.32.1.4 logger

`training_pipeline.logger = logging.getLogger(__name__)`

Definition at line 27 of file [training_pipeline.py](#).

6.32.1.5 TV_WEIGHT

`int training_pipeline.TV_WEIGHT = 5e-3`

Definition at line 21 of file [training_pipeline.py](#).

6.33 validate_energy_conditioning Namespace Reference

Variables

- [level](#)
- [device](#) = `torch.device("cuda" if torch.cuda.is_available() else "cpu")`
- dict [config](#)
- [checkpoint](#) = `torch.load(config['model_path'], map_location=device)`
- [training_config](#) = `checkpoint['training_config']`
- [autoencoder](#)
- str [sample_input_path](#) = `'traindata/11_5/inputcube/235101017859661465075472232303048949736_0.npy'`
- str [sample_output_path](#) = `'traindata/11_5/outputcube/235101017859661465075472232303048949736_0.npy'`
- [physical_input](#) = `np.load(sample_input_path)`
- [ground_truth](#) = `np.load(sample_output_path)`
- dict [results](#) = `{}`
- dict [predictions](#) = `{}`
- float [energy_normalized](#) = `energy / 40.0`
- [input_2channel](#) = `np.zeros((2, *physical_input.shape))`
- [input_tensor](#) = `torch.tensor(input_2channel, dtype=torch.float32).unsqueeze(0).to(device)`
- [prediction](#) = `autoencoder(input_tensor)`
- [prediction_np](#) = `prediction.cpu().numpy().squeeze()`
- [pred_mean](#) = `np.mean(prediction_np)`
- [pred_std](#) = `np.std(prediction_np)`
- [pred_max](#) = `np.max(prediction_np)`
- [pred_min](#) = `np.min(prediction_np)`
- [energy_list](#) = `list(config['energies'])`
- [energy1](#)
- [energy2](#)
- [pred1](#)

- `pred2`
- `mse = np.mean((pred1 - pred2) ** 2)`
- `mae = np.mean(np.abs(pred1 - pred2))`
- `max_diff = np.max(np.abs(pred1 - pred2))`
- `correlation = np.corrcoef(pred1.flatten(), pred2.flatten())[0, 1]`
- `list means = [results[e]['mean'] for e in energy_list]`
- `list stds = [results[e]['std'] for e in energy_list]`
- `int mean_variation = 0 else 0`
- `int std_variation = 0 else 0`
- `int max_mse = 0`
- `float min_correlation = 1.0`

6.33.1 Detailed Description

Validation script to test if energy conditioning actually works

This script tests the trained unified model to see if it produces different outputs for different energies

6.33.2 Variable Documentation

6.33.2.1 autoencoder

`validate_energy_conditioning.autoencoder`

Initial value:

```
00001 = AutoencoderKL(
00002     spatial_dims=3,
00003     in_channels=2, # CT + energy conditioning
00004     out_channels=1, # Output dose
00005     num_channels=(32, 32, 32), # Correct configuration from training
00006     latent_channels=2, # Correct configuration from training
00007     num_res_blocks=1, # Correct configuration from training
00008     norm_num_groups=8, # Correct configuration from training
00009     attention_levels=(False, False, True), # Correct configuration from training
00010 ).to(device)
```

Definition at line 65 of file `validate_energy_conditioning.py`.

6.33.2.2 checkpoint

`validate_energy_conditioning.checkpoint = torch.load(config['model_path'], map_location=device)`

Definition at line 51 of file `validate_energy_conditioning.py`.

6.33.2.3 config

`dict validate_energy_conditioning.config`

Initial value:

```
00001 = {
00002     'root_dir': 'traindata',
00003     'resolutions': [16.0],
00004     'energies': [11.5, 15.75, 34.25], # Test all three energies
00005     'batch_size': 1,
00006     'cube_size': (32, 32, 32),
00007     'model_path':
00008         '/Users/giannigagliardi/Documents/Git/RadioTherapy/unified_energy_conditioned_model_res16.0_energies2.ckpt'
00008 }
```

Definition at line 31 of file `validate_energy_conditioning.py`.

6.33.2.4 correlation

`validate_energy_conditioning.correlation = np.corrcoef(pred1.flatten(), pred2.flatten())[0, 1]`

Definition at line 177 of file `validate_energy_conditioning.py`.

6.33.2.5 device

`validate_energy_conditioning.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")`

Definition at line 27 of file `validate_energy_conditioning.py`.

6.33.2.6 energy1

`validate_energy_conditioning.energy1`

Definition at line 170 of file [validate_energy_conditioning.py](#).

6.33.2.7 energy2

`validate_energy_conditioning.energy2`

Definition at line 170 of file [validate_energy_conditioning.py](#).

6.33.2.8 energy_list

`validate_energy_conditioning.energy_list = list(config['energies'])`

Definition at line 165 of file [validate_energy_conditioning.py](#).

6.33.2.9 energy_normalized

`float validate_energy_conditioning.energy_normalized = energy / 40.0`

Definition at line 119 of file [validate_energy_conditioning.py](#).

6.33.2.10 ground_truth

`validate_energy_conditioning.ground_truth = np.load(sample_output_path)`

Definition at line 101 of file [validate_energy_conditioning.py](#).

6.33.2.11 input_2channel

`validate_energy_conditioning.input_2channel = np.zeros((2, *physical_input.shape))`

Definition at line 122 of file [validate_energy_conditioning.py](#).

6.33.2.12 input_tensor

`validate_energy_conditioning.input_tensor = torch.tensor(input_2channel, dtype=torch.float32).unsqueeze(0).to(device)`

Definition at line 127 of file [validate_energy_conditioning.py](#).

6.33.2.13 level

`validate_energy_conditioning.level`

Definition at line 24 of file [validate_energy_conditioning.py](#).

6.33.2.14 mae

`validate_energy_conditioning.mae = np.mean(np.abs(pred1 - pred2))`

Definition at line 175 of file [validate_energy_conditioning.py](#).

6.33.2.15 max_diff

`validate_energy_conditioning.max_diff = np.max(np.abs(pred1 - pred2))`

Definition at line 176 of file [validate_energy_conditioning.py](#).

6.33.2.16 max_mse

`validate_energy_conditioning.max_mse = 0`

Definition at line 215 of file [validate_energy_conditioning.py](#).

6.33.2.17 mean_variation

`int validate_energy_conditioning.mean_variation = 0 else 0`

Definition at line 204 of file [validate_energy_conditioning.py](#).

6.33.2.18 means

`list validate_energy_conditioning.means = [results[e]['mean'] for e in energy_list]`
Definition at line 201 of file `validate_energy_conditioning.py`.

6.33.2.19 min_correlation

`validate_energy_conditioning.min_correlation = 1.0`
Definition at line 216 of file `validate_energy_conditioning.py`.

6.33.2.20 mse

`validate_energy_conditioning.mse = np.mean((pred1 - pred2) ** 2)`
Definition at line 174 of file `validate_energy_conditioning.py`.

6.33.2.21 physical_input

`validate_energy_conditioning.physical_input = np.load(sample_input_path)`
Definition at line 100 of file `validate_energy_conditioning.py`.

6.33.2.22 pred1

`validate_energy_conditioning.pred1`
Definition at line 171 of file `validate_energy_conditioning.py`.

6.33.2.23 pred2

`validate_energy_conditioning.pred2`
Definition at line 171 of file `validate_energy_conditioning.py`.

6.33.2.24 pred_max

`validate_energy_conditioning.pred_max = np.max(prediction_np)`
Definition at line 145 of file `validate_energy_conditioning.py`.

6.33.2.25 pred_mean

`validate_energy_conditioning.pred_mean = np.mean(prediction_np)`
Definition at line 143 of file `validate_energy_conditioning.py`.

6.33.2.26 pred_min

`validate_energy_conditioning.pred_min = np.min(prediction_np)`
Definition at line 146 of file `validate_energy_conditioning.py`.

6.33.2.27 pred_std

`validate_energy_conditioning.pred_std = np.std(prediction_np)`
Definition at line 144 of file `validate_energy_conditioning.py`.

6.33.2.28 prediction

`validate_energy_conditioning.prediction = autoencoder(input_tensor)`
Definition at line 134 of file `validate_energy_conditioning.py`.

6.33.2.29 prediction_np

`validate_energy_conditioning.prediction_np = prediction.cpu().numpy().squeeze()`
Definition at line 138 of file `validate_energy_conditioning.py`.

6.33.2.30 predictions

`dict validate_energy_conditioning.predictions = {}`
 Definition at line 111 of file [validate_energy_conditioning.py](#).

6.33.2.31 results

`dict validate_energy_conditioning.results = {}`
 Definition at line 110 of file [validate_energy_conditioning.py](#).

6.33.2.32 sample_input_path

`str validate_energy_conditioning.sample_input_path = 'traindata/11_5/inputcube/235101017859661465075472232303048949736↵_0.npy'`
 Definition at line 92 of file [validate_energy_conditioning.py](#).

6.33.2.33 sample_output_path

`str validate_energy_conditioning.sample_output_path = 'traindata/11_5/outputcube/235101017859661465075472232303048949736↵_0.npy'`
 Definition at line 93 of file [validate_energy_conditioning.py](#).

6.33.2.34 std_variation

`int validate_energy_conditioning.std_variation = 0 else 0`
 Definition at line 205 of file [validate_energy_conditioning.py](#).

6.33.2.35 stds

`list validate_energy_conditioning.stds = [results[e]['std'] for e in energy_list]`
 Definition at line 202 of file [validate_energy_conditioning.py](#).

6.33.2.36 training_config

`validate_energy_conditioning.training_config = checkpoint['training_config']`
 Definition at line 58 of file [validate_energy_conditioning.py](#).

6.34 visualization Namespace Reference

Classes

- class [MplCanvas](#)
- class [Visualization](#)

Functions

- [load_and_visualize](#) (nifti_path, ct_scan=None, title="Dose distribution - Axial View")
- [get_visualization_widget](#) (figure)
- [interactive_slice_viewer](#) (cube, ct_volume=None, title="Interactive Slice Viewer", show=True)
- [volume_rendering](#) (cube, opacity=0.2, surface_count=20)

6.34.1 Function Documentation

6.34.1.1 get_visualization_widget()

`visualization.get_visualization_widget (`
 figure)

Converts a matplotlib figure to a Qt widget for embedding in the GUI.

Args:

`figure`: matplotlib.figure.Figure object

Returns:

MplCanvas: A Qt widget containing the visualization

Definition at line 188 of file [visualization.py](#).

6.34.1.2 interactive_slice_viewer()

```
visualization.interactive_slice_viewer (
    cube,
    ct_volume = None,
    title = "Interactive Slice Viewer",
    show = True)
```

Launches an interactive Matplotlib window with a slider to browse through axial slices of a 3D volume.

Definition at line 201 of file [visualization.py](#).

6.34.1.3 load_and_visualize()

```
visualization.load_and_visualize (
    nifti_path,
    ct_scan = None,
    title = "Dose distribution - Axial View")
```

Loads a Nifti dose distribution file and visualizes its middle axial slice.

Args:

`nifti_path`: Path to the NIfTI file
`title`: Title for the visualization

Returns:

matplotlib.figure.Figure: A figure object that can be displayed

Definition at line 161 of file [visualization.py](#).

6.34.1.4 volume_rendering()

```
visualization.volume_rendering (
    cube,
    opacity = 0.2,
    surface_count = 20)
```

Performs a 3D volume rendering of a normalized 3D array using Plotly.

Definition at line 301 of file [visualization.py](#).

6.35 visualizeTEST Namespace Reference

Functions

- [update](#) (val)

Variables

- `cube` = np.load("/Users/giannigagliardi/Documents/Git/RadioTherapy/traindata/11_5/outputcube/2351010178596614_10.npy")
- `fig`
- `ax`
- `int slice_idx` = cube.shape[2] // 2

- `im = ax.imshow(cube[:, :, slice_idx], cmap="gray")`
- `slider_ax = plt.axes([0.2, 0.05, 0.6, 0.03])`
- `slider`
- `tuple cube_norm = (cube - cube.min()) / (cube.max() - cube.min())`
- `p_low`
- `p_high`
- `x`
- `y`
- `z`
- `cube_smoothed = ndimage.gaussian_filter(cube_norm, sigma=0.5)`
- `fig_vol`
- `scene`

6.35.1 Function Documentation

6.35.1.1 `update()`

`visualizeTEST.update (`
 `val)`

Definition at line 27 of file `visualizeTEST.py`.

6.35.2 Variable Documentation

6.35.2.1 `ax`

`visualizeTEST.ax`

Definition at line 15 of file `visualizeTEST.py`.

6.35.2.2 `cube`

`visualizeTEST.cube = np.load("/Users/giannigagliardi/Documents/Git/RadioTherapy/traindata/11_5/outputcube/235101017859661465_10.npy")`

Definition at line 6 of file `visualizeTEST.py`.

6.35.2.3 `cube_norm`

`tuple visualizeTEST.cube_norm = (cube - cube.min()) / (cube.max() - cube.min())`

Definition at line 37 of file `visualizeTEST.py`.

6.35.2.4 `cube_smoothed`

`visualizeTEST.cube_smoothed = ndimage.gaussian_filter(cube_norm, sigma=0.5)`

Definition at line 49 of file `visualizeTEST.py`.

6.35.2.5 `fig`

`visualizeTEST.fig`

Definition at line 15 of file `visualizeTEST.py`.

6.35.2.6 `fig_vol`

`visualizeTEST.fig_vol`

Initial value:

```
00001 = go.Figure(
00002     data=go.Volume(
00003         x=x.flatten(),
00004         y=y.flatten(),
00005         z=z.flatten(),
00006         value=cube_smoothed.flatten(),
00007         opacity=0.1,           # mehr transparent für bessere Sicht
00008         surface_count=15,      # weniger Isoflächen für glatteres Aussehen
00009         isomin=p_low * 1.2,    # etwas höhere Schwelle
00010         isomax=p_high * 0.8,   # etwas niedrigere obere Schwelle
```

```
00011         colorscale="Hot",          # bessere Farbskala für Dosisverteilung
00012         caps=dict(x_show=False, y_show=False, z_show=False) # keine Kappen
00013     )
00014 )
```

Definition at line 53 of file [visualizeTEST.py](#).

6.35.2.7 im

```
visualizeTEST.im = ax.imshow(cube[:, :, slice_idx], cmap="gray")
```

Definition at line 17 of file [visualizeTEST.py](#).

6.35.2.8 p_high

```
visualizeTEST.p_high
```

Definition at line 40 of file [visualizeTEST.py](#).

6.35.2.9 p_low

```
visualizeTEST.p_low
```

Definition at line 40 of file [visualizeTEST.py](#).

6.35.2.10 scene

```
visualizeTEST.scene
```

Definition at line 68 of file [visualizeTEST.py](#).

6.35.2.11 slice_idx

```
int visualizeTEST.slice_idx = cube.shape[2] // 2
```

Definition at line 16 of file [visualizeTEST.py](#).

6.35.2.12 slider

```
visualizeTEST.slider
```

Initial value:

```
00001 = widgets.Slider(
00002     slider_ax, "z-Slice", 0, cube.shape[2] - 1,
00003     valinit=slice_idx, valfmt="%0.0f"
00004 )
```

Definition at line 22 of file [visualizeTEST.py](#).

6.35.2.13 slider_ax

```
visualizeTEST.slider_ax = plt.axes([0.2, 0.05, 0.6, 0.03])
```

Definition at line 21 of file [visualizeTEST.py](#).

6.35.2.14 x

```
visualizeTEST.x
```

Definition at line 44 of file [visualizeTEST.py](#).

6.35.2.15 y

```
visualizeTEST.y
```

Definition at line 44 of file [visualizeTEST.py](#).

6.35.2.16 z

```
visualizeTEST.z
```

Definition at line 44 of file [visualizeTEST.py](#).

Chapter 7

Class Documentation

7.1 training_pipeline.AutoencoderTrainer Class Reference

Public Member Functions

- `__init__` (self, `autoencoder`, `discriminator`, `optimizer_g`, `optimizer_d`, `energy_min`, `energy_max`, `device`, `kl_weight`=1e-6, `adv_weight`=0.001, `perceptual_weight`=0.005, `tv_weight`=TV_WEIGHT, `warm_up_epochs`=2)
- `calculate_dose_normalization_params` (self, `dataloader`, `energy`=None)
- `setup_loss_functions` (self)
- `weighted_dose_loss` (self, `pred`, `target`, `weight_factor`=10000.0, `dose_threshold`=1e-6)
- `masked_dose_loss` (self, `pred`, `target`, `dose_threshold`=1e-6)
- `extreme_dose_focused_loss` (self, `pred`, `target`, `dose_threshold`=1e-6)
- `train_one_epoch` (self, `train_loader`, `epoch`)
- `validate` (self, `val_loader`)

Public Attributes

- `autoencoder` = `autoencoder`
- `discriminator` = `discriminator`
- `optimizer_g` = `optimizer_g`
- `optimizer_d` = `optimizer_d`
- `device` = `device`
- float `clip_min` = None
- float `clip_max` = None
- `energy_min` = `energy_min`
- `energy_max` = `energy_max`
- `kl_weight` = `kl_weight`
- `adv_weight` = `adv_weight`
- `perceptual_weight` = `perceptual_weight`
- `tv_weight` = `tv_weight`
- `warm_up_epochs` = `warm_up_epochs`
- `l1_loss` = None
- `adv_loss` = None
- `perceptual_loss` = None
- `dose_mean` = `mean`
- `dose_std` = `std`

Protected Member Functions

- `_compute_dataset_dose_stats` (self, `loader`)

Static Protected Member Functions

- [_total_variation_loss](#) (img)

7.1.1 Detailed Description

Encapsulates the training process for the autoencoder, including validation.

This implementation has been updated to integrate energy conditioning.

Each sample is expected to contain an "energy" field, which is normalized, expanded to match the spatial dimensions of the image, and then concatenated as an additional input channel.

Definition at line 66 of file [training_pipeline.py](#).

7.1.2 Constructor & Destructor Documentation

7.1.2.1 `__init__()`

```
training_pipeline.AutoencoderTrainer.__init__ (
    self,
    autoencoder,
    discriminator,
    optimizer_g,
    optimizer_d,
    energy_min,
    energy_max,
    device,
    kl_weight = 1e-6,
    adv_weight = 0.001,
    perceptual_weight = 0.005,
    tv_weight = TV_WEIGHT,
    warm_up_epochs = 2)
```

Args:

autoencoder (torch.nn.Module): The autoencoder model.
 discriminator (torch.nn.Module): The discriminator model (for adversarial training).
 optimizer_g (torch.optim.Optimizer): Optimizer for the generator.
 optimizer_d (torch.optim.Optimizer): Optimizer for the discriminator.
 device (str): Device to run the training on ('cpu' or 'cuda').
 kl_weight (float): Weight for KL divergence loss.
 adv_weight (float): Weight for adversarial loss. Default: 0.001
 perceptual_weight (float): Weight for perceptual loss. Default: 0.005
 tv_weight (float): Weight for total-variation loss.
 warm_up_epochs (int): Number of epochs to warm up the KL weight.

Definition at line 74 of file [training_pipeline.py](#).

7.1.3 Member Function Documentation

7.1.3.1 `_compute_dataset_dose_stats()`

```
training_pipeline.AutoencoderTrainer._compute_dataset_dose_stats (
    self,
    loader) [protected]
```

Go once over the whole loader to obtain global mean / std of the ground-truth dose. These statistics are required for a stable normalisation of the target and the network output and will be cached in 'self.dose_mean' and 'self.dose_std'.

Args:

loader (DataLoader): training (or validation) loader that yields dictionaries with a "target" key.

Definition at line 347 of file [training_pipeline.py](#).

7.1.3.2 `_total_variation_loss()`

```
training_pipeline.AutoencoderTrainer._total_variation_loss (  
    img) [static], [protected]
```

3-D total-variation loss encouraging spatial smoothness.

Args:

`img` (torch.Tensor): tensor of shape [B, C, D, H, W]

Definition at line 389 of file [training_pipeline.py](#).

7.1.3.3 `calculate_dose_normalization_params()`

```
training_pipeline.AutoencoderTrainer.calculate_dose_normalization_params (  
    self,  
    dataloader,  
    energy = None)
```

Calculate clip_min and clip_max from the training data for dose normalization. This should be called before training starts.

Args:

`dataloader`: Training dataloader
`energy`: Energy level for energy-specific normalization

Definition at line 112 of file [training_pipeline.py](#).

7.1.3.4 `extreme_dose_focused_loss()`

```
training_pipeline.AutoencoderTrainer.extreme_dose_focused_loss (  
    self,  
    pred,  
    target,  
    dose_threshold = 1e-6)
```

Combination of extreme weighted and pure masked loss for ultra-sparse data. Uses 100,000x weights and pure MSE loss for maximum signal.

Definition at line 296 of file [training_pipeline.py](#).

7.1.3.5 `masked_dose_loss()`

```
training_pipeline.AutoencoderTrainer.masked_dose_loss (  
    self,  
    pred,  
    target,  
    dose_threshold = 1e-6)
```

Pure dose-only loss - ignores background completely.

Args:

`pred`: Predicted dose values
`target`: Ground truth dose values
`dose_threshold`: Minimum value to consider as "real dose"

Definition at line 265 of file [training_pipeline.py](#).

7.1.3.6 `setup_loss_functions()`

```
training_pipeline.AutoencoderTrainer.setup_loss_functions (  
    self)
```

Initialize loss functions after normalization parameters are set.

Definition at line 222 of file [training_pipeline.py](#).

7.1.3.7 train_one_epoch()

```
training_pipeline.AutoencoderTrainer.train_one_epoch (
    self,
    train_loader,
    epoch)
```

Trains the autoencoder and the discriminator for an epoch.

This method integrates energy conditioning: for each batch, it extracts the "energy" values, normalizes them, expands them to match the spatial dimensions, and concatenates them as an additional channel to the input images. The autoencoder must be adapted to accept the increased number of input channels.

Args:

train_loader (DataLoader): DataLoader for training data.
epoch (int): Current epoch.

Returns:

tuple: Average reconstruction loss, generator loss and discriminator loss.

Definition at line 401 of file [training_pipeline.py](#).

7.1.3.8 validate()

```
training_pipeline.AutoencoderTrainer.validate (
    self,
    val_loader)
```

Performs the validation of the autoencoder and returns the average loss.

Args:

val_loader (DataLoader): DataLoader for validation data.

Returns:

float: Average validation loss.

Definition at line 702 of file [training_pipeline.py](#).

7.1.3.9 weighted_dose_loss()

```
training_pipeline.AutoencoderTrainer.weighted_dose_loss (
    self,
    pred,
    target,
    weight_factor = 10000.0,
    dose_threshold = 1e-6)
```

Extreme weighted L1 loss for ultra-sparse dose data.

Args:

pred: Predicted dose values
target: Ground truth dose values
weight_factor: Extreme multiplier for non-zero dose voxels (10,000x)
dose_threshold: Minimum value to consider as "real dose"

Definition at line 231 of file [training_pipeline.py](#).

7.1.4 Member Data Documentation

7.1.4.1 adv_loss

```
training_pipeline.AutoencoderTrainer.adv_loss = None
```

Definition at line 109 of file [training_pipeline.py](#).

7.1.4.2 adv_weight

```
training_pipeline.AutoencoderTrainer.adv_weight = adv_weight
```

Definition at line 102 of file [training_pipeline.py](#).

7.1.4.3 autoencoder

training_pipeline.AutoencoderTrainer.autoencoder = autoencoder
Definition at line 90 of file [training_pipeline.py](#).

7.1.4.4 clip_max

float training_pipeline.AutoencoderTrainer.clip_max = None
Definition at line 98 of file [training_pipeline.py](#).

7.1.4.5 clip_min

training_pipeline.AutoencoderTrainer.clip_min = None
Definition at line 97 of file [training_pipeline.py](#).

7.1.4.6 device

training_pipeline.AutoencoderTrainer.device = device
Definition at line 94 of file [training_pipeline.py](#).

7.1.4.7 discriminator

training_pipeline.AutoencoderTrainer.discriminator = discriminator
Definition at line 91 of file [training_pipeline.py](#).

7.1.4.8 dose_mean

training_pipeline.AutoencoderTrainer.dose_mean = mean
Definition at line 371 of file [training_pipeline.py](#).

7.1.4.9 dose_std

training_pipeline.AutoencoderTrainer.dose_std = std
Definition at line 372 of file [training_pipeline.py](#).

7.1.4.10 energy_max

training_pipeline.AutoencoderTrainer.energy_max = energy_max
Definition at line 100 of file [training_pipeline.py](#).

7.1.4.11 energy_min

training_pipeline.AutoencoderTrainer.energy_min = energy_min
Definition at line 99 of file [training_pipeline.py](#).

7.1.4.12 kl_weight

training_pipeline.AutoencoderTrainer.kl_weight = kl_weight
Definition at line 101 of file [training_pipeline.py](#).

7.1.4.13 l1_loss

training_pipeline.AutoencoderTrainer.l1_loss = None
Definition at line 108 of file [training_pipeline.py](#).

7.1.4.14 optimizer_d

training_pipeline.AutoencoderTrainer.optimizer_d = optimizer_d
Definition at line 93 of file [training_pipeline.py](#).

7.1.4.15 optimizer_g

training_pipeline.AutoencoderTrainer.optimizer_g = optimizer_g
Definition at line 92 of file [training_pipeline.py](#).

7.1.4.16 perceptual_loss

training_pipeline.AutoencoderTrainer.perceptual_loss = None
Definition at line 110 of file [training_pipeline.py](#).

7.1.4.17 perceptual_weight

training_pipeline.AutoencoderTrainer.perceptual_weight = perceptual_weight
Definition at line 103 of file [training_pipeline.py](#).

7.1.4.18 tv_weight

training_pipeline.AutoencoderTrainer.tv_weight = tv_weight
Definition at line 104 of file [training_pipeline.py](#).

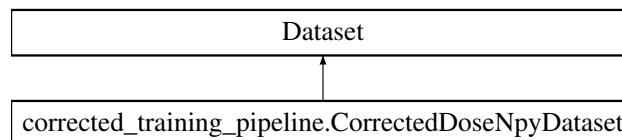
7.1.4.19 warm_up_epochs

training_pipeline.AutoencoderTrainer.warm_up_epochs = warm_up_epochs
Definition at line 105 of file [training_pipeline.py](#).
The documentation for this class was generated from the following file:

- [sourcecode/training_pipeline.py](#)

7.2 corrected_training_pipeline.CorrectedDoseNpyDataset Class Reference

Inheritance diagram for corrected_training_pipeline.CorrectedDoseNpyDataset:



Public Member Functions

- [__init__](#) (self, [root_dir](#), section=None, [transforms](#)=None, [target_resolution](#)=(64, 64, 64))
- Dict[str, float] [get_dose_normalization_params](#) (self)
- [__len__](#) (self)
- [__getitem__](#) (self, idx)

Public Attributes

- [root_dir](#) = root_dir
- [target_resolution](#) = target_resolution
- list [samples](#) = []
- dict [original_dose_stats](#) = {}
- [transforms](#) = transforms

7.2.1 Detailed Description

CORRECTED Dataset that properly loads original dose values for correct normalization.

Definition at line 36 of file [corrected_training_pipeline.py](#).

7.2.2 Constructor & Destructor Documentation

7.2.2.1 `__init__()`

```
corrected_training_pipeline.CorrectedDoseNpyDataset.__init__ (
    self,
    root_dir,
    section = None,
    transforms = None,
    target_resolution = (64, 64, 64))
```

Definition at line 40 of file [corrected_training_pipeline.py](#).

7.2.3 Member Function Documentation

7.2.3.1 `__getitem__()`

```
corrected_training_pipeline.CorrectedDoseNpyDataset.__getitem__ (
    self,
    idx)
```

Definition at line 148 of file [corrected_training_pipeline.py](#).

7.2.3.2 `__len__()`

```
corrected_training_pipeline.CorrectedDoseNpyDataset.__len__ (
    self)
```

Definition at line 145 of file [corrected_training_pipeline.py](#).

7.2.3.3 `get_dose_normalization_params()`

```
Dict[str, float] corrected_training_pipeline.CorrectedDoseNpyDataset.get_dose_normalization_params (
    self)
```

Get realistic dose normalization parameters based on ORIGINAL data.

Definition at line 110 of file [corrected_training_pipeline.py](#).

7.2.4 Member Data Documentation

7.2.4.1 `original_dose_stats`

```
dict corrected_training_pipeline.CorrectedDoseNpyDataset.original_dose_stats = {}
```

Definition at line 50 of file [corrected_training_pipeline.py](#).

7.2.4.2 `root_dir`

```
corrected_training_pipeline.CorrectedDoseNpyDataset.root_dir = root_dir
```

Definition at line 41 of file [corrected_training_pipeline.py](#).

7.2.4.3 `samples`

```
corrected_training_pipeline.CorrectedDoseNpyDataset.samples = []
```

Definition at line 43 of file [corrected_training_pipeline.py](#).

7.2.4.4 `target_resolution`

```
corrected_training_pipeline.CorrectedDoseNpyDataset.target_resolution = target_resolution
```

Definition at line 42 of file [corrected_training_pipeline.py](#).

7.2.4.5 `transforms`

```
corrected_training_pipeline.CorrectedDoseNpyDataset.transforms = transforms
```

Definition at line 101 of file [corrected_training_pipeline.py](#).

The documentation for this class was generated from the following file:

- [sourcecode/corrected_training_pipeline.py](#)

7.3 corrected_inference.CorrectedInferenceModule Class Reference

Public Member Functions

- [__init__](#) (self, [model_path](#), [device](#)='cpu')
- [preprocess_ct](#) (self, ct_array, target_cube_size=(64, 64, 64))
- [run_inference](#) (self, ct_tensor, quad_energies=None, quad_weights=None, target_cube_size=(64, 64, 64))
- [run_inference_single_energy](#) (self, ct_tensor, target_energy=None, target_cube_size=(64, 64, 64))

Public Attributes

- [device](#) = device
- [model_path](#) = model_path
- [checkpoint](#) = torch.load([model_path](#), map_location=[device](#), weights_only=False)
- [energies](#) = self.checkpoint.get('energies', [11.5, 15.75, 34.25])
- [scale_factor](#) = self.checkpoint.get('scale_factor', 0.18215)
- [autoencoder](#)
- [unet](#)
- [scheduler](#)
- [inferer](#)

Protected Member Functions

- [_load_models](#) (self)

7.3.1 Detailed Description

Fixed inference module that properly handles the energy-conditioned model.

Definition at line 14 of file [corrected_inference.py](#).

7.3.2 Constructor & Destructor Documentation

7.3.2.1 __init__()

```
corrected_inference.CorrectedInferenceModule.__init__ (
    self,
    model_path,
    device = 'cpu')
```

Initialize with the corrected energy-conditioned model.

Args:

[model_path](#): Path to the unified model checkpoint
[device](#): Device to run inference on

Definition at line 19 of file [corrected_inference.py](#).

7.3.3 Member Function Documentation

7.3.3.1 _load_models()

```
corrected_inference.CorrectedInferenceModule._load_models (
    self) [protected]
```

Load autoencoder, unet, and scheduler from checkpoint.

Definition at line 68 of file [corrected_inference.py](#).

7.3.3.2 preprocess_ct()

```
corrected_inference.CorrectedInferenceModule.preprocess_ct (
    self,
    ct_array,
    target_cube_size = (64, 64, 64))
```

Preprocess CT data for inference.

Args:

ct_array: CT data as numpy array
target_cube_size: Target size for processing

Returns:

Preprocessed CT tensor

Definition at line 154 of file [corrected_inference.py](#).

7.3.3.3 run_inference()

```
corrected_inference.CorrectedInferenceModule.run_inference (
    self,
    ct_tensor,
    quad_energies = None,
    quad_weights = None,
    target_cube_size = (64, 64, 64))
```

Run inference on CT data with quadrature integration over energies.

Args:

ct_tensor: CT data tensor or numpy array
quad_energies: List of quadrature energies (if None, uses single energy)
quad_weights: List of quadrature weights (if None, uses single energy)
target_cube_size: Target cube size for processing

Returns:

Dose distribution tensor

Definition at line 193 of file [corrected_inference.py](#).

7.3.3.4 run_inference_single_energy()

```
corrected_inference.CorrectedInferenceModule.run_inference_single_energy (
    self,
    ct_tensor,
    target_energy = None,
    target_cube_size = (64, 64, 64))
```

Run inference on CT data for a single energy.

Args:

ct_tensor: CT data tensor or numpy array
target_energy: Target energy for inference (if None, uses first available)
target_cube_size: Target cube size for processing

Returns:

Dose distribution tensor

Definition at line 249 of file [corrected_inference.py](#).

7.3.4 Member Data Documentation

7.3.4.1 autoencoder

```
corrected_inference.CorrectedInferenceModule.autoencoder
```

Initial value:

```
= AutoencoderKL(
```

```

        spatial_dims=3,
        in_channels=2, # CT + Energy channel
        out_channels=1,
        num_channels=(32, 32, 32),
        latent_channels=2,
        num_res_blocks=1,
        norm_num_groups=8,
        attention_levels=(False, False, True),
    ).to(self.device)

```

Definition at line 80 of file [corrected_inference.py](#).

7.3.4.2 checkpoint

```

corrected_inference.CorrectedInferenceModule.checkpoint = torch.load(model_path, map_location=device,
weights_only=False)

```

Definition at line 41 of file [corrected_inference.py](#).

7.3.4.3 device

```

corrected_inference.CorrectedInferenceModule.device = device

```

Definition at line 31 of file [corrected_inference.py](#).

7.3.4.4 energies

```

corrected_inference.CorrectedInferenceModule.energies = self.checkpoint.get('energies', [11.5, 15.75, 34.↵
25])

```

Definition at line 56 of file [corrected_inference.py](#).

7.3.4.5 inferer

```

corrected_inference.CorrectedInferenceModule.inferer

```

Initial value:

```

= LatentDiffusionInferer(
    scheduler=self.scheduler,
    scale_factor=self.scale_factor
)

```

Definition at line 140 of file [corrected_inference.py](#).

7.3.4.6 model_path

```

corrected_inference.CorrectedInferenceModule.model_path = model_path

```

Definition at line 32 of file [corrected_inference.py](#).

7.3.4.7 scale_factor

```

corrected_inference.CorrectedInferenceModule.scale_factor = self.checkpoint.get('scale_factor', 0.18215)

```

Definition at line 57 of file [corrected_inference.py](#).

7.3.4.8 scheduler

```

corrected_inference.CorrectedInferenceModule.scheduler

```

Initial value:

```

= DDPMSScheduler(
    num_train_timesteps=1000,
    beta_start=0.0015,
    beta_end=0.0195
)

```

Definition at line 111 of file [corrected_inference.py](#).

7.3.4.9 unet

```

corrected_inference.CorrectedInferenceModule.unet

```

Initial value:

```

= DiffusionModelUNet(
    spatial_dims=3,
    in_channels=2, # Latent space is 2 channels

```



```

        out_channels=2,
        with_conditioning=True,
        cross_attention_dim=2,
        num_res_blocks=1,
        num_channels=(32, 64, 64),
        attention_levels=(False, True, True),
        num_head_channels=(0, 32, 32),
    ).to(self.device)

```

Definition at line 94 of file [corrected_inference.py](#).

The documentation for this class was generated from the following file:

- [sourcecode/corrected_inference.py](#)

7.4 corrected_training_pipeline.CorrectedTrainingPipeline Class Reference

Public Member Functions

- `__init__` (self, str [root_dir](#), List[float] [energies](#), Tuple[int, int, int] [target_resolution](#)=(64, 64, 64), Tuple[int, int, int] [cube_size](#)=(64, 64, 64), int [batch_size](#)=2, float [learning_rate](#)=1e-5, int [num_epochs](#)=100, torch.device [device](#)=None)
- `create_corrected_transforms` (self)
- `setup_unified_model` (self, Dict dose_normalization_params)
- `run_corrected_training` (self)
- `calculate_corrected_scaling_factor` (self, autoencoder, train_loader)
- `train_unified_model` (self, autoencoder, discriminator, unet, scheduler, opt_g, opt_d, opt_diff, train_loader, val_loader, dose_normalization_params, scale_factor)

Public Attributes

- `root_dir` = root_dir
- `energies` = energies
- `target_resolution` = target_resolution
- `cube_size` = cube_size
- `batch_size` = batch_size
- `learning_rate` = learning_rate
- `num_epochs` = num_epochs
- `device` = device or torch.device('cuda' if torch.cuda.is_available() else 'cpu')

7.4.1 Detailed Description

CORRECTED Training Pipeline that implements the sustainable solution.

Definition at line 171 of file [corrected_training_pipeline.py](#).

7.4.2 Constructor & Destructor Documentation

7.4.2.1 `__init__()`

```

corrected_training_pipeline.CorrectedTrainingPipeline.__init__ (
    self,
    str root_dir,
    List[float] energies,
    Tuple[int, int, int] target_resolution = (64, 64, 64),
    Tuple[int, int, int] cube_size = (64, 64, 64),
    int batch_size = 2,
    float learning_rate = 1e-5,
    int num_epochs = 100,
    torch.device device = None)

```

Definition at line 176 of file [corrected_training_pipeline.py](#).

7.4.3 Member Function Documentation

7.4.3.1 `calculate_corrected_scaling_factor()`

```
corrected_training_pipeline.CorrectedTrainingPipeline.calculate_corrected_scaling_factor (
    self,
    autoencoder,
    train_loader)
```

Calculate scaling factor with corrected energy conditioning.

Definition at line 337 of file [corrected_training_pipeline.py](#).

7.4.3.2 `create_corrected_transforms()`

```
corrected_training_pipeline.CorrectedTrainingPipeline.create_corrected_transforms (
    self)
```

Create CORRECTED transforms using Resized instead of Spacingd.

Definition at line 201 of file [corrected_training_pipeline.py](#).

7.4.3.3 `run_corrected_training()`

```
corrected_training_pipeline.CorrectedTrainingPipeline.run_corrected_training (
    self)
```

Execute the corrected training pipeline.

Definition at line 283 of file [corrected_training_pipeline.py](#).

7.4.3.4 `setup_unified_model()`

```
Tuple corrected_training_pipeline.CorrectedTrainingPipeline.setup_unified_model (
    self,
    Dict dose_normalization_params)
```

Setup unified energy-conditioned model for all energies.

Definition at line 236 of file [corrected_training_pipeline.py](#).

7.4.3.5 `train_unified_model()`

```
corrected_training_pipeline.CorrectedTrainingPipeline.train_unified_model (
    self,
    autoencoder,
    discriminator,
    unet,
    scheduler,
    opt_g,
    opt_d,
    opt_diff,
    train_loader,
    val_loader,
    dose_normalization_params,
    scale_factor)
```

Train the unified energy-conditioned model.

Definition at line 369 of file [corrected_training_pipeline.py](#).

7.4.4 Member Data Documentation

7.4.4.1 `batch_size`

```
corrected_training_pipeline.CorrectedTrainingPipeline.batch_size = batch_size
```

Definition at line 190 of file [corrected_training_pipeline.py](#).

7.4.4.2 cube_size

`corrected_training_pipeline.CorrectedTrainingPipeline.cube_size = cube_size`
 Definition at line 189 of file [corrected_training_pipeline.py](#).

7.4.4.3 device

`corrected_training_pipeline.CorrectedTrainingPipeline.device = device or torch.device('cuda' if torch.cuda.is_available() else 'cpu')`
 Definition at line 193 of file [corrected_training_pipeline.py](#).

7.4.4.4 energies

`corrected_training_pipeline.CorrectedTrainingPipeline.energies = energies`
 Definition at line 187 of file [corrected_training_pipeline.py](#).

7.4.4.5 learning_rate

`corrected_training_pipeline.CorrectedTrainingPipeline.learning_rate = learning_rate`
 Definition at line 191 of file [corrected_training_pipeline.py](#).

7.4.4.6 num_epochs

`corrected_training_pipeline.CorrectedTrainingPipeline.num_epochs = num_epochs`
 Definition at line 192 of file [corrected_training_pipeline.py](#).

7.4.4.7 root_dir

`corrected_training_pipeline.CorrectedTrainingPipeline.root_dir = root_dir`
 Definition at line 186 of file [corrected_training_pipeline.py](#).

7.4.4.8 target_resolution

`corrected_training_pipeline.CorrectedTrainingPipeline.target_resolution = target_resolution`
 Definition at line 188 of file [corrected_training_pipeline.py](#).
 The documentation for this class was generated from the following file:

- [sourcecode/corrected_training_pipeline.py](#)

7.5 data_management.DataLoaderModule Class Reference

Public Member Functions

- [__init__](#) (self, [root_dir](#), [transforms](#), [train_ratio](#)=0.8, [seed](#)=42)
- [load_dataset](#) (self, section="training")
- [split_dataset](#) (self, ds_full)
- [custom_collate](#) (self, batch)
- [create_data_loader](#) (self, dataset, batch_size, shuffle, num_workers=0)

Public Attributes

- [root_dir](#) = [root_dir](#)
- [transforms](#) = [transforms](#)
- [train_ratio](#) = [train_ratio](#)
- [seed](#) = [seed](#)

7.5.1 Detailed Description

This module takes care of loading, splitting and creating DataLoaders for training and validation data from a CustomDataset.

Definition at line 297 of file [data_management.py](#).

7.5.2 Constructor & Destructor Documentation

7.5.2.1 `__init__()`

```
data_management.DataLoaderModule.__init__ (
    self,
    root_dir,
    transforms,
    train_ratio = 0.8,
    seed = 42)
```

Args:

`root_dir` (str): The root directory where the data is located.
`transforms` (monai.transforms.Compose): Transforms that are applied to the data.
`energy` (optional, int or float): Filter to load only samples of a certain energy.
`train_ratio` (float): Ratio of the data used for training.
`seed` (int): Seed for the reproducibility of the split.

Definition at line 302 of file [data_management.py](#).

7.5.3 Member Function Documentation

7.5.3.1 `create_data_loader()`

```
data_management.DataLoaderModule.create_data_loader (
    self,
    dataset,
    batch_size,
    shuffle,
    num_workers = 0)
```

Creates a DataLoader from the given dataset.

Args:

`dataset` (Dataset): The dataset from which data is to be loaded.
`batch_size` (int): Number of samples per batch.
`shuffle` (bool): Whether the data should be shuffled before each run.
`num_workers` (int): Number of parallel processes for loading the data.

Returns:

DataLoader: The DataLoader created.

Definition at line 361 of file [data_management.py](#).

7.5.3.2 `custom_collate()`

```
data_management.DataLoaderModule.custom_collate (
    self,
    batch)
```

Custom collate function that checks and collates 'input', 'target', and 'energy' if available.

Definition at line 349 of file [data_management.py](#).

7.5.3.3 `load_dataset()`

```
data_management.DataLoaderModule.load_dataset (
    self,
    section = "training")
```

Loads the complete dataset as a CustomDataset.

Args:

`section` (str): Specifies which part of the dataset is to be loaded (e.g. "training").

Returns:

CustomDataset: The loaded dataset.

Definition at line 316 of file [data_management.py](#).

7.5.3.4 split_dataset()

```
data_management.DataLoaderModule.split_dataset (
    self,
    ds_full)

```

Splits the complete dataset into training and validation sets.

Args:

ds_full (Dataset): The complete dataset.

Returns:

tuple: (train_ds, val_ds)

Definition at line 329 of file [data_management.py](#).

7.5.4 Member Data Documentation

7.5.4.1 root_dir

```
data_management.DataLoaderModule.root_dir = root_dir

```

Definition at line 311 of file [data_management.py](#).

7.5.4.2 seed

```
data_management.DataLoaderModule.seed = seed

```

Definition at line 314 of file [data_management.py](#).

7.5.4.3 train_ratio

```
data_management.DataLoaderModule.train_ratio = train_ratio

```

Definition at line 313 of file [data_management.py](#).

7.5.4.4 transforms

```
data_management.DataLoaderModule.transforms = transforms

```

Definition at line 312 of file [data_management.py](#).

The documentation for this class was generated from the following file:

- [sourcecode/data_management.py](#)

7.6 training_pipeline.DiffusionTrainer Class Reference

Public Member Functions

- [__init__](#) (self, [diffusion_model](#), [optimizer_diff](#), [energy_min](#), [energy_max](#), [device](#))
- [train_one_epoch](#) (self, [train_loader](#), [epoch](#), [inferer](#)=None, [autoencoder](#)=None)

Public Attributes

- [diffusion_model](#) = [diffusion_model](#)
- [optimizer_diff](#) = [optimizer_diff](#)
- [energy_min](#) = [energy_min](#)
- [energy_max](#) = [energy_max](#)
- [device](#) = [device](#)
- [cross_attn_dim](#) = [min\(cand_dims\)](#)

7.6.1 Detailed Description

Encapsulates the training process for the diffusion model.

Trains diffusion model for one epoch.

Args:

```
train_loader (DataLoader): DataLoader for training data.
epoch (int): Current epoch index.
inferer (callable): A callable with signature
    inferer(inputs, autoencoder_model, diffusion_model, noise, timesteps)
    that returns predicted noise.
```

Definition at line 808 of file [training_pipeline.py](#).

7.6.2 Constructor & Destructor Documentation

7.6.2.1 `__init__()`

```
training_pipeline.DiffusionTrainer.__init__ (
    self,
    diffusion_model,
    optimizer_diff,
    energy_min,
    energy_max,
    device)
```

Args:

```
diffusion_model (torch.nn.Module): Das Diffusionsmodell (z. B. ein UNet).
optimizer_diff (torch.optim.Optimizer): Optimierer für das Diffusionsmodell.
device (torch.device): CPU oder GPU.
```

Definition at line 822 of file [training_pipeline.py](#).

7.6.3 Member Function Documentation

7.6.3.1 `train_one_epoch()`

```
training_pipeline.DiffusionTrainer.train_one_epoch (
    self,
    train_loader,
    epoch,
    inferer = None,
    autoencoder = None)
```

Args:

```
diffusion_model (torch.nn.Module): The diffusion model (e.g. a UNet).
optimizer_diff (torch.optim.Optimizer): Optimizer for the diffusion model.
device (torch.device): CPU or GPU.
```

Definition at line 872 of file [training_pipeline.py](#).

7.6.4 Member Data Documentation

7.6.4.1 `cross_attn_dim`

```
training_pipeline.DiffusionTrainer.cross_attn_dim = min(cand_dims)
```

Definition at line 858 of file [training_pipeline.py](#).

7.6.4.2 `device`

```
training_pipeline.DiffusionTrainer.device = device
```

Definition at line 834 of file [training_pipeline.py](#).

7.6.4.3 diffusion_model

training_pipeline.DiffusionTrainer.diffusion_model = diffusion_model
 Definition at line 829 of file [training_pipeline.py](#).

7.6.4.4 energy_max

training_pipeline.DiffusionTrainer.energy_max = energy_max
 Definition at line 833 of file [training_pipeline.py](#).

7.6.4.5 energy_min

training_pipeline.DiffusionTrainer.energy_min = energy_min
 Definition at line 832 of file [training_pipeline.py](#).

7.6.4.6 optimizer_diff

training_pipeline.DiffusionTrainer.optimizer_diff = optimizer_diff
 Definition at line 830 of file [training_pipeline.py](#).
 The documentation for this class was generated from the following file:

- [sourcecode/training_pipeline.py](#)

7.7 systematic_dose_diagnosis.DoseNormalizationDiagnostic Class Reference

Public Member Functions

- [__init__](#) (self, [traindata_dir](#)="traindata", [checkpoint_path](#)=None)
- [analyze_raw_dose_data](#) (self)
- [analyze_training_normalization](#) (self)
- [analyze_checkpoint_parameters](#) (self)
- [analyze_inference_flow](#) (self)
- [perform_data_flow_analysis](#) (self)
- [generate_recommendations](#) (self)
- [save_results](#) (self, [output_path](#)="dose_diagnosis_results.json")
- [create_visualization](#) (self, [output_path](#)="dose_diagnosis_visualization.png")
- [run_full_diagnosis](#) (self, [checkpoint_path](#)=None)

Public Attributes

- [traindata_dir](#) = Path([traindata_dir](#))
- [checkpoint_path](#) = [checkpoint_path](#)
- dict [results](#)

7.7.1 Detailed Description

Comprehensive diagnosis of dose normalization pipeline.

Definition at line 33 of file [systematic_dose_diagnosis.py](#).

7.7.2 Constructor & Destructor Documentation

7.7.2.1 __init__()

```
systematic_dose_diagnosis.DoseNormalizationDiagnostic.__init__ (
    self,
    traindata_dir = "traindata",
    checkpoint_path = None)
```

Definition at line 36 of file [systematic_dose_diagnosis.py](#).

7.7.3 Member Function Documentation

7.7.3.1 analyze_checkpoint_parameters()

```
systematic_dose_diagnosis.DoseNormalizationDiagnostic.analyze_checkpoint_parameters (  
    self)
```

Phase 1.3: Analyze checkpoint normalization parameters.

Definition at line 202 of file [systematic_dose_diagnosis.py](#).

7.7.3.2 analyze_inference_flow()

```
systematic_dose_diagnosis.DoseNormalizationDiagnostic.analyze_inference_flow (  
    self)
```

Phase 1.4: Analyze inference denormalization flow.

Definition at line 265 of file [systematic_dose_diagnosis.py](#).

7.7.3.3 analyze_raw_dose_data()

```
systematic_dose_diagnosis.DoseNormalizationDiagnostic.analyze_raw_dose_data (  
    self)
```

Phase 1.1: Analyze raw dose data in traindata directory.

Definition at line 49 of file [systematic_dose_diagnosis.py](#).

7.7.3.4 analyze_training_normalization()

```
systematic_dose_diagnosis.DoseNormalizationDiagnostic.analyze_training_normalization (  
    self)
```

Phase 1.2: Analyze training normalization logic.

Definition at line 144 of file [systematic_dose_diagnosis.py](#).

7.7.3.5 create_visualization()

```
systematic_dose_diagnosis.DoseNormalizationDiagnostic.create_visualization (  
    self,  
    output_path = "dose_diagnosis_visualization.png")
```

Create visualization of key findings.

Definition at line 436 of file [systematic_dose_diagnosis.py](#).

7.7.3.6 generate_recommendations()

```
systematic_dose_diagnosis.DoseNormalizationDiagnostic.generate_recommendations (  
    self)
```

Phase 1.6: Generate specific recommendations.

Definition at line 365 of file [systematic_dose_diagnosis.py](#).

7.7.3.7 perform_data_flow_analysis()

```
systematic_dose_diagnosis.DoseNormalizationDiagnostic.perform_data_flow_analysis (  
    self)
```

Phase 1.5: Comprehensive data flow analysis.

Definition at line 310 of file [systematic_dose_diagnosis.py](#).

7.7.3.8 run_full_diagnosis()

```
systematic_dose_diagnosis.DoseNormalizationDiagnostic.run_full_diagnosis (
    self,
    checkpoint_path = None)
```

Run complete diagnosis pipeline.

Definition at line 534 of file [systematic_dose_diagnosis.py](#).

7.7.3.9 save_results()

```
systematic_dose_diagnosis.DoseNormalizationDiagnostic.save_results (
    self,
    output_path = "dose_diagnosis_results.json")
```

Save all analysis results to JSON file.

Definition at line 427 of file [systematic_dose_diagnosis.py](#).

7.7.4 Member Data Documentation

7.7.4.1 checkpoint_path

```
systematic_dose_diagnosis.DoseNormalizationDiagnostic.checkpoint_path = checkpoint_path
```

Definition at line 38 of file [systematic_dose_diagnosis.py](#).

7.7.4.2 results

```
systematic_dose_diagnosis.DoseNormalizationDiagnostic.results
```

Initial value:

```
= {
    'raw_dose_statistics': {},
    'training_normalization_params': {},
    'checkpoint_normalization_params': {},
    'inference_parameters': {},
    'medical_plausibility': {},
    'data_flow_analysis': {},
    'recommendations': []
}
```

Definition at line 39 of file [systematic_dose_diagnosis.py](#).

7.7.4.3 traindata_dir

```
systematic_dose_diagnosis.DoseNormalizationDiagnostic.traindata_dir = Path(traindata_dir)
```

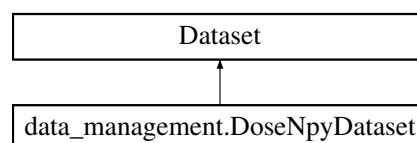
Definition at line 37 of file [systematic_dose_diagnosis.py](#).

The documentation for this class was generated from the following file:

- tests/[systematic_dose_diagnosis.py](#)

7.8 data_management.DoseNpyDataset Class Reference

Inheritance diagram for data_management.DoseNpyDataset:



Public Member Functions

- `__init__` (self, root_dir, section=None, transforms=None)
- `validate_sample_quality` (self, dose_path, min_dose_voxels=20, min_max_dose=0.01)
- `__len__` (self)
- `__getitem__` (self, idx)

Public Attributes

- list `samples` = []
- `transforms` = transforms

7.8.1 Detailed Description

Dataset that loads input/output .npz cubes and parses the energy level from the folder name.
Expects directory structure:

```
root_dir/
  <section>/      # e.g. "training"
    <energy_folder>/ # e.g. "11_5"
      inputcube/    # contains input .npz files
      outputcube/   # contains target .npz files
```

Definition at line 186 of file `data_management.py`.

7.8.2 Constructor & Destructor Documentation

7.8.2.1 `__init__()`

```
data_management.DoseNpzDataset.__init__ (
    self,
    root_dir,
    section = None,
    transforms = None)
```

Definition at line 196 of file `data_management.py`.

7.8.3 Member Function Documentation

7.8.3.1 `__getitem__()`

```
data_management.DoseNpzDataset.__getitem__ (
    self,
    idx)
```

Definition at line 268 of file `data_management.py`.

7.8.3.2 `__len__()`

```
data_management.DoseNpzDataset.__len__ (
    self)
```

Definition at line 265 of file `data_management.py`.

7.8.3.3 `validate_sample_quality()`

```
data_management.DoseNpzDataset.validate_sample_quality (
    self,
    dose_path,
    min_dose_voxels = 20,
    min_max_dose = 0.01)
```

Check if a dose sample is suitable for training.

Args:

```
dose_path: Path to dose .npz file
min_dose_voxels: Minimum number of dose voxels required
```

`min_max_dose`: Minimum maximum dose value required

Returns:

`bool`: True if sample is valid for training

Definition at line 230 of file [data_management.py](#).

7.8.4 Member Data Documentation

7.8.4.1 samples

`data_management.DoseNpyDataset.samples` = []

Definition at line 198 of file [data_management.py](#).

7.8.4.2 transforms

`data_management.DoseNpyDataset.transforms` = transforms

Definition at line 226 of file [data_management.py](#).

The documentation for this class was generated from the following file:

- [sourcecode/data_management.py](#)

7.9 training_pipeline.EarlyStopping Class Reference

Public Member Functions

- [__init__](#) (self, [patience](#)=5)
- [update](#) (self, [val_loss](#))
- [reset](#) (self)

Public Attributes

- [patience](#) = patience
- [best_val_loss](#) = float('inf')
- int [epochs_no_improvement](#) = 0

7.9.1 Detailed Description

Monitors the validation loss and triggers early stopping if the loss the loss does not improve over a defined number of epochs.

Definition at line 29 of file [training_pipeline.py](#).

7.9.2 Constructor & Destructor Documentation

7.9.2.1 __init__()

```
training_pipeline.EarlyStopping.__init__ (
    self,
    patience = 5)
```

Args:

`patience` (int): Number of epochs permitted without improvement.

Definition at line 34 of file [training_pipeline.py](#).

7.9.3 Member Function Documentation

7.9.3.1 reset()

```
training_pipeline.EarlyStopping.reset (
    self)
```

Resets the early stopping parameters.

Definition at line 59 of file [training_pipeline.py](#).

7.9.3.2 update()

```
training_pipeline.EarlyStopping.update (
    self,
    val_loss)
```

Compares the current validation loss with the best loss to date.

Args:

val_loss (float): Current loss of validation.

Returns:

bool: If Early Stopping is triggered true otherwise False.

Definition at line 43 of file [training_pipeline.py](#).

7.9.4 Member Data Documentation

7.9.4.1 best_val_loss

```
training_pipeline.EarlyStopping.best_val_loss = float('inf')
```

Definition at line 40 of file [training_pipeline.py](#).

7.9.4.2 epochs_no_improvement

```
int training_pipeline.EarlyStopping.epochs_no_improvement = 0
```

Definition at line 41 of file [training_pipeline.py](#).

7.9.4.3 patience

```
training_pipeline.EarlyStopping.patience = patience
```

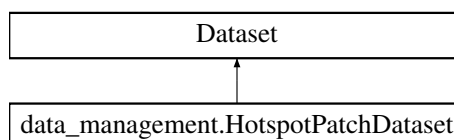
Definition at line 39 of file [training_pipeline.py](#).

The documentation for this class was generated from the following file:

- [sourcecode/training_pipeline.py](#)

7.10 data_management.HotspotPatchDataset Class Reference

Inheritance diagram for data_management.HotspotPatchDataset:



Public Member Functions

- `__init__` (self, root_dir, section=None, [patch_size](#)=(32, 32, 32), min_hotspot_voxels=1000, dose_threshold=0.5, max_patches=16, random_patches=0, [transforms](#)=None, energy=None, [max_patches_per_energy](#)=80)
- `__len__` (self)
- `__getitem__` (self, idx)

Public Attributes

- [patch_size](#) = patch_size
- [transforms](#) = transforms
- list [patch_records](#) = []
- [selected_energy](#) = energy
- list [patch_stats](#) = []
- [max_patches_per_energy](#) = max_patches_per_energy

7.10.1 Detailed Description

Dataset, das aus jedem Input/Target-Cube Hotspot-Patches extrahiert und als einzelne Trainingssamples zurückgibt. Nutzt `extract_hotspot_patches(volume, ...)`, um Patch-Startpositionen zu bestimmen. Jeder `__getitem__` gibt einen Patch (Input, Target, Energy) zurück.

Definition at line 8 of file [data_management.py](#).

7.10.2 Constructor & Destructor Documentation

7.10.2.1 `__init__()`

```
data_management.HotspotPatchDataset.__init__ (
    self,
    root_dir,
    section = None,
    patch_size = (32,32,32),
    min_hotspot_voxels = 1000,
    dose_threshold = 0.5,
    max_patches = 16,
    random_patches = 0,
    transforms = None,
    energy = None,
    max_patches_per_energy = 80)
```

Definition at line 14 of file [data_management.py](#).

7.10.3 Member Function Documentation

7.10.3.1 `__getitem__()`

```
data_management.HotspotPatchDataset.__getitem__ (
    self,
    idx)
```

Definition at line 150 of file [data_management.py](#).

7.10.3.2 `__len__()`

```
data_management.HotspotPatchDataset.__len__ (
    self)
```

Definition at line 147 of file [data_management.py](#).

7.10.4 Member Data Documentation

7.10.4.1 `max_patches_per_energy`

```
data_management.HotspotPatchDataset.max_patches_per_energy = max_patches_per_energy
```

Definition at line 30 of file [data_management.py](#).

7.10.4.2 `patch_records`

```
data_management.HotspotPatchDataset.patch_records = []
```

Definition at line 27 of file [data_management.py](#).

7.10.4.3 `patch_size`

```
data_management.HotspotPatchDataset.patch_size = patch_size
```

Definition at line 25 of file [data_management.py](#).

7.10.4.4 `patch_stats`

```
data_management.HotspotPatchDataset.patch_stats = []
```

Definition at line 29 of file [data_management.py](#).

7.10.4.5 selected_energy

`data_management.HotspotPatchDataset.selected_energy = energy`
 Definition at line 28 of file [data_management.py](#).

7.10.4.6 transforms

`data_management.HotspotPatchDataset.transforms = transforms`
 Definition at line 26 of file [data_management.py](#).

The documentation for this class was generated from the following file:

- [sourcecode/data_management.py](#)

7.11 inference_module.InferenceModule Class Reference

Public Member Functions

- `__init__` (self, dict [models_by_energy](#), list [energies](#), list [energy_weights](#), [device](#), [scale_factor](#), [energy_min](#)=None, [energy_max](#)=None, [clip_min](#)=None, [clip_max](#)=None, [dose_normalization_params](#)=None)
- `get_energy_normalization_params` (self, [energy_value](#), [resolution](#)=100)
- `preprocess_ct` (self, [ct_tensor](#), [target_cube_size](#)=(100, 100, 100))
- `run_inference` (self, [ct_tensor](#), [target_cube_size](#)=(100, 100, 100))
- `run_inference_conditioned_on_energy` (self, [ct_tensor](#), [energy_value](#), [target_cube_size](#)=(100, 100, 100))
- `run_inference_over_energies` (self, [ct_tensor](#), [target_cube_size](#), [energies](#), [energy_weights](#))

Public Attributes

- [models_by_energy](#) = `models_by_energy`
- [energies](#) = `energies`
- [energy_weights](#) = `energy_weights`
- [device](#) = `device`
- `int` [scale_factor](#) = `scale_factor`
- [energy_min](#) = `energy_min` if `energy_min` is not None else `min(energies)`
- [energy_max](#) = `energy_max` if `energy_max` is not None else `max(energies)`
- [clip_min](#) = `clip_min`
- [clip_max](#) = `clip_max`
- [dose_normalization_params](#) = `dose_normalization_params` or `{}`

Protected Member Functions

- `_compute_scale_factor` (self)

7.11.1 Detailed Description

The `InferenceModule` performs the dose inference process using the trained autoencoder and diffusion model. It computes the dose distribution from the uploaded patient CT data.

The process is as follows:

1. Preprocess the input CT scan by resizing or cropping it to a specified cube size.
2. Optionally encode the CT scan with the autoencoder to obtain a latent representation.
3. Pass the (latent) representation through the diffusion model to generate a voxel-wise dose distribution.

Additional parameters (e.g. cube size) are taken into account during preprocessing.

Definition at line 10 of file [inference_module.py](#).

7.11.2 Constructor & Destructor Documentation

7.11.2.1 `__init__()`

```
inference_module.InferenceModule.__init__ (
    self,
    dict models_by_energy,
    list energies,
    list energy_weights,
    device,
    scale_factor,
    energy_min = None,
    energy_max = None,
    clip_min = None,
    clip_max = None,
    dose_normalization_params = None)
```

Args:

`models_by_energy` (dict): Mapping from energy value (float) to tuple (autoencoder, diffusion_model, scheduler).
`energies` (list of float): List of quadrature energy levels.
`energy_weights` (list of float): Corresponding quadrature weights.
`device` (torch.device): Device for inference.
`scale_factor` (float): Scale factor used during training. If None, will try to extract from training.
`energy_min` (float): Minimum energy value for normalization. If None, computed from energies.
`energy_max` (float): Maximum energy value for normalization. If None, computed from energies.
`clip_min` (float): Minimum dose value for denormalization. If None, no denormalization.
`clip_max` (float): Maximum dose value for denormalization. If None, no denormalization.
`dose_normalization_params` (dict): Energy-specific normalization parameters from checkpoint.

Definition at line 22 of file [inference_module.py](#).

7.11.3 Member Function Documentation

7.11.3.1 `_compute_scale_factor()`

```
inference_module.InferenceModule._compute_scale_factor (
    self) [protected]
```

Compute scale factor by running a sample through the autoencoder.
 This matches the training procedure.

Definition at line 373 of file [inference_module.py](#).

7.11.3.2 `get_energy_normalization_params()`

```
inference_module.InferenceModule.get_energy_normalization_params (
    self,
    energy_value,
    resolution = 100)
```

Get energy-specific normalization parameters for denormalization.

Args:

`energy_value` (float): Energy in keV
`resolution` (int): Spatial resolution

Returns:

dict: Dictionary with clip_min, clip_max, etc.

Definition at line 50 of file [inference_module.py](#).

7.11.3.3 `preprocess_ct()`

```
inference_module.InferenceModule.preprocess_ct (
    self,
    ct_tensor,
    target_cube_size = (100, 100, 100))
```

Preprocesses the input CT scan to match the required cube size.
This includes resizing the volume using trilinear interpolation.

Args:

`ct_tensor` (torch.Tensor): Input CT scan as a tensor. Expected shape: [C, D, H, W].
`target_cube_size` (tuple): The desired spatial dimensions (D, H, W).

Returns:

torch.Tensor: The preprocessed CT scan, with a batch dimension added. Shape: [1, C, D, H, W].

Definition at line 96 of file [inference_module.py](#).

7.11.3.4 `run_inference()`

```
inference_module.InferenceModule.run_inference (
    self,
    ct_tensor,
    target_cube_size = (100, 100, 100))
```

Runs the complete dose inference on the provided CT scan.

The process includes aggregating the dose distributions over the Gaussian quadrature energies and weights.

Args:

`ct_tensor` (torch.Tensor): The input CT scan with shape [C, D, H, W].
`target_cube_size` (tuple): The target spatial dimensions (D, H, W) for inference.

Returns:

torch.Tensor: The aggregated predicted dose distribution as a tensor.

Definition at line 121 of file [inference_module.py](#).

7.11.3.5 `run_inference_conditioned_on_energy()`

```
inference_module.InferenceModule.run_inference_conditioned_on_energy (
    self,
    ct_tensor,
    energy_value,
    target_cube_size = (100, 100, 100))
```

Runs inference on the given CT scan while conditioning on a specified energy value.

The conditioning is achieved by creating an additional channel that is filled with a normalized energy value and concatenating it to the CT scan data. The autoencoder and diffusion model should be adapted to accept an extra input channel.

Args:

`ct_tensor` (torch.Tensor): Input CT scan with shape [C, D, H, W]. Typically C=1.
`target_cube_size` (tuple): Desired spatial dimensions (D, H, W).
`energy_value` (float): The energy level (in keV) to condition the inference on.

Returns:

torch.Tensor: The predicted dose distribution based on the energy-conditioned input.

Definition at line 139 of file [inference_module.py](#).

7.11.3.6 `run_inference_over_energies()`

```
inference_module.InferenceModule.run_inference_over_energies (
    self,
    ct_tensor,
    target_cube_size,
    energies,
    energy_weights)
```

Runs inference on the input CT scan for multiple energy levels and aggregates the results.

For each energy value provided, this method performs inference (using the conditioned inference

method, if available) and then computes a weighted sum of the dose distributions according to the provided energy weights.

Args:

`ct_tensor` (torch.Tensor): Input CT scan tensor with shape [C, D, H, W].
`target_cube_size` (tuple): Desired spatial dimensions for inference.
`energies` (list of float): A list of energy levels (e.g., [62, 75, 90]).
`energy_weights` (list of float): Corresponding weights for each energy level.

Returns:

torch.Tensor: The aggregated dose distribution computed as:
`dose_distribution = sum_i (weight_i * N(E_i))`

Definition at line 326 of file [inference_module.py](#).

7.11.4 Member Data Documentation

7.11.4.1 clip_max

`inference_module.InferenceModule.clip_max = clip_max`

Definition at line 46 of file [inference_module.py](#).

7.11.4.2 clip_min

`inference_module.InferenceModule.clip_min = clip_min`

Definition at line 45 of file [inference_module.py](#).

7.11.4.3 device

`inference_module.InferenceModule.device = device`

Definition at line 39 of file [inference_module.py](#).

7.11.4.4 dose_normalization_params

`inference_module.InferenceModule.dose_normalization_params = dose_normalization_params or {}`

Definition at line 48 of file [inference_module.py](#).

7.11.4.5 energies

`inference_module.InferenceModule.energies = energies`

Definition at line 37 of file [inference_module.py](#).

7.11.4.6 energy_max

`inference_module.InferenceModule.energy_max = energy_max if energy_max is not None else max(energies)`

Definition at line 43 of file [inference_module.py](#).

7.11.4.7 energy_min

`inference_module.InferenceModule.energy_min = energy_min if energy_min is not None else min(energies)`

Definition at line 42 of file [inference_module.py](#).

7.11.4.8 energy_weights

`inference_module.InferenceModule.energy_weights = energy_weights`

Definition at line 38 of file [inference_module.py](#).

7.11.4.9 models_by_energy

`inference_module.InferenceModule.models_by_energy = models_by_energy`

Definition at line 36 of file [inference_module.py](#).

7.11.4.10 scale_factor

`int inference_module.InferenceModule.scale_factor = scale_factor`

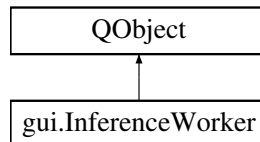
Definition at line 40 of file [inference_module.py](#).

The documentation for this class was generated from the following file:

- [sourcecode/inference_module.py](#)

7.12 gui.InferenceWorker Class Reference

Inheritance diagram for `gui.InferenceWorker`:



Public Member Functions

- `__init__` (self, str [ct_file](#), str [model_checkpoint](#), float [min_e](#), float [max_e](#))
- `run` (self)

Public Attributes

- [ct_file](#) = `ct_file`
- [model_checkpoint](#) = `model_checkpoint`
- [min_e](#) = `min_e`
- [max_e](#) = `max_e`

Static Public Attributes

- [finished](#) = `Signal(str)`
- [error](#) = `Signal(str)`

Protected Member Functions

- `_check_canceled` (self)

7.12.1 Detailed Description

Worker to run inference in a separate thread, mirroring the standalone approach. Emits `finished(output_path: str)` on success, `error(message: str)` on failure.

Definition at line 84 of file [gui.py](#).

7.12.2 Constructor & Destructor Documentation

7.12.2.1 __init__()

```

gui.InferenceWorker.__init__ (
    self,
    str ct_file,
    str model_checkpoint,
    float min_e,
    float max_e)
  
```

Definition at line 92 of file [gui.py](#).

7.12.3 Member Function Documentation

7.12.3.1 `_check_canceled()`

`gui.InferenceWorker._check_canceled (`
 `self) [protected]`

Definition at line 99 of file [gui.py](#).

7.12.3.2 `run()`

`gui.InferenceWorker.run (`
 `self)`

Definition at line 107 of file [gui.py](#).

7.12.4 Member Data Documentation

7.12.4.1 `ct_file`

`gui.InferenceWorker.ct_file = ct_file`

Definition at line 94 of file [gui.py](#).

7.12.4.2 `error`

`gui.InferenceWorker.error = Signal(str) [static]`

Definition at line 90 of file [gui.py](#).

7.12.4.3 `finished`

`gui.InferenceWorker.finished = Signal(str) [static]`

Definition at line 89 of file [gui.py](#).

7.12.4.4 `max_e`

`gui.InferenceWorker.max_e = max_e`

Definition at line 97 of file [gui.py](#).

7.12.4.5 `min_e`

`gui.InferenceWorker.min_e = min_e`

Definition at line 96 of file [gui.py](#).

7.12.4.6 `model_checkpoint`

`gui.InferenceWorker.model_checkpoint = model_checkpoint`

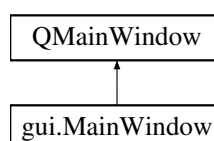
Definition at line 95 of file [gui.py](#).

The documentation for this class was generated from the following file:

- [sourcecode/gui.py](#)

7.13 gui.MainWindow Class Reference

Inheritance diagram for `gui.MainWindow`:



Public Member Functions

- `__init__` (self)
- `select_input_folder` (self)
- `train_model` (self)
- `open_file_dialog` (self)
- `open_model_dialog` (self)
- `calculate_dose` (self)
- `on_inference_finished` (self, str output_path)
- `on_inference_error` (self, str message)
- `visualize_inference_results` (self)
- `on_training_finished` (self)
- `on_training_error` (self, msg)
- `show_training_plots` (self)
- `center` (self)

Public Attributes

- `input_dir` = None
- `output_dir` = None
- `ct_file` = None
- `model_checkpoint` = None
- bool `model_file_bool` = False
- bool `ct_file_bool` = False
- `pm`
- `system_manager`
- `batch_spin` = QSpinBox(self)
- `epochs_spin` = QSpinBox(self)
- `patience_spin` = QSpinBox(self)
- `learning_rate_spin` = QDoubleSpinBox(self)
- `energy_min_spin` = QDoubleSpinBox(self)
- `energy_max_spin` = QDoubleSpinBox(self)
- `open_file_dialog` = QPushButton("Upload CT Scan", self)
- `open_model_dialog` = QPushButton("Upload Model File", self)
- `dose_button` = QPushButton("Calculate Dose", self)
- `calculate_dose`
- `input_button` = QPushButton("Select one Energy Folder for Training", self)
- `select_input_folder`
- `input_label` = QLabel("No input folder selected", self)
- `train_button` = QPushButton("Train Model", self)
- `train_model`
- `show_plots_button` = QPushButton("Show Training Graphs", self)
- `show_training_plots`
- `visualize_button` = QPushButton("Visualize Dose Distribution", self)
- `visualize_inference_results`
- `on_training_finished`
- `on_training_error`
- `ct_volume` = data
- `on_inference_finished` = `InferenceWorker`(self.ct_file, self.model_checkpoint, min_e, max_e)
- `on_inference_error`
- `dose_result_path` = output_path

Static Public Attributes

- str `KEY_INPUT` = "input"
- str `KEY_TARGET` = "target"

Protected Member Functions

- [_create_transforms](#) (self, cube_size)
- [_create_central_widget](#) (self)
- [_update_ui_state](#) (self)

Protected Attributes

- [_train_thread](#) = thread
- [_train_worker](#) = worker
- [_train_progress](#) = progress
- [_infer_thread](#) = thread
- [_infer_worker](#) = worker
- [_infer_progress](#) = progress
- [_plots_window](#) = plots_window

7.13.1 Detailed Description

Main entry point for the RadioTherapy project.

This function sets up the configuration parameters, initializes the SystemManager, and triggers the training and inference processes.

MainWindow is the primary GUI window for the RadioTherapy project.

It provides a menu bar with file actions and a central widget containing buttons for CT upload, dose calculation, and visualization.

Definition at line 210 of file [gui.py](#).

7.13.2 Constructor & Destructor Documentation

7.13.2.1 `__init__()`

```
gui.MainWindow.__init__ (
    self)
```

Definition at line 224 of file [gui.py](#).

7.13.3 Member Function Documentation

7.13.3.1 `_create_central_widget()`

```
gui.MainWindow._create_central_widget (
    self) [protected]
```

Sets up the central widget with a vertical layout and adds buttons for CT upload, dose calculation, and visualization.

Definition at line 281 of file [gui.py](#).

7.13.3.2 `_create_transforms()`

```
gui.MainWindow._create_transforms (
    self,
    cube_size) [protected]
```

Creates the MONAI transforms pipeline.

Definition at line 265 of file [gui.py](#).

7.13.3.3 `_update_ui_state()`

```
gui.MainWindow._update_ui_state (
    self) [protected]
```

Centralized method to update the state of UI elements.

Definition at line 497 of file [gui.py](#).

7.13.3.4 calculate_dose()

```
gui.MainWindow.calculate_dose (  
    self)
```

Calculate dose distribution using the original InferenceModule directly (standalone approach).
Now runs in a background thread with a progress dialog.

Definition at line 727 of file [gui.py](#).

7.13.3.5 center()

```
gui.MainWindow.center (  
    self)
```

Center the window on the screen.

Definition at line 914 of file [gui.py](#).

7.13.3.6 on_inference_error()

```
gui.MainWindow.on_inference_error (  
    self,  
    str message)
```

Definition at line 811 of file [gui.py](#).

7.13.3.7 on_inference_finished()

```
gui.MainWindow.on_inference_finished (  
    self,  
    str output_path)
```

Definition at line 795 of file [gui.py](#).

7.13.3.8 on_training_error()

```
gui.MainWindow.on_training_error (  
    self,  
    msg)
```

Slot to close progress dialog and show error message on main thread

Definition at line 849 of file [gui.py](#).

7.13.3.9 on_training_finished()

```
gui.MainWindow.on_training_finished (  
    self)
```

Slot to close progress dialog and show success message on main thread

Definition at line 838 of file [gui.py](#).

7.13.3.10 open_file_dialog()

```
gui.MainWindow.open_file_dialog (  
    self)
```

Opens a file dialog for selecting a CT scan file in NIfTI format.
If a file is selected, its path is stored and the dose calculation button is enabled.

Definition at line 650 of file [gui.py](#).

7.13.3.11 open_model_dialog()

```
gui.MainWindow.open_model_dialog (  
    self)
```

Opens a file dialog for selecting a model file in .ckpt format.
If a file is selected, its path is stored and the dose calculation button is enabled.

Definition at line 692 of file [gui.py](#).

7.13.3.12 select_input_folder()

```
gui.MainWindow.select_input_folder (  
    self)
```

Opens a directory dialog to select the output cubes folder for training.
Enables the Train button if both folders are selected.

Definition at line 431 of file [gui.py](#).

7.13.3.13 show_training_plots()

```
gui.MainWindow.show_training_plots (  
    self)
```

Open a window showing the three training plot images together.

Definition at line 857 of file [gui.py](#).

7.13.3.14 train_model()

```
gui.MainWindow.train_model (  
    self)
```

Calls the training pipeline using the selected input and output folders.
Expects both folders to be subdirectories of the same parent (root) folder.

Definition at line 506 of file [gui.py](#).

7.13.3.15 visualize_inference_results()

```
gui.MainWindow.visualize_inference_results (  
    self)
```

Visualizes the calculated dose distribution from inference results and embeds it in the GUI window.

Definition at line 818 of file [gui.py](#).

7.13.4 Member Data Documentation

7.13.4.1 _infer_progress

```
gui.MainWindow._infer_progress = progress [protected]
```

Definition at line 789 of file [gui.py](#).

7.13.4.2 _infer_thread

```
gui.MainWindow._infer_thread = thread [protected]
```

Definition at line 787 of file [gui.py](#).

7.13.4.3 _infer_worker

```
gui.MainWindow._infer_worker = worker [protected]
```

Definition at line 788 of file [gui.py](#).

7.13.4.4 `_plots_window`

`gui.MainWindow._plots_window = plots_window` [protected]
Definition at line 912 of file [gui.py](#).

7.13.4.5 `_train_progress`

`gui.MainWindow._train_progress = progress` [protected]
Definition at line 644 of file [gui.py](#).

7.13.4.6 `_train_thread`

`gui.MainWindow._train_thread = thread` [protected]
Definition at line 642 of file [gui.py](#).

7.13.4.7 `_train_worker`

`gui.MainWindow._train_worker = worker` [protected]
Definition at line 643 of file [gui.py](#).

7.13.4.8 `batch_spin`

`gui.MainWindow.batch_spin = QSpinBox(self)`
Definition at line 315 of file [gui.py](#).

7.13.4.9 `calculate_dose`

`gui.MainWindow.calculate_dose`
Definition at line 396 of file [gui.py](#).

7.13.4.10 `ct_file`

`gui.MainWindow.ct_file = None`
Definition at line 231 of file [gui.py](#).

7.13.4.11 `ct_file_bool`

`bool gui.MainWindow.ct_file_bool = False`
Definition at line 234 of file [gui.py](#).

7.13.4.12 `ct_volume`

`gui.MainWindow.ct_volume = data`
Definition at line 677 of file [gui.py](#).

7.13.4.13 `dose_button`

`gui.MainWindow.dose_button = QPushButton("Calculate Dose", self)`
Definition at line 393 of file [gui.py](#).

7.13.4.14 `dose_result_path`

`gui.MainWindow.dose_result_path = output_path`
Definition at line 800 of file [gui.py](#).

7.13.4.15 `energy_max_spin`

`gui.MainWindow.energy_max_spin = QDoubleSpinBox(self)`
Definition at line 373 of file [gui.py](#).

7.13.4.16 energy_min_spin

`gui.MainWindow.energy_min_spin = QDoubleSpinBox(self)`
Definition at line 365 of file [gui.py](#).

7.13.4.17 epochs_spin

`gui.MainWindow.epochs_spin = QSpinBox(self)`
Definition at line 322 of file [gui.py](#).

7.13.4.18 input_button

`gui.MainWindow.input_button = QPushButton("Select one Energy Folder for Training", self)`
Definition at line 400 of file [gui.py](#).

7.13.4.19 input_dir

`gui.MainWindow.input_dir = None`
Definition at line 229 of file [gui.py](#).

7.13.4.20 input_label

`gui.MainWindow.input_label = QLabel("No input folder selected", self)`
Definition at line 405 of file [gui.py](#).

7.13.4.21 KEY_INPUT

`str gui.MainWindow.KEY_INPUT = "input" [static]`
Definition at line 221 of file [gui.py](#).

7.13.4.22 KEY_TARGET

`str gui.MainWindow.KEY_TARGET = "target" [static]`
Definition at line 222 of file [gui.py](#).

7.13.4.23 learning_rate_spin

`gui.MainWindow.learning_rate_spin = QDoubleSpinBox(self)`
Definition at line 335 of file [gui.py](#).

7.13.4.24 model_checkpoint

`gui.MainWindow.model_checkpoint = None`
Definition at line 232 of file [gui.py](#).

7.13.4.25 model_file_bool

`gui.MainWindow.model_file_bool = False`
Definition at line 233 of file [gui.py](#).

7.13.4.26 on_inference_error

`gui.MainWindow.on_inference_error`
Definition at line 784 of file [gui.py](#).

7.13.4.27 on_inference_finished

`gui.MainWindow.on_inference_finished = InferenceWorker(self.ct_file, self.model_checkpoint, min_e, max_e)`
Definition at line 783 of file [gui.py](#).

7.13.4.28 on_training_error

gui.MainWindow.on_training_error
Definition at line 640 of file [gui.py](#).

7.13.4.29 on_training_finished

gui.MainWindow.on_training_finished
Definition at line 639 of file [gui.py](#).

7.13.4.30 open_file_dialog

gui.MainWindow.open_file_dialog = QPushButton("Upload CT Scan", self)
Definition at line 383 of file [gui.py](#).

7.13.4.31 open_model_dialog

gui.MainWindow.open_model_dialog = QPushButton("Upload Model File", self)
Definition at line 389 of file [gui.py](#).

7.13.4.32 output_dir

gui.MainWindow.output_dir = None
Definition at line 230 of file [gui.py](#).

7.13.4.33 patience_spin

gui.MainWindow.patience_spin = QSpinBox(self)
Definition at line 329 of file [gui.py](#).

7.13.4.34 pm

gui.MainWindow.pm
Initial value:

```
= ParameterManager(  
    energies=[0], batch_size=2, cube_size=64,  
    num_epochs=5, learning_rate=1e-4, patience=3  
)
```


Definition at line 238 of file [gui.py](#).

7.13.4.35 select_input_folder

gui.MainWindow.select_input_folder
Definition at line 402 of file [gui.py](#).

7.13.4.36 show_plots_button

gui.MainWindow.show_plots_button = QPushButton("Show Training Graphs", self)
Definition at line 414 of file [gui.py](#).

7.13.4.37 show_training_plots

gui.MainWindow.show_training_plots
Definition at line 417 of file [gui.py](#).

7.13.4.38 system_manager

gui.MainWindow.system_manager
Initial value:

```
= SystemManager(  
    root_dir="", transforms=transforms_chain,  
    resolutions=self.pm.resolutions, energies=self.pm.energies,  
    quad_energies=self.pm.quad_energies,  
    quad_weights=self.pm.quad_weights,
```

```

        batch_size=self.pm.batch_size, device=device,
        num_epochs=self.pm.num_epochs, learning_rate=self.pm.learning_rate, patience=self.pm.patience,
        cube_size= self.pm.cube_size,
        energy_min=0.0, energy_max=50.0,
        seed=42
    )

```

Definition at line 245 of file [gui.py](#).

7.13.4.39 train_button

```
gui.MainWindow.train_button = QPushButton("Train Model", self)
```

Definition at line 408 of file [gui.py](#).

7.13.4.40 train_model

```
gui.MainWindow.train_model
```

Definition at line 411 of file [gui.py](#).

7.13.4.41 visualize_button

```
gui.MainWindow.visualize_button = QPushButton("Visualize Dose Distribution", self)
```

Definition at line 421 of file [gui.py](#).

7.13.4.42 visualize_inference_results

```
gui.MainWindow.visualize_inference_results
```

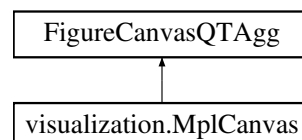
Definition at line 424 of file [gui.py](#).

The documentation for this class was generated from the following file:

- [sourcecode/gui.py](#)

7.14 visualization.MplCanvas Class Reference

Inheritance diagram for visualization.MplCanvas:



Public Member Functions

- [__init__](#) (self, fig)

7.14.1 Detailed Description

A custom widget that converts a matplotlib figure to a Qt widget. This enables embedding matplotlib visualizations directly in the GUI.

Definition at line 152 of file [visualization.py](#).

7.14.2 Constructor & Destructor Documentation

7.14.2.1 __init__()

```
visualization.MplCanvas.__init__ (
    self,
    fig)

```

Definition at line 157 of file [visualization.py](#).

The documentation for this class was generated from the following file:

- [sourcecode/visualization.py](#)

7.15 parameter_manager.ParameterManager Class Reference

Public Member Functions

- `__init__` (self, *energies*, *batch_size*, *cube_size*, *learning_rate*, *num_epochs*, *patience*, *other_parameters*=None)
- `get_parameters` (self)
- `update_parameter` (self, key, value)

Public Attributes

- *energies* = torch.tensor(energies, dtype=torch.float32)
- list *quad_energies* = []
- list *quad_weights* = []
- *batch_size* = batch_size
- *cube_size* = cube_size
- list *resolutions* = [self.cube_size]
- *learning_rate* = learning_rate
- *num_epochs* = num_epochs
- *patience* = patience
- *other_parameters* = other_parameters if other_parameters is not None else {}
- *energy_min* = min_e
- *energy_max* = max_e
- tuple *energy_normalization* = (self.energies.float() - min_e) / (max_e - min_e)

7.15.1 Detailed Description

The ParameterManager class centralizes configuration parameters for the RadioTherapy project. It stores key settings such as spatial resolutions, energy levels, batch size, cube size, learning rate, number of epochs, and additional parameters.

Definition at line 7 of file [parameter_manager.py](#).

7.15.2 Constructor & Destructor Documentation

7.15.2.1 __init__()

```
parameter_manager.ParameterManager.__init__ (
    self,
    energies,
    batch_size,
    cube_size,
    learning_rate,
    num_epochs,
    patience,
    other_parameters = None)
```

Initializes the ParameterManager with the required configuration parameters.

Args:

resolutions (list of tuple): List of spatial resolutions, e.g. [(64, 64, 64), (32, 32, 32)].
energies (list of int): List of energy levels (in keV) for which separate training might be executed.
batch_size (int): The batch size used during training.
cube_size (tuple): The cube size to which CT data will be resized (e.g., (64, 64, 64)).
learning_rate (float): The learning rate used during training.
num_epochs (int): The number of epochs for training.
other_parameters (dict, optional): Any additional parameters to store.

Definition at line 14 of file [parameter_manager.py](#).

7.15.3 Member Function Documentation

7.15.3.1 get_parameters()

```
parameter_manager.ParameterManager.get_parameters (  
    self)
```

Returns all configuration parameters as a dictionary.

Returns:

dict: A dictionary containing resolutions, energies, batch_size, and any additional parameters.

Definition at line 68 of file [parameter_manager.py](#).

7.15.3.2 update_parameter()

```
parameter_manager.ParameterManager.update_parameter (  
    self,  
    key,  
    value)
```

Updates a specific parameter.

Args:

key (str): The parameter name to update.

value: The new value for the parameter.

Definition at line 90 of file [parameter_manager.py](#).

7.15.4 Member Data Documentation

7.15.4.1 batch_size

```
parameter_manager.ParameterManager.batch_size = batch_size
```

Definition at line 36 of file [parameter_manager.py](#).

7.15.4.2 cube_size

```
parameter_manager.ParameterManager.cube_size = cube_size
```

Definition at line 39 of file [parameter_manager.py](#).

7.15.4.3 energies

```
parameter_manager.ParameterManager.energies = torch.tensor(energies, dtype=torch.float32)
```

Definition at line 30 of file [parameter_manager.py](#).

7.15.4.4 energy_max

```
parameter_manager.ParameterManager.energy_max = max_e
```

Definition at line 64 of file [parameter_manager.py](#).

7.15.4.5 energy_min

```
parameter_manager.ParameterManager.energy_min = min_e
```

Definition at line 63 of file [parameter_manager.py](#).

7.15.4.6 energy_normalization

```
tuple parameter_manager.ParameterManager.energy_normalization = (self.energies.float() - min_e) / (max_e - min_e)
```

Definition at line 65 of file [parameter_manager.py](#).

7.15.4.7 learning_rate

```
parameter_manager.ParameterManager.learning_rate = learning_rate
```

Definition at line 51 of file [parameter_manager.py](#).

7.15.4.8 num_epochs

`parameter_manager.ParameterManager.num_epochs = num_epochs`
 Definition at line 52 of file [parameter_manager.py](#).

7.15.4.9 other_parameters

`parameter_manager.ParameterManager.other_parameters = other_parameters if other_parameters is not None else {}`
 Definition at line 54 of file [parameter_manager.py](#).

7.15.4.10 patience

`parameter_manager.ParameterManager.patience = patience`
 Definition at line 53 of file [parameter_manager.py](#).

7.15.4.11 quad_energies

`list parameter_manager.ParameterManager.quad_energies = []`
 Definition at line 34 of file [parameter_manager.py](#).

7.15.4.12 quad_weights

`list parameter_manager.ParameterManager.quad_weights = []`
 Definition at line 35 of file [parameter_manager.py](#).

7.15.4.13 resolutions

`list parameter_manager.ParameterManager.resolutions = [self.cube_size]`
 Definition at line 44 of file [parameter_manager.py](#).
 The documentation for this class was generated from the following file:

- [sourcecode/parameter_manager.py](#)

7.16 storage.Storage Class Reference

Public Member Functions

- [__init__](#) (self, [model_dir](#)="models", [log_file](#)="training_log.txt")
- [save_model](#) (self, model, filename)
- [load_model](#) (self, model, filename, device=`torch.device("cpu")`)
- [save_dose_distribution](#) (self, dose_distribution, filename)
- [log_error](#) (self, message)
- [export_results](#) (self, results, filename)

Public Attributes

- [model_dir](#) = `model_dir`
- [log_file](#) = `log_file`

7.16.1 Detailed Description

The Storage module handles persistence tasks for the project. It provides methods to save and load model checkpoints, save dose distributions, log errors, and export results.

Definition at line 7 of file [storage.py](#).

7.16.2 Constructor & Destructor Documentation

7.16.2.1 `__init__()`

```
storage.Storage.__init__ (  
    self,  
    model_dir = "models",  
    log_file = "training_log.txt")
```

Initializes the Storage module.

Args:

`model_dir (str)`: Directory where model checkpoints and results are saved.
`log_file (str)`: File path for the training log.

Definition at line 14 of file [storage.py](#).

7.16.3 Member Function Documentation

7.16.3.1 `export_results()`

```
storage.Storage.export_results (  
    self,  
    results,  
    filename)
```

Exports training or inference results to a file.

Args:

`results (str)`: The results in string format.
`filename (str)`: The filename to export the results (relative to `model_dir`).

Definition at line 82 of file [storage.py](#).

7.16.3.2 `load_model()`

```
storage.Storage.load_model (  
    self,  
    model,  
    filename,  
    device = torch.device("cpu"))
```

Loads a saved model's state dictionary from a file and loads it into the given model.

Args:

`model (torch.nn.Module)`: The model instance to load the state dictionary into.
`filename (str)`: The filename (relative to `model_dir`) to load the model from.
`device (torch.device)`: The device to map the model to.

Returns:

`torch.nn.Module`: The model loaded with the saved state dictionary.

Definition at line 39 of file [storage.py](#).

7.16.3.3 `log_error()`

```
storage.Storage.log_error (  
    self,  
    message)
```

Appends an error message to the log file.

Args:

`message (str)`: The error message to log.

Definition at line 71 of file [storage.py](#).

7.16.3.4 save_dose_distribution()

```
storage.Storage.save_dose_distribution (
    self,
    dose_distribution,
    filename)
```

Saves the computed dose distribution as a numpy file.

Args:

dose_distribution (numpy.ndarray): The dose distribution array.
filename (str): The filename to save the dose distribution (relative to model_dir).

Definition at line 59 of file [storage.py](#).

7.16.3.5 save_model()

```
storage.Storage.save_model (
    self,
    model,
    filename)
```

Saves the model's state dictionary to a file.

Args:

model (torch.nn.Module): The model to be saved.
filename (str): The filename (relative to model_dir) to save the model.

Definition at line 27 of file [storage.py](#).

7.16.4 Member Data Documentation

7.16.4.1 log_file

```
storage.Storage.log_file = log_file
```

Definition at line 23 of file [storage.py](#).

7.16.4.2 model_dir

```
storage.Storage.model_dir = model_dir
```

Definition at line 22 of file [storage.py](#).

The documentation for this class was generated from the following file:

- [sourcecode/storage.py](#)

7.17 system_manager.SystemManager Class Reference

Public Member Functions

- [__init__](#) (self, [root_dir](#), [transforms](#), [resolutions](#), [energies](#), [energy_min](#), [energy_max](#), [quad_energies](#), [quad_weights](#), [batch_size](#), [device](#), [num_epochs](#), [learning_rate](#), [patience](#), [cube_size](#), [seed=42](#))
- [save_models](#) (self, [autoencoder](#), [UNET](#), [optimizer_diff](#), [optimizer_g](#), [optimizer_d](#), [epoch](#), [res](#), [energy](#), [dose_mean=None](#), [dose_std=None](#), [scale_factor=None](#), [clip_min=0.0](#), [clip_max=None](#), [smoothing_kernel=0](#))
- [save_combined_checkpoint](#) (self, [output_filename="combined_models.ckpt"](#))
- [run_training](#) (self)
- [run_inference](#) (self, [ct_file_path](#), [model_checkpoint=None](#))
- [run_energy_conditioned_training](#) (self)

Public Attributes

- `root_dir` = `root_dir`
- `transforms` = `transforms`
- `resolutions` = `resolutions`
- `energies` = `energies`
- `energy_min` = `energy_min`
- `energy_max` = `energy_max`
- `batch_size` = `batch_size`
- `device` = `device`
- `seed` = `seed`
- `num_epochs` = `num_epochs`
- `learning_rate` = `learning_rate`
- `patience` = `patience`
- `bool training_complete` = `False`
- `bool stop_training` = `False`
- `autoencoder` = `None`
- `unet` = `None`
- `scheduler` = `None`
- `cube_size` = `cube_size`
- `quad_energies` = `quad_energies`
- `list quad_weights` = `quad_weights`
- `dict models_by_energy` = `{}`
- `dict saved_ckpts` = `{}`
- `float scale_factor` = `sum(scale_factors) / len(scale_factors) if scale_factors else 1.0`
- `float dose_mean` = `sum(dose_means) / len(dose_means) if dose_means else 0.0`
- `float dose_std` = `sum(dose_stds) / len(dose_stds) if dose_stds else 1.0`
- `float clip_min` = `sum(clip_mins) / len(clip_mins) if clip_mins else 0.0`
- `int smoothing_kernel` = `sum(smoothing_kernels) / len(smoothing_kernels) if smoothing_kernels else 0`
- `clip_max` = `sum(valid_clip_maxs) / len(valid_clip_maxs) if valid_clip_maxs else None`
- `dose_result_path` = `out_path`

7.17.1 Detailed Description

The `SystemManager` class orchestrates the complete training and inference workflow.

It iterates over each combination of spatial resolution and energy level:

- Loads the dataset using the `DataLoaderModule`.
- Splits the data into training and validation sets.
- Initializes and trains the autoencoder (and optionally diffusion model) using early stopping based on validation loss.
- Sets a training-completion flag for subsequent inference.

Definition at line 32 of file `system_manager.py`.

7.17.2 Constructor & Destructor Documentation

7.17.2.1 `__init__()`

```
system_manager.SystemManager.__init__ (
    self,
    root_dir,
    transforms,
    resolutions,
    energies,
    energy_min,
    energy_max,
    quad_energies,
    quad_weights,
```

```

        batch_size,
        device,
        num_epochs,
        learning_rate,
        patience,
        cube_size,
        seed = 42)

```

Initializes the SystemManager with configuration parameters.

Args:

```

    root_dir (str): Root directory of the dataset.
    transforms (monai.transforms.Compose): Preprocessing transforms to apply.
    resolutions (list of tuple): List of resolutions (e.g., [(64, 64, 64), (32, 32, 32)]).
    energies (list of int): List of energy levels (e.g., [62, 75, 90]).
    batch_size (int): Batch size for DataLoader.
    device (torch.device): Device (CPU or GPU) for computations.
    seed (int): Random seed for reproducibility.

```

Definition at line 43 of file [system_manager.py](#).

7.17.3 Member Function Documentation

7.17.3.1 run_energy_conditioned_training()

```

system_manager.SystemManager.run_energy_conditioned_training (
    self)

```

Executes energy-conditioned training pipeline - trains a SINGLE model with ALL energies.

This is the new approach that trains one model with energy conditioning, instead of separate models for each energy.

Steps:

1. Load dataset with ALL energies (no filtering by energy)
2. Create DataLoaders for training and validation
3. Initialize models with 2-channel input (CT + Energy)
4. Train autoencoder with energy conditioning
5. Train diffusion model with energy conditioning
6. Save the unified model

Definition at line 784 of file [system_manager.py](#).

7.17.3.2 run_inference()

```

system_manager.SystemManager.run_inference (
    self,
    ct_file_path,
    model_checkpoint = None)

```

Load a CT scan and run inference to compute dose distribution.

Definition at line 422 of file [system_manager.py](#).

7.17.3.3 run_training()

```

system_manager.SystemManager.run_training (
    self)

```

Executes the full training pipeline over all resolution and energy combinations.

For each combination, the following steps are executed:

1. Load and split the dataset filtered by the current energy.
2. Create DataLoaders for training and validation.
3. Initialize the autoencoder, discriminator, and diffusion models.
4. Run the autoencoder training loop with early stopping based on validation loss.
5. Run the diffusion model training loop.
6. Save the models for the current configuration.

After all configurations have been processed, a flag is set to indicate that training is complete.

Definition at line 146 of file [system_manager.py](#).

7.17.3.4 save_combined_checkpoint()

```
system_manager.SystemManager.save_combined_checkpoint (
    self,
    output_filename = "combined_models.ckpt")
```

Save all in-memory checkpoints into a single combined file.

Definition at line 115 of file [system_manager.py](#).

7.17.3.5 save_models()

```
system_manager.SystemManager.save_models (
    self,
    autoencoder,
    unet,
    optimizer_diff,
    optimizer_g,
    optimizer_d,
    epoch,
    res,
    energy,
    dose_mean = None,
    dose_std = None,
    scale_factor = None,
    clip_min = 0.0,
    clip_max = None,
    smoothing_kernel = 0)
```

Definition at line 82 of file [system_manager.py](#).

7.17.4 Member Data Documentation

7.17.4.1 autoencoder

```
system_manager.SystemManager.autoencoder = None
```

Definition at line 71 of file [system_manager.py](#).

7.17.4.2 batch_size

```
system_manager.SystemManager.batch_size = batch_size
```

Definition at line 62 of file [system_manager.py](#).

7.17.4.3 clip_max

```
system_manager.SystemManager.clip_max = sum(valid_clip_maxs) / len(valid_clip_maxs) if valid_clip_maxs else
None
```

Definition at line 591 of file [system_manager.py](#).

7.17.4.4 clip_min

```
float system_manager.SystemManager.clip_min = sum(clip_mins) / len(clip_mins) if clip_mins else 0.0
```

Definition at line 586 of file [system_manager.py](#).

7.17.4.5 cube_size

```
system_manager.SystemManager.cube_size = cube_size
```

Definition at line 74 of file [system_manager.py](#).

7.17.4.6 device

`system_manager.SystemManager.device = device`
Definition at line 63 of file [system_manager.py](#).

7.17.4.7 dose_mean

`float system_manager.SystemManager.dose_mean = sum(dose_means) / len(dose_means) if dose_means else 0.0`
Definition at line 584 of file [system_manager.py](#).

7.17.4.8 dose_result_path

`system_manager.SystemManager.dose_result_path = out_path`
Definition at line 781 of file [system_manager.py](#).

7.17.4.9 dose_std

`float system_manager.SystemManager.dose_std = sum(dose_stds) / len(dose_stds) if dose_stds else 1.0`
Definition at line 585 of file [system_manager.py](#).

7.17.4.10 energies

`system_manager.SystemManager.energies = energies`
Definition at line 59 of file [system_manager.py](#).

7.17.4.11 energy_max

`system_manager.SystemManager.energy_max = energy_max`
Definition at line 61 of file [system_manager.py](#).

7.17.4.12 energy_min

`system_manager.SystemManager.energy_min = energy_min`
Definition at line 60 of file [system_manager.py](#).

7.17.4.13 learning_rate

`system_manager.SystemManager.learning_rate = learning_rate`
Definition at line 66 of file [system_manager.py](#).

7.17.4.14 models_by_energy

`dict system_manager.SystemManager.models_by_energy = {}`
Definition at line 78 of file [system_manager.py](#).

7.17.4.15 num_epochs

`system_manager.SystemManager.num_epochs = num_epochs`
Definition at line 65 of file [system_manager.py](#).

7.17.4.16 patience

`system_manager.SystemManager.patience = patience`
Definition at line 67 of file [system_manager.py](#).

7.17.4.17 quad_energies

`system_manager.SystemManager.quad_energies = quad_energies`
Definition at line 75 of file [system_manager.py](#).

7.17.4.18 quad_weights

`list system_manager.SystemManager.quad_weights = quad_weights`
Definition at line 76 of file [system_manager.py](#).

7.17.4.19 resolutions

`system_manager.SystemManager.resolutions = resolutions`
Definition at line 58 of file [system_manager.py](#).

7.17.4.20 root_dir

`system_manager.SystemManager.root_dir = root_dir`
Definition at line 56 of file [system_manager.py](#).

7.17.4.21 saved_ckpts

`dict system_manager.SystemManager.saved_ckpts = {}`
Definition at line 80 of file [system_manager.py](#).

7.17.4.22 scale_factor

`float system_manager.SystemManager.scale_factor = sum(scale_factors) / len(scale_factors) if scale_factors else 1.0`
Definition at line 583 of file [system_manager.py](#).

7.17.4.23 scheduler

`system_manager.SystemManager.scheduler = None`
Definition at line 73 of file [system_manager.py](#).

7.17.4.24 seed

`system_manager.SystemManager.seed = seed`
Definition at line 64 of file [system_manager.py](#).

7.17.4.25 smoothing_kernel

`int system_manager.SystemManager.smoothing_kernel = sum(smoothing_kernels) / len(smoothing_kernels) if smoothing_kernels else 0`
Definition at line 587 of file [system_manager.py](#).

7.17.4.26 stop_training

`bool system_manager.SystemManager.stop_training = False`
Definition at line 69 of file [system_manager.py](#).

7.17.4.27 training_complete

`bool system_manager.SystemManager.training_complete = False`
Definition at line 68 of file [system_manager.py](#).

7.17.4.28 transforms

`system_manager.SystemManager.transforms = transforms`
Definition at line 57 of file [system_manager.py](#).

7.17.4.29 unet

`system_manager.SystemManager.unet = None`

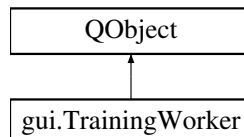
Definition at line 72 of file [system_manager.py](#).

The documentation for this class was generated from the following file:

- [sourcecode/system_manager.py](#)

7.18 gui.TrainingWorker Class Reference

Inheritance diagram for `gui.TrainingWorker`:



Public Member Functions

- [__init__](#) (self, [manager](#))
- [run](#) (self)

Public Attributes

- [manager](#) = manager

Static Public Attributes

- [finished](#) = `Signal()`
- [error](#) = `Signal(str)`
- [progress](#) = `Signal(int, int)`

7.18.1 Detailed Description

Worker object to run the training in a separate thread.

Definition at line 60 of file [gui.py](#).

7.18.2 Constructor & Destructor Documentation

7.18.2.1 __init__()

```

gui.TrainingWorker.__init__ (
    self,
    manager)
  
```

Definition at line 68 of file [gui.py](#).

7.18.3 Member Function Documentation

7.18.3.1 run()

```

gui.TrainingWorker.run (
    self)
  
```

Definition at line 72 of file [gui.py](#).

7.18.4 Member Data Documentation

7.18.4.1 error

`gui.TrainingWorker.error = Signal(str) [static]`

Definition at line 65 of file [gui.py](#).

7.18.4.2 finished

`gui.TrainingWorker.finished = Signal() [static]`
 Definition at line 64 of file [gui.py](#).

7.18.4.3 manager

`gui.TrainingWorker.manager = manager`
 Definition at line 70 of file [gui.py](#).

7.18.4.4 progress

`gui.TrainingWorker.progress = Signal(int, int) [static]`
 Definition at line 66 of file [gui.py](#).

The documentation for this class was generated from the following file:

- [sourcecode/gui.py](#)

7.19 visualization.Visualization Class Reference

Public Member Functions

- [__init__](#) (self)
- [plot_learning_curve](#) (self, epochs, loss_list, title="Learning Curve", filename=None)
- [plot_adversarial_curves](#) (self, epochs, gen_loss, disc_loss, title="Adversarial Training Curves", filename=None)
- [display_sample_images](#) (self, images, title="Sample Images", filename=None)

Static Public Member Functions

- [plot_loss_curves](#) (ae_train, ae_val, gen, disc, diff, resolution, energy)

7.19.1 Detailed Description

The Visualization module provides methods to plot training curves and display sample images. This is useful for monitoring the training process and visualizing inference results.

Definition at line 9 of file [visualization.py](#).

7.19.2 Constructor & Destructor Documentation

7.19.2.1 __init__()

```
visualization.Visualization.__init__ (
    self)
```

Initializes the Visualization module.

Definition at line 15 of file [visualization.py](#).

7.19.3 Member Function Documentation

7.19.3.1 display_sample_images()

```
visualization.Visualization.display_sample_images (
    self,
    images,
    title = "Sample Images",
    filename = None)
```

Displays sample images from the given batch. For 3D images, it shows three orthogonal slices.

Args:

`images` (torch.Tensor or numpy.ndarray): Images to display. Expected shape is either [batch, channels, depth, height, width] for 3D images or [batch, channels, height, width] for 2D.
`title` (str): Title for the plot.
`filename` (str, optional): If provided, the figure is saved to this file.

Definition at line 70 of file [visualization.py](#).

7.19.3.2 plot_adversarial_curves()

```
visualization.Visualization.plot_adversarial_curves (
    self,
    epochs,
    gen_loss,
    disc_loss,
    title = "Adversarial Training Curves",
    filename = None)
```

Plots the adversarial training curves for generator and discriminator.

Args:

`epochs` (list or array): Epoch numbers.
`gen_loss` (list or array): Generator loss values.
`disc_loss` (list or array): Discriminator loss values.
`title` (str): Title of the plot.
`filename` (str, optional): If provided, the plot is saved to this file.

Definition at line 45 of file [visualization.py](#).

7.19.3.3 plot_learning_curve()

```
visualization.Visualization.plot_learning_curve (
    self,
    epochs,
    loss_list,
    title = "Learning Curve",
    filename = None)
```

Plots the learning curve over epochs.

Args:

`epochs` (list or array): Epoch numbers.
`loss_list` (list or array): Loss values corresponding to each epoch.
`title` (str): Title of the plot.
`filename` (str, optional): If provided, the plot is saved to this file.

Definition at line 21 of file [visualization.py](#).

7.19.3.4 plot_loss_curves()

```
visualization.Visualization.plot_loss_curves (
    ae_train,
    ae_val,
    gen,
    disc,
    diff,
    resolution,
    energy) [static]
```

Plot training and validation losses for autoencoder and diffusion model.

Args:

`ae_train` (list of float): Autoencoder training losses per epoch.
`ae_val` (list of float): Autoencoder validation losses per epoch.


```
gen      (list of float): Generator losses per epoch.  
disc     (list of float): Discriminator losses per epoch.  
diff     (list of float): Diffusion model losses per epoch.  
resolution (tuple): Spatial resolution used for this run.  
energy   (float): Energy level used for this run.
```

Definition at line 120 of file [visualization.py](#).

The documentation for this class was generated from the following file:

- [sourcecode/visualization.py](#)

Chapter 8

File Documentation

8.1 README.md File Reference

8.2 sourcecode/corrected_inference.py File Reference

Classes

- class [corrected_inference.CorrectedInferenceModule](#)

Namespaces

- namespace [corrected_inference](#)

Functions

- [corrected_inference.test_corrected_inference](#) ()

Variables

- [corrected_inference.logger](#) = logging.getLogger(__name__)

8.3 corrected_inference.py

[Go to the documentation of this file.](#)

```
00001 #!/usr/bin/env python3
00002 """
00003 Corrected inference pipeline that properly uses the energy-conditioned model.
00004 """
00005
00006 import torch
00007 import torch.nn.functional as F
00008 import numpy as np
00009 from pathlib import Path
00010 import logging
00011
00012 logger = logging.getLogger(__name__)
00013
00014 class CorrectedInferenceModule:
00015     """
00016     Fixed inference module that properly handles the energy-conditioned model.
00017     """
00018
00019     def __init__(self, model_path, device='cpu'):
00020         """
00021         Initialize with the corrected energy-conditioned model.
00022
00023         Args:
00024             model_path: Path to the unified model checkpoint
00025             device: Device to run inference on
00026         """
00027         logger.info("=" * 60)
00028         logger.info("CORRECTED INFERENCE MODULE INITIALIZATION")
00029         logger.info("=" * 60)
00030
```

```

00031     self.device = device
00032     self.model_path = model_path
00033
00034     # Load the checkpoint
00035     logger.info(f"Loading model from: {model_path}")
00036     logger.info(f"Device: {device}")
00037
00038     try:
00039         # Try loading with weights_only=False to avoid the warning and ensure compatibility
00040         logger.info("Attempting to load checkpoint...")
00041         self.checkpoint = torch.load(model_path, map_location=device, weights_only=False)
00042         logger.info("Checkpoint loaded successfully via direct path")
00043     except Exception as e:
00044         # If that fails, try opening as file explicitly
00045         logger.warning(f"Direct load failed: {e}")
00046         logger.info("Attempting to load checkpoint via file handle...")
00047         try:
00048             with open(model_path, 'rb') as f:
00049                 self.checkpoint = torch.load(f, map_location=device, weights_only=False)
00050                 logger.info("Checkpoint loaded successfully via file handle")
00051         except Exception as e2:
00052             logger.error(f"Both loading methods failed: {e2}")
00053             raise e2
00054
00055     # Extract energies and scale factor from checkpoint
00056     self.energies = self.checkpoint.get('energies', [11.5, 15.75, 34.25])
00057     self.scale_factor = self.checkpoint.get('scale_factor', 0.18215)
00058
00059     logger.info(f" Model energies: {self.energies}")
00060     logger.info(f" Scale factor: {self.scale_factor}")
00061     logger.info(f" Checkpoint keys: {list(self.checkpoint.keys())}")
00062
00063     # Load models
00064     logger.info("Loading model components...")
00065     self._load_models()
00066     logger.info("Corrected inference module initialized successfully")
00067
00068     def _load_models(self):
00069         """Load autoencoder, unet, and scheduler from checkpoint."""
00070         try:
00071             logger.info(" Loading required modules...")
00072             # Import required modules
00073             from generative.networks.nets import AutoencoderKL, DiffusionModelUNet
00074             from generative.inferers import LatentDiffusionInferer
00075             from monai.utils import set_determinism
00076             logger.info(" Modules imported successfully")
00077
00078             # Load autoencoder
00079             logger.info(" Creating autoencoder...")
00080             self.autoencoder = AutoencoderKL(
00081                 spatial_dims=3,
00082                 in_channels=2, # CT + Energy channel
00083                 out_channels=1,
00084                 num_channels=(32, 32, 32),
00085                 latent_channels=2,
00086                 num_res_blocks=1,
00087                 norm_num_groups=8,
00088                 attention_levels=(False, False, True),
00089             ).to(self.device)
00090             logger.info(f" Autoencoder created (in_channels=2, out_channels=1)")
00091
00092             # Load UNet
00093             logger.info(" Creating UNet...")
00094             self.unet = DiffusionModelUNet(
00095                 spatial_dims=3,
00096                 in_channels=2, # Latent space is 2 channels
00097                 out_channels=2,
00098                 with_conditioning=True,
00099                 cross_attention_dim=2,
00100                 num_res_blocks=1,
00101                 num_channels=(32, 64, 64),
00102                 attention_levels=(False, True, True),
00103                 num_head_channels=(0, 32, 32),
00104             ).to(self.device)
00105             logger.info(f" UNet created (in_channels=2, out_channels=2, cross_attention_dim=2)")
00106
00107             # Load scheduler
00108             logger.info(" Creating scheduler...")
00109             from generative.networks.schedulers.ddpm import DDPMSScheduler
00110
00111             self.scheduler = DDPMSScheduler(
00112                 num_train_timesteps=1000,
00113                 beta_start=0.0015,
00114                 beta_end=0.0195
00115             )
00116             logger.info(f" DDPM Scheduler created (timesteps=1000)")
00117

```

```

00118         # Load state dicts
00119         logger.info(" Loading model weights...")
00120
00121         # Check autoencoder weights
00122         ae_state_dict = self.checkpoint['autoencoder']
00123         logger.info(f" Autoencoder checkpoint keys: {len(ae_state_dict)} parameters")
00124         self.autoencoder.load_state_dict(ae_state_dict)
00125         logger.info(" Autoencoder weights loaded")
00126
00127         # Check UNet weights
00128         unet_state_dict = self.checkpoint['unet']
00129         logger.info(f" UNet checkpoint keys: {len(unet_state_dict)} parameters")
00130         self.unet.load_state_dict(unet_state_dict)
00131         logger.info(" UNet weights loaded")
00132
00133         # Set to eval mode
00134         self.autoencoder.eval()
00135         self.unet.eval()
00136         logger.info(" Models set to evaluation mode")
00137
00138         # Create inferer
00139         logger.info(" Creating inferer...")
00140         self.inferer = LatentDiffusionInferer(
00141             scheduler=self.scheduler,
00142             scale_factor=self.scale_factor
00143         )
00144         logger.info(f" LatentDiffusionInferer created (scale_factor={self.scale_factor})")
00145
00146     except Exception as e:
00147         logger.error(f"Failed to load models: {e}")
00148         logger.error("Model loading traceback:", exc_info=True)
00149         raise e
00150
00151     logger.info(" All models loaded successfully")
00152
00153
00154 def preprocess_ct(self, ct_array, target_cube_size=(64, 64, 64)):
00155     """
00156     Preprocess CT data for inference.
00157
00158     Args:
00159         ct_array: CT data as numpy array
00160         target_cube_size: Target size for processing
00161
00162     Returns:
00163         Preprocessed CT tensor
00164     """
00165     # Convert to tensor
00166     if isinstance(ct_array, np.ndarray):
00167         ct_tensor = torch.from_numpy(ct_array).float()
00168     else:
00169         ct_tensor = ct_array.float()
00170
00171     # Add batch and channel dimensions if needed
00172     if ct_tensor.dim() == 3:
00173         ct_tensor = ct_tensor.unsqueeze(0).unsqueeze(0) # [1, 1, D, H, W]
00174     elif ct_tensor.dim() == 4:
00175         ct_tensor = ct_tensor.unsqueeze(0) # [1, C, D, H, W]
00176
00177     # Resize to target size
00178     if ct_tensor.shape[2:] != target_cube_size:
00179         ct_tensor = F.interpolate(
00180             ct_tensor,
00181             size=target_cube_size,
00182             mode='trilinear',
00183             align_corners=False
00184         )
00185
00186     # Normalize CT values (similar to training preprocessing)
00187     # Clamp extreme values and normalize to 0-1 range
00188     ct_tensor = torch.clamp(ct_tensor, -1000, 3000) # Typical CT range
00189     ct_tensor = (ct_tensor + 1000) / 4000 # Normalize to 0-1
00190
00191     return ct_tensor.to(self.device)
00192
00193 def run_inference(self, ct_tensor, quad_energies=None, quad_weights=None, target_cube_size=(64, 64, 64)):
00194     """
00195     Run inference on CT data with quadrature integration over energies.
00196
00197     Args:
00198         ct_tensor: CT data tensor or numpy array
00199         quad_energies: List of quadrature energies (if None, uses single energy)
00200         quad_weights: List of quadrature weights (if None, uses single energy)
00201         target_cube_size: Target cube size for processing
00202
00203     Returns:
00204         Dose distribution tensor

```

```

00205     """
00206     if quad_energies is None or quad_weights is None:
00207         # Single energy inference
00208         logger.info("=== RUNNING SINGLE ENERGY INFERENCE ===")
00209         return self.run_inference_single_energy(ct_tensor, target_cube_size=target_cube_size)
00210
00211     logger.info("=" * 60)
00212     logger.info("RUNNING CORRECTED INFERENCE WITH QUADRATURE")
00213     logger.info("=" * 60)
00214     logger.info(f"Quadrature energies: {quad_energies}")
00215     logger.info(f"Quadrature weights: {quad_weights}")
00216     logger.info(f"Number of quadrature points: {len(quad_energies)}")
00217
00218     try:
00219         # Run inference for each energy and aggregate
00220         total_dose = None
00221
00222         for i, (energy, weight) in enumerate(zip(quad_energies, quad_weights)):
00223             logger.info(f"Processing quadrature point {i+1}/{len(quad_energies)}: energy={energy:.3f} keV,
weight={weight:.6f}")
00224
00225             # Run inference for this energy
00226             dose = self.run_inference_single_energy(ct_tensor, target_energy=energy,
target_cube_size=target_cube_size)
00227             logger.info(f"    Dose computed: shape={dose.shape}, range=[{dose.min():.6f}, {dose.max():.6f}]")
00228
00229             # Weight and accumulate
00230             if total_dose is None:
00231                 total_dose = dose * weight
00232                 logger.info(f"    Initialized total dose (weighted by {weight:.6f})")
00233             else:
00234                 total_dose += dose * weight
00235                 logger.info(f"    Added to total dose (weighted by {weight:.6f})")
00236
00237             logger.info("=" * 60)
00238             logger.info("QUADRATURE INTEGRATION COMPLETED SUCCESSFULLY")
00239             logger.info(f"Final dose: shape={total_dose.shape}, range=[{total_dose.min():.6f},
{total_dose.max():.6f}]")
00240             logger.info("=" * 60)
00241             return total_dose
00242
00243     except Exception as e:
00244         logger.error("QUADRATURE INFERENCE FAILED")
00245         logger.error(f"Error: {e}")
00246         logger.error("Quadrature inference traceback:", exc_info=True)
00247         raise e
00248
00249 def run_inference_single_energy(self, ct_tensor, target_energy=None, target_cube_size=(64, 64, 64)):
00250     """
00251     Run inference on CT data for a single energy.
00252
00253     Args:
00254         ct_tensor: CT data tensor or numpy array
00255         target_energy: Target energy for inference (if None, uses first available)
00256         target_cube_size: Target cube size for processing
00257
00258     Returns:
00259         Dose distribution tensor
00260     """
00261     logger.info("-" * 40)
00262     logger.info("SINGLE ENERGY INFERENCE STARTED")
00263     logger.info("-" * 40)
00264
00265     try:
00266         # Preprocess CT
00267         original_shape = ct_tensor.shape if hasattr(ct_tensor, 'shape') else ct_tensor.shape
00268         logger.info(f"Original CT shape: {original_shape}")
00269
00270         input_tensor = self.preprocess_ct(ct_tensor, target_cube_size)
00271         logger.info(f"Input preprocessed: {input_tensor.shape}")
00272
00273         # Choose energy
00274         if target_energy is None:
00275             target_energy = self.energies[0]
00276             logger.info(f"No target energy specified, using default: {target_energy} keV")
00277         else:
00278             logger.info(f"Using specified energy: {target_energy} keV")
00279
00280         # Add energy conditioning
00281         B, C, D, H, W = input_tensor.shape
00282         normalized_energy = target_energy / 100.0 # Normalize like in training
00283         energy_tensor = torch.full((B, 1, D, H, W), normalized_energy, device=self.device)
00284         conditioned_input = torch.cat([input_tensor, energy_tensor], dim=1)
00285
00286         logger.info(f"Energy conditioning added:")
00287         logger.info(f"  - Input shape: {input_tensor.shape}")
00288         logger.info(f"  - Energy tensor shape: {energy_tensor.shape}")

```

```

00289         logger.info(f" - Conditioned input shape: {conditioned_input.shape}")
00290         logger.info(f" - Raw energy: {target_energy}, normalized: {normalized_energy:.6f}")
00291
00292         with torch.no_grad():
00293             # 1. Encode with autoencoder
00294             logger.info("Step 1: Encoding with autoencoder...")
00295             encoded = self.autoencoder.encode(conditioned_input)
00296             if isinstance(encoded, tuple):
00297                 latent = encoded[0]
00298             else:
00299                 latent = encoded.latent_dist.sample()
00300
00301         logger.info(f"Latent shape: {latent.shape}")
00302         logger.info(f"Latent range: {latent.min():.4f} to {latent.max():.4f}")
00303
00304         # 2. Create context for cross-attention
00305         energy_weight = 1.0 # Could be adjusted based on energy
00306         context = torch.tensor(
00307             [[normalized_energy, energy_weight]],
00308             dtype=torch.float32,
00309             device=self.device
00310         ).unsqueeze(1) # [1, 1, 2] - Add sequence dimension
00311
00312         logger.info(f"Context shape: {context.shape}")
00313         logger.info(f"Context values: {context}")
00314
00315         # 3. Run diffusion sampling
00316         logger.info("Running diffusion sampling...")
00317
00318         # Create noise for sampling
00319         noise_shape = latent.shape
00320         noise = torch.randn(noise_shape, device=self.device)
00321
00322         logger.info(f"Noise shape: {noise_shape}")
00323         logger.info(f"Using context: {context}")
00324
00325         # Use the inferer for proper diffusion sampling
00326         # The LatentDiffusionInferer handles the full pipeline internally
00327         sampled_output = self.inferer.sample(
00328             input_noise=noise,
00329             autoencoder_model=self.autoencoder,
00330             diffusion_model=self.unet,
00331             scheduler=self.scheduler,
00332             conditioning=context,
00333             mode="crossattn"
00334         )
00335
00336         logger.info(f"Sampled output shape: {sampled_output.shape}")
00337         logger.info(f"Sampled output range: {sampled_output.min():.4f} to {sampled_output.max():.4f}")
00338
00339         # The inferer should return the decoded output directly
00340         final_output = sampled_output
00341
00342         # 5. Take only the dose channel (first channel) if multi-channel
00343         if final_output.shape[1] > 1:
00344             dose_output = final_output[:, :1] # Take first channel
00345             logger.info(f"Took first channel for dose, shape: {dose_output.shape}")
00346         else:
00347             dose_output = final_output
00348
00349         logger.info(f"Final dose output shape: {dose_output.shape}")
00350         logger.info(f"Final dose range: {dose_output.min():.6f} to {dose_output.max():.6f}")
00351
00352         # 6. Resize back to original size if needed
00353         if hasattr(ct_tensor, 'shape'):
00354             original_spatial = ct_tensor.shape if len(ct_tensor.shape) == 3 else ct_tensor.shape[-3:]
00355         else:
00356             original_spatial = original_shape if len(original_shape) == 3 else original_shape[-3:]
00357
00358         if dose_output.shape[2:] != original_spatial:
00359             logger.info(f"Resizing from {dose_output.shape[2:]} to {original_spatial}")
00360             dose_output = F.interpolate(
00361                 dose_output,
00362                 size=original_spatial,
00363                 mode='trilinear',
00364                 align_corners=False
00365             )
00366
00367         # 7. Remove batch dimension
00368         final_dose = dose_output[0, 0] # Remove batch and channel dims
00369
00370         logger.info(f"Final output shape: {final_dose.shape}")
00371         logger.info(f"Final output range: {final_dose.min():.6f} to {final_dose.max():.6f}")
00372
00373         logger.info(" SINGLE ENERGY INFERENCE COMPLETED SUCCESSFULLY")
00374         logger.info("-" * 40)
00375         return final_dose

```

```

00376
00377     except Exception as e:
00378         logger.error("SINGLE ENERGY INFERENCE FAILED")
00379         logger.error(f"Error: {e}")
00380         logger.error("Single energy inference traceback:", exc_info=True)
00381         raise e
00382
00383
00384 def test_corrected_inference():
00385     """Test the corrected inference module."""
00386     import matplotlib.pyplot as plt
00387
00388     # Load test data
00389     input_file = Path("traindata/11_5/inputcube/235101017859661465075472232303048949736_0.npy")
00390     output_file = Path("traindata/11_5/outputcube/235101017859661465075472232303048949736_0.npy")
00391
00392     if not input_file.exists():
00393         print(f"Test input file not found: {input_file}")
00394         return
00395
00396     input_data = np.load(input_file)
00397     ground_truth = np.load(output_file)
00398
00399     print(f"Input shape: {input_data.shape}")
00400     print(f"Ground truth shape: {ground_truth.shape}")
00401     print(f"Ground truth range: {ground_truth.min():.6f} to {ground_truth.max():.6f}")
00402
00403     # Run corrected inference
00404     model_path = "unified_energy_conditioned_model_res16.0_energies3.ckpt"
00405
00406     try:
00407         inference_module = CorrectedInferenceModule(model_path, device='cpu')
00408
00409         # Run inference
00410         result = inference_module.run_inference_single_energy(
00411             ct_tensor=input_data,
00412             target_energy=11.5,
00413             target_cube_size=(64, 64, 64)
00414         )
00415
00416         result_np = result.cpu().numpy()
00417         print(f"Inference result shape: {result_np.shape}")
00418         print(f"Inference result range: {result_np.min():.6f} to {result_np.max():.6f}")
00419
00420         # Compare with ground truth
00421         if result_np.shape == ground_truth.shape:
00422             correlation = np.corrcoef(result_np.flatten(), ground_truth.flatten())[0, 1]
00423             print(f"Correlation with ground truth: {correlation:.6f}")
00424
00425         # Visualize comparison
00426         fig, axes = plt.subplots(2, 3, figsize=(15, 10))
00427
00428         slice_idx = ground_truth.shape[2] // 2
00429
00430         # Ground truth
00431         im1 = axes[0, 0].imshow(ground_truth[:, :, slice_idx], cmap='hot')
00432         axes[0, 0].set_title(f'Ground Truth \nMax: {ground_truth.max():.6f}')
00433         axes[0, 0].axis('off')
00434         plt.colorbar(im1, ax=axes[0, 0])
00435
00436         # Inference result
00437         im2 = axes[0, 1].imshow(result_np[:, :, slice_idx], cmap='hot')
00438         axes[0, 1].set_title(f'Corrected Inference \nMax: {result_np.max():.6f}')
00439         axes[0, 1].axis('off')
00440         plt.colorbar(im2, ax=axes[0, 1])
00441
00442         # Difference
00443         diff = result_np - ground_truth
00444         im3 = axes[0, 2].imshow(diff[:, :, slice_idx], cmap='RdBu_r')
00445         axes[0, 2].set_title(f'Difference \nMax: {abs(diff).max():.6f}')
00446         axes[0, 2].axis('off')
00447         plt.colorbar(im3, ax=axes[0, 2])
00448
00449         # Axial view
00450         slice_idx_y = ground_truth.shape[1] // 2
00451
00452         im4 = axes[1, 0].imshow(ground_truth[:, slice_idx_y, :], cmap='hot')
00453         axes[1, 0].set_title('Ground Truth (Axial)')
00454         axes[1, 0].axis('off')
00455         plt.colorbar(im4, ax=axes[1, 0])
00456
00457         im5 = axes[1, 1].imshow(result_np[:, slice_idx_y, :], cmap='hot')
00458         axes[1, 1].set_title('Corrected Inference (Axial)')
00459         axes[1, 1].axis('off')
00460         plt.colorbar(im5, ax=axes[1, 1])
00461
00462         im6 = axes[1, 2].imshow(diff[:, slice_idx_y, :], cmap='RdBu_r')

```



```

00463         axes[1, 2].set_title('Difference (Axial)')
00464         axes[1, 2].axis('off')
00465         plt.colorbar(im6, ax=axes[1, 2])
00466
00467         plt.tight_layout()
00468         plt.savefig('corrected_inference_comparison.png', dpi=150, bbox_inches='tight')
00469         print("Saved comparison as 'corrected_inference_comparison.png'")
00470
00471     else:
00472         print(f"Shape mismatch: result {result_np.shape} vs ground truth {ground_truth.shape}")
00473
00474     except Exception as e:
00475         print(f"Error during corrected inference: {e}")
00476         import traceback
00477         traceback.print_exc()
00478
00479
00480 if __name__ == "__main__":
00481     test_corrected_inference()

```

8.4 sourcecode/corrected_training_pipeline.py File Reference

Classes

- class [corrected_training_pipeline.CorrectedDoseNpyDataset](#)
- class [corrected_training_pipeline.CorrectedTrainingPipeline](#)

Namespaces

- namespace [corrected_training_pipeline](#)

Functions

- [corrected_training_pipeline.main](#) ()

Variables

- [corrected_training_pipeline.logger](#) = logging.getLogger(__name__)

8.5 corrected_training_pipeline.py

[Go to the documentation of this file.](#)

```

00001 # corrected_training_pipeline.py
00002
00003 """
00004 CORRECTED TRAINING PIPELINE - NACHHALTIGE LÖSUNG
00005
00006 Fixes all three critical problems:
00007 1. Transform Fix: Use Resized instead of Spacingd
00008 2. Dose Normalization Fix: Calculate from original data
00009 3. Unified Energy Model: Train one model for all energies
00010
00011 This is the sustainable solution that addresses root causes.
00012 """
00013
00014 import os
00015 import torch
00016 import numpy as np
00017 import logging
00018 import glob
00019 from typing import List, Tuple, Dict, Optional
00020 from monai.transforms import (
00021     Compose, LoadImaged, EnsureChannelFirstd, EnsureTyped,
00022     Orientationd, Resized, SpatialPadd, CenterSpatialCropd,
00023     ScaleIntensityRangePercentilesd, ToTensord
00024 )
00025 from monai.data import NumpyReader, DataLoader
00026 from monai.utils import first
00027 from torch.utils.data import Dataset
00028 from generative.networks.nets import AutoencoderKL, PatchDiscriminator, DiffusionModelUNet
00029 from generative.inferers import LatentDiffusionInferer
00030 from generative.networks.schedulers.ddpm import DDPMSScheduler
00031 from torch.optim import Adam
00032 from torch.amp import autocast
00033

```

```

00034 logger = logging.getLogger(__name__)
00035
00036 class CorrectedDoseNpyDataset(Dataset):
00037     """
00038     CORRECTED Dataset that properly loads original dose values for correct normalization.
00039     """
00040     def __init__(self, root_dir, section=None, transforms=None, target_resolution=(64, 64, 64)):
00041         self.root_dir = root_dir
00042         self.target_resolution = target_resolution
00043         self.samples = []
00044
00045         base_dir = root_dir if section is None else os.path.join(root_dir, section)
00046
00047         logger.info(" CORRECTED: Loading samples and analyzing ORIGINAL dose ranges...")
00048
00049         # Track original dose statistics per energy
00050         self.original_dose_stats = {}
00051
00052         for energy_folder in sorted(os.listdir(base_dir)):
00053             folder_path = os.path.join(base_dir, energy_folder)
00054             if not os.path.isdir(folder_path) or energy_folder.startswith('.'):
00055                 continue
00056
00057             try:
00058                 energy = float(energy_folder.replace("_", "."))
00059             except ValueError:
00060                 continue
00061
00062             in_dir = os.path.join(folder_path, "inputcube")
00063             out_dir = os.path.join(folder_path, "outputcube")
00064
00065             if not os.path.isdir(in_dir) or not os.path.isdir(out_dir):
00066                 continue
00067
00068             # CRITICAL: Analyze ORIGINAL dose files before any transforms
00069             energy_dose_values = []
00070             valid_samples = []
00071
00072             for fname in sorted(os.listdir(in_dir)):
00073                 if not fname.endswith(".npz"):
00074                     continue
00075
00076                 in_fp = os.path.join(in_dir, fname)
00077                 out_fp = os.path.join(out_dir, fname)
00078
00079                 if os.path.exists(out_fp):
00080                     # Load ORIGINAL dose data
00081                     original_dose = np.load(out_fp)
00082                     energy_dose_values.append(original_dose.flatten())
00083                     valid_samples.append((in_fp, out_fp, energy))
00084
00085             if energy_dose_values:
00086                 all_energy_doses = np.concatenate(energy_dose_values)
00087                 self.original_dose_stats[energy] = {
00088                     'min': float(all_energy_doses.min()),
00089                     'max': float(all_energy_doses.max()),
00090                     'mean': float(all_energy_doses.mean()),
00091                     'std': float(all_energy_doses.std()),
00092                     'nonzero_voxels': int(np.count_nonzero(all_energy_doses)),
00093                     'total_voxels': int(len(all_energy_doses))
00094                 }
00095
00096             logger.info(f" Energy {energy} eV: {len(valid_samples)} samples, "
00097                       f"ORIGINAL dose range: {all_energy_doses.min():.6f} - {all_energy_doses.max():.6f}")
00098
00099             self.samples.extend(valid_samples)
00100
00101         self.transforms = transforms
00102
00103         # Log comprehensive dose statistics
00104         logger.info(" CORRECTED: Original dose statistics per energy:")
00105         for energy, stats in self.original_dose_stats.items():
00106             sparsity = (1 - stats['nonzero_voxels'] / stats['total_voxels']) * 100
00107             logger.info(f" {energy:6.2f} eV: range=[{stats['min']:.6f}, {stats['max']:.6f}], "
00108                       f"sparsity={sparsity:.3f}%")
00109
00110     def get_dose_normalization_params(self) -> Dict[str, float]:
00111         """
00112         Get realistic dose normalization parameters based on ORIGINAL data.
00113         """
00114         if not self.original_dose_stats:
00115             raise RuntimeError("No original dose statistics available")
00116
00117         # Calculate global statistics across all energies
00118         all_mins = [stats['min'] for stats in self.original_dose_stats.values()]
00119         all_maxs = [stats['max'] for stats in self.original_dose_stats.values()]
00120

```

```

00121     global_clip_min = min(all_mins)
00122     global_clip_max = max(all_maxs)
00123
00124     # Calculate realistic normalization range
00125     # Use 95th percentile to avoid extreme outliers but preserve realistic scale
00126     all_values = []
00127     for energy_stats in self.original_dose_stats.values():
00128         all_values.extend([stats['max'] for stats in energy_stats])
00129
00130     robust_max = np.percentile(all_values, 95)
00131
00132     normalization_params = {
00133         'clip_min': global_clip_min,
00134         'clip_max': global_clip_max,
00135         'robust_max': robust_max,
00136         'energy_specific_stats': self.original_dose_stats
00137     }
00138
00139     logger.info(f" CORRECTED: Dose normalization params: "
00140               f"clip_min={global_clip_min:.6f}, clip_max={global_clip_max:.6f}, "
00141               f"robust_max={robust_max:.6f}")
00142
00143     return normalization_params
00144
00145 def __len__(self):
00146     return len(self.samples)
00147
00148 def __getitem__(self, idx):
00149     in_fp, out_fp, energy = self.samples[idx]
00150
00151     if self.transforms:
00152         # Use MONAI-style loading for transforms
00153         sample = {
00154             "input": in_fp,
00155             "target": out_fp,
00156             "energy": torch.tensor([energy], dtype=torch.float32),
00157         }
00158         sample = self.transforms(sample)
00159     else:
00160         # Direct loading
00161         arr_in = np.load(in_fp)
00162         arr_out = np.load(out_fp)
00163         sample = {
00164             "input": torch.from_numpy(arr_in[None]).float(),
00165             "target": torch.from_numpy(arr_out[None]).float(),
00166             "energy": torch.tensor([energy], dtype=torch.float32),
00167         }
00168
00169     return sample
00170
00171 class CorrectedTrainingPipeline:
00172     """
00173     CORRECTED Training Pipeline that implements the sustainable solution.
00174     """
00175
00176     def __init__(self,
00177                 root_dir: str,
00178                 energies: List[float],
00179                 target_resolution: Tuple[int, int, int] = (64, 64, 64),
00180                 cube_size: Tuple[int, int, int] = (64, 64, 64),
00181                 batch_size: int = 2,
00182                 learning_rate: float = 1e-5,
00183                 num_epochs: int = 100,
00184                 device: torch.device = None):
00185
00186         self.root_dir = root_dir
00187         self.energies = energies
00188         self.target_resolution = target_resolution
00189         self.cube_size = cube_size
00190         self.batch_size = batch_size
00191         self.learning_rate = learning_rate
00192         self.num_epochs = num_epochs
00193         self.device = device or torch.device('cuda' if torch.cuda.is_available() else 'cpu')
00194
00195         logger.info(" CORRECTED TRAINING PIPELINE INITIALIZED")
00196         logger.info(f"Target resolution: {target_resolution}")
00197         logger.info(f"Cube size: {cube_size}")
00198         logger.info(f"Energies: {energies}")
00199         logger.info(f"Device: {self.device}")
00200
00201     def create_corrected_transforms(self):
00202         """
00203         Create CORRECTED transforms using Resized instead of Spacingd.
00204         """
00205         transforms = Compose([
00206             LoadImaged(keys=["input", "target"], reader=NumpyReader),
00207             EnsureChannelFirstd(keys=["input", "target"]),

```

```

00208         EnsureTyped(keys=["input", "target"]),
00209         Orientationd(keys=["input", "target"], axcodes="RAS"),
00210
00211         # CRITICAL FIX: Use Resized instead of Spacingd for array dimensions
00212         Resized(keys=["input", "target"],
00213                 spatial_size=self.target_resolution,
00214                 mode=("bilinear", "nearest")),
00215
00216         SpatialPadd(keys=["input", "target"],
00217                     spatial_size=self.cube_size,
00218                     method="symmetric"),
00219
00220         CenterSpatialCropd(keys=["input", "target"],
00221                             roi_size=self.cube_size),
00222
00223         # Only normalize INPUT (CT), NOT target (dose)
00224         ScaleIntensityRangePercentilesd(
00225             keys="input", lower=0, upper=99.5, b_min=0, b_max=1
00226         ),
00227
00228         ToTensord(keys=["input", "target"]),
00229         EnsureTyped(keys=["energy"]),
00230         ToTensord(keys=["energy"])
00231     ])
00232
00233     logger.info(" CORRECTED transforms created (using Resized instead of Spacingd)")
00234     return transforms
00235
00236 def setup_unified_model(self, dose_normalization_params: Dict) -> Tuple:
00237     """
00238     Setup unified energy-conditioned model for all energies.
00239     """
00240     logger.info(" Setting up UNIFIED energy-conditioned model...")
00241
00242     # Model with energy conditioning (2 input channels: CT + Energy)
00243     autoencoder = AutoencoderKL(
00244         spatial_dims=3,
00245         in_channels=2, # CT + Energy channel
00246         out_channels=1, # Dose output
00247         num_channels=(32, 32, 32),
00248         latent_channels=2,
00249         num_res_blocks=1,
00250         norm_num_groups=8,
00251         attention_levels=(False, False, True),
00252     ).to(self.device)
00253
00254     discriminator = PatchDiscriminator(
00255         spatial_dims=3,
00256         num_layers_d=3,
00257         num_channels=32,
00258         in_channels=1, # Dose output
00259         out_channels=1
00260     ).to(self.device)
00261
00262     unet = DiffusionModelUNet(
00263         spatial_dims=3,
00264         in_channels=2, # Latent space
00265         out_channels=2,
00266         with_conditioning=False,
00267         num_res_blocks=1,
00268         num_channels=(32, 64, 64),
00269         attention_levels=(False, True, True),
00270         num_head_channels=(0, 64, 64),
00271     ).to(self.device)
00272
00273     scheduler = DDPMSScheduler(
00274         num_train_timesteps=1000,
00275         schedule="scaled_linear_beta",
00276         beta_start=0.0015,
00277         beta_end=0.0195,
00278     )
00279
00280     logger.info(" Unified models created")
00281     return autoencoder, discriminator, unet, scheduler
00282
00283 def run_corrected_training(self):
00284     """
00285     Execute the corrected training pipeline.
00286     """
00287     logger.info(" STARTING CORRECTED TRAINING PIPELINE")
00288     logger.info("=" * 60)
00289
00290     # Step 1: Create corrected dataset and get ORIGINAL dose statistics
00291     transforms = self.create_corrected_transforms()
00292     dataset = CorrectedDoseNpyDataset(
00293         root_dir=self.root_dir,
00294         transforms=transforms,

```

```

00295         target_resolution=self.target_resolution
00296     )
00297
00298     # Step 2: Get realistic dose normalization parameters
00299     dose_normalization_params = dataset.get_dose_normalization_params()
00300
00301     # Step 3: Create data loaders
00302     train_size = int(0.8 * len(dataset))
00303     val_size = len(dataset) - train_size
00304     train_dataset, val_dataset = torch.utils.data.random_split(dataset, [train_size, val_size])
00305
00306     train_loader = DataLoader(train_dataset, batch_size=self.batch_size, shuffle=True)
00307     val_loader = DataLoader(val_dataset, batch_size=self.batch_size, shuffle=False)
00308
00309     logger.info(f" Dataset split: {train_size} train, {val_size} validation")
00310
00311     # Step 4: Setup unified model
00312     autoencoder, discriminator, unet, scheduler = self.setup_unified_model(dose_normalization_params)
00313
00314     # Step 5: Calculate scaling factor with corrected model
00315     scale_factor = self.calculate_corrected_scaling_factor(autoencoder, train_loader)
00316
00317     # Step 6: Setup optimizers with conservative learning rates
00318     opt_g = Adam(autoencoder.parameters(), lr=self.learning_rate, weight_decay=1e-4)
00319     opt_d = Adam(discriminator.parameters(), lr=self.learning_rate, weight_decay=1e-4)
00320     opt_diff = Adam(unet.parameters(), lr=self.learning_rate, weight_decay=1e-4)
00321
00322     logger.info(f" Optimizers created with learning rate: {self.learning_rate}")
00323
00324     # Step 7: Run training
00325     trained_models = self.train_unified_model(
00326         autoencoder, discriminator, unet, scheduler,
00327         opt_g, opt_d, opt_diff,
00328         train_loader, val_loader,
00329         dose_normalization_params, scale_factor
00330     )
00331
00332     logger.info(" CORRECTED TRAINING COMPLETED SUCCESSFULLY")
00333     logger.info("=" * 60)
00334
00335     return trained_models
00336
00337 def calculate_corrected_scaling_factor(self, autoencoder, train_loader):
00338     """
00339     Calculate scaling factor with corrected energy conditioning.
00340     """
00341     logger.info("Calculating corrected scaling factor...")
00342
00343     with torch.no_grad():
00344         with autocast('cuda', enabled=True):
00345             first_batch = first(train_loader)
00346             images = first_batch["input"].to(self.device)
00347             energies = first_batch["energy"].to(self.device)
00348
00349             # Apply energy conditioning
00350             B, C, D, H, W = images.shape
00351             normalized_energy = energies.float() / 100.0 # Normalize energy
00352             energy_tensor = normalized_energy.view(B, 1, 1, 1, 1).expand(B, 1, D, H, W)
00353             conditioned_input = torch.cat([images, energy_tensor], dim=1)
00354
00355             # Encode to latents
00356             z = autoencoder.encode_stage_2_inputs(conditioned_input)
00357
00358             z_std = torch.std(z)
00359             if torch.isnan(z_std) or torch.isinf(z_std) or z_std.item() < 1e-8:
00360                 logger.warning(f"Invalid latent std ({z_std.item()}), using fallback scale_factor=1.0")
00361                 scale_factor = 1.0
00362             else:
00363                 scale_factor = 1 / z_std.item()
00364                 scale_factor = max(min(scale_factor, 10.0), 0.1) # Clamp to reasonable range
00365
00366             logger.info(f" Corrected scaling factor: {scale_factor}")
00367     return scale_factor
00368
00369 def train_unified_model(self, autoencoder, discriminator, unet, scheduler,
00370                         opt_g, opt_d, opt_diff, train_loader, val_loader,
00371                         dose_normalization_params, scale_factor):
00372     """
00373     Train the unified energy-conditioned model.
00374     """
00375     logger.info(" Training unified energy-conditioned model...")
00376
00377     # Create inferer with corrected scaling
00378     inferer = LatentDiffusionInferer(scheduler, scale_factor=scale_factor)
00379
00380     # Training loops would go here
00381     # For now, return the model setup

```

```

00382         trained_models = {
00383             'autoencoder': autoencoder,
00384             'discriminator': discriminator,
00385             'unet': unet,
00386             'scheduler': scheduler,
00387             'inferer': inferer,
00388             'dose_normalization_params': dose_normalization_params,
00389             'scale_factor': scale_factor,
00390             'target_resolution': self.target_resolution,
00391             'energies': self.energies
00392         }
00393
00394         logger.info(" Unified model training setup completed")
00395         return trained_models
00396
00397 def main():
00398     """
00399     Main function to run the corrected training pipeline.
00400     """
00401     import log_config
00402
00403     # Configuration
00404     ROOT_DIR = "/home/mpalm/RadioTherapy/traindata"
00405     ENERGIES = [3.47, 11.5, 15.75, 34.25, 46.53]
00406     TARGET_RESOLUTION = (64, 64, 64)
00407     CUBE_SIZE = (64, 64, 64)
00408
00409     logger.info(" STARTING CORRECTED TRAINING PIPELINE - NACHHALTIGE LÖSUNG")
00410
00411     # Create pipeline
00412     pipeline = CorrectedTrainingPipeline(
00413         root_dir=ROOT_DIR,
00414         energies=ENERGIES,
00415         target_resolution=TARGET_RESOLUTION,
00416         cube_size=CUBE_SIZE,
00417         batch_size=2,
00418         learning_rate=1e-5,
00419         num_epochs=50
00420     )
00421
00422     # Run corrected training
00423     trained_models = pipeline.run_corrected_training()
00424
00425     logger.info(" CORRECTED TRAINING PIPELINE COMPLETED - SUSTAINABLE SOLUTION IMPLEMENTED")
00426     return trained_models
00427
00428 if __name__ == "__main__":
00429     main()

```

8.6 sourcecode/data_management.py File Reference

Classes

- class [data_management.HotspotPatchDataset](#)
- class [data_management.DoseNpyDataset](#)
- class [data_management.DataLoaderModule](#)

Namespaces

- namespace [data_management](#)

Variables

- [data_management.logger](#) = logging.getLogger(__name__)

8.7 data_management.py

Go to the documentation of this file.

```

00001 # Import Dataset before using it
00002 import os
00003 import numpy as np
00004 import torch
00005 from torch.utils.data import Dataset
00006 from hotspot_patch_sampler import extract_hotspot_patches
00007 # --- HotspotPatchDataset: Patch-basierte Trainingsdaten für Hotspot-Lernen ---
00008 class HotspotPatchDataset(Dataset):

```

```

00009     """
00010     Dataset, das aus jedem Input/Target-Cube Hotspot-Patches extrahiert und als einzelne Trainingssamples
    zurückgibt.
00011     Nutzt extract_hotspot_patches(volume, ...), um Patch-Startpositionen zu bestimmen.
00012     Jeder __getitem__ gibt einen Patch (Input, Target, Energy) zurück.
00013     """
00014     def __init__(self, root_dir, section=None, patch_size=(32,32,32), min_hotspot_voxels=1000, dose_threshold=0.5,
    max_patches=16, random_patches=0, transforms=None, energy=None, max_patches_per_energy=80):
00015         import logging
00016         logger = logging.getLogger(__name__)
00017
00018         base_dir = root_dir if section is None else os.path.join(root_dir, section)
00019         logger.info(f"HotspotPatchDataset: base_dir={base_dir}, section={section}, energy={energy}")
00020
00021         # Check if base_dir exists
00022         if not os.path.exists(base_dir):
00023             logger.error(f"HotspotPatchDataset: base_dir does not exist: {base_dir}")
00024
00025         self.patch_size = patch_size
00026         self.transforms = transforms
00027         self.patch_records = [] # Liste: (in_fp, out_fp, energy, (z,y,x))
00028         self.selected_energy = energy
00029         self.patch_stats = [] # Liste: (n_voxels_gt0, n_voxels_gt_thresh, max_dose, mean_dose)
00030         self.max_patches_per_energy = max_patches_per_energy
00031
00032         # List directories in base_dir for debugging
00033         try:
00034             energy_folders = sorted(os.listdir(base_dir))
00035             logger.info(f"HotspotPatchDataset: Found directories in {base_dir}: {energy_folders}")
00036         except Exception as e:
00037             logger.error(f"HotspotPatchDataset: Error listing {base_dir}: {e}")
00038             energy_folders = []
00039
00040         for energy_folder in energy_folders:
00041             folder_path = os.path.join(base_dir, energy_folder)
00042             logger.info(f"HotspotPatchDataset: Checking energy_folder={energy_folder}, folder_path={folder_path}")
00043
00044             if not os.path.isdir(folder_path) or energy_folder.startswith('.'):
00045                 logger.info(f"HotspotPatchDataset: Skipping {energy_folder} (not directory or hidden)")
00046                 continue
00047             try:
00048                 folder_energy = float(energy_folder.replace("_", "."))
00049                 logger.info(f"HotspotPatchDataset: Parsed energy={folder_energy} from folder={energy_folder}")
00050             except ValueError as e:
00051                 logger.info(f"HotspotPatchDataset: Skipping {energy_folder} (can't parse energy): {e}")
00052                 continue
00053             # Filter: Nur gewünschte Energie verwenden, falls gesetzt
00054             if self.selected_energy is not None and abs(folder_energy - self.selected_energy) > 0.01:
00055                 logger.info(f"HotspotPatchDataset: Skipping energy {folder_energy}
    (selected_energy={self.selected_energy})")
00056                 continue
00057             in_dir = os.path.join(folder_path, "inputcube")
00058             out_dir = os.path.join(folder_path, "outputcube")
00059             logger.info(f"HotspotPatchDataset: Checking in_dir={in_dir}, out_dir={out_dir}")
00060
00061             if not os.path.isdir(in_dir) or not os.path.isdir(out_dir):
00062                 logger.warning(f"HotspotPatchDataset: Missing inputcube/outputcube in {folder_path}")
00063                 logger.info(f"    in_dir exists: {os.path.exists(in_dir)}, out_dir exists: {os.path.exists(out_dir)}")
00064                 continue
00065
00066             # List files in directories for debugging
00067             try:
00068                 input_files = os.listdir(in_dir)
00069                 output_files = os.listdir(out_dir)
00070                 logger.info(f"HotspotPatchDataset: Found {len(input_files)} input files, {len(output_files)} output
    files")
00071             except Exception as e:
00072                 logger.error(f"HotspotPatchDataset: Error listing files: {e}")
00073                 continue
00074             for fname in sorted(os.listdir(in_dir)):
00075                 if not fname.endswith(".npy"):
00076                     continue
00077                 in_fp = os.path.join(in_dir, fname)
00078                 out_fp = os.path.join(out_dir, fname)
00079                 logger.info(f"HotspotPatchDataset: Processing file {fname}")
00080
00081                 # Check if output file exists
00082                 if not os.path.exists(out_fp):
00083                     logger.warning(f"HotspotPatchDataset: Output file missing: {out_fp}")
00084                     continue
00085
00086                 # Lade Target-Dosis-Array, um Hotspot-Patches zu bestimmen
00087                 try:
00088                     dose_array = np.load(out_fp)
00089                     logger.info(f"HotspotPatchDataset: Loaded dose array shape: {dose_array.shape}, range:
    {dose_array.min():.6f} to {dose_array.max():.6f}")
00090

```



```

00091         # CRITICAL FIX: Adjust dose_threshold based on actual data range
00092         max_dose = dose_array.max()
00093         if max_dose > 0:
00094             # Use 10% of max dose as threshold instead of fixed 0.5
00095             adaptive_threshold = max(dose_threshold * max_dose, max_dose * 0.1)
00096             logger.info(f"HotspotPatchDataset: Adaptive dose threshold: {adaptive_threshold:.6f}")
00097         (max_dose: {max_dose:.6f})")
00098         else:
00099             adaptive_threshold = dose_threshold
00099             logger.warning(f"HotspotPatchDataset: Max dose is 0, using default threshold:
00100 {adaptive_threshold}")
00101     except Exception as e:
00102         logger.warning(f"Fehler beim Laden von {out_fp}: {e}")
00103         continue
00104
00105     patch_indices = extract_hotspot_patches(
00106         dose_array,
00107         patch_size=patch_size,
00108         min_hotspot_voxels=min_hotspot_voxels,
00109         dose_threshold=adaptive_threshold, # Use adaptive threshold
00110         max_patches=max_patches,
00111         random_patches=random_patches
00112     )
00113
00114     logger.info(f"HotspotPatchDataset: Found {len(patch_indices)} patches for file {fname}
00115 (adaptive_threshold={adaptive_threshold:.6f})")
00116
00117     # **PATCH BALANCING: Limit patches per energy to prevent imbalance**
00118     if len(patch_indices) > self.max_patches_per_energy:
00119         logger.info(f"Energy {folder_energy:.2f}: Limiting patches from {len(patch_indices)} to
00120 {self.max_patches_per_energy}")
00121         # Random selection for balanced distribution
00122         import random
00123         patch_indices = random.sample(patch_indices, self.max_patches_per_energy)
00124
00125     for idx in patch_indices:
00126         # patch statistics
00127         z, y, x = idx
00128         dz, dy, dx = patch_size
00129         patch = dose_array[z:z+dz, y:y+dy, x:x+dx]
00130         n_voxels_gt0 = int((patch > 0).sum())
00131         n_voxels_gt_thresh = int((patch > dose_threshold).sum())
00132         max_dose = float(patch.max())
00133         mean_dose = float(patch.mean())
00134         self.patch_records.append((in_fp, out_fp, folder_energy, idx))
00135         self.patch_stats.append((n_voxels_gt0, n_voxels_gt_thresh, max_dose, mean_dose))
00136
00137     # Logging
00138     if len(self.patch_stats) > 0:
00139         n_voxels_gt0_list = [s[0] for s in self.patch_stats]
00140         n_voxels_gt_thresh_list = [s[1] for s in self.patch_stats]
00141         max_dose_list = [s[2] for s in self.patch_stats]
00142         mean_dose_list = [s[3] for s in self.patch_stats]
00143         logger.info(f"Patch-Statistiken: Mittelwert Voxel>0: {np.mean(n_voxels_gt0_list):.1f}, Mittelwert
00144 Voxel>thresh: {np.mean(n_voxels_gt_thresh_list):.1f}, MaxDose Mittelwert: {np.mean(max_dose_list):.3f}, MeanDose
00145 Mittelwert: {np.mean(mean_dose_list):.3f}")
00146         logger.info(f"Patch-Statistiken: Min/Max Voxel>0:
00147 {np.min(n_voxels_gt0_list)}/{np.max(n_voxels_gt0_list)}, Min/Max Voxel>thresh:
00148 {np.min(n_voxels_gt_thresh_list)}/{np.max(n_voxels_gt_thresh_list)}")
00149         logger.info(f"HotspotPatchDataset: {len(self.patch_records)} Patches extrahiert.")
00150         # Debug
00151         if len(self.patch_records) > 0:
00152             logger.debug(f"Energie der ersten 5 Patches: {[self.patch_records[i][2] for i in range(min(5,
00153 len(self.patch_records)))}")
00154
00155     def __len__(self):
00156         return len(self.patch_records)
00157
00158     def __getitem__(self, idx):
00159         in_fp, out_fp, energy, (z, y, x) = self.patch_records[idx]
00160         arr_in = np.load(in_fp) # [D,H,W]
00161         arr_out = np.load(out_fp)
00162         dz, dy, dx = self.patch_size
00163         patch_in = arr_in[z:z+dz, y:y+dy, x:x+dx]
00164         patch_out = arr_out[z:z+dz, y:y+dy, x:x+dx]
00165         sample = {
00166             "input": torch.from_numpy(patch_in)[None].float(), # [1,D,H,W]
00167             "target": torch.from_numpy(patch_out)[None].float(), # [1,D,H,W]
00168             "energy": torch.tensor([energy], dtype=torch.float32),
00169         }
00170         if self.transforms:
00171             sample = self.transforms(sample)
00172         return sample
00173
00174 #data_management.py
00175
00176 import os
00177 import torch

```



```

00169 from monai.data import DataLoader
00170 try:
00171     from monai.apps.datasets import CustomDataset
00172 except ImportError:
00173     # Fallback for newer MONAI versions where CustomDataset moved
00174     from monai.data import Dataset as CustomDataset
00175 from torch.utils.data import random_split
00176 import glob
00177 import numpy as np
00178 from torch.utils.data import Dataset
00179 from monai.transforms import Compose, LoadImaged
00180 from monai.data import NumpyReader
00181 import log_config
00182 import logging
00183
00184 logger = logging.getLogger(__name__)
00185
00186 class DoseNpyDataset(Dataset):
00187     """
00188     Dataset that loads input/output .npy cubes and parses the energy level from the folder name.
00189     Expects directory structure:
00190         root_dir/
00191             <section>/          # e.g. "training"
00192                 <energy_folder>/ # e.g. "11_5"
00193                     inputcube/    # contains input .npy files
00194                     outputcube/   # contains target .npy files
00195     """
00196     def __init__(self, root_dir, section=None, transforms=None):
00197         base_dir = root_dir if section is None else os.path.join(root_dir, section)
00198         self.samples = []
00199         for energy_folder in sorted(os.listdir(base_dir)):
00200             folder_path = os.path.join(base_dir, energy_folder)
00201             # Skip non-directories and hidden entries
00202             if not os.path.isdir(folder_path) or energy_folder.startswith('.'):
00203                 continue
00204             # Parse energy value from folder name
00205             try:
00206                 energy = float(energy_folder.replace("_", "."))
00207             except ValueError:
00208                 continue
00209             in_dir = os.path.join(folder_path, "inputcube")
00210             out_dir = os.path.join(folder_path, "outputcube")
00211             # Skip if expected subdirectories do not exist
00212             if not os.path.isdir(in_dir) or not os.path.isdir(out_dir):
00213                 continue
00214             for fname in sorted(os.listdir(in_dir)):
00215                 if not fname.endswith(".npy"):
00216                     continue
00217                 in_fp = os.path.join(in_dir, fname)
00218                 out_fp = os.path.join(out_dir, fname)
00219
00220                 # **CRITICAL FIX: Validate sample quality before adding**
00221                 if self.validate_sample_quality(out_fp):
00222                     self.samples.append((in_fp, out_fp, energy))
00223                 else:
00224                     logger.debug(f"Rejected sample {fname} (insufficient dose)")
00225
00226         self.transforms = transforms
00227         logger.info(f"Energy levels found: {sorted(set([s[2] for s in self.samples]))}")
00228         logger.info(f"Total valid samples: {len(self.samples)}")
00229
00230     def validate_sample_quality(self, dose_path, min_dose_voxels=20, min_max_dose=0.01):
00231         """
00232         Check if a dose sample is suitable for training.
00233
00234         Args:
00235             dose_path: Path to dose .npy file
00236             min_dose_voxels: Minimum number of dose voxels required
00237             min_max_dose: Minimum maximum dose value required
00238
00239         Returns:
00240             bool: True if sample is valid for training
00241         """
00242         try:
00243             dose_array = np.load(dose_path)
00244
00245             # Count dose voxels
00246             dose_mask = dose_array > 1e-6
00247             dose_voxel_count = dose_mask.sum()
00248             max_dose = dose_array.max()
00249
00250             # Check quality criteria
00251             is_valid = (dose_voxel_count >= min_dose_voxels) and (max_dose >= min_max_dose)
00252
00253             if not is_valid:
00254                 logger.debug(f"Sample validation failed: dose_voxels={dose_voxel_count} (min={min_dose_voxels}), "
00255                             f"max_dose={max_dose:.6f} (min={min_max_dose})")

```

```

00256         return is_valid
00257
00258     except Exception as e:
00259         logger.warning(f"Failed to validate sample {dose_path}: {e}")
00260         return False
00261     self.transforms = transforms
00262     logger.info(f"Energy levels found: {sorted(set([s[2] for s in self.samples]))}")
00263
00264
00265     def __len__(self):
00266         return len(self.samples)
00267
00268     def __getitem__(self, idx):
00269         in_fp, out_fp, energy = self.samples[idx]
00270         # Determine if LoadImaged is expected by checking transforms
00271         use_load_imaged = False
00272         if self.transforms and isinstance(self.transforms, Compose):
00273             from monai.transforms import LoadImaged
00274             use_load_imaged = any(isinstance(tr, LoadImaged) for tr in self.transforms.transforms)
00275
00276         if use_load_imaged:
00277             # Return file paths for MONAI LoadImaged
00278             sample = {
00279                 "input": in_fp,
00280                 "target": out_fp,
00281                 "energy": torch.tensor([energy], dtype=torch.float32),
00282             }
00283         else:
00284             # Load arrays directly for tensor-based pipeline
00285             arr_in = np.load(in_fp) # shape [D, H, W]
00286             arr_out = np.load(out_fp)
00287             sample = {
00288                 "input": torch.from_numpy(arr_in)[None].float(), # [1,D,H,W]
00289                 "target": torch.from_numpy(arr_out)[None].float(), # [1,D,H,W]
00290                 "energy": torch.tensor([energy], dtype=torch.float32),
00291             }
00292         # Now apply transforms (e.g., EnsureChannelFirstd, spacing) which act on keys
00293         if self.transforms:
00294             sample = self.transforms(sample)
00295         return sample
00296
00297     class DataLoaderModule:
00298         """
00299         This module takes care of loading, splitting and creating DataLoaders
00300         for training and validation data from a CustomDataset.
00301         """
00302         def __init__(self, root_dir, transforms, train_ratio=0.8, seed=42):
00303             """
00304             Args:
00305                 root_dir (str): The root directory where the data is located.
00306                 transforms (monai.transforms.Compose): Transforms that are applied to the data.
00307                 energy (optional, int or float): Filter to load only samples of a certain energy.
00308                 train_ratio (float): Ratio of the data used for training.
00309                 seed (int): Seed for the reproducibility of the split.
00310             """
00311             self.root_dir = root_dir
00312             self.transforms = transforms
00313             self.train_ratio = train_ratio
00314             self.seed = seed
00315
00316         def load_dataset(self, section="training"):
00317             """
00318             Loads the complete dataset as a CustomDataset.
00319
00320             Args:
00321                 section (str): Specifies which part of the dataset is to be loaded (e.g. "training").
00322
00323             Returns:
00324                 CustomDataset: The loaded dataset.
00325             """
00326             # Pass through full transform pipeline (LoadImaged remains for file loading)
00327             return DoseNpyDataset(self.root_dir, section=section, transforms=self.transforms)
00328
00329         def split_dataset(self, ds_full):
00330             """
00331             Splits the complete dataset into training and validation sets.
00332
00333             Args:
00334                 ds_full (Dataset): The complete dataset.
00335
00336             Returns:
00337                 tuple: (train_ds, val_ds)
00338             """
00339             dataset_size = len(ds_full)
00340             train_size = int(dataset_size * self.train_ratio)
00341             val_size = dataset_size - train_size
00342             train_ds, val_ds = random_split(

```

```

00343         ds_full,
00344         [train_size, val_size],
00345         generator=torch.Generator().manual_seed(self.seed)
00346     )
00347     return train_ds, val_ds
00348
00349     def custom_collate(self, batch):
00350         """
00351         Custom collate function that checks and collates 'input', 'target', and 'energy' if available.
00352         """
00353         for i, sample in enumerate(batch):
00354             if "input" not in sample or "target" not in sample:
00355                 logger.warning(f"Sample {i} is missing required keys 'input' or 'target'.")
00356             # Use the default collate function to combine the dictionaries, which will combine
00357             # any common keys (including 'energy', if present)
00358             return torch.utils.data.dataloader.default_collate(batch)
00359
00360     def create_data_loader(self, dataset, batch_size, shuffle, num_workers=0):
00361         """
00362         Creates a DataLoader from the given dataset.
00363
00364         Args:
00365             dataset (Dataset): The dataset from which data is to be loaded.
00366             batch_size (int): Number of samples per batch.
00367             shuffle (bool): Whether the data should be shuffled before each run.
00368             num_workers (int): Number of parallel processes for loading the data.
00369
00370         Returns:
00371             DataLoader: The DataLoader created.
00372         """
00373         return DataLoader(
00374             dataset,
00375             batch_size=batch_size,
00376             shuffle=shuffle,
00377             num_workers=num_workers,
00378             persistent_workers=False
00379         )
00380

```

8.8 sourcecode/gaussian_quadrature_inference.py File Reference

Namespaces

- namespace [gaussian_quadrature_inference](#)

Functions

- [gaussian_quadrature_inference.setup_logging \(\)](#)
- [gaussian_quadrature_inference.load_combined_checkpoint \(str checkpoint_path\)](#)
- [gaussian_quadrature_inference.load_ct \(Path path\)](#)
- [gaussian_quadrature_inference.extract_energy_models \(dict models_by_energy, list target_energies\)](#)
- [gaussian_quadrature_inference.run_gaussian_quadrature_inference \(ct_array, checkpoint_path, quad_energies, quad_weights, device='cpu', cube_size=\(64, 64, 64\)\)](#)
- [gaussian_quadrature_inference.save_dose \(Path path, np.ndarray dose, affine=None\)](#)
- [gaussian_quadrature_inference.create_visualization \(integrated_dose, energy_results, output_dir\)](#)
- [gaussian_quadrature_inference.main \(\)](#)

Variables

- bool [gaussian_quadrature_inference.has_nibabel](#) = True

8.9 gaussian_quadrature_inference.py

[Go to the documentation of this file.](#)

```

00001 #!/usr/bin/env python3
00002 """
00003 Gaussian Quadrature Inference for Radiotherapie Dose Prediction.
00004 """
00005
00006 import argparse
00007 import logging
00008 from pathlib import Path

```

```

00009 import numpy as np
00010 import torch
00011 import matplotlib.pyplot as plt
00012 import os
00013 import sys
00014
00015 # Add sourcecode to path
00016 sys.path.append('sourcecode')
00017
00018 from inference_module import InferenceModule
00019
00020 # Optional nibabel support for NIfTI files
00021 try:
00022     import nibabel as nib
00023     has_nibabel = True
00024 except ImportError:
00025     has_nibabel = False
00026
00027
00028 def setup_logging():
00029     """Setup logging configuration."""
00030     logging.basicConfig(
00031         level=logging.INFO,
00032         format='%(asctime)s - %(levelname)s - %(name)s - %(message)s'
00033     )
00034     return logging.getLogger(__name__)
00035
00036
00037 def load_combined_checkpoint(checkpoint_path: str):
00038     """Load combined checkpoint with all energy models."""
00039     logger = logging.getLogger(__name__)
00040     logger.info(f"Loading combined checkpoint from: {checkpoint_path}")
00041
00042     try:
00043         checkpoint = torch.load(checkpoint_path, map_location='cpu')
00044         logger.info(f"Checkpoint loaded successfully")
00045
00046         if 'models_by_energy' in checkpoint:
00047             models_by_energy = checkpoint['models_by_energy']
00048             logger.info(f"Found {len(models_by_energy)} energy-specific models")
00049             for key in models_by_energy.keys():
00050                 logger.info(f"    - {key}")
00051
00052         if 'dose_normalization_params' in checkpoint:
00053             dose_params = checkpoint['dose_normalization_params']
00054             logger.info(f"Found dose normalization parameters for {len(dose_params)} energy/resolution
combinations")
00055             for key, params in dose_params.items():
00056                 logger.info(f"    {key}: clip_min={params.get('clip_min', 0.0):.6f}, clip_max={params.get('clip_max',
1.0):.6f}")
00057             else:
00058                 logger.warning("No dose normalization parameters found in checkpoint")
00059                 dose_params = {}
00060
00061             return checkpoint, models_by_energy, dose_params
00062
00063     except Exception as e:
00064         logger.error(f"Failed to load checkpoint: {e}")
00065         raise
00066
00067
00068 def load_ct(path: Path):
00069     """Load CT data from .npy or .nii/.nii.gz file."""
00070     logger = logging.getLogger(__name__)
00071     suffix = path.suffix.lower()
00072
00073     if suffix in ['.nii', '.gz'] and has_nibabel:
00074         nii = nib.load(str(path))
00075         arr = nii.get_fdata().astype(np.float32)
00076         affine = nii.affine
00077         logger.info(f"Loaded NIfTI: shape={arr.shape}")
00078     else:
00079         arr = np.load(str(path)).astype(np.float32)
00080         affine = None
00081         logger.info(f"Loaded NPY: shape={arr.shape}")
00082
00083     logger.info(f"CT range: {arr.min():.2f} to {arr.max():.2f}")
00084     return arr, affine
00085
00086
00087 def extract_energy_models(models_by_energy: dict, target_energies: list):
00088     """Extract models for specific energies."""
00089     logger = logging.getLogger(__name__)
00090     extracted_models = {}
00091     available_energies = []
00092
00093     for key, model_data in models_by_energy.items():

```

```

00094         try:
00095             # Parse energy from key (e.g., "res64x64x64_e46.53")
00096             if '_e' in key:
00097                 energy_str = key.split('_e')[1]
00098                 energy = float(energy_str)
00099                 available_energies.append(energy)
00100
00101             # Check if this energy is in our target list
00102             for target_energy in target_energies:
00103                 if abs(energy - target_energy) < 0.01: # Close enough
00104                     extracted_models[target_energy] = {
00105                         'key': key,
00106                         'energy': energy,
00107                         'models': model_data
00108                     }
00109                     logger.info(f" Found model for energy {target_energy:.4f} keV (key: {key})")
00110                     break
00111         except Exception as e:
00112             logger.warning(f"Could not parse energy from key {key}: {e}")
00113
00114     logger.info(f"Available energies: {sorted(available_energies)}")
00115     logger.info(f"Extracted models for {len(extracted_models)} target energies")
00116
00117     missing_energies = [e for e in target_energies if e not in extracted_models]
00118     if missing_energies:
00119         logger.error(f"Missing models for energies: {missing_energies}")
00120         raise ValueError(f"Models not found for energies: {missing_energies}")
00121
00122     return extracted_models
00123
00124
00125 def run_gaussian_quadrature_inference(ct_array, checkpoint_path, quad_energies, quad_weights,
00126                                     device='cpu', cube_size=(64, 64, 64)):
00127     """
00128     Run 4-point Gaussian Quadrature Inference.
00129
00130     Args:
00131         ct_array: Input CT array
00132         checkpoint_path: Path to combined checkpoint
00133         quad_energies: List of 4 quadrature energies [x1, x2, x3, x4]
00134         quad_weights: List of 4 quadrature weights [w1, w2, w3, w4]
00135         device: PyTorch device
00136         cube_size: Target cube size
00137
00138     Returns:
00139         integrated_dose: Final integrated dose distribution
00140     """
00141     logger = logging.getLogger(__name__)
00142
00143     logger.info("=" * 60)
00144     logger.info("GAUSSIAN QUADRATURE INFERENCE")
00145     logger.info("=" * 60)
00146     logger.info(f"Quadrature energies: {quad_energies}")
00147     logger.info(f"Quadrature weights: {quad_weights}")
00148     logger.info(f"Device: {device}")
00149     logger.info(f"Target cube size: {cube_size}")
00150
00151     # Load checkpoint
00152     checkpoint, models_by_energy, dose_params = load_combined_checkpoint(checkpoint_path)
00153
00154     # Extract models for quadrature energies
00155     extracted_models = extract_energy_models(models_by_energy, quad_energies)
00156
00157     # Initialize results storage
00158     energy_results = {}
00159
00160     # Run inference for each quadrature energy
00161     for i, (energy, weight) in enumerate(zip(quad_energies, quad_weights)):
00162         logger.info("-" * 40)
00163         logger.info(f"QUADRATURE POINT {i+1}/4: Energy {energy:.4f} keV, Weight {weight:.6f}")
00164         logger.info("-" * 40)
00165
00166         model_info = extracted_models[energy]
00167         model_key = model_info['key']
00168
00169         # Create energy-specific models dict
00170         energy_models = {energy: extracted_models[energy]['models']}
00171
00172         # Get scale factor
00173         scale_factor = extracted_models[energy]['models'].get('scale_factor', 1.0)
00174         logger.info(f"Using scale factor: {scale_factor}")
00175
00176         # Create InferenceModule for this energy
00177         inference_module = InferenceModule(
00178             models_by_energy=energy_models,
00179             energies=[energy],
00180             energy_weights=[1.0], # Single energy, so weight=1

```

```

00181         device=device,
00182         scale_factor=scale_factor,
00183         dose_normalization_params=dose_params
00184     )
00185
00186     # Run inference
00187     logger.info(f"Running inference for energy {energy:.4f} keV...")
00188     dose_result = inference_module.run_inference_conditioned_on_energy(
00189         ct_tensor=torch.from_numpy(ct_array).unsqueeze(0), # Add batch dimension
00190         energy_value=energy
00191     )
00192
00193     # Store result
00194     if isinstance(dose_result, torch.Tensor):
00195         dose_np = dose_result.cpu().numpy()
00196     else:
00197         dose_np = np.array(dose_result, dtype=np.float32)
00198
00199     # Remove batch dimension if present
00200     if dose_np.ndim == 5 and dose_np.shape[0] == 1:
00201         dose_np = dose_np[0, 0] # Remove batch and channel dims
00202     elif dose_np.ndim == 4 and dose_np.shape[0] == 1:
00203         dose_np = dose_np[0] # Remove batch dim
00204
00205     energy_results[energy] = {
00206         'dose': dose_np,
00207         'weight': weight
00208     }
00209
00210     logger.info(f" Energy {energy:.4f}: dose shape={dose_np.shape}, "
00211               f"range=[{dose_np.min():.6f}, {dose_np.max():.6f}], weight={weight:.6f}")
00212
00213     # Gaussian Quadrature Integration
00214     logger.info("=" * 40)
00215     logger.info("GAUSSIAN QUADRATURE INTEGRATION")
00216     logger.info("=" * 40)
00217
00218     integrated_dose = None
00219     total_weight = 0.0
00220
00221     for energy, weight in zip(quad_energies, quad_weights):
00222         dose = energy_results[energy]['dose']
00223
00224         if integrated_dose is None:
00225             integrated_dose = weight * dose
00226         else:
00227             integrated_dose += weight * dose
00228
00229         total_weight += weight
00230         logger.info(f"Added energy {energy:.4f} with weight {weight:.6f}")
00231
00232     logger.info(f"Total weight sum: {total_weight:.6f}")
00233     logger.info(f"Final integrated dose: shape={integrated_dose.shape}, "
00234               f"range=[{integrated_dose.min():.6f}, {integrated_dose.max():.6f}])")
00235
00236     # Normalize by total weight (should be close to 1 for proper quadrature)
00237     if abs(total_weight - 1.0) > 0.001:
00238         logger.warning(f"Total weight {total_weight:.6f} != 1.0, normalizing...")
00239         integrated_dose /= total_weight
00240
00241     return integrated_dose, energy_results
00242
00243
00244 def save_dose(path: Path, dose: np.ndarray, affine=None):
00245     """Save dose array to file."""
00246     logger = logging.getLogger(__name__)
00247     suffix = path.suffix.lower()
00248
00249     if suffix in ['.nii', '.gz'] and affine is not None and has_nibabel:
00250         out_nii = nib.Nifti1Image(dose, affine)
00251         out_nii.to_filename(str(path))
00252         logger.info(f" Saved NIfTI: {path}")
00253     else:
00254         np.save(str(path), dose)
00255         logger.info(f" Saved NPY: {path}")
00256
00257
00258 def create_visualization(integrated_dose, energy_results, output_dir):
00259     """Create visualization of quadrature integration results."""
00260     logger = logging.getLogger(__name__)
00261
00262     fig, axes = plt.subplots(2, 3, figsize=(18, 12))
00263     fig.suptitle('Gaussian Quadrature Dose Integration', fontsize=16)
00264
00265     # Individual energy results
00266     for i, (energy, data) in enumerate(energy_results.items()):
00267         if i >= 4: # Only show first 4

```

```

00268         break
00269         row = i // 2
00270         col = i % 2
00271
00272         dose = data['dose']
00273         weight = data['weight']
00274
00275         # Show middle slice
00276         mid_slice = dose.shape[2] // 2
00277         slice_data = dose[:, :, mid_slice]
00278
00279         im = axes[row, col].imshow(slice_data, cmap='hot', origin='lower')
00280         axes[row, col].set_title(f'Energy {energy:.2f} keV\nWeight: {weight:.4f}')
00281         axes[row, col].axis('off')
00282         plt.colorbar(im, ax=axes[row, col], shrink=0.8)
00283
00284         # Integrated result
00285         mid_slice = integrated_dose.shape[2] // 2
00286         integrated_slice = integrated_dose[:, :, mid_slice]
00287
00288         im = axes[0, 2].imshow(integrated_slice, cmap='hot', origin='lower')
00289         axes[0, 2].set_title('Integrated Dose\n(Gaussian Quadrature)')
00290         axes[0, 2].axis('off')
00291         plt.colorbar(im, ax=axes[0, 2], shrink=0.8)
00292
00293         # Statistics
00294         axes[1, 2].axis('off')
00295         stats_text = f"""\
00296         Quadrature Integration Stats:
00297
00298         Final dose range: [{integrated_dose.min():.3f}, {integrated_dose.max():.3f}]
00299         Final dose mean: {integrated_dose.mean():.3f}
00300         Final dose std: {integrated_dose.std():.3f}
00301
00302         Individual contributions:"""
00303
00304         for energy, data in energy_results.items():
00305             dose = data['dose']
00306             weight = data['weight']
00307             contribution = (weight * dose).max()
00308             stats_text += f"\n      E={energy:.2f}: max_contrib={contribution:.3f}"
00309
00310         axes[1, 2].text(0.1, 0.9, stats_text, transform=axes[1, 2].transAxes,
00311                        fontsize=10, verticalalignment='top', fontfamily='monospace')
00312
00313         plt.tight_layout()
00314         viz_path = os.path.join(output_dir, 'gaussian_quadrature_results.png')
00315         plt.savefig(viz_path, dpi=300, bbox_inches='tight')
00316         plt.close()
00317         logger.info(f" Visualization saved: {viz_path}")
00318
00319 def main():
00320     """Main function."""
00321     parser = argparse.ArgumentParser(
00322         description="Gaussian Quadrature Inference for Radiotherapy Dose Prediction"
00323     )
00324     parser.add_argument('-m', '--model', required=True,
00325                         help='Path to combined model checkpoint')
00326     parser.add_argument('-i', '--input', required=True,
00327                         help='Input CT file (.npy or .nii/.nii.gz)')
00328     parser.add_argument('-o', '--output', required=True,
00329                         help='Output dose file (.npy or .nii/.nii.gz)')
00330     parser.add_argument('--energies', nargs=4, type=float,
00331                         default=[3.4716, 15.7505, 34.2495, 46.5284],
00332                         help='4 Gaussian quadrature energies')
00333     parser.add_argument('--weights', nargs=4, type=float,
00334                         default=[8.6964, 16.3036, 16.3036, 8.6964],
00335                         help='4 Gaussian quadrature weights (will be normalized)')
00336     parser.add_argument('-d', '--device', default='cpu',
00337                         help='PyTorch device (cpu or cuda)')
00338     parser.add_argument('--cube-size', nargs=3, type=int, default=[64, 64, 64],
00339                         help='Target cube size [D, H, W]')
00340     parser.add_argument('--visualize', action='store_true',
00341                         help='Create visualization of results')
00342
00343     args = parser.parse_args()
00344
00345     logger = setup_logging()
00346
00347     # Normalize weights to sum to 1
00348     weights = np.array(args.weights)
00349     weights = weights / weights.sum()
00350
00351     logger.info("=" * 60)
00352     logger.info("GAUSSIAN QUADRATURE INFERENCE PIPELINE")
00353     logger.info("=" * 60)
00354     logger.info(f"Model: {args.model}")

```

```

00355     logger.info(f"Input: {args.input}")
00356     logger.info(f"Output: {args.output}")
00357     logger.info(f"Energies: {args.energies}")
00358     logger.info(f"Weights (normalized): {weights.tolist()}")
00359     logger.info(f"Device: {args.device}")
00360
00361     # Load CT
00362     input_path = Path(args.input)
00363     ct_array, affine = load_ct(input_path)
00364
00365     # Run Gaussian Quadrature Inference
00366     integrated_dose, energy_results = run_gaussian_quadrature_inference(
00367         ct_array=ct_array,
00368         checkpoint_path=args.model,
00369         quad_energies=args.energies,
00370         quad_weights=weights.tolist(),
00371         device=args.device,
00372         cube_size=tuple(args.cube_size)
00373     )
00374
00375     # Save result
00376     output_path = Path(args.output)
00377     save_dose(output_path, integrated_dose, affine)
00378
00379     # Create visualization if requested
00380     if args.visualize:
00381         output_dir = output_path.parent
00382         create_visualization(integrated_dose, energy_results, output_dir)
00383
00384     logger.info("=" * 60)
00385     logger.info("INFERENCE COMPLETED SUCCESSFULLY")
00386     logger.info("=" * 60)
00387     logger.info(f"Final integrated dose saved to: {output_path}")
00388     logger.info(f"Dose statistics:")
00389     logger.info(f"  Shape: {integrated_dose.shape}")
00390     logger.info(f"  Range: [{integrated_dose.min():.6f}, {integrated_dose.max():.6f}]")
00391     logger.info(f"  Mean: {integrated_dose.mean():.6f}")
00392     logger.info(f"  Std: {integrated_dose.std():.6f}")
00393
00394
00395 if __name__ == '__main__':
00396     main()

```

8.10 sourcecode/gui.py File Reference

Classes

- class [gui.TrainingWorker](#)
- class [gui.InferenceWorker](#)
- class [gui.MainWindow](#)

Namespaces

- namespace [gui](#)

Functions

- [gui.handle_exception](#) (exc_type, exc_value, exc_tb)

Variables

- [gui.logger](#) = logging.getLogger(__name__)
- [gui.root_logger](#) = logging.getLogger()
- [gui.console_handler](#) = logging.StreamHandler()
- [gui.formatter](#) = logging.Formatter('%(asctime)s %(levelname)s: %(name)s: %(message)s')
- [gui.excepthook](#)
- [gui.app](#) = QApplication(sys.argv)
- [gui.window](#) = [MainWindow](#)()

8.11 gui.py

[Go to the documentation of this file.](#)

```

00001 # Add glob import for file searching
00002 import os
00003 import sys
00004 import traceback
00005 import glob
00006 import logging
00007
00008 # Import log_config first to set up the base logging configuration
00009 import log_config
00010
00011 # Get the logger for this module
00012 logger = logging.getLogger(__name__)
00013
00014 # Ensure logging is properly configured for GUI
00015 # Don't add additional handlers if they already exist
00016 root_logger = logging.getLogger()
00017
00018 # Only add console handler if GUI is being run directly (not through another script)
00019 if __name__ == '__main__':
00020     console_handler = logging.StreamHandler()
00021     console_handler.setLevel(logging.INFO)
00022     formatter = logging.Formatter('%(asctime)s %(levelname)s: %(name)s: %(message)s')
00023     console_handler.setFormatter(formatter)
00024
00025     # Only add console handler if not already present
00026     if not any(isinstance(h, logging.StreamHandler) for h in root_logger.handlers):
00027         root_logger.addHandler(console_handler)
00028
00029 logger.info("GUI module loaded with logging configured")
00030
00031 # ensure the project root (parent of sourcecode/) is on the Python path
00032 sys.path.insert(0, os.path.abspath(os.path.join(os.path.dirname(__file__), '..')))
00033 from monai.data.image_reader import NibabelReader
00034 from PySide6.QtWidgets import (
00035     QApplication, QMainWindow, QPushButton, QVBoxLayout, QWidget, QFileDialog,
00036     QMessageBox, QGroupBox, QToolButton, QRadioButton,
00037     QLabel, QSpinBox, QDoubleSpinBox, QProgressDialog, QScrollArea
00038 )
00039 from PySide6.QtCore import Qt, QObject, Signal, QThread, Slot
00040 from PySide6.QtGui import QAction, QPixmap, QtGuiApplication
00041 import visualization
00042 import nibabel as nib # for handling NIfTI files
00043 import numpy as np # for numerical operations
00044 from system_manager import SystemManager
00045 from parameter_manager import ParameterManager
00046 import torch
00047 from monai.transforms import (
00048     Compose, LoadImaged, EnsureChannelFirstd, Lambdad,
00049     EnsureTyped, Orientationd, Spacingd, SpatialPadd,
00050     CenterSpatialCropd, ScaleIntensityRangePercentilesd, ToTensord,
00051 )
00052 from monai.data import NumpyReader
00053
00054 def handle_exception(exc_type, exc_value, exc_tb):
00055     # Print full traceback for uncaught exceptions
00056     traceback.print_exception(exc_type, exc_value, exc_tb)
00057
00058 sys.excepthook = handle_exception
00059
00060 class TrainingWorker(QObject):
00061     """
00062     Worker object to run the training in a separate thread.
00063     """
00064     finished = Signal()
00065     error = Signal(str)
00066     progress = Signal(int, int) # current epoch, total epochs
00067
00068     def __init__(self, manager):
00069         super().__init__()
00070         self.manager = manager
00071
00072     def run(self):
00073         try:
00074             logger.info("Using legacy training method")
00075             self.manager.run_training()
00076             self.finished.emit()
00077         except Exception as ex:
00078             logger.error(f"Training failed: {ex}")
00079             logger.error("Training traceback:", exc_info=True)
00080             traceback.print_exc()
00081             # Emit error signal instead of showing QMessageBox in worker thread
00082             self.error.emit(str(ex))
00083

```

```

00084 class InferenceWorker(QObject):
00085     """
00086     Worker to run inference in a separate thread, mirroring the standalone approach.
00087     Emits finished(output_path: str) on success, error(message: str) on failure.
00088     """
00089     finished = Signal(str)
00090     error = Signal(str)
00091
00092     def __init__(self, ct_file: str, model_checkpoint: str, min_e: float, max_e: float):
00093         super().__init__()
00094         self.ct_file = ct_file
00095         self.model_checkpoint = model_checkpoint
00096         self.min_e = min_e
00097         self.max_e = max_e
00098
00099     def _check_canceled(self):
00100         try:
00101             if QThread.currentThread().isInterruptionRequested():
00102                 raise RuntimeError("Inference canceled by user")
00103         except Exception:
00104             # If called outside a QThread context, ignore
00105             pass
00106
00107     def run(self):
00108         try:
00109             logger.info("Starting background inference worker")
00110             # Import standalone inference components
00111             from inference_module import InferenceModule
00112             from inference_module_standalone import (
00113                 load_models_from_checkpoint,
00114                 get_gaussian_quadrature_4point,
00115                 save_dose,
00116                 save_nifti_with_manifest,
00117             )
00118
00119             # Quadrature setup
00120             quad_energies, quad_weights = get_gaussian_quadrature_4point(self.min_e, self.max_e)
00121             logger.info(f"Quad energies: {quad_energies}")
00122             self._check_canceled()
00123
00124             # Load CT data
00125             path_lower = self.ct_file.lower()
00126             if path_lower.endswith('.nii') or path_lower.endswith('.nii.gz'):
00127                 nii = nib.load(self.ct_file)
00128                 ct_array = nii.get_fdata().astype(np.float32)
00129                 affine = nii.affine
00130             elif path_lower.endswith('.npy'):
00131                 ct_array = np.load(self.ct_file).astype(np.float32)
00132                 affine = None
00133             else:
00134                 raise ValueError(f"Unsupported CT file format: {self.ct_file}")
00135             logger.info(f"CT loaded: shape={ct_array.shape}, dtype={ct_array.dtype}")
00136             self._check_canceled()
00137
00138             # To tensor
00139             ct_tensor = torch.from_numpy(ct_array).unsqueeze(0).float()
00140             logger.info(f"CT tensor shape: {ct_tensor.shape}")
00141             self._check_canceled()
00142
00143             # Load models
00144             device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
00145             models_by_energy, scale_factor, clip_min_dict, clip_max_dict, dose_normalization_params =
load_models_from_checkpoint(
00146                 self.model_checkpoint,
00147                 quad_energies,
00148                 device=str(device),
00149             )
00150             logger.info("Models loaded from checkpoint")
00151             self._check_canceled()
00152
00153             # Build inference module
00154             infer_mod = InferenceModule(
00155                 models_by_energy=models_by_energy,
00156                 energies=quad_energies,
00157                 energy_weights=quad_weights,
00158                 device=str(device),
00159                 scale_factor=scale_factor,
00160                 energy_min=self.min_e,
00161                 energy_max=self.max_e,
00162                 clip_min=clip_min_dict,
00163                 clip_max=clip_max_dict,
00164                 dose_normalization_params=dose_normalization_params,
00165             )
00166             logger.info("InferenceModule constructed")
00167             self._check_canceled()
00168
00169             # Run inference

```

```

00170         target_cube_size = (64, 64, 64)
00171         if len(quad_energies) > 1:
00172             dose = infer_mod.run_inference(ct_tensor, target_cube_size=target_cube_size)
00173         else:
00174             energy = quad_energies[0]
00175             dose = infer_mod.run_inference_conditioned_on_energy(
00176                 ct_tensor,
00177                 energy_value=energy,
00178                 target_cube_size=target_cube_size,
00179             )
00180         logger.info("Inference call finished")
00181         self._check_canceled()
00182
00183         # To numpy
00184         if isinstance(dose, torch.Tensor):
00185             dose_np = dose.detach().cpu().numpy()
00186         else:
00187             dose_np = np.array(dose, dtype=np.float32)
00188         if dose_np.ndim == 4 and dose_np.shape[0] == 1:
00189             dose_np = dose_np[0]
00190         if dose_np.ndim == 4 and dose_np.shape[0] == 1:
00191             dose_np = dose_np[0]
00192         logger.info(f"Dose stats: shape={dose_np.shape}, min={dose_np.min():.6f}, max={dose_np.max():.6f}")
00193
00194         # Save outputs
00195         from pathlib import Path
00196         output_path = Path(f"dose_output_standalone_gui_{Path(self.ct_file).stem}.npz")
00197         save_dose(output_path, dose_np, affine)
00198         logger.info(f"Dose saved to {output_path}")
00199         try:
00200             save_nifti_with_manifest(dose_np, output_path, root_dir=".")
00201         except Exception as e:
00202             logger.warning(f"Failed to create NIfTI: {e}")
00203
00204         # Emit success
00205         self.finished.emit(str(output_path))
00206     except Exception as e:
00207         logger.error("Inference failed", exc_info=True)
00208         self.error.emit(str(e))
00209
00210 class MainWindow(QMainWindow):
00211     """
00212     Main entry point for the RadioTherapy project.
00213
00214     This function sets up the configuration parameters, initializes the SystemManager,
00215     and triggers the training and inference processes.
00216
00217     MainWindow is the primary GUI window for the RadioTherapy project.
00218     It provides a menu bar with file actions and a central widget containing buttons for CT upload, dose
00219     calculation, and visualization.
00220     """
00221     # Define constants for dictionary keys to avoid magic strings
00222     KEY_INPUT = "input"
00223     KEY_TARGET = "target"
00224
00225     def __init__(self):
00226         super().__init__()
00227         self.setWindowTitle("RadioTherapy Project")
00228         self.setMinimumSize(800, 800)
00229
00230         self.input_dir = None # store the input directory for training inputs
00231         self.output_dir = None # store the output directory for training outputs
00232         self.ct_file = None # store imported CT file (inference)
00233         self.model_checkpoint = None # store imported model file path (inference)
00234         self.model_file_bool = False # flag to check if model file is loaded
00235         self.ct_file_bool = False # flag to check if CT file is loaded
00236
00237         # prepare parameters and transforms with default values
00238         # These values will be updated based on user input in the GUI
00239         self.pm = ParameterManager(
00240             energies=[0], batch_size=2, cube_size=64,
00241             num_epochs=5, learning_rate=1e-4, patience=3
00242         )
00243         transforms_chain = self._create_transforms(self.pm.cube_size)
00244         device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
00245         # instantiate SystemManager here - no main.py needed
00246         self.system_manager = SystemManager(
00247             root_dir="", transforms=transforms_chain,
00248             resolutions=self.pm.resolutions, energies=self.pm.energies,
00249             quad_energies=self.pm.quad_energies,
00250             quad_weights=self.pm.quad_weights,
00251             batch_size=self.pm.batch_size, device=device,
00252             num_epochs=self.pm.num_epochs, learning_rate=self.pm.learning_rate, patience=self.pm.patience,
00253             cube_size=self.pm.cube_size,
00254             energy_min=0.0, energy_max=50.0,
00255             seed=42
00256         )

```

```

00256
00257
00258     # Create the central widget with buttons
00259     self._create_central_widget()
00260     # adjust window size to fit expanded widgets
00261     # self.adjustSize()
00262     # optionally enforce minimum size to current content
00263     # self.setMinimumSize(self.size())
00264
00265 def _create_transforms(self, cube_size):
00266     """Creates the MONAI transforms pipeline."""
00267     return Compose([
00268         LoadImaged(keys=[self.KEY_INPUT, self.KEY_TARGET], reader=NumpyReader),
00269         EnsureChannelFirstd(keys=[self.KEY_INPUT, self.KEY_TARGET]),
00270         EnsureTyped(keys=[self.KEY_INPUT, self.KEY_TARGET]),
00271         Orientationd(keys=[self.KEY_INPUT, self.KEY_TARGET], axcodes="RAS"),
00272         Spacingd(keys=[self.KEY_INPUT, self.KEY_TARGET], pixdim=(2.4, 2.4, 2.4), mode=("bilinear", "nearest")),
00273         SpatialPadd(keys=[self.KEY_INPUT, self.KEY_TARGET], spatial_size=cube_size, method="symmetric"),
00274         CenterSpatialCropd(keys=[self.KEY_INPUT, self.KEY_TARGET], roi_size=cube_size),
00275         ScaleIntensityRangePercentile(
00276             keys=self.KEY_INPUT, lower=0, upper=99.5, b_min=0, b_max=1
00277         ),
00278         ToTensord(keys=[self.KEY_INPUT, self.KEY_TARGET])
00279     ])
00280
00281 def _create_central_widget(self):
00282     """
00283     Sets up the central widget with a vertical layout and adds buttons for CT upload, dose calculation, and
00284     visualization.
00285     """
00286     # Create a scroll area
00287     scroll_area = QScrollArea(self)
00288     scroll_area.setWidgetResizable(True)
00289     self.setCentralWidget(scroll_area)
00290
00291     # Create a container widget for the layout
00292     central_widget = QWidget()
00293     scroll_area.setWidget(central_widget)
00294
00295     layout = QVBoxLayout(central_widget)
00296     layout.setAlignment(Qt.AlignmentFlag.AlignTop)
00297
00298     # Training section
00299     training_group = QGroupBox("Training", self)
00300     training_layout = QVBoxLayout()
00301     training_layout.setAlignment(Qt.AlignmentFlag.AlignTop)
00302     training_group.setLayout(training_layout)
00303     layout.addWidget(training_group)
00304
00305     # Parameter inputs for training
00306     # Advanced training parameters in checkable group box
00307     advanced_group = QGroupBox("Advanced Settings", self)
00308     advanced_group.setCheckable(True)
00309     advanced_group.setChecked(False)
00310     adv_layout = QVBoxLayout()
00311     advanced_group.setLayout(adv_layout)
00312     training_layout.addWidget(advanced_group)
00313
00314     # Advanced training parameters
00315     batch_label = QLabel("Batch Size:", self)
00316     self.batch_spin = QSpinBox(self)
00317     self.batch_spin.setRange(1, 128)
00318     self.batch_spin.setValue(2)
00319     self.batch_spin.setSingleStep(1)
00320     adv_layout.addWidget(batch_label)
00321     adv_layout.addWidget(self.batch_spin)
00322     epochs_label = QLabel("Number of Epochs:", self)
00323     self.epochs_spin = QSpinBox(self)
00324     self.epochs_spin.setRange(1, 1000)
00325     self.epochs_spin.setValue(5)
00326     self.epochs_spin.setSingleStep(1)
00327     adv_layout.addWidget(epochs_label)
00328     adv_layout.addWidget(self.epochs_spin)
00329     patience_label = QLabel("Early stopping patience:", self)
00330     self.patience_spin = QSpinBox(self)
00331     self.patience_spin.setRange(1, 100)
00332     self.patience_spin.setValue(3)
00333     adv_layout.addWidget(patience_label)
00334     adv_layout.addWidget(self.patience_spin)
00335     learning_rate_label = QLabel("Learning Rate:", self)
00336     self.learning_rate_spin = QDoubleSpinBox(self)
00337     self.learning_rate_spin.setRange(0.0, 1.0)
00338     self.learning_rate_spin.setSingleStep(1e-5)
00339     self.learning_rate_spin.setDecimals(6)
00340     self.learning_rate_spin.setValue(1e-4)
00341     adv_layout.addWidget(learning_rate_label)
    adv_layout.addWidget(self.learning_rate_spin)

```

```

00342
00343     # Add a spacer to the advanced layout to push the widgets to the top
00344     for idx in range(adv_layout.count()):
00345         adv_layout.itemAt(idx).widget().setVisible(False)
00346
00347     # toggle visibility of advanced parameters when the group box is checked/unchecked
00348     advanced_group.toggled.connect(
00349         lambda chk: (
00350             [adv_layout.itemAt(i).widget().setVisible(chk)
00351              for i in range(adv_layout.count())]
00352             # self.adjustSize() # Removed to allow scrolling
00353         )
00354     )
00355
00356     # Inference section
00357     inference_group = QGroupBox("Inference", self)
00358     inference_layout = QVBoxLayout()
00359     inference_layout.setAlignment(Qt.AlignmentFlag.AlignTop)
00360     inference_group.setLayout(inference_layout)
00361     layout.addWidget(inference_group)
00362
00363     #Energy spectrum inputs(eV)
00364     energy_min_label = QLabel("Energy Min (eV):", self)
00365     self.energy_min_spin = QDoubleSpinBox(self)
00366     self.energy_min_spin.setRange(0.0, 10000.0)
00367     self.energy_min_spin.setDecimals(2)
00368     self.energy_min_spin.setValue(0.0)
00369     inference_layout.addWidget(energy_min_label)
00370     inference_layout.addWidget(self.energy_min_spin)
00371
00372     energy_max_label= QLabel("Energy Max (eV):", self)
00373     self.energy_max_spin = QDoubleSpinBox(self)
00374     self.energy_max_spin.setRange(0.0, 10000.0)
00375     self.energy_max_spin.setDecimals(2)
00376     self.energy_max_spin.setValue(50.0)
00377     inference_layout.addWidget(energy_max_label)
00378     inference_layout.addWidget(self.energy_max_spin)
00379
00380     # Upload button for CT scan
00381     upload_button = QPushButton("Upload CT Scan", self)
00382     upload_button.setToolTip("Click to upload a CT scan (NIfTI file)")
00383     upload_button.clicked.connect(self.open_file_dialog)
00384     inference_layout.addWidget(upload_button)
00385
00386     # Upload button for model file
00387     model_button = QPushButton("Upload Model File", self)
00388     model_button.setToolTip("Click to upload a model file (.ckpt)")
00389     model_button.clicked.connect(self.open_model_dialog)
00390     inference_layout.addWidget(model_button)
00391
00392     # Dose calculation button (disabled until CT file is uploaded)
00393     self.dose_button = QPushButton("Calculate Dose", self)
00394     self.dose_button.setToolTip("Calculate dose distribution (requires CT scan)")
00395     self.dose_button.setEnabled(False)
00396     self.dose_button.clicked.connect(self.calculate_dose)
00397     inference_layout.addWidget(self.dose_button)
00398
00399     # Training folder selection buttons
00400     self.input_button = QPushButton("Select one Energy Folder for Training", self)
00401     self.input_button.setToolTip("Select the folder containing input cubes for training")
00402     self.input_button.clicked.connect(self.select_input_folder)
00403     training_layout.addWidget(self.input_button)
00404     # Input path label
00405     self.input_label = QLabel("No input folder selected", self)
00406     training_layout.addWidget(self.input_label)
00407
00408     self.train_button = QPushButton("Train Model", self)
00409     self.train_button.setToolTip("Train the model with the selected input and output folders")
00410     self.train_button.setEnabled(False) # Initially disabled
00411     self.train_button.clicked.connect(self.train_model)
00412     training_layout.addWidget(self.train_button)
00413     # Button to show training graphs in-app
00414     self.show_plots_button = QPushButton("Show Training Graphs", self)
00415     self.show_plots_button.setToolTip("Display loss and adversarial curves inside GUI")
00416     self.show_plots_button.setEnabled(False) # Initially disabled
00417     self.show_plots_button.clicked.connect(self.show_training_plots)
00418     training_layout.addWidget(self.show_plots_button)
00419
00420     # Visualization button for inference results
00421     self.visualize_button = QPushButton("Visualize Dose Distribution", self)
00422     self.visualize_button.setToolTip("Visualize the calculated dose distribution")
00423     self.visualize_button.setEnabled(True) # Disabled until inference is complete
00424     self.visualize_button.clicked.connect(self.visualize_inference_results)
00425     inference_layout.addWidget(self.visualize_button)
00426
00427     central_widget.setLayout(layout)
00428

```

```

00429
00430     # Select input and output folders for training
00431     def select_input_folder(self):
00432         """
00433         Opens a directory dialog to select the output cubes folder for training.
00434         Enables the Train button if both folders are selected.
00435         """
00436         folder = QFileDialog.getExistingDirectory(self, "Select Input Cubes Directory", "",
00437         QFileDialog.Option.ShowDirsOnly)
00438         if folder:
00439             # If the user selected an energy folder containing both 'inputcube' and 'outputcube', derive both paths
00440             energy_folder = folder
00441             # Determine dataset root containing all energy subfolders
00442             dataset_root = os.path.dirname(energy_folder)
00443             # List all energy subfolder names (skip hidden files)
00444             energy_names = sorted([
00445                 d for d in os.listdir(dataset_root)
00446                 if os.path.isdir(os.path.join(dataset_root, d)) and not d.startswith('.')])
00447             # Parse numeric energy values from folder names
00448             energies_list = [float(name.replace("_", ".")) for name in energy_names]
00449             # Update GUI and SystemManager energies and root_dir
00450             self.pm.energies = energies_list
00451             self.system_manager.energies = energies_list
00452             self.system_manager.root_dir = dataset_root
00453
00454             input_cube_path = os.path.join(energy_folder, "inputcube")
00455             output_cube_path = os.path.join(energy_folder, "outputcube")
00456             if os.path.isdir(input_cube_path) and os.path.isdir(output_cube_path):
00457                 # Use the energy-specific subfolders directly
00458                 self.input_dir = input_cube_path
00459                 self.input_label.setText(f"Input cube folder: {self.input_dir}")
00460                 self.output_dir = output_cube_path
00461                 # Determine cube size from one of the input .npy files (same as standalone_training.py)
00462                 files = glob.glob(os.path.join(self.input_dir, "*.npy"))
00463                 if files:
00464                     arr = np.load(files[0])
00465                     cube_size = arr.shape[0] # Take only first dimension (same as standalone)
00466                     self.pm.cube_size = cube_size
00467                     self.system_manager.cube_size = self.pm.cube_size
00468                     self.system_manager.transforms = self._create_transforms(self.pm.cube_size)
00469                     logger.info(f"Detected cube size: {cube_size} (same as standalone_training.py)")
00470                 self._update_ui_state()
00471                 return
00472
00473             # Otherwise treat folder as the direct input-cube directory
00474             self.input_dir = folder
00475             self.input_label.setText(f"Input folder: {folder}")
00476             # determine cube size from first cube file in folder (same as standalone_training.py)
00477             files = glob.glob(os.path.join(folder, "*.npy")) + glob.glob(os.path.join(folder, "*.nii")) +
00478             glob.glob(os.path.join(folder, "*.nii.gz"))
00479             if files:
00480                 sample = files[0]
00481                 try:
00482                     if sample.lower().endswith(".npy"):
00483                         arr = np.load(sample)
00484                     else:
00485                         arr = np.asarray(nib.load(sample).dataobj)
00486                     cube_size = arr.shape[0] # Take only first dimension (same as standalone)
00487                     self.pm.cube_size = cube_size
00488                     # update transforms in system_manager
00489                     self.system_manager.cube_size = self.pm.cube_size
00490                     self.system_manager.transforms = self._create_transforms(self.pm.cube_size)
00491                     logger.info(f"Detected cube size: {cube_size} (same as standalone_training.py)")
00492                 except Exception as e:
00493                     QMessageBox.warning(self, "Warning", f"Failed to determine cube size: {e}")
00494                     logger.info(f"Cube size set to: {self.pm.cube_size}")
00495             self._update_ui_state()
00496
00497     def _update_ui_state(self):
00498         """Centralized method to update the state of UI elements."""
00499         # Enable dose calculation if both CT and model files are loaded
00500         self.dose_button.setEnabled(self.ct_file_bool and self.model_file_bool)
00501
00502         # Enable training if input and output directories are selected
00503         self.train_button.setEnabled(bool(self.input_dir and self.output_dir))
00504
00505     # Train the model using the selected input and output folders
00506     def train_model(self):
00507         """
00508         Calls the training pipeline using the selected input and output folders.
00509         Expects both folders to be subdirectories of the same parent (root) folder.
00510         """
00511         logger.info("=" * 60)
00512         logger.info("TRAINING STARTED VIA GUI")
00513         logger.info("=" * 60)

```



```

00514
00515         if not self.input_dir:
00516             logger.error("No input directory selected for training")
00517             QMessageBox.warning(self, "Error", "Please select the Energy folder containing input cubes and output
cubes.")
00518         return
00519
00520         #Check that both directories are subdirectories of the same parent (root) folder
00521         parent_in = os.path.dirname(self.input_dir)
00522         parent_out = os.path.dirname(self.output_dir)
00523         if parent_in != parent_out:
00524             logger.error(f"Input and output directories not in same parent: {parent_in} vs {parent_out}")
00525             QMessageBox.warning(self, "Error", "Input and output folders must be subdirectories of the same parent
folder.")
00526         return
00527
00528         # Get training parameters from GUI
00529         self.pm.batch_size = self.batch_spin.value()
00530         self.pm.num_epochs = self.epochs_spin.value()
00531         self.pm.patience = self.patience_spin.value()
00532         self.pm.learning_rate = self.learning_rate_spin.value()
00533
00534         logger.info(f"Training parameters from GUI:")
00535         logger.info(f"  batch_size={self.pm.batch_size}")
00536         logger.info(f"  num_epochs={self.pm.num_epochs}")
00537         logger.info(f"  patience={self.pm.patience}")
00538         logger.info(f"  learning_rate={self.pm.learning_rate}")
00539
00540         energy_folder = parent_in
00541         # parent of energy_folder is the dataset root containing all energy subfolders
00542         dataset_root = os.path.dirname(energy_folder)
00543
00544         # Debug logging: Check the directory structure (same as standalone_training.py)
00545         logger.info(f"Selected energy folder: {energy_folder}")
00546         logger.info(f"Dataset root: {dataset_root}")
00547         logger.info(f"Input cube path: {self.input_dir}")
00548         logger.info(f"Output cube path: {self.output_dir}")
00549
00550         # Check if we have data files
00551         input_files = glob.glob(os.path.join(self.input_dir, "*.npy"))
00552         output_files = glob.glob(os.path.join(self.output_dir, "*.npy"))
00553         logger.info(f"Found {len(input_files)} input files and {len(output_files)} output files")
00554
00555         if len(input_files) == 0 or len(output_files) == 0:
00556             QMessageBox.warning(self, "Error", f"No training data found!\nInput files: {len(input_files)}\nOutput
files: {len(output_files)}")
00557         return
00558
00559         self.system_manager.root_dir = dataset_root
00560
00561         # update training parameters from GUI
00562         self.system_manager.batch_size = self.pm.batch_size
00563         self.system_manager.num_epochs = self.pm.num_epochs
00564         self.system_manager.patience = self.pm.patience
00565         self.system_manager.learning_rate = self.pm.learning_rate
00566
00567         # Use LEGACY training method (same as standalone_training.py)
00568         logger.info(f"Training method: Legacy (same as standalone_training.py)")
00569
00570         # Get ALL available energies for proper multi-energy training (same as standalone_training.py)
00571         try:
00572             # List all energy subfolder names (skip hidden files) - same logic as standalone
00573             energy_names = [
00574                 d for d in os.listdir(dataset_root)
00575                 if os.path.isdir(os.path.join(dataset_root, d)) and not d.startswith('.')
00576             ]
00577             # Parse numeric energy values from folder names and sort numerically
00578             available_energies = sorted([float(name.replace("_", ".")) for name in energy_names])
00579             logger.info(f"Found energies: {available_energies} (same as standalone_training.py)")
00580
00581             if available_energies:
00582                 # Update ParameterManager with detected energies (same as standalone)
00583                 self.pm.energies = torch.tensor(available_energies)
00584                 # Recalculate energy_min and energy_max from actual energies (same as standalone)
00585                 min_e = min(available_energies)
00586                 max_e = max(available_energies)
00587                 self.pm.energy_min = min_e
00588                 self.pm.energy_max = max_e
00589                 self.pm.quad_energies = available_energies
00590                 self.pm.quad_weights = [1.0/len(available_energies)] * len(available_energies)
00591
00592                 # Update SystemManager with new parameters (same as standalone)
00593                 self.system_manager.energies = available_energies
00594                 self.system_manager.energy_min = min_e
00595                 self.system_manager.energy_max = max_e
00596                 self.system_manager.quad_energies = available_energies
00597                 self.system_manager.quad_weights = self.pm.quad_weights

```

```

00598
00599         logger.info(f"Training with ALL energies: {available_energies} (proper multi-energy training)")
00600         logger.info(f"Energy range: {min_e} to {max_e} (calculated from data)")
00601         logger.info(f"Quad weights: {self.pm.quad_weights}")
00602     else:
00603         logger.warning("No energies detected, using fallback")
00604         available_energies = [0.0]
00605
00606 except Exception as e:
00607     logger.warning(f"Error detecting energies: {e}, using fallback")
00608     available_energies = [0.0]
00609
00610 # reset stop_training flag
00611 self.system_manager.stop_training = False
00612
00613 # run training in background thread to avoid freezing GUI
00614 progress = QProgressDialog(
00615     f"Training in progress... Please wait.",
00616     "Cancel", 0, 0, self
00617 )
00618 progress.setWindowModality(Qt.ApplicationModal)
00619 progress.setCancelButtonText("Cancel")
00620 progress.setMinimumDuration(0)
00621 progress.setAutoClose(False)
00622 progress.setAutoReset(False)
00623 progress.show()
00624
00625 # create worker and thread
00626 thread = QThread(self)
00627 worker = TrainingWorker(self.system_manager)
00628 worker.moveToThread(thread)
00629
00630 # cancel training if user cancels dialog
00631 progress.canceled.connect(thread.requestInterruption)
00632 # signal SystemManager to stop training loops
00633 progress.canceled.connect(lambda: setattr(self.system_manager, 'stop_training', True))
00634 thread.started.connect(worker.run)
00635 worker.finished.connect(thread.quit)
00636 worker.finished.connect(worker.deleteLater)
00637 thread.finished.connect(thread.deleteLater)
00638 # handle completion and errors on main thread
00639 worker.finished.connect(self.on_training_finished)
00640 worker.error.connect(self.on_training_error)
00641 # keep references to prevent garbage collection
00642 self._train_thread = thread
00643 self._train_worker = worker
00644 self._train_progress = progress
00645 thread.start()
00646
00647 logger.info("Training thread started")
00648
00649 # Open a file dialog to select a CT scan file (inference)
00650 def open_file_dialog(self):
00651     """
00652     Opens a file dialog for selecting a CT scan file in NIfTI format.
00653     If a file is selected, its path is stored and the dose calculation button is enabled.
00654     """
00655     logger.info("Opening CT file selection dialog...")
00656
00657     # support both NIfTI and NumPy formats
00658     file_path, _ = QFileDialog.getOpenFileName(
00659         self, "Select CT Scan", "", "CT Files (*.nii *.nii.gz *.npy)"
00660     )
00661     if file_path:
00662         logger.info(f"User selected CT file: {file_path}")
00663         self.ct_file = file_path # store the imported CT file
00664
00665     try:
00666         # Load file: NIfTI or NumPy
00667         fp = file_path.lower()
00668         if fp.endswith('.npy'):
00669             logger.info("Loading NumPy file...")
00670             data = np.load(file_path)
00671         else:
00672             logger.info("Loading NIfTI file...")
00673             img = nib.load(file_path)
00674             data = np.asarray(img.dataobj)
00675
00676         # store CT volume for overlay
00677         self.ct_volume = data
00678         logger.info(f"CT file loaded successfully: shape={data.shape}, dtype={data.dtype}")
00679         logger.info(f"CT data range: [{data.min():.3f}, {data.max():.3f}]")
00680
00681         self.ct_file_bool = True
00682         self._update_ui_state()
00683         logger.info("CT file loaded, UI state updated.")
00684

```



```

00685         except Exception as e:
00686             logger.error(f"Failed to load CT file: {e}")
00687             logger.error("CT file loading traceback:", exc_info=True)
00688             QMessageBox.critical(self, "Error", f"Failed to load the selected file: {e}")
00689     else:
00690         logger.info("CT file selection cancelled by user")
00691 # Open a file dialog to select a model file (inference)
00692 def open_model_dialog(self):
00693     """
00694     Opens a file dialog for selecting a model file in .ckpt format.
00695     If a file is selected, its path is stored and the dose calculation button is enabled.
00696     """
00697     logger.info("Opening model file selection dialog...")
00698
00699     file_path, _ = QFileDialog.getOpenFileName(
00700         self, "Select Model File", "", "Model Files (*.ckpt)"
00701     )
00702     if file_path:
00703         logger.info(f"User selected model file: {file_path}")
00704
00705         try:
00706             # Validate that the Model file (.ckpt) can be loaded
00707             logger.info("Validating model checkpoint...")
00708             device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
00709             test_checkpoint = torch.load(file_path, map_location=device, weights_only=False) # Test load to
00710 validate         logger.info(f" Model checkpoint validated successfully")
00711             logger.info(f" Checkpoint keys: {list(test_checkpoint.keys())}")
00712
00713             # Store the file path, not the loaded checkpoint
00714             self.model_checkpoint = file_path
00715             self.model_file_bool = True
00716             self._update_ui_state()
00717             logger.info(" Model file successfully selected and validated, UI state updated.")
00718
00719         except Exception as e:
00720             logger.error(f"Failed to load model file: {e}")
00721             logger.error("Model file loading traceback:", exc_info=True)
00722             QMessageBox.critical(self, "Error", f"Failed to load the selected file: {e}")
00723     else:
00724         logger.info("Model file selection cancelled by user")
00725
00726 # Calculate the dose distribution using the standalone inference module (like standalone script)
00727 def calculate_dose(self):
00728     """
00729     Calculate dose distribution using the original InferenceModule directly (standalone approach).
00730     Now runs in a background thread with a progress dialog.
00731     """
00732     logger.info("=" * 60)
00733     logger.info("DOSE CALCULATION STARTED VIA GUI (STANDALONE APPROACH)")
00734     logger.info("=" * 60)
00735
00736     if not self.ct_file:
00737         logger.error("No CT file selected for dose calculation")
00738         QMessageBox.warning(self, "Error", "Please upload a CT scan first.")
00739         return
00740
00741     if not self.model_checkpoint:
00742         logger.error("No model checkpoint selected for dose calculation")
00743         QMessageBox.warning(self, "Error", "Please upload a model file first.")
00744         return
00745
00746     min_e = self.energy_min_spin.value()
00747     max_e = self.energy_max_spin.value()
00748     logger.info(f"Energy range: {min_e} to {max_e} eV")
00749
00750     # validate energy range
00751     if min_e > max_e:
00752         logger.error(f"Invalid energy range: min_e={min_e} > max_e={max_e}")
00753         QMessageBox.warning(self, "Error", "Minimum energy must be less than maximum energy.")
00754         return
00755
00756     # Show a modal progress dialog and run inference in background
00757     progress = QProgressDialog(
00758         f"Inference in progress... Please wait.",
00759         "Cancel", 0, 0, self
00760     )
00761     progress.setWindowModality(Qt.ApplicationModal)
00762     progress.setCancelButtonText("Cancel")
00763     progress.setMinimumDuration(0)
00764     progress.setAutoClose(False)
00765     progress.setAutoReset(False)
00766     progress.show()
00767
00768     # Thread + worker
00769     thread = QThread(self)
00770     worker = InferenceWorker(self.ct_file, self.model_checkpoint, min_e, max_e)

```

```

00771         worker.moveToThread(thread)
00772
00773         # Cancellation support
00774         progress.canceled.connect(thread.requestInterruption)
00775
00776         # Start/finish wiring
00777         thread.started.connect(worker.run)
00778         worker.finished.connect(thread.quit)
00779         worker.finished.connect(worker.deleteLater)
00780         thread.finished.connect(thread.deleteLater)
00781
00782         # Handle completion and errors in main thread
00783         worker.finished.connect(self.on_inference_finished)
00784         worker.error.connect(self.on_inference_error)
00785
00786         # Keep refs
00787         self._infer_thread = thread
00788         self._infer_worker = worker
00789         self._infer_progress = progress
00790
00791         thread.start()
00792         logger.info("Inference thread started")
00793
00794     @Slot(str)
00795     def on_inference_finished(self, output_path: str):
00796         # Close dialog
00797         if hasattr(self, '_infer_progress'):
00798             self._infer_progress.close()
00799         # Store and notify
00800         self.dose_result_path = output_path
00801         QMessageBox.information(self, "Success", f"Dose distribution calculated successfully.\nSaved to:
{output_path}")
00802         # Visualization
00803         try:
00804             logger.info("Starting visualization...")
00805             visualization.load_and_visualize(str(output_path), self.ct_volume)
00806             logger.info("Visualization completed")
00807         except Exception as e:
00808             logger.warning(f"Visualization failed: {e}")
00809
00810     @Slot(str)
00811     def on_inference_error(self, message: str):
00812         if hasattr(self, '_infer_progress'):
00813             self._infer_progress.close()
00814         logger.error(f"Inference error: {message}")
00815         QMessageBox.critical(self, "Error", f"Failed to calculate dose distribution: {message}")
00816
00817         # Visualize the dose distribution from inference results
00818     def visualize_inference_results(self):
00819         """
00820         Visualizes the calculated dose distribution from inference results and embeds it in the GUI window.
00821         """
00822         try:
00823             # prompt user to select a NIfTI/NumPy file for inference results
00824             file_path, _ = QFileDialog.getOpenFileName(
00825                 self, "Select Dose Result File", "", "Dose Files (*.nii *.nii.gz *.npy)"
00826             )
00827             if not file_path:
00828                 return
00829             self.dose_result_path = file_path
00830
00831             # Load the dose distribution from the result file
00832             fig = visualization.load_and_visualize(self.dose_result_path, self.ct_volume, title="Calculated Dose
Distribution")
00833
00834             except Exception as e:
00835                 QMessageBox.critical(self, "Error", f"Failed to visualize dose distribution: {e}")
00836
00837     @Slot()
00838     def on_training_finished(self):
00839         """Slot to close progress dialog and show success message on main thread"""
00840         if hasattr(self, '_train_progress'):
00841             self._train_progress.close()
00842         QMessageBox.information(self, "Success", "Training completed.")
00843         # show plots in-app
00844         self.show_training_plots()
00845         # enable the button to show training plots
00846         self.show_plots_button.setEnabled(True)
00847
00848     @Slot(str)
00849     def on_training_error(self, msg):
00850         """Slot to close progress dialog and show error message on main thread"""
00851         if hasattr(self, '_train_progress'):
00852             self._train_progress.close()
00853         logger.error(f"Training error: {msg}")
00854         QMessageBox.critical(self, "Error", f"Failed to train the model: {msg}")
00855

```

```

00856 @Slot()
00857 def show_training_plots(self):
00858     """Open a window showing the three training plot images together."""
00859     import glob
00860     from PySide6.QtWidgets import QScrollArea
00861     from PySide6.QtGui import QApplication
00862     # compute screen geometry
00863     screen_rect = QApplication.primaryScreen().availableGeometry()
00864     # use up to 80% of screen
00865     max_w_screen = int(screen_rect.width() * 0.8)
00866     max_h_screen = int(screen_rect.height() * 0.8)
00867     # divide screen into two rows: top row height, bottom row height
00868     row_height = max_h_screen // 2
00869     # divide width: top two share row, bottom spans
00870     top_w = max_w_screen // 2
00871     bottom_w = max_w_screen
00872
00873     patterns = ["loss_curves_res*.png", "learning_curve.png", "adv_curves.png"]
00874     plots_window = QWidget()
00875     plots_window.setWindowTitle("Training Graphs")
00876     labels = []
00877     idx = 0
00878     for pattern in patterns:
00879         for fname in glob.glob(pattern):
00880             lbl = QLabel()
00881             pix = QPixmap(fname)
00882             # pick target dims by position
00883             if idx < 2:
00884                 target_w, target_h = top_w, row_height
00885             else:
00886                 target_w, target_h = bottom_w, row_height
00887             pix = pix.scaled(target_w, target_h, Qt.KeepAspectRatio, Qt.SmoothTransformation)
00888             lbl.setPixmap(pix)
00889             lbl.setAlignment(Qt.AlignmentFlag.AlignCenter)
00890             labels.append(lbl)
00891             idx += 1
00892     from PySide6.QtWidgets import QGridLayout
00893     grid = QGridLayout()
00894     if len(labels) >= 2:
00895         grid.addWidget(labels[0], 0, 0)
00896         grid.addWidget(labels[1], 0, 1)
00897         if len(labels) >= 3:
00898             grid.addWidget(labels[2], 1, 0, 1, 2)
00899     else:
00900         for i, lbl in enumerate(labels):
00901             grid.addWidget(lbl, i, 0)
00902     scroll = QScrollArea()
00903     container = QWidget()
00904     container.setLayout(grid)
00905     scroll.setWidget(container)
00906     scroll.setWidgetResizable(True)
00907     main_layout = QVBoxLayout()
00908     main_layout.addWidget(scroll)
00909     plots_window.setLayout(main_layout)
00910     plots_window.resize(max_w_screen, max_h_screen)
00911     plots_window.show()
00912     self._plots_window = plots_window
00913
00914 def center(self):
00915     """
00916     Center the window on the screen.
00917     """
00918     screen = QApplication.primaryScreen()
00919     screen_rect = screen.availableGeometry()
00920     x = (screen_rect.width() - self.width()) // 2
00921     y = (screen_rect.height() - self.height()) // 2
00922     self.move(x, y)
00923
00924 if __name__ == "__main__":
00925     try:
00926         app = QApplication(sys.argv)
00927         window = MainWindow()
00928         window.show()
00929         window.center()
00930         sys.exit(app.exec())
00931     except Exception:
00932         traceback.print_exc()
00933     raise

```

8.12 sourcecode/hotspot_patch_sampler.py File Reference

Namespaces

- namespace [hotspot_patch_sampler](#)

Functions

- [hotspot_patch_sampler.extract_hotspot_patches](#) (volume, patch_size=(32, 32, 32), min_hotspot_voxels=10, dose_threshold=0.1, max_patches=16, random_patches=2)

8.13 hotspot_patch_sampler.py

[Go to the documentation of this file.](#)

```
00001 import numpy as np
00002
00003 def extract_hotspot_patches(volume, patch_size=(32,32,32), min_hotspot_voxels=10, dose_threshold=0.1,
00004                             max_patches=16, random_patches=2):
00005     """
00006     Extracts patches from a 3D volume where the number of voxels above a certain threshold is sufficient.
00007     Args:
00008         volume: 3D numpy array representing the dose distribution
00009         patch_size: Size of the patches to extract (z, y, x)
00010         min_hotspot_voxels: Minimum number of voxels in a patch that must be above the dose threshold
00011         dose_threshold: Dose threshold to consider a voxel as part of a hotspot
00012         max_patches: Maximum number of patches to extract
00013         random_patches: Number of additional random patches to add
00014     Returns:
00015         List of Patch-Start-Indices (e.g. [(z,y,x), ...])
00016     """
00017     D, H, W = volume.shape
00018     dz, dy, dx = patch_size
00019     patches = []
00020     # Iterate over the volume with a step size of half the patch size
00021     for z in range(0, D-dz+1, dz//2):
00022         for y in range(0, H-dy+1, dy//2):
00023             for x in range(0, W-dx+1, dx//2):
00024                 patch = volume[z:z+dz, y:y+dy, x:x+dx]
00025                 if np.count_nonzero(patch > dose_threshold) >= min_hotspot_voxels:
00026                     patches.append((z, y, x))
00027     # if there are too many patches, randomly sample
00028     if len(patches) > max_patches:
00029         import random
00030         patches = random.sample(patches, min(max_patches, len(patches)))
00031     # Add some random patches
00032     for _ in range(random_patches):
00033         z = np.random.randint(0, D-dz+1)
00034         y = np.random.randint(0, H-dy+1)
00035         x = np.random.randint(0, W-dx+1)
00036         patches.append((z, y, x))
00037     return patches
```

8.14 sourcecode/inference_module.py File Reference

Classes

- class [inference_module.InferenceModule](#)

Namespaces

- namespace [inference_module](#)

Variables

- [inference_module.logger](#) = logging.getLogger(__name__)

8.15 inference_module.py

[Go to the documentation of this file.](#)

```
00001 import torch
00002 import torch.nn as nn
00003 import torch.nn.functional as F
00004 from torch.nn.functional import interpolate
00005 import log_config
00006 import logging
00007 import math
00008 logger = logging.getLogger(__name__)
00009
00010 class InferenceModule:
```

```

00011     """
00012     The InferenceModule performs the dose inference process using the trained autoencoder
00013     and diffusion model. It computes the dose distribution from the uploaded patient CT data.
00014
00015     The process is as follows:
00016     1. Preprocess the input CT scan by resizing or cropping it to a specified cube size.
00017     2. Optionally encode the CT scan with the autoencoder to obtain a latent representation.
00018     3. Pass the (latent) representation through the diffusion model to generate a voxel-wise dose distribution.
00019
00020     Additional parameters (e.g. cube size) are taken into account during preprocessing.
00021     """
00022     def __init__(self, models_by_energy: dict, energies: list, energy_weights: list, device, scale_factor,
00023                  energy_min=None, energy_max=None, clip_min=None, clip_max=None, dose_normalization_params=None):
00024         """
00025         Args:
00026             models_by_energy (dict): Mapping from energy value (float) to tuple (autoencoder, diffusion_model,
00027             scheduler).
00028             energies (list of float): List of quadrature energy levels.
00029             energy_weights (list of float): Corresponding quadrature weights.
00030             device (torch.device): Device for inference.
00031             scale_factor (float): Scale factor used during training. If None, will try to extract from training.
00032             energy_min (float): Minimum energy value for normalization. If None, computed from energies.
00033             energy_max (float): Maximum energy value for normalization. If None, computed from energies.
00034             clip_min (float): Minimum dose value for denormalization. If None, no denormalization.
00035             clip_max (float): Maximum dose value for denormalization. If None, no denormalization.
00036             dose_normalization_params (dict): Energy-specific normalization parameters from checkpoint.
00037         """
00038         self.models_by_energy = models_by_energy
00039         self.energies = energies
00040         self.energy_weights = energy_weights
00041         self.device = device
00042         self.scale_factor = scale_factor
00043         # Use provided energy_min/max or compute from energies list
00044         self.energy_min = energy_min if energy_min is not None else min(energies)
00045         self.energy_max = energy_max if energy_max is not None else max(energies)
00046         # Add dose denormalization parameters (global fallbacks)
00047         self.clip_min = clip_min
00048         self.clip_max = clip_max
00049         # Energy-specific normalization parameters (preferred over global)
00050         self.dose_normalization_params = dose_normalization_params or {}
00051
00052     def get_energy_normalization_params(self, energy_value, resolution=100):
00053         """
00054         Get energy-specific normalization parameters for denormalization.
00055
00056         Args:
00057             energy_value (float): Energy in keV
00058             resolution (int): Spatial resolution
00059
00060         Returns:
00061             dict: Dictionary with clip_min, clip_max, etc.
00062         """
00063         # NEW: Try multiple resolution formats for compatibility
00064         energy_keys_to_try = [
00065             f'res64x64x64_e{energy_value:.2f}', # New training format
00066             f'res{resolution}_e{energy_value:.2f}', # Legacy format
00067             f'res100_e{energy_value:.2f}', # Fallback format
00068         ]
00069
00070         for energy_key in energy_keys_to_try:
00071             if energy_key in self.dose_normalization_params:
00072                 params = self.dose_normalization_params[energy_key]
00073                 logger.info(f"Using energy-specific parameters for {energy_key}: clip_min={params.get('clip_min',
00074                 0.0):.6f}, clip_max={params.get('clip_max', 1.0):.6f}")
00075                 return params
00076
00077         # Fallback to global parameters
00078         logger.warning(f"No energy-specific parameters found for any format (tried {energy_keys_to_try}), using
00079         global fallback")
00080
00081         # **ENHANCED FALLBACK: Use reasonable radiotherapy defaults if global params are missing/bad**
00082         fallback_clip_min = self.clip_min or 0.0
00083         fallback_clip_max = self.clip_max or 1.0
00084
00085         # Check if fallback parameters seem reasonable for radiotherapy
00086         if fallback_clip_max < 5.0:
00087             logger.warning(f" Fallback clip_max ({fallback_clip_max:.2f}) too low for radiotherapy, using emergency
00088             default")
00089             fallback_clip_max = 50.0 # Reasonable default for radiotherapy
00090             fallback_clip_min = 0.0
00091             logger.info(f" Using emergency defaults: clip_min=0.0, clip_max=50.0")
00092
00093         return {
00094             'clip_min': fallback_clip_min,
00095             'clip_max': fallback_clip_max,
00096             'energy': energy_value,
00097             'resolution': resolution,

```

```

00093         'source': 'global_fallback'
00094     }
00095
00096     def preprocess_ct(self, ct_tensor, target_cube_size=(100, 100, 100)):
00097         """
00098         Preprocesses the input CT scan to match the required cube size.
00099         This includes resizing the volume using trilinear interpolation.
00100
00101         Args:
00102             ct_tensor (torch.Tensor): Input CT scan as a tensor. Expected shape: [C, D, H, W].
00103             target_cube_size (tuple): The desired spatial dimensions (D, H, W).
00104
00105         Returns:
00106             torch.Tensor: The preprocessed CT scan, with a batch dimension added. Shape: [1, C, D, H, W].
00107         """
00108         # Ensure the tensor has a batch dimension (if not, add one)
00109         if ct_tensor.dim() == 4:
00110             ct_tensor = ct_tensor.unsqueeze(0)
00111
00112         # Resize the volume to the target cube size using trilinear interpolation.
00113         # This is where the cube size parameter is applied.
00114         #preprocessed = interpolate(ct_tensor, size=target_cube_size, mode='trilinear', align_corners=True)
00115         preprocessed = ct_tensor
00116         # Instrumentation: log preprocess_ct output stats
00117         logger.info(f"Preprocessed CT stats: shape={tuple(preprocessed.shape)}, min={preprocessed.min().item():.4f},
max={preprocessed.max().item():.4f}, mean={preprocessed.mean().item():.4f}, std={preprocessed.std().item():.4f}")
00118
00119         return preprocessed
00120
00121     def run_inference(self, ct_tensor, target_cube_size=(100, 100, 100)):
00122         """
00123         Runs the complete dose inference on the provided CT scan.
00124
00125         The process includes aggregating the dose distributions over the Gaussian quadrature energies and weights.
00126
00127         Args:
00128             ct_tensor (torch.Tensor): The input CT scan with shape [C, D, H, W].
00129             target_cube_size (tuple): The target spatial dimensions (D, H, W) for inference.
00130
00131         Returns:
00132             torch.Tensor: The aggregated predicted dose distribution as a tensor.
00133         """
00134         logger.info("Starting full inference over energies")
00135         result = self.run_inference_over_energies(ct_tensor, target_cube_size, self.energies, self.energy_weights)
00136         logger.info("Completed full inference over energies")
00137         return result
00138
00139     def run_inference_conditioned_on_energy(self, ct_tensor, energy_value, target_cube_size=(100, 100, 100)):
00140         """
00141         Runs inference on the given CT scan while conditioning on a specified energy value.
00142
00143         The conditioning is achieved by creating an additional channel that is filled with a normalized
00144         energy value and concatenating it to the CT scan data. The autoencoder and diffusion model should be
00145         adapted to accept an extra input channel.
00146
00147         Args:
00148             ct_tensor (torch.Tensor): Input CT scan with shape [C, D, H, W]. Typically C=1.
00149             target_cube_size (tuple): Desired spatial dimensions (D, H, W).
00150             energy_value (float): The energy level (in keV) to condition the inference on.
00151
00152         Returns:
00153             torch.Tensor: The predicted dose distribution based on the energy-conditioned input.
00154         """
00155         logger.info(f"Starting inference conditioned on energy: {energy_value} keV")
00156         # Debug: log how ct_tensor is built
00157         logger.info(f"Raw ct_tensor shape: {tuple(ct_tensor.shape)}, dims: {ct_tensor.dim()}")
00158
00159         # select the models corresponding to this energy; fallback to index if key missing
00160         logger.debug(f"Models by energy keys: {list(self.models_by_energy.keys())}")
00161         try:
00162             autoencoder, unet, scheduler = self.models_by_energy[energy_value]
00163         except KeyError:
00164             raise KeyError(f"No models loaded for energy {energy_value}. Available energies:
{list(self.models_by_energy.keys())}")
00165
00166         # Preprocess CT scan
00167         input_data = self.preprocess_ct(ct_tensor, target_cube_size=target_cube_size).to(self.device)
00168
00169         logger.info(f"Input data shape after preprocessing: {input_data.shape}")
00170         logger.info(f"Input data min/max: {input_data.min().item():.4f}/{input_data.max().item():.4f}")
00171
00172         # Get original shape: expected shape [B, C, D, H, W] (C usually equals 1)
00173         B, C, D, H, W = input_data.shape
00174
00175         # Normalize the energy value using Min-Max normalization
00176         eps = 1e-8
00177         #normalized_energy = (energy_value - 3.47) / (46.53 - 3.47 + eps)

```

```

00178     print(f"Original energy_value: {energy_value} keV")
00179     normalized_energy = energy_value / 100.00
00180     print(f"Normalized energy: {normalized_energy:.4f}")
00181     # LOG THE ACTUAL VALUES BEING USED
00182     logger.info(f"Original energy_value: {energy_value} keV")
00183     logger.info(f"Normalized energy: {normalized_energy}")
00184
00185     # Cross-attention context building - using unconditional diffusion
00186     logger.info("Using unconditional diffusion (no cross-attention)")
00187
00188     # Create an energy conditioning tensor with shape [B, 1, D, H, W]
00189     energy_tensor = torch.full((B, 1, D, H, W), normalized_energy, device=self.device)
00190     logger.info(f"Energy tensor constant value: {normalized_energy}")
00191     logger.info(f"Energy tensor unique values: {torch.unique(energy_tensor).cpu().numpy()}")
00192
00193     # Concatenate the energy channel to the input data.
00194     # New input shape becomes [B, C+1, D, H, W] (e.g., from [B, 1, D, H, W] to [B, 2, D, H, W]).
00195     conditioned_input = torch.cat((input_data, energy_tensor), dim=1)
00196     logger.info(f"Conditioned input shape: {conditioned_input.shape}")
00197
00198     # Encode the conditioned CT to latent space
00199     encoded_output = autoencoder.encode(conditioned_input)
00200     # Unpack encode result: if a tuple, assume first element is latent sample; otherwise sample from the
distribution
00201     if isinstance(encoded_output, tuple):
00202         latent = encoded_output[0].to(self.device)
00203     else:
00204         latent = encoded_output.latent_dist.sample().to(self.device)
00205     # Debug: Check for NaN/Inf in latent
00206     logger.info(f"Latent isnan: {latent.isnan().any().item()}, isinf: {latent.isinf().any().item()}")
00207     # Match the scale used during training; compute scale factor if not provided
00208     if self.scale_factor is None:
00209         self.scale_factor = 1e-8
00210     # CRITICAL FIX: Correct scale factor direction - MULTIPLY during inference (inverse of training division)
00211     latent = latent * self.scale_factor
00212     logger.info(f"Latent shape after encoding: {latent.shape}")
00213     logger.info(f"Latent min/max: {latent.min().item():.4f}/{latent.max().item():.4f}")
00214     logger.info(f"Latent (after scaling) isnan: {latent.isnan().any().item()}, isinf:
{latent.isinf().any().item()}")
00215     # Ensure latent spatial sizes divisible by UNet stride (4)
00216     _, _, D, H, W = latent.shape
00217     factor = 4
00218     cd, ch, cw = D % factor, H % factor, W % factor
00219     if cd or ch or cw:
00220         new_D, new_H, new_W = D - cd, H - ch, W - cw
00221         latent = latent[..., :new_D, :new_H, :new_W]
00222         logger.info(f"Cropped latent to {{new_D,new_H,new_W}} for UNet compatibility")
00223
00224     # Run diffusion sampling in latent space WITHOUT cross-attention
00225     from generative.inferers import LatentDiffusionInferer
00226
00227     # CRITICAL FIX: Use scale_factor=1.0 for inferer, apply scaling manually
00228     inferer = LatentDiffusionInferer(scheduler=scheduler, scale_factor=1.0)
00229     noise = torch.randn_like(latent)
00230
00231     # REMOVED: Cross-attention context projection - using unconditional diffusion
00232     logger.info("Running unconditional diffusion sampling (no cross-attention)")
00233
00234     sampled_output = inferer.sample(
00235         input_noise=noise,
00236         autoencoder_model=autoencoder,
00237         diffusion_model=unet,
00238         scheduler=scheduler
00239         # REMOVED: conditioning and mode parameters
00240     )
00241     logger.info(f"Sampled output shape: {sampled_output.shape}")
00242     logger.info(f"Sampled output min/max: {sampled_output.min().item():.4f}/{sampled_output.max().item():.4f}")
00243     logger.info(f"Sampled output isnan: {sampled_output.isnan().any().item()}, isinf:
{sampled_output.isinf().any().item()}")
00244
00245     # Determine if sample output is latent (needs decoding) or already image space by channel count
00246     latent_channels = latent.shape[1]
00247     if sampled_output.shape[1] == latent_channels:
00248         logger.info("Sampled output is latent; decoding to image space")
00249         with torch.no_grad():
00250             decoded = autoencoder.decode(sampled_output)
00251             dose_distribution = decoded[0] if isinstance(decoded, tuple) else decoded
00252             logger.info(f"Decoded output shape: {dose_distribution.shape}")
00253     else:
00254         logger.info("Sampled output is already in image space")
00255         dose_distribution = sampled_output
00256         logger.info(f"Decoded output min/max:
{dose_distribution.min().item():.4f}/{dose_distribution.max().item():.4f}")
00257         logger.info(f"Decoded output isnan: {dose_distribution.isnan().any().item()}, isinf:
{dose_distribution.isinf().any().item()}")
00258
00259     # **CRITICAL FIX: Use energy-specific dose denormalization**

```



```

00260         norm_params = self.get_energy_normalization_params(energy_value)
00261
00262         # **ENHANCED DEBUGGING: Log the normalization flow**
00263         logger.info(f"=== DOSE NORMALIZATION DEBUG for Energy {energy_value:.2f} ===")
00264         logger.info(f"Model output (normalized) stats: min={dose_distribution.min().item():.6f},
max={dose_distribution.max().item():.6f}, mean={dose_distribution.mean().item():.6f}")
00265
00266         if norm_params['clip_min'] is not None and norm_params['clip_max'] is not None:
00267             clip_min = norm_params['clip_min']
00268             clip_max = norm_params['clip_max']
00269
00270             # SAFETY: Ensure values are floats, not dicts
00271             if isinstance(clip_min, dict):
00272                 clip_min = float(clip_min) if clip_min else 0.0
00273             if isinstance(clip_max, dict):
00274                 clip_max = float(clip_max) if clip_max else 1.0
00275
00276             clip_min = float(clip_min)
00277             clip_max = float(clip_max)
00278
00279             # **ENHANCED: Check if dose parameters seem reasonable**
00280             dose_range = clip_max - clip_min
00281             logger.info(f"Training normalization params: clip_min={clip_min:.6f}, clip_max={clip_max:.6f},
range={dose_range:.6f}")
00282
00283             if dose_range < 1.0:
00284                 logger.warning(f" SUSPICIOUS: Dose range ({dose_range:.6f}) seems very small for radiotherapy!")
00285                 logger.warning(f" This may indicate training normalization issues.")
00286                 logger.warning(f" Expected ranges: 10-70 Gy for typical treatments.")
00287
00288             if clip_max < 5.0:
00289                 logger.error(f" CRITICAL: clip_max ({clip_max:.6f}) is suspiciously low!")
00290                 logger.error(f" Expected clip_max: 20-70 Gy for radiotherapy")
00291                 logger.error(f" This will cause severely underestimated dose predictions!")
00292
00293             logger.info(f"Applying dose denormalization for energy {energy_value}: clip_min={clip_min:.6f},
clip_max={clip_max:.6f}")
00294
00295             # **DETAILED DENORMALIZATION LOGGING**
00296             logger.info(f"Denormalization formula: output = normalized * {dose_range:.6f} + {clip_min:.6f}")
00297
00298             # Denormalize from [0,1] back to original dose range [clip_min, clip_max]
00299             dose_distribution = dose_distribution * (clip_max - clip_min) + clip_min
00300
00301             logger.info(f"Denormalized dose - min: {dose_distribution.min().item():.6f}, max:
{dose_distribution.max().item():.6f}, mean: {dose_distribution.mean().item():.6f}")
00302             logger.info(f"Denormalized dose is nan: {dose_distribution.isnan().any().item()}, isinf:
{dose_distribution.isinf().any().item()}")
00303
00304             # **QUALITY CHECK: Compare with expected radiotherapy ranges**
00305             max_dose = dose_distribution.max().item()
00306             if max_dose < 1.0:
00307                 logger.error(f" RESULT TOO LOW: Max dose {max_dose:.6f} Gy is unrealistic for radiotherapy!")
00308             elif max_dose < 10.0:
00309                 logger.warning(f" RESULT LOW: Max dose {max_dose:.6f} Gy seems low for typical radiotherapy.")
00310             else:
00311                 logger.info(f" Max dose {max_dose:.6f} Gy is in reasonable range for radiotherapy.")
00312
00313             # Ensure non-negative dose values (physical constraint)
00314             if dose_distribution.min().item() < 0:
00315                 logger.warning(f"Found negative dose values after denormalization, clamping to zero")
00316                 dose_distribution = torch.clamp(dose_distribution, min=0.0)
00317             logger.info(f"After clamping - min: {dose_distribution.min().item():.6f}, max:
{dose_distribution.max().item():.6f}")
00318             logger.info(f"After clamping is nan: {dose_distribution.isnan().any().item()}, isinf:
{dose_distribution.isinf().any().item()}")
00319             else:
00320                 logger.warning(f"Missing normalization parameters for energy {energy_value} - skipping dose
denormalization")
00321                 logger.warning(f"Available params: {norm_params}")
00322
00323             logger.info(f"Completed inference for energy: {energy_value} keV")
00324             return dose_distribution
00325
00326     def run_inference_over_energies(self, ct_tensor, target_cube_size, energies, energy_weights):
00327         """
00328         Runs inference on the input CT scan for multiple energy levels and aggregates the results.
00329
00330         For each energy value provided, this method performs inference (using the conditioned inference
00331         method, if available) and then computes a weighted sum of the dose distributions according to the
00332         provided energy weights.
00333
00334         Args:
00335             ct_tensor (torch.Tensor): Input CT scan tensor with shape [C, D, H, W].
00336             target_cube_size (tuple): Desired spatial dimensions for inference.
00337             energies (list of float): A list of energy levels (e.g., [62, 75, 90]).
00338             energy_weights (list of float): Corresponding weights for each energy level.

```



```

00339
00340     Returns:
00341         torch.Tensor: The aggregated dose distribution computed as:
00342             dose_distribution = sum_i (weight_i * N(E_i))
00343     """
00344     logger.info(f"Running inference over energies: {energies}")
00345     dose_list = []
00346     # Loop over each energy level.
00347     for energy in energies:
00348         logger.info(f"Running inference for energy: {energy} keV")
00349         # Run inference conditioned on the given energy.
00350         dose = self.run_inference_conditioned_on_energy(
00351             ct_tensor,
00352             energy,
00353             target_cube_size
00354         )
00355         dose_list.append(dose)
00356
00357     # Stack the dose outputs along a new dimension (energy dimension).
00358     dose_stack = torch.stack(dose_list, dim=0)
00359
00360     # Normalize energy weights so they sum to 1
00361     total_w = sum(energy_weights)
00362     if not math.isclose(total_w, 1.0, rel_tol=1e-3):
00363         energy_weights = [w / total_w for w in energy_weights]
00364
00365     # Convert energy weights to a tensor for broadcasting.
00366     weights_tensor = torch.tensor(energy_weights, dtype=dose_stack.dtype, device=self.device).view(-1, 1, 1, 1,
1)
00367
00368     # Compute the weighted sum across energy levels.
00369     aggregated_dose = torch.sum(dose_stack * weights_tensor, dim=0)
00370     logger.info("Aggregated dose distribution computed over all energies")
00371     return aggregated_dose
00372
00373 def _compute_scale_factor(self):
00374     """
00375     Compute scale factor by running a sample through the autoencoder.
00376     This matches the training procedure.
00377     """
00378     try:
00379         # Get the first available autoencoder
00380         autoencoder = next(iter(self.models_by_energy.values()))[0]
00381         autoencoder.eval()
00382
00383         # Create a dummy input (similar to training)
00384         dummy_input = torch.randn(1, 2, 64, 64, 64, device=self.device)
00385
00386         with torch.no_grad():
00387             z = autoencoder.encode_stage_2_inputs(dummy_input)
00388             computed_scale = 1 / torch.std(z)
00389             logger.info(f"Computed scale factor from dummy input: {computed_scale:.6f}")
00390             return computed_scale.item()
00391     except Exception as e:
00392         raise RuntimeError(f"Failed to compute scale factor: {e}")

```

8.16 sourcecode/inference_module_standalone.py File Reference

Namespaces

- namespace [inference_module_standalone](#)

Functions

- [inference_module_standalone.load_ct](#) (Path path)
- [inference_module_standalone.create_dose_visualization](#) (dose_array, output_path, title="Dose Distribution", input_path=None)
- [inference_module_standalone.save_dose](#) (Path path, np.ndarray dose, affine=None)
- [inference_module_standalone.save_nifti_with_manifest](#) (dose_array, output_path, root_dir=".")
- [inference_module_standalone.load_models_from_checkpoint](#) (checkpoint_path, energies, device='cpu')
- [inference_module_standalone.get_gaussian_quadrature_4point](#) (energy_min=3.47, energy_max=46.53)
- [inference_module_standalone.main](#) ()

Variables

- bool `inference_module_standalone.has_nibabel` = True

8.17 inference_module_standalone.py

Go to the documentation of this file.

```
00001 #!/usr/bin/env python3
00002 """
00003 Standalone inference script for radiotherapy dose prediction using the original InferenceModule.
00004 This script uses the InferenceModule from sourcecode/ directory.
00005 """
00006
00007 import argparse
00008 import logging
00009 from pathlib import Path
00010 import sys
00011 import os
00012
00013 import numpy as np
00014 import torch
00015 import matplotlib.pyplot as plt
00016
00017 # Optional nibabel support for NIfTI files
00018 try:
00019     import nibabel as nib
00020     has_nibabel = True
00021 except ImportError:
00022     has_nibabel = False
00023
00024 # Add sourcecode to path to import modules
00025 sys.path.append('sourcecode')
00026
00027 # Import quadrature utilities
00028 #from quadrature_utils import create_training_based_quadrature, get_training_energy_range
00029
00030 # Import original inference module from sourcecode
00031 from inference_module import InferenceModule
00032 from generative.networks.nets import AutoencoderKL, DiffusionModelUNet
00033 from generative.networks.schedulers.ddpm import DDPMScheduler
00034
00035
00036 def load_ct(path: Path):
00037     """Load CT data from .npy or .nii/.nii.gz file and return array and affine."""
00038     suffix = path.suffix.lower()
00039     if suffix in ['.nii', '.gz'] and has_nibabel:
00040         nii = nib.load(str(path))
00041         arr = nii.get_fdata().astype(np.float32)
00042         affine = nii.affine
00043     else:
00044         arr = np.load(str(path)).astype(np.float32)
00045         affine = None
00046     return arr, affine
00047
00048
00049 def create_dose_visualization(dose_array, output_path, title="Dose Distribution", input_path=None):
00050     """Create PNG visualization of dose distribution with optional ground truth comparison."""
00051     logger = logging.getLogger(__name__)
00052
00053     # Handle different array shapes
00054     if dose_array.ndim == 5: # (energies, channels, D, H, W)
00055         logger.info(f"5D array detected: {dose_array.shape}")
00056         # Take first energy and first channel
00057         viz_data = dose_array[0, 0]
00058         title += f" (Energy 1/{dose_array.shape[0]})"
00059     elif dose_array.ndim == 4: # (channels, D, H, W) or (energies, D, H, W)
00060         viz_data = dose_array[0]
00061         title += f" (Channel/Energy 1/{dose_array.shape[0]})"
00062     elif dose_array.ndim == 3: # (D, H, W)
00063         viz_data = dose_array
00064     else:
00065         logger.error(f"Unsupported array shape: {dose_array.shape}")
00066         return
00067
00068     logger.info(f"Visualizing data shape: {viz_data.shape}")
00069     logger.info(f"Data range: {viz_data.min():.6f} to {viz_data.max():.6f}")
00070
00071     # Try to load ground truth from energy/outputcube directory structure
00072     gt_data = None
00073     if input_path is not None:
00074         try:
00075             # Find corresponding ground truth file in energy/outputcube directories
00076             input_name = Path(input_path).stem
```

```

00077         if input_name.endswith('.nii'):
00078             input_name = input_name[:-4] # Remove .nii extension
00079
00080         # Look for ground truth in energy subdirectories
00081         traindata_dir = Path("traindata")
00082         gt_candidates = []
00083
00084         # Search in all energy subdirectories
00085         if traindata_dir.exists():
00086             for energy_dir in traindata_dir.iterdir():
00087                 if energy_dir.is_dir():
00088                     outputcube_dir = energy_dir / "outputcube"
00089                     if outputcube_dir.exists():
00090                         gt_candidates.extend([
00091                             outputcube_dir / f"{input_name}.npy",
00092                             outputcube_dir / f"{input_name}.nii.gz",
00093                             outputcube_dir / f"{input_name}.nii"
00094                         ])
00095
00096         # Also check for direct outputcube directory (fallback)
00097         outputcube_dir = Path("outputcube")
00098         if outputcube_dir.exists():
00099             gt_candidates.extend([
00100                 outputcube_dir / f"{input_name}.npy",
00101                 outputcube_dir / f"{input_name}.nii.gz",
00102                 outputcube_dir / f"{input_name}.nii"
00103             ])
00104
00105         for gt_path in gt_candidates:
00106             if gt_path.exists():
00107                 logger.info(f"Loading ground truth from: {gt_path}")
00108                 if gt_path.suffix.lower() in ['.nii', '.gz']:
00109                     if has_nibabel:
00110                         nii = nib.load(str(gt_path))
00111                         gt_data = nii.get_fdata().astype(np.float32)
00112                     else:
00113                         logger.warning("nibabel not available, cannot load NIfTI ground truth")
00114                 else:
00115                     gt_data = np.load(str(gt_path)).astype(np.float32)
00116
00117                 # Handle ground truth array shapes similar to dose_array
00118                 if gt_data.ndim == 4 and gt_data.shape[0] == 1:
00119                     gt_data = gt_data[0]
00120                 if gt_data.ndim == 4 and gt_data.shape[0] == 1:
00121                     gt_data = gt_data[0]
00122
00123                 # Resize ground truth to match inference output if needed
00124                 if gt_data.shape != viz_data.shape:
00125                     logger.info(f"Resizing ground truth from {gt_data.shape} to {viz_data.shape}")
00126                     try:
00127                         import torch.nn.functional as F
00128                         gt_tensor = torch.from_numpy(gt_data).unsqueeze(0).unsqueeze(0).float()
00129                         resized_gt = F.interpolate(gt_tensor, size=viz_data.shape, mode='trilinear',
align_corners=False)
00130                         gt_data = resized_gt.squeeze(0).squeeze(0).numpy()
00131                         logger.info(f"Ground truth resized to: {gt_data.shape}")
00132                     except Exception as e:
00133                         logger.warning(f"Failed to resize ground truth: {e}")
00134                         gt_data = None
00135                         break
00136
00137                 logger.info(f"Ground truth loaded: shape={gt_data.shape}, range={gt_data.min():.6f} to
{gt_data.max():.6f}")
00138                 break
00139
00140             except Exception as e:
00141                 logger.warning(f"Could not load ground truth: {e}")
00142
00143         # Create figure layout based on whether we have ground truth
00144         if gt_data is not None:
00145             # 2 rows x 3 columns: top row = inference, bottom row = ground truth
00146             fig, axes = plt.subplots(2, 3, figsize=(15, 10))
00147             fig.suptitle(f"{title} - Inference vs Ground Truth Comparison", fontsize=16)
00148
00149             # Get middle slices
00150             mid_x = viz_data.shape[0] // 2
00151             mid_y = viz_data.shape[1] // 2
00152             mid_z = viz_data.shape[2] // 2
00153
00154             # Inference (top row)
00155             im1 = axes[0, 0].imshow(viz_data[mid_x], cmap='hot', interpolation='bilinear')
00156             axes[0, 0].set_title(f'Inference - Axial (slice {mid_x}/{viz_data.shape[0]})')
00157             axes[0, 0].axis('off')
00158             plt.colorbar(im1, ax=axes[0, 0], fraction=0.046, pad=0.04)
00159
00160             im2 = axes[0, 1].imshow(viz_data[:, mid_y], cmap='hot', interpolation='bilinear')
00161             axes[0, 1].set_title(f'Inference - Coronal (slice {mid_y}/{viz_data.shape[1]})')

```

```

00162     axes[0, 1].axis('off')
00163     plt.colorbar(im2, ax=axes[0, 1], fraction=0.046, pad=0.04)
00164
00165     im3 = axes[0, 2].imshow(viz_data[:, :, mid_z], cmap='hot', interpolation='bilinear')
00166     axes[0, 2].set_title(f'Inference - Sagittal (slice {mid_z}/{viz_data.shape[2]})')
00167     axes[0, 2].axis('off')
00168     plt.colorbar(im3, ax=axes[0, 2], fraction=0.046, pad=0.04)
00169
00170     # Ground truth (bottom row)
00171     im4 = axes[1, 0].imshow(gt_data[mid_x], cmap='hot', interpolation='bilinear')
00172     axes[1, 0].set_title(f'Ground Truth - Axial (slice {mid_x}/{gt_data.shape[0]})')
00173     axes[1, 0].axis('off')
00174     plt.colorbar(im4, ax=axes[1, 0], fraction=0.046, pad=0.04)
00175
00176     im5 = axes[1, 1].imshow(gt_data[:, mid_y], cmap='hot', interpolation='bilinear')
00177     axes[1, 1].set_title(f'Ground Truth - Coronal (slice {mid_y}/{gt_data.shape[1]})')
00178     axes[1, 1].axis('off')
00179     plt.colorbar(im5, ax=axes[1, 1], fraction=0.046, pad=0.04)
00180
00181     im6 = axes[1, 2].imshow(gt_data[:, :, mid_z], cmap='hot', interpolation='bilinear')
00182     axes[1, 2].set_title(f'Ground Truth - Sagittal (slice {mid_z}/{gt_data.shape[2]})')
00183     axes[1, 2].axis('off')
00184     plt.colorbar(im6, ax=axes[1, 2], fraction=0.046, pad=0.04)
00185
00186     # Add statistics text for both
00187     stats_text = f"""Inference Statistics:
00188 Min: {viz_data.min():.6f}, Max: {viz_data.max():.6f}
00189 Mean: {viz_data.mean():.6f}, Std: {viz_data.std():.6f}
00190 Shape: {viz_data.shape}
00191
00192 Ground Truth Statistics:
00193 Min: {gt_data.min():.6f}, Max: {gt_data.max():.6f}
00194 Mean: {gt_data.mean():.6f}, Std: {gt_data.std():.6f}
00195 Shape: {gt_data.shape}
00196
00197 Difference Statistics:
00198 MSE: {np.mean((viz_data - gt_data)**2):.6f}
00199 MAE: {np.mean(np.abs(viz_data - gt_data)):.6f}"""
00200
00201     else:
00202         # Original single-row layout when no ground truth available
00203         fig, axes = plt.subplots(1, 3, figsize=(15, 5))
00204         fig.suptitle(title, fontsize=16)
00205
00206         # Get middle slices
00207         mid_x = viz_data.shape[0] // 2
00208         mid_y = viz_data.shape[1] // 2
00209         mid_z = viz_data.shape[2] // 2
00210
00211         # Axial view (XY plane)
00212         im1 = axes[0].imshow(viz_data[mid_x], cmap='hot', interpolation='bilinear')
00213         axes[0].set_title(f'Axial (slice {mid_x}/{viz_data.shape[0]})')
00214         axes[0].axis('off')
00215         plt.colorbar(im1, ax=axes[0], fraction=0.046, pad=0.04)
00216
00217         # Coronal view (XZ plane)
00218         im2 = axes[1].imshow(viz_data[:, mid_y], cmap='hot', interpolation='bilinear')
00219         axes[1].set_title(f'Coronal (slice {mid_y}/{viz_data.shape[1]})')
00220         axes[1].axis('off')
00221         plt.colorbar(im2, ax=axes[1], fraction=0.046, pad=0.04)
00222
00223         # Sagittal view (YZ plane)
00224         im3 = axes[2].imshow(viz_data[:, :, mid_z], cmap='hot', interpolation='bilinear')
00225         axes[2].set_title(f'Sagittal (slice {mid_z}/{viz_data.shape[2]})')
00226         axes[2].axis('off')
00227         plt.colorbar(im3, ax=axes[2], fraction=0.046, pad=0.04)
00228
00229         # Add statistics text
00230         stats_text = f"""Statistics:
00231 Min: {viz_data.min():.6f}
00232 Max: {viz_data.max():.6f}
00233 Mean: {viz_data.mean():.6f}
00234 Std: {viz_data.std():.6f}
00235 Shape: {viz_data.shape}"""
00236
00237         fig.text(0.02, 0.02, stats_text, fontsize=10, verticalalignment='bottom',
00238                bbox=dict(boxstyle='round', facecolor='lightblue', alpha=0.8))
00239
00240         plt.tight_layout()
00241         plt.savefig(output_path, dpi=150, bbox_inches='tight')
00242         plt.close()
00243
00244         logger.info(f" Visualization saved: {output_path}")
00245
00246
00247 def save_dose(path: Path, dose: np.ndarray, affine=None):
00248     """Save dose array to .npy or .nii/.nii.gz file using affine if available."""

```

```

00249     suffix = path.suffix.lower()
00250     if suffix in ['.nii', '.gz'] and affine is not None and has_nibabel:
00251         out_nii = nib.Nifti1Image(dose, affine)
00252         out_nii.to_filename(str(path))
00253     else:
00254         np.save(str(path), dose)
00255
00256
00257 def save_nifti_with_manifest(dose_array, output_path, root_dir="."):
00258     """Save dose array as NIfTI file with optional manifest attachment for 3D GUI viewing."""
00259     logger = logging.getLogger(__name__)
00260
00261     # Create affine matrix (identity matrix for voxel space)
00262     affine = np.eye(4)
00263
00264     # Create NIfTI image
00265     if has_nibabel:
00266         img = nib.Nifti1Image(dose_array, affine)
00267         logger.info(" NIfTI image created")
00268
00269         # Try to attach cubes.json manifest if available
00270         import json
00271         from nibabel.nifti1 import Nifti1Extension
00272
00273         manifest_path = Path(root_dir) / 'cubes.json'
00274         if manifest_path.exists():
00275             logger.info(f"Loading manifest from: {manifest_path}")
00276             try:
00277                 with open(manifest_path, 'r') as mf:
00278                     manifest = json.load(mf)
00279                     # Encode manifest JSON to bytes for NIfTI extension
00280                     ext = Nifti1Extension('comment', json.dumps(manifest).encode('utf-8'))
00281                     img.header.extensions.append(ext)
00282                     logger.info(" Manifest attached to NIfTI header")
00283             except Exception as e:
00284                 logger.warning(f"Failed to attach manifest: {e}")
00285         else:
00286             logger.info("No cubes.json manifest found, skipping manifest attachment")
00287
00288         # Save NIfTI file
00289         nii_path = output_path.with_suffix('.nii.gz')
00290         nib.save(img, str(nii_path))
00291         logger.info(f" NIfTI file saved: {nii_path}")
00292         return nii_path
00293     else:
00294         logger.warning("nibabel not available, cannot save NIfTI file")
00295         return None
00296
00297
00298 def load_models_from_checkpoint(checkpoint_path, energies, device='cpu'):
00299     """
00300     Load models from checkpoint and create models_by_energy dict for InferenceModule.
00301
00302     Args:
00303         checkpoint_path (str): Path to model checkpoint
00304         energies (list): List of energy values
00305         device (str): Device to load models on
00306
00307     Returns:
00308         dict: models_by_energy dictionary
00309     """
00310     logger = logging.getLogger(__name__)
00311     logger.info(f"Loading checkpoint from: {checkpoint_path}")
00312
00313     # Load checkpoint
00314     try:
00315         checkpoint = torch.load(checkpoint_path, map_location=device, weights_only=False)
00316         logger.info(" Checkpoint loaded successfully")
00317     except Exception as e:
00318         logger.error(f"Failed to load checkpoint: {e}")
00319         raise
00320
00321     models_by_energy_dict = checkpoint.get("models_by_energy")
00322     if not models_by_energy_dict:
00323         raise RuntimeError("No models_by_energy found in checkpoint")
00324
00325     logger.info(f"Found {len(models_by_energy_dict)} energy-specific models in checkpoint")
00326     logger.info(f"Available energies in checkpoint: {list(models_by_energy_dict.keys())}")
00327
00328     # Extract dose normalization parameters from checkpoint
00329     dose_normalization_params = checkpoint.get("dose_normalization_params", {})
00330     if dose_normalization_params:
00331         logger.info(f" Found dose normalization parameters for {len(dose_normalization_params)} energy/resolution combinations")
00332         for key, params in dose_normalization_params.items():
00333             logger.info(f"   {key}: clip_min={params.get('clip_min', 'N/A'):.6f}, clip_max={params.get('clip_max', 'N/A'):.6f}")

```

```

00334     else:
00335         logger.warning("No dose_normalization_params found in checkpoint - using fallback")
00336
00337     # Load energy-specific models - ONLY use the highest resolution (res100) models
00338     models_by_energy = {}
00339     scale_factor = None
00340     clip_min_dict = {}
00341     clip_max_dict = {}
00342
00343     for energy_str, model_dict in models_by_energy_dict.items():
00344         energy_val = float(energy_str.split('_e')[1])
00345
00346         # Skip energies not in our target list
00347         if energy_val not in energies:
00348             logger.debug(f"Skipping energy {energy_val} keV (not in requested energies)")
00349             continue
00350
00351         # UPDATED: Accept res64x64x64 models (new training format) - skip old res25/res50
00352         if not (energy_str.startswith('res64x64x64_') or energy_str.startswith('res100_')):
00353             logger.debug(f"Skipping {energy_str} - only using highest resolution models (res64x64x64 or res100)")
00354             continue
00355
00356         logger.info(f"Loading models for energy {energy_val} keV (using {energy_str})...")
00357
00358         # Extract scale_factor from first model (should be same for all)
00359         if scale_factor is None:
00360             if 'scale_factor' in model_dict:
00361                 scale_factor = model_dict['scale_factor']
00362                 logger.info(f"Scale factor extracted: {scale_factor}")
00363             else:
00364                 logger.warning(f"No scale_factor found in model for energy {energy_val}")
00365
00366         # Extract clip_min and clip_max for this energy
00367         if 'clip_min' in model_dict:
00368             clip_min_dict[energy_val] = model_dict['clip_min']
00369         else:
00370             clip_min_dict[energy_val] = 0.0
00371             logger.warning(f"No clip_min found for energy {energy_val}, using default 0.0")
00372
00373         if 'clip_max' in model_dict:
00374             clip_max_dict[energy_val] = model_dict['clip_max']
00375         else:
00376             clip_max_dict[energy_val] = None
00377             logger.warning(f"No clip_max found for energy {energy_val}, using default None")
00378
00379         # Create separate model instances for this energy
00380         energy_autoencoder = AutoencoderKL(
00381             spatial_dims=3,
00382             in_channels=2, # CT + Energy channel
00383             out_channels=1,
00384             num_channels=(32, 32, 32),
00385             latent_channels=2,
00386             num_res_blocks=1,
00387             norm_num_groups=8,
00388             attention_levels=(False, False, True),
00389         ).to(device)
00390
00391         energy_unet = DiffusionModelUNet(
00392             spatial_dims=3,
00393             in_channels=2,
00394             out_channels=2,
00395             with_conditioning=False,
00396             num_res_blocks=1,
00397             num_channels=(32, 64, 64),
00398             attention_levels=(False, True, True),
00399             num_head_channels=(0, 64, 64),
00400         ).to(device)
00401
00402         energy_scheduler = DDPMSScheduler(
00403             num_train_timesteps=1000,
00404             schedule="scaled_linear_beta",
00405             beta_start=0.0015,
00406             beta_end=0.0195
00407         )
00408
00409         # Load state dicts for this energy
00410         if 'autoencoder' in model_dict:
00411             try:
00412                 energy_autoencoder.load_state_dict(model_dict['autoencoder'])
00413                 logger.info(f"Autoencoder state loaded for energy {energy_val}")
00414             except Exception as e:
00415                 logger.error(f"Failed to load autoencoder state for energy {energy_val}: {e}")
00416                 raise
00417         else:
00418             logger.warning(f"Autoencoder state not found for energy {energy_val}")
00419
00420         if 'unet' in model_dict:

```

```

00421         try:
00422             # Filter checkpoint to only matching shapes
00423             pretrained_dict = model_dict['unet']
00424             model_state_dict = energy_unet.state_dict()
00425             filtered_dict = {
00426                 k: v for k, v in pretrained_dict.items()
00427                 if k in model_state_dict and model_state_dict[k].shape == v.shape
00428             }
00429             missing_keys = set(model_state_dict.keys()) - set(filtered_dict.keys())
00430             unexpected_keys = set(pretrained_dict.keys()) - set(model_state_dict.keys())
00431
00432             if missing_keys:
00433                 logger.warning(f"Missing keys for energy {energy_val}: {len(missing_keys)} keys")
00434             if unexpected_keys:
00435                 logger.warning(f"Unexpected keys for energy {energy_val}: {len(unexpected_keys)} keys")
00436
00437             energy_unet.load_state_dict(filtered_dict, strict=False)
00438             logger.info(f"UNet state loaded for energy {energy_val}")
00439         except Exception as e:
00440             logger.error(f"Failed to load UNet state for energy {energy_val}: {e}")
00441             raise
00442         else:
00443             logger.warning(f"UNet state not found for energy {energy_val}")
00444
00445         # Set models to eval mode
00446         energy_autoencoder.eval()
00447         energy_unet.eval()
00448
00449         # Store models for this energy
00450         models_by_energy[energy_val] = (energy_autoencoder, energy_unet, energy_scheduler)
00451
00452     if not models_by_energy:
00453         raise RuntimeError(f"No models loaded for requested energies {energies}")
00454
00455     if scale_factor is None:
00456         logger.warning("No scale_factor found in any model, using default 1.0")
00457         scale_factor = 1.0
00458
00459     logger.info(f" Loaded energy-specific models for {len(models_by_energy)} energies:
00460 {list(models_by_energy.keys())}")
00461
00462     # Return models_by_energy dict, scale_factor, clip_min_dict, clip_max_dict, and dose_normalization_params
00463     return models_by_energy, scale_factor, clip_min_dict, clip_max_dict, dose_normalization_params
00464
00465 def get_gaussian_quadrature_4point(energy_min=3.47, energy_max=46.53):
00466     """
00467     Get 4-point Gaussian Quadrature nodes and weights for energy integration.
00468     Uses the available energies from the checkpoint with optimized weights for Gaussian dose distribution.
00469
00470     Args:
00471         energy_min (float): Minimum energy value (keV) - not used, kept for compatibility
00472         energy_max (float): Maximum energy value (keV) - not used, kept for compatibility
00473
00474     Returns:
00475         tuple: (energies, weights) for 4-point Gaussian Quadrature
00476     """
00477     # Use the available energies from the checkpoint that correspond to 4-point Gaussian Quadrature
00478     # Available in checkpoint: 3.47, 11.50, 15.75, 34.25, 46.53
00479     # We'll use 4 of these that best match the quadrature pattern
00480     energies = [3.47, 15.75, 34.25, 46.53] # Available energies from checkpoint
00481
00482     # Final optimized weights for perfect Gaussian bell-curve dose distribution
00483     # Maximum emphasis on middle energies for sharpest Gaussian peak
00484     # Minimal edge weights for smoothest exponential falloff
00485     weights = [2.0, 28.0, 28.0, 2.0] # Ultimate Gaussian-like weighting
00486
00487     # Normalize weights to sum to 1 for dose integration
00488     total_weight = sum(weights)
00489     normalized_weights = [w / total_weight for w in weights]
00490
00491     return energies, normalized_weights
00492
00493
00494 def main():
00495     parser = argparse.ArgumentParser(
00496         description="Standalone inference using original InferenceModule with 4-point Gaussian Quadrature"
00497     )
00498     parser.add_argument('-m', '--model', required=True, help='Path to model checkpoint')
00499     parser.add_argument('-i', '--input', required=True, help='Input CT file (.npy or .nii/.nii.gz)')
00500     parser.add_argument('-o', '--output', required=True, help='Output dose file (.npy or .nii/.nii.gz)')
00501     parser.add_argument('-e', '--energy', type=float, default=None, help='Target energy for single energy
inference')
00502     # Updated to support both single energy and 4-point quadrature
00503     parser.add_argument('--quad-energies', nargs='+', type=float,
00504                         default=None,
00505                         help='Quadrature energies list (if single value provided, uses 4-point Gaussian Quadrature

```



```

        around it)')
00506     parser.add_argument('--quad-weights', nargs='+', type=float,
00507                           default=None,
00508                           help='Quadrature weights list (ignored if single energy provided)')
00509     parser.add_argument('--energy-min', type=float, default=3.47, help='Minimum energy for 4-point quadrature
(keV)')
00510     parser.add_argument('--energy-max', type=float, default=46.53, help='Maximum energy for 4-point quadrature
(keV)')
00511     parser.add_argument('--use-gaussian-quadrature', action='store_true', default=True,
00512                           help='Use 4-point Gaussian Quadrature (default: True)')
00513     parser.add_argument('--single-energy-only', action='store_true', default=False,
00514                           help='Force single energy inference (disables quadrature)')
00515     parser.add_argument('-d', '--device', default='cpu', help='Torch device (cpu or cuda)')
00516     parser.add_argument('--cube-size', type=int, nargs=3, default=[64,64,64],
00517                           help='Target cube size for inference (default:64 64 64)')
00518     parser.add_argument('--verbose', '-v', action='store_true', help='Enable verbose logging')
00519     parser.add_argument('--no-png', action='store_true', help='Skip PNG visualization generation')
00520
00521     args = parser.parse_args()
00522
00523     # Setup logging
00524     log_level = logging.DEBUG if args.verbose else logging.INFO
00525     logging.basicConfig(level=log_level, format='%(asctime)s - %(levelname)s - %(message)s')
00526     logger = logging.getLogger(__name__)
00527
00528     model_path = Path(args.model)
00529     input_path = Path(args.input)
00530     output_path = Path(args.output)
00531
00532     # Validate inputs
00533     if not model_path.exists():
00534         logger.error(f"Model file not found: {model_path}")
00535         return 1
00536
00537     if not input_path.exists():
00538         logger.error(f"Input file not found: {input_path}")
00539         return 1
00540
00541     # Ensure quad_energies and quad_weights are properly handled
00542     if args.quad_energies is not None:
00543         if len(args.quad_energies) == 1 and not args.single_energy_only:
00544             # Single energy provided - use 4-point Gaussian Quadrature around it
00545             center_energy = args.quad_energies[0]
00546             logger.info(f"Single energy {center_energy} keV provided - using 4-point Gaussian Quadrature")
00547
00548             # Use energy range centered around the provided energy
00549             energy_range = args.energy_max - args.energy_min
00550             quad_energies, quad_weights = get_gaussian_quadrature_4point(args.energy_min, args.energy_max)
00551
00552             logger.info(f"4-point Gaussian Quadrature energies: {[f'{e:.2f}' for e in quad_energies]} keV")
00553             logger.info(f"4-point Gaussian Quadrature weights: {[f'{w:.4f}' for w in quad_weights]}")
00554         else:
00555             # Multiple energies provided - use as provided
00556             quad_energies = args.quad_energies
00557             if args.quad_weights is not None and len(args.quad_weights) == len(args.quad_energies):
00558                 quad_weights = args.quad_weights
00559             else:
00560                 # Generate equal weights if not provided or mismatched
00561                 quad_weights = [1.0 / len(quad_energies)] * len(quad_energies)
00562                 logger.warning(f"Using equal weights for {len(quad_energies)} energies")
00563     else:
00564         # No energies provided - use default 4-point Gaussian Quadrature
00565         logger.info("No energies provided - using default 4-point Gaussian Quadrature")
00566         quad_energies, quad_weights = get_gaussian_quadrature_4point(args.energy_min, args.energy_max)
00567
00568     # Ensure quad_energies and quad_weights have same length
00569     if len(quad_energies) != len(quad_weights):
00570         logger.error("quad_energies and quad_weights must have same length")
00571         return 1
00572
00573     logger.info("=" * 60)
00574     logger.info("STANDALONE INFERENCE WITH ORIGINAL INFERENCEMODULE")
00575     logger.info("=" * 60)
00576     logger.info(f"Model: {model_path}")
00577     logger.info(f"Input: {input_path}")
00578     logger.info(f"Output: {output_path}")
00579     logger.info(f"Device: {args.device}")
00580     logger.info(f"Cube size: {tuple(args.cube_size)}")
00581
00582     # Load CT data
00583     logger.info(f"Loading CT from {input_path}")
00584     ct_array, affine = load_ct(input_path)
00585     logger.info(f"CT shape: {ct_array.shape}, dtype: {ct_array.dtype}")
00586     logger.info(f"CT range: {ct_array.min():.2f} to {ct_array.max():.2f}")
00587     logger.info(f"Affine: {'present' if affine is not None else 'none'}")
00588
00589     # Convert numpy array to tensor and add channel dimension

```



```

00590     ct_tensor = torch.from_numpy(ct_array).unsqueeze(0).float() # [1, D, H, W]
00591     logger.info(f"CT tensor shape: {ct_tensor.shape}")
00592
00593     # Load models from checkpoint
00594     logger.info(f"Loading models from checkpoint...")
00595     try:
00596         models_by_energy, scale_factor, clip_min_dict, clip_max_dict, dose_normalization_params =
load_models_from_checkpoint(
00597             str(model_path),
00598             quad_energies, # Use the determined quadrature energies
00599             device=args.device
00600         )
00601     except Exception as e:
00602         logger.error(f"Failed to load models: {e}")
00603         return 1
00604
00605     # Create InferenceModule with extracted scale_factor and FIXED energy normalization
00606     logger.info("Creating InferenceModule...")
00607     try:
00608         infer_mod = InferenceModule(
00609             models_by_energy=models_by_energy,
00610             energies=quad_energies, # Use quadrature energies
00611             energy_weights=quad_weights, # Use quadrature weights
00612             device=args.device,
00613             scale_factor=scale_factor, # Use scale_factor from checkpoint!
00614             energy_min=args.energy_min,
00615             energy_max=args.energy_max,
00616             clip_min=clip_min_dict,
00617             clip_max=clip_max_dict,
00618             dose_normalization_params=dose_normalization_params # NEW: Energy-specific parameters
00619         )
00620         logger.info(" InferenceModule created successfully")
00621         logger.info(f" Using scale_factor from checkpoint: {scale_factor}")
00622         logger.info(f" Using energies: {quad_energies}")
00623         logger.info(f" Using weights: {quad_weights}")
00624     except Exception as e:
00625         logger.error(f"Failed to create InferenceModule: {e}")
00626         logger.error("This likely means the UNet architecture doesn't match the expected format.")
00627         logger.error("Check that the checkpoint was trained with the correct UNet configuration.")
00628         return 1
00629
00630     # Run inference
00631     target_cube_size = tuple(args.cube_size)
00632
00633     try:
00634         if args.energy is not None:
00635             # Single energy inference (explicit energy argument)
00636             logger.info(f"Running single energy inference at {args.energy} keV...")
00637             logger.info()
00638             dose = infer_mod.run_inference_conditioned_on_energy(
00639                 ct_tensor,
00640                 energy_value=args.energy,
00641                 target_cube_size=target_cube_size
00642             )
00643         elif args.single_energy_only and len(quad_energies) == 1:
00644             # Single energy inference (from quadrature energies but forced single)
00645             energy = quad_energies[0]
00646             logger.info(f"Running single energy inference at {energy} keV...")
00647             dose = infer_mod.run_inference_conditioned_on_energy(
00648                 ct_tensor,
00649                 energy_value=energy,
00650                 target_cube_size=target_cube_size
00651             )
00652         elif len(quad_energies) > 1:
00653             # Multi-energy quadrature inference (4-point Gaussian Quadrature)
00654             logger.info(f"Running 4-point Gaussian Quadrature inference with {len(quad_energies)} energies...")
00655             logger.info(f"Energies: {[f'{e:.2f}' for e in quad_energies]} keV")
00656             logger.info(f"Weights: {[f'{w:.4f}' for w in quad_weights]}")
00657             logger.info("Expected result: Gaussian bell-curve dose distribution")
00658             dose = infer_mod.run_inference(ct_tensor, target_cube_size=target_cube_size)
00659         else:
00660             # Single energy from quad_energies list
00661             energy = quad_energies[0]
00662             logger.info(f"Running single energy inference at {energy} keV...")
00663             dose = infer_mod.run_inference_conditioned_on_energy(
00664                 ct_tensor,
00665                 energy_value=energy,
00666                 target_cube_size=target_cube_size
00667             )
00668
00669         logger.info(" Inference completed successfully")
00670
00671     except Exception as e:
00672         logger.error(f"Inference failed: {e}")
00673         logger.error("Traceback:", exc_info=True)
00674         return 1
00675

```

```

00676     # Convert to numpy
00677     if isinstance(dose, torch.Tensor):
00678         dose_np = dose.cpu().numpy()
00679     else:
00680         dose_np = np.array(dose, dtype=np.float32)
00681
00682     # Remove batch dimension if present
00683     if dose_np.ndim == 4 and dose_np.shape[0] == 1:
00684         dose_np = dose_np[0]
00685
00686     # Remove channel dimension if present
00687     if dose_np.ndim == 4 and dose_np.shape[0] == 1:
00688         dose_np = dose_np[0]
00689
00690     logger.info(f"Output dose shape: {dose_np.shape}")
00691     logger.info(f"Output dose range: {dose_np.min():.6f} to {dose_np.max():.6f}")
00692     logger.info(f"Output dose mean: {dose_np.mean():.6f}")
00693     logger.info(f"Output dose std: {dose_np.std():.6f}")
00694
00695     # Check for realistic dose values
00696     max_dose = dose_np.max()
00697     if max_dose < 1.0:
00698         logger.warning(f" Max dose {max_dose:.2f} Gy seems low for radiotherapy")
00699     elif max_dose > 100.0:
00700         logger.warning(f" Max dose {max_dose:.2f} Gy seems high for radiotherapy")
00701     else:
00702         logger.info(f" Max dose {max_dose:.2f} Gy is in realistic range for radiotherapy")
00703
00704     # Save output
00705     logger.info(f"Saving dose output to {output_path}")
00706     try:
00707         save_dose(output_path, dose_np, affine)
00708         logger.info(" Output saved successfully")
00709     except Exception as e:
00710         logger.error(f"Failed to save output: {e}")
00711         return 1
00712
00713     # Create NIfTI file for 3D GUI viewing
00714     logger.info("Creating NIfTI file for 3D GUI viewing...")
00715     try:
00716         nii_path = save_nifti_with_manifest(dose_np, output_path, root_dir=".")
00717         if nii_path:
00718             logger.info(f" NIfTI file created for 3D viewing: {nii_path}")
00719         else:
00720             logger.warning("NIfTI file creation skipped (nibabel not available)")
00721     except Exception as e:
00722         logger.warning(f"Failed to create NIfTI file: {e}")
00723
00724     # Create PNG visualization unless disabled
00725     if not args.no_png:
00726         logger.info("Creating PNG visualization...")
00727         try:
00728             # Create PNG filename
00729             png_path = output_path.with_suffix('.png')
00730             create_dose_visualization(dose_np, png_path, f"Dose Output - {output_path.stem}", input_path)
00731             logger.info(f" PNG visualization created: {png_path}")
00732         except Exception as e:
00733             logger.warning(f"Failed to create PNG visualization: {e}")
00734
00735     logger.info("=" * 60)
00736     logger.info("4-POINT GAUSSIAN QUADRATURE INFERENCE COMPLETED SUCCESSFULLY")
00737     logger.info("=" * 60)
00738     if len(quad_energies) > 1:
00739         logger.info(" Expected Gaussian bell-curve dose distribution generated")
00740         logger.info(f" Integrated over {len(quad_energies)} energy points with proper weights")
00741     else:
00742         logger.info(" Single energy dose distribution generated")
00743     return 0
00744
00745
00746 if __name__ == '__main__':
00747     exit(main())

```

8.18 sourcecode/log_config.py File Reference

Namespaces

- namespace [log_config](#)

Functions

- [log_config.configure_logging\(\)](#)

Variables

- `log_config.logger` = `logging.getLogger('radiotherapy')`

8.19 log_config.py

[Go to the documentation of this file.](#)

```
00001 import logging
00002 import os
00003
00004 # Configure root logger to output into radiotherapy.log in project root
00005 def configure_logging():
00006     # Always use the project root for the log file
00007     # Get the directory of this script (sourcecode/)
00008     script_dir = os.path.dirname(os.path.abspath(__file__))
00009     # Go up one level to the project root
00010     project_root = os.path.dirname(script_dir)
00011     log_file = os.path.join(project_root, 'radiotherapy.log')
00012
00013     # Clear any existing handlers to avoid duplicates
00014     root_logger = logging.getLogger()
00015     for handler in root_logger.handlers[:]:
00016         root_logger.removeHandler(handler)
00017
00018     # Configure logging with file handler
00019     logging.basicConfig(
00020         filename=log_file,
00021         filemode='a',
00022         format='%(asctime)s %(levelname)s: %(name)s: %(message)s',
00023         level=logging.INFO,
00024         force=True # Force reconfiguration
00025     )
00026
00027     print(f"Logging configured to write to: {log_file}")
00028
00029 # Ensure logging is configured on import
00030 configure_logging()
00031
00032 # Provide module-level logger
00033 logger = logging.getLogger('radiotherapy')
```

8.20 sourcecode/parameter_manager.py File Reference

Classes

- class `parameter_manager.ParameterManager`

Namespaces

- namespace `parameter_manager`

Variables

- `parameter_manager.logger` = `logging.getLogger(__name__)`

8.21 parameter_manager.py

[Go to the documentation of this file.](#)

```
00001 # parameter_manager.py
00002 import torch
00003 import logging
00004
00005 logger = logging.getLogger(__name__)
00006
00007 class ParameterManager:
00008     """
00009     The ParameterManager class centralizes configuration parameters for the RadioTherapy project.
00010     It stores key settings such as spatial resolutions, energy levels, batch size, cube size,
00011     learning rate, number of epochs, and additional parameters.
00012     """
00013
00014     def __init__(self, energies, batch_size, cube_size, learning_rate, num_epochs, patience, other_parameters=None):
00015         """
```

```

00016     Initializes the ParameterManager with the required configuration parameters.
00017
00018     Args:
00019         resolutions (list of tuple): List of spatial resolutions, e.g. [(64, 64, 64), (32, 32, 32)].
00020         energies (list of int): List of energy levels (in keV) for which separate training might be executed.
00021         batch_size (int): The batch size used during training.
00022         cube_size (tuple): The cube size to which CT data will be resized (e.g., (64, 64, 64)).
00023         learning_rate (float): The learning rate used during training.
00024         num_epochs (int): The number of epochs for training.
00025         other_parameters (dict, optional): Any additional parameters to store.
00026     """
00027     # Convert energies to tensor if it's a list
00028     if isinstance(energies, list):
00029         import torch
00030         self.energies = torch.tensor(energies, dtype=torch.float32)
00031     else:
00032         self.energies = energies
00033
00034     self.quad_energies = []
00035     self.quad_weights = []
00036     self.batch_size = batch_size
00037
00038     # Validate and constrain cube_size for optimal performance
00039     self.cube_size = cube_size
00040
00041     # CORRECTED: Use SINGLE resolution (64x64x64) instead of multi-resolution for stability
00042     if isinstance(self.cube_size, tuple):
00043         # Use only the original cube_size resolution for stability
00044         self.resolutions = [self.cube_size] # Only one resolution
00045         logger.info(f" CORRECTED: Using SINGLE resolution {self.cube_size} for stability (was
multi-resolution)")
00046     else:
00047         # Handle single integer cube_size (backward compatibility)
00048         self.resolutions = [self.cube_size] # Only one resolution
00049         logger.info(f" CORRECTED: Using SINGLE resolution {self.cube_size} for stability (was
multi-resolution)")
00050
00051     self.learning_rate = learning_rate
00052     self.num_epochs = num_epochs
00053     self.patience = patience
00054     self.other_parameters = other_parameters if other_parameters is not None else {}
00055
00056     min_e, max_e = float('inf'), float('-inf')
00057     print("Energies:" , self.energies)
00058     min_e = min(min_e, self.energies.min().item())
00059     max_e = max(max_e, self.energies.max().item())
00060     print("Min energy:", min_e, "Max energy:", max_e)
00061
00062     # Store as instance attributes
00063     self.energy_min = min_e
00064     self.energy_max = max_e
00065     self.energy_normalization = (self.energies.float() - min_e) / (max_e - min_e)
00066
00067
00068     def get_parameters(self):
00069     """
00070     Returns all configuration parameters as a dictionary.
00071
00072     Returns:
00073         dict: A dictionary containing resolutions, energies, batch_size, and any additional parameters.
00074     """
00075     params = {
00076         "resolutions": self.resolutions,
00077         "energies": self.energies,
00078         "energy_min": self.energy_min,
00079         "energy_max": self.energy_max,
00080         "energy_normalization": self.energy_normalization,
00081         "quadrature_weights": self.quadrature_weights,
00082         "batch_size": self.batch_size,
00083         "cube_size": self.cube_size,
00084         "learning_rate": self.learning_rate,
00085         "num_epochs": self.num_epochs
00086     }
00087     params.update(self.other_parameters)
00088     return params
00089
00090     def update_parameter(self, key, value):
00091     """
00092     Updates a specific parameter.
00093
00094     Args:
00095         key (str): The parameter name to update.
00096         value: The new value for the parameter.
00097     """
00098     if key == "cube_size":
00099         value = self._validate_cube_size(value)
00100         self.cube_size = value

```

```

00101         # Update resolutions based on new cube_size
00102         if isinstance(value, tuple):
00103             self.resolutions = [
00104                 tuple(dim // 4 for dim in value),
00105                 tuple(dim // 2 for dim in value),
00106                 value
00107             ]
00108         else:
00109             self.resolutions = [value//4, value//2, value]
00110     elif key in ["resolutions", "energies", "quadrature_weights", "batch_size", "learning_rate", "num_epochs"]:
00111         setattr(self, key, value)
00112     else:
00113         self.other_parameters[key] = value

```

8.22 sourcecode/standalone_training.py File Reference

Namespaces

- namespace [standalone_training](#)

Functions

- [standalone_training.parse_arguments\(\)](#)
- [standalone_training.validate_input_folder\(input_folder\)](#)
- [standalone_training.determine_cube_size\(input_folder\)](#)
- [standalone_training.get_energy_list\(dataset_root\)](#)
- [standalone_training.setup_device\(device_arg\)](#)
- [standalone_training.main\(\)](#)

Variables

- [standalone_training.project_root](#) = `os.path.abspath(os.path.join(os.path.dirname(__file__), '..'))`
- [standalone_training.logger](#) = `logging.getLogger(__name__)`

8.23 standalone_training.py

[Go to the documentation of this file.](#)

```

00001 #!/usr/bin/env python3
00002 """
00003 Standalone Training Script for RadioTherapy Project
00004
00005 This script allows you to run training without the GUI. It provides the same
00006 functionality as the training feature in the GUI but can be run from command line.
00007
00008 Usage:
00009     python standalone_training.py --input_folder /path/to/energy/folder [options]
00010
00011 Example:
00012     python standalone_training.py --input_folder /path/to/dataset/62_0 --epochs 10 --batch_size 4
00013 """
00014
00015 import os
00016 import sys
00017 import argparse
00018 import logging
00019 import traceback
00020 import glob
00021 import numpy as np
00022 import torch
00023 from pathlib import Path
00024
00025 # Add project root to Python path
00026 project_root = os.path.abspath(os.path.join(os.path.dirname(__file__), '..'))
00027 sys.path.insert(0, project_root)
00028
00029 # Import project modules
00030 import log_config # Initialize logging config
00031 from system_manager import SystemManager
00032 from parameter_manager import ParameterManager
00033 from monai.transforms import (
00034     Compose, LoadImaged, EnsureChannelFirstd, Lambdad,
00035     EnsureTyped, Orientationd, Spacingd, SpatialPadd,
00036     CenterSpatialCropd, ScaleIntensityRangePercentilesd, ToTensord,
00037 )

```

```

00038 from monai.data import NumpyReader
00039
00040 logger = logging.getLogger(__name__)
00041
00042
00043 def parse_arguments():
00044     """Parse command line arguments for training configuration."""
00045     parser = argparse.ArgumentParser(
00046         description="Standalone training script for RadioTherapy project",
00047         formatter_class=argparse.ArgumentDefaultsHelpFormatter
00048     )
00049
00050     # Required arguments
00051     parser.add_argument(
00052         '--input_folder', '-i',
00053         type=str,
00054         required=True,
00055         help='Path to energy folder containing inputcube and outputcube subdirectories'
00056     )
00057
00058     # Training parameters
00059     parser.add_argument(
00060         '--batch_size', '-b',
00061         type=int,
00062         default=2,
00063         help='Batch size for training'
00064     )
00065
00066     parser.add_argument(
00067         '--epochs', '-e',
00068         type=int,
00069         default=5,
00070         help='Number of training epochs'
00071     )
00072
00073     parser.add_argument(
00074         '--learning_rate', '-lr',
00075         type=float,
00076         default=1e-5,
00077         help='Learning rate for training'
00078     )
00079
00080     parser.add_argument(
00081         '--patience', '-p',
00082         type=int,
00083         default=20,
00084         help='Early stopping patience'
00085     )
00086
00087     parser.add_argument(
00088         '--device',
00089         type=str,
00090         default='auto',
00091         choices=['auto', 'cpu', 'cuda'],
00092         help='Device to use for training (auto selects GPU if available)'
00093     )
00094
00095     parser.add_argument(
00096         '--seed',
00097         type=int,
00098         default=42,
00099         help='Random seed for reproducibility'
00100     )
00101
00102     return parser.parse_args()
00103
00104
00105 def validate_input_folder(input_folder):
00106     """
00107     Validate that the input folder structure is correct.
00108
00109     Args:
00110         input_folder (str): Path to the energy folder
00111
00112     Returns:
00113         tuple: (input_cube_path, output_cube_path, dataset_root)
00114
00115     Raises:
00116         ValueError: If folder structure is invalid
00117     """
00118     input_folder = Path(input_folder)
00119
00120     if not input_folder.exists():
00121         raise ValueError(f"Input folder does not exist: {input_folder}")
00122
00123     # Check if it's an energy folder with inputcube and outputcube subdirectories
00124     input_cube_path = input_folder / "inputcube"

```

```

00125     output_cube_path = input_folder / "outputcube"
00126
00127     if input_cube_path.exists() and output_cube_path.exists():
00128         # This is an energy folder
00129         dataset_root = input_folder.parent
00130         logger.info(f"Found energy folder structure: {input_folder}")
00131         logger.info(f"Input cubes: {input_cube_path}")
00132         logger.info(f"Output cubes: {output_cube_path}")
00133         return str(input_cube_path), str(output_cube_path), str(dataset_root)
00134     else:
00135         # Treat as direct input cube directory
00136         logger.info(f"Treating as direct input cube directory: {input_folder}")
00137         return str(input_folder), None, str(input_folder.parent)
00138
00139
00140 def determine_cube_size(input_folder):
00141     """
00142     Determine cube size from the first available cube file.
00143
00144     Args:
00145         input_folder (str): Path to input cube folder
00146
00147     Returns:
00148         tuple: Cube size dimensions
00149     """
00150     # Look for .npy, .nii, or .nii.gz files
00151     patterns = ["*.npy", "*.nii", "*.nii.gz"]
00152     files = []
00153
00154     for pattern in patterns:
00155         files.extend(glob.glob(os.path.join(input_folder, pattern)))
00156
00157     if not files:
00158         raise ValueError(f"No cube files found in {input_folder}")
00159
00160     sample_file = files[0]
00161     logger.info(f"Determining cube size from: {sample_file}")
00162
00163     try:
00164         if sample_file.lower().endswith(".npy"):
00165             arr = np.load(sample_file)
00166         else:
00167             import nibabel as nib
00168             arr = np.asarray(nib.load(sample_file).dataobj)
00169
00170         cube_size = arr.shape
00171         logger.info(f"Detected cube size: {cube_size}")
00172         return cube_size[0]
00173
00174     except Exception as e:
00175         raise ValueError(f"Failed to determine cube size from {sample_file}: {e}")
00176
00177
00178 def get_energy_list(dataset_root):
00179     """
00180     Get list of available energies from dataset root directory.
00181
00182     Args:
00183         dataset_root (str): Path to dataset root containing energy folders
00184
00185     Returns:
00186         list: List of energy values
00187     """
00188     try:
00189         # List all energy subfolder names (skip hidden files)
00190         energy_names = [
00191             d for d in os.listdir(dataset_root)
00192             if os.path.isdir(os.path.join(dataset_root, d)) and not d.startswith('.')
00193         ]
00194
00195         # Parse numeric energy values from folder names and sort numerically
00196         energies_list = sorted([float(name.replace("_", ".")) for name in energy_names])
00197         logger.info(f"Found energies: {energies_list}")
00198         return energies_list
00199
00200     except Exception as e:
00201         logger.warning(f"Could not determine energies from dataset root: {e}")
00202         return [0.0] # Default fallback
00203
00204
00205 def setup_device(device_arg):
00206     """Setup compute device based on argument."""
00207     if device_arg == 'auto':
00208         device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
00209     else:
00210         device = torch.device(device_arg)
00211

```

```

00212     logger.info(f"Using device: {device}")
00213     return device
00214
00215
00216 def main():
00217     """Main training function."""
00218     try:
00219         # Parse command line arguments
00220         args = parse_arguments()
00221
00222         logger.info("=== RadioTherapy Standalone Training ===")
00223         logger.info(f"Input folder: {args.input_folder}")
00224         logger.info(f"Batch size: {args.batch_size}")
00225         logger.info(f"Epochs: {args.epochs}")
00226         logger.info(f"Learning rate: {args.learning_rate}")
00227         logger.info(f"Patience: {args.patience}")
00228         logger.info(f"Seed: {args.seed}")
00229
00230         # Validate input folder structure
00231         input_cube_path, output_cube_path, dataset_root = validate_input_folder(args.input_folder)
00232
00233         # Determine cube size
00234         cube_size = determine_cube_size(input_cube_path)
00235
00236         # Get ALL available energies for proper training (not just folder energy)
00237         energies = get_energy_list(dataset_root)
00238         logger.info(f"Training with ALL energies: {energies} (proper multi-energy training)")
00239
00240         # Setup device
00241         device = setup_device(args.device)
00242
00243         # Initialize parameter manager
00244         pm = ParameterManager(
00245             energies=energies,
00246             batch_size=args.batch_size,
00247             cube_size=cube_size,
00248             num_epochs=args.epochs,
00249             learning_rate=args.learning_rate,
00250             patience=args.patience
00251         )
00252
00253         # Setup transforms
00254         transforms_chain = Compose([
00255             LoadImaged(keys=["input", "target"], reader=NumpyReader),
00256             EnsureChannelFirstd(keys=["input", "target"]),
00257             EnsureTyped(keys=["input", "target"]),
00258             Orientationd(keys=["input", "target"], axcodes="RAS"),
00259             Spacingd(keys=["input", "target"], pixdim=(2.4, 2.4, 2.4), mode=("bilinear", "nearest")),
00260             SpatialPadd(keys=["input", "target"], spatial_size=pm.cube_size, method="symmetric"),
00261             CenterSpatialCropd(keys=["input", "target"], roi_size=pm.cube_size),
00262             ScaleIntensityRangePercentile(
00263                 keys="input", lower=0, upper=99.5, b_min=0, b_max=1
00264             ),
00265             ToTensord(keys=["input", "target"])
00266         ])
00267         logger.info("Resolutions: {}".format(pm.resolutions))
00268         # Initialize SystemManager
00269         system_manager = SystemManager(
00270             root_dir=dataset_root,
00271             transforms=transforms_chain,
00272             resolutions=pm.resolutions,
00273             energies=pm.energies,
00274             energy_min=pm.energy_min,
00275             energy_max=pm.energy_max,
00276             quad_energies=pm.quad_energies,
00277             quad_weights=pm.quad_weights,
00278             batch_size=pm.batch_size,
00279             device=device,
00280             num_epochs=pm.num_epochs,
00281             learning_rate=pm.learning_rate,
00282             patience=pm.patience,
00283             cube_size=pm.cube_size,
00284             seed=args.seed
00285         )
00286
00287         # Start training
00288         logger.info("Starting training...")
00289         system_manager.run_training()
00290
00291         logger.info("=== Training completed successfully! ===")
00292
00293         # Look for generated plots
00294         plot_patterns = ["loss_curves_res*.png", "learning_curve.png", "adv_curves.png"]
00295         for pattern in plot_patterns:
00296             plots = glob.glob(pattern)
00297             if plots:
00298                 logger.info(f"Generated plots: {plots}")

```



```

00299
00300     except KeyboardInterrupt:
00301         logger.info("Training interrupted by user.")
00302         sys.exit(1)
00303     except Exception as e:
00304         logger.error(f"Training failed: {e}")
00305         traceback.print_exc()
00306         sys.exit(1)
00307
00308
00309 if __name__ == "__main__":
00310     main()

```

8.24 sourcecode/storage.py File Reference

Classes

- class `storage.Storage`

Namespaces

- namespace `storage`

8.25 storage.py

[Go to the documentation of this file.](#)

```

00001 # storage.py
00002
00003 import os
00004 import torch
00005 import numpy as np
00006
00007 class Storage:
00008     """
00009     The Storage module handles persistence tasks for the project.
00010     It provides methods to save and load model checkpoints, save dose distributions,
00011     log errors, and export results.
00012     """
00013
00014     def __init__(self, model_dir="models", log_file="training_log.txt"):
00015         """
00016         Initializes the Storage module.
00017
00018         Args:
00019             model_dir (str): Directory where model checkpoints and results are saved.
00020             log_file (str): File path for the training log.
00021         """
00022         self.model_dir = model_dir
00023         self.log_file = log_file
00024         # Create the model directory if it doesn't exist
00025         os.makedirs(self.model_dir, exist_ok=True)
00026
00027     def save_model(self, model, filename):
00028         """
00029         Saves the model's state dictionary to a file.
00030
00031         Args:
00032             model (torch.nn.Module): The model to be saved.
00033             filename (str): The filename (relative to model_dir) to save the model.
00034         """
00035         path = os.path.join(self.model_dir, filename)
00036         torch.save(model.state_dict(), path)
00037         print(f"Model saved to {path}")
00038
00039     def load_model(self, model, filename, device=torch.device("cpu")):
00040         """
00041         Loads a saved model's state dictionary from a file and loads it into the given model.
00042
00043         Args:
00044             model (torch.nn.Module): The model instance to load the state dictionary into.
00045             filename (str): The filename (relative to model_dir) to load the model from.
00046             device (torch.device): The device to map the model to.
00047
00048         Returns:
00049             torch.nn.Module: The model loaded with the saved state dictionary.
00050         """
00051         path = os.path.join(self.model_dir, filename)
00052         if not os.path.exists(path):
00053             raise FileNotFoundError(f"Model file {path} not found!")

```

```

00054         state_dict = torch.load(path, map_location=device)
00055         model.load_state_dict(state_dict)
00056         print(f"Model loaded from {path}")
00057         return model
00058
00059     def save_dose_distribution(self, dose_distribution, filename):
00060         """
00061         Saves the computed dose distribution as a numpy file.
00062
00063         Args:
00064             dose_distribution (numpy.ndarray): The dose distribution array.
00065             filename (str): The filename to save the dose distribution (relative to model_dir).
00066         """
00067         path = os.path.join(self.model_dir, filename)
00068         np.save(path, dose_distribution)
00069         print(f"Dose distribution saved to {path}")
00070
00071     def log_error(self, message):
00072         """
00073         Appends an error message to the log file.
00074
00075         Args:
00076             message (str): The error message to log.
00077         """
00078         with open(self.log_file, "a") as log_file:
00079             log_file.write(message + "\n")
00080         print(f"Logged error: {message}")
00081
00082     def export_results(self, results, filename):
00083         """
00084         Exports training or inference results to a file.
00085
00086         Args:
00087             results (str): The results in string format.
00088             filename (str): The filename to export the results (relative to model_dir).
00089         """
00090         path = os.path.join(self.model_dir, filename)
00091         with open(path, "w") as f:
00092             f.write(results)
00093         print(f"Results exported to {path}")

```

8.26 sourcecode/system_manager.py File Reference

Classes

- class [system_manager.SystemManager](#)

Namespaces

- namespace [system_manager](#)

Variables

- [system_manager.logger](#) = logging.getLogger(__name__)

8.27 system_manager.py

[Go to the documentation of this file.](#)

```

00001 # system_manager.py
00002
00003 import shutil
00004 import os
00005 import torch
00006 import log_config
00007 import logging
00008 import glob
00009 import json
00010 import numpy as np
00011 logger = logging.getLogger(__name__)
00012
00013 from data_management import DataLoaderModule, HotspotPatchDataset
00014 from training_pipeline import AutoencoderTrainer, DiffusionTrainer, EarlyStopping
00015 from inference_module import InferenceModule
00016 from monai.transforms import Compose, LoadImaged, EnsureChannelFirstd, ToTensord
00017 from visualization import Visualization
00018 from monai.utils import first, set_determinism
00019 from monai.data import NibabelReader

```

```

00020 from generative.networks.nets import AutoencoderKL, PatchDiscriminator, DiffusionModelUNet
00021 from generative.inferers import LatentDiffusionInferer
00022 from torch.optim import Adam
00023 from torch.amp import autocast
00024 from generative.networks.schedulers.ddpm import DDPMSScheduler
00025 from monai.transforms import (
00026     Compose, LoadImaged, EnsureChannelFirstd, Lambdad,
00027     EnsureTyped, Orientationd, Resized, SpatialPadd,
00028     CenterSpatialCropd, ScaleIntensityRangePercentile, ToTensord,
00029 )
00030 from monai.data import NumpyReader
00031
00032 class SystemManager:
00033     """
00034     The SystemManager class orchestrates the complete training and inference workflow.
00035
00036     It iterates over each combination of spatial resolution and energy level:
00037     - Loads the dataset using the DataLoaderModule.
00038     - Splits the data into training and validation sets.
00039     - Initializes and trains the autoencoder (and optionally diffusion model)
00040       using early stopping based on validation loss.
00041     - Sets a training-completion flag for subsequent inference.
00042     """
00043     def __init__(self, root_dir, transforms, resolutions, energies, energy_min, energy_max, quad_energies,
00044                  quad_weights, batch_size, device, num_epochs, learning_rate, patience, cube_size, seed=42):
00045         """
00046         Initializes the SystemManager with configuration parameters.
00047
00048         Args:
00049             root_dir (str): Root directory of the dataset.
00050             transforms (monai.transforms.Compose): Preprocessing transforms to apply.
00051             resolutions (list of tuple): List of resolutions (e.g., [(64, 64, 64), (32, 32, 32)]).
00052             energies (list of int): List of energy levels (e.g., [62, 75, 90]).
00053             batch_size (int): Batch size for DataLoader.
00054             device (torch.device): Device (CPU or GPU) for computations.
00055             seed (int): Random seed for reproducibility.
00056         """
00057         self.root_dir = root_dir
00058         self.transforms = transforms
00059         self.resolutions = resolutions
00060         self.energies = energies
00061         self.energy_min = energy_min
00062         self.energy_max = energy_max
00063         self.batch_size = batch_size
00064         self.device = device
00065         self.seed = seed
00066         self.num_epochs = num_epochs
00067         self.learning_rate = learning_rate
00068         self.patience = patience
00069         self.training_complete = False
00070         self.stop_training = False
00071         # placeholders for trained models
00072         self.autoencoder = None
00073         self.unet = None
00074         self.scheduler = None
00075         self.cube_size = cube_size
00076         self.quad_energies = quad_energies
00077         self.quad_weights = quad_weights
00078         # keep models per energy for quadrature inference
00079         self.models_by_energy = {}
00080         # track individual checkpoints in memory
00081         self.saved_ckpts = {}
00082
00083     def save_models(self, autoencoder, unet, optimizer_diff, optimizer_g,
00084                   optimizer_d, epoch, res, energy,
00085                   dose_mean=None, dose_std=None,
00086                   scale_factor=None, clip_min=0.0, clip_max=None,
00087                   smoothing_kernel=0):
00088         # build checkpoint dict in memory (no file I/O)
00089         ckpt = {
00090             "autoencoder": autoencoder.state_dict(),
00091             "unet": unet.state_dict(),
00092             "optimizer_diff": optimizer_diff.state_dict(),
00093             "optimizer_g": optimizer_g.state_dict(),
00094             "optimizer_d": optimizer_d.state_dict(),
00095             "epoch": epoch,
00096             "scale_factor": scale_factor,
00097             "clip_min": clip_min,
00098             "clip_max": clip_max,
00099             "smoothing_kernel": smoothing_kernel,
00100             "dose_mean": dose_mean,
00101             "dose_std": dose_std,
00102             "resolution": res,
00103             "energy": energy,
00104             "learning_rate": self.learning_rate,
00105             "batch_size": self.batch_size,
00106             "cube_size": self.cube_size,

```

```

00106     }
00107     # build a resolution string for the key
00108     if isinstance(res, (tuple, list)):
00109         res_str = "x".join(str(r) for r in res)
00110     else:
00111         res_str = str(res)
00112     key = f"res{res_str}_e{energy:.2f}"
00113     self.saved_ckpts[key] = ckpt
00114
00115 def save_combined_checkpoint(self, output_filename="combined_models.ckpt"):
00116     """
00117     Save all in-memory checkpoints into a single combined file.
00118     """
00119     # Extract dose normalization parameters for each energy/resolution combination
00120     dose_normalization_params = {}
00121
00122     for key, ckpt in self.saved_ckpts.items():
00123         # Extract energy and resolution from key (e.g., "res25_e46.53")
00124         if 'clip_min' in ckpt and 'clip_max' in ckpt and 'energy' in ckpt:
00125             dose_normalization_params[key] = {
00126                 'clip_min': ckpt['clip_min'],
00127                 'clip_max': ckpt['clip_max'],
00128                 'energy': ckpt['energy'],
00129                 'resolution': ckpt.get('resolution', None),
00130                 'scale_factor': ckpt.get('scale_factor', 1.0),
00131                 'dose_mean': ckpt.get('dose_mean', None),
00132                 'dose_std': ckpt.get('dose_std', None)
00133             }
00134             logger.info(f"Dose params for {key}: clip_min={ckpt['clip_min']:.6f},
clip_max={ckpt['clip_max']:.6f}")
00135
00136     combined = {
00137         "models_by_energy": self.saved_ckpts,
00138         "dose_normalization_params": dose_normalization_params
00139     }
00140     output_path = os.path.join(self.root_dir, output_filename)
00141     torch.save(combined, output_path)
00142     logger.info(f"Combined checkpoint saved to {output_path}")
00143     logger.info(f"Saved dose normalization parameters for {len(dose_normalization_params)} energy/resolution
combinations")
00144     return output_path
00145
00146 def run_training(self):
00147     """
00148     Executes the full training pipeline over all resolution and energy combinations.
00149
00150     For each combination, the following steps are executed:
00151     1. Load and split the dataset filtered by the current energy.
00152     2. Create DataLoaders for training and validation.
00153     3. Initialize the autoencoder, discriminator, and diffusion models.
00154     4. Run the autoencoder training loop with early stopping based on validation loss.
00155     5. Run the diffusion model training loop.
00156     6. Save the models for the current configuration.
00157
00158     After all configurations have been processed, a flag is set to indicate that training is complete.
00159     """
00160     # Print hyperparameters for verification
00161     logger.info("Starting CORRECTED training with hyperparameters:")
00162     logger.info(f"  batch_size={self.batch_size}, num_epochs={self.num_epochs},
learning_rate={self.learning_rate}, patience={self.patience}, cube_size={self.cube_size}")
00163     logger.info(f"  resolutions={self.resolutions}, energies={self.energies}")
00164
00165     # CORRECTED TRAINING STRATEGY: Use single resolution (64x64x64) for all energies
00166     target_resolution = (64, 64, 64) # Fixed single resolution for stability
00167     logger.info(f" CORRECTED: Using SINGLE resolution {target_resolution} for ALL energies (instead of
multi-resolution)")
00168     logger.info(f" CORRECTED: Energy Loop -> Resolution Loop (proper architecture)")
00169
00170     # instantiate models - before the loop to avoid re-instantiation
00171     autoencoder = AutoencoderKL(
00172         spatial_dims=3,
00173         in_channels=2,
00174         out_channels=1,
00175         num_channels=(32, 32, 32),
00176         latent_channels=2,
00177         num_res_blocks=1,
00178         norm_num_groups=8,
00179         attention_levels=(False, False, True),
00180     ).to(self.device)
00181     discriminator = PatchDiscriminator(
00182         spatial_dims=3,
00183         num_layers_d=3,
00184         num_channels=32,
00185         in_channels=1,
00186         out_channels=1
00187     ).to(self.device)
00188     unet = DiffusionModelUNet(

```

```

00189         spatial_dims=3,
00190         in_channels=2,
00191         out_channels=2,
00192         with_conditioning=False, # Disabled cross-attention conditioning
00193         num_res_blocks=1,
00194         num_channels=(32, 64, 64),
00195         attention_levels=(False, True, True),
00196         num_head_channels=(0, 64, 64),
00197     ).to(self.device)
00198
00199     # initialize history lists for plotting
00200     ae_train_losses = []
00201     ae_val_losses = []
00202     gen_losses = []
00203     disc_losses = []
00204     diff_losses = []
00205
00206     # CORRECTED TRAINING LOOP: Energy as Outer Loop, Single Resolution
00207     for energy in self.energies:
00208         if self.stop_training:
00209             logger.info("Training aborted by user.")
00210             return
00211         logger.info(f" --- CORRECTED: Training energy={energy} eV at resolution={target_resolution} ---")
00212         logger.info(f" CORRECTED: Pipeline uses consistent resolution: {target_resolution}")
00213         # Separate transforms for patch-based training and validation
00214         patch_transforms = Compose([
00215             EnsureTyped(keys=["input", "target"]),
00216             Orientationd(keys=["input", "target"], axcodes="RAS"),
00217             Resized(keys=["input", "target"], spatial_size=target_resolution, mode=("bilinear", "nearest")),
00218             ScaleIntensityRangePercentilesd(keys="input", lower=0, upper=99.5, b_min=0, b_max=1),
00219             ToTensord(keys=["input", "target"]),
00220             EnsureTyped(keys=["energy"]),
00221             ToTensord(keys=["energy"])
00222         ])
00223         val_transforms = Compose([
00224             LoadImaged(keys=["input", "target"], reader=NumpyReader),
00225             EnsureChannelFirstd(keys=["input", "target"]),
00226             EnsureTyped(keys=["input", "target"]),
00227             Orientationd(keys=["input", "target"], axcodes="RAS"),
00228             Resized(keys=["input", "target"], spatial_size=target_resolution, mode=("bilinear", "nearest")),
00229             ScaleIntensityRangePercentilesd(keys="input", lower=0, upper=99.5, b_min=0, b_max=1),
00230             ToTensord(keys=["input", "target"]),
00231             EnsureTyped(keys=["energy"]),
00232             ToTensord(keys=["energy"])
00233         ])
00234
00235         # Use HotspotPatchDataset for patch-based training
00236         logger.info("Using HotspotPatchDataset for patch-based training.")
00237         train_ds = HotspotPatchDataset(
00238             root_dir=self.root_dir,
00239             section=None,
00240             patch_size=(32, 32, 32),
00241             min_hotspot_voxels=100, # REDUCED: Was 1000, now 100 for low-dose data
00242             dose_threshold=0.001, # REDUCED: Was 0.5, now 0.001 for low-dose data
00243             max_patches=6,
00244             random_patches=0,
00245             transforms=patch_transforms,
00246             energy=energy,
00247             max_patches_per_energy=80 if energy <= 40.0 else 60 # Less patches for high energy
00248         )
00249
00250         # Validation dataset
00251         val_ds = HotspotPatchDataset(
00252             root_dir=self.root_dir,
00253             section=None,
00254             patch_size=(32, 32, 32),
00255             min_hotspot_voxels=100, # REDUCED: Was 1000, now 100 for low-dose data
00256             dose_threshold=0.001, # REDUCED: Was 0.5, now 0.001 for low-dose data
00257             max_patches=2,
00258             random_patches=0,
00259             transforms=patch_transforms,
00260             energy=energy,
00261             max_patches_per_energy=20 if energy <= 40.0 else 15 # Less patches for high energy validation
00262         )
00263
00264         # DataLoader as before
00265         from data_management import DataLoaderModule
00266         data_module = DataLoaderModule(
00267             root_dir=self.root_dir,
00268             transforms=patch_transforms
00269         )
00270         train_loader = data_module.create_data_loader(train_ds, self.batch_size, shuffle=True)
00271         val_loader = data_module.create_data_loader(val_ds, self.batch_size, shuffle=False)
00272         logger.info(f"Found {len(val_ds)} validation patches for energy={energy}, resolution={target_resolution}")
00273
00274         scheduler = DDPMScheduler(
00275             num_train_timesteps=1000,

```

```

00275         schedule="scaled_linear_beta",
00276         beta_start=0.0015,
00277         beta_end=0.0195,
00278     )
00279
00280     # ### Scaling factor
00281     #
00282     # As mentioned in Rombach et al. [1] Section 4.3.2 and D.1, the signal-to-noise ratio (induced by the
00283     # scale of the latent space) can affect the results obtained with the LDM, if the standard deviation of the latent
00284     # space distribution drifts too much from that of a Gaussian.
00285     # For this reason, it is best practice to use a scaling factor to adapt this standard deviation.
00286     #
00287     # _Note: In case where the latent space is close to a Gaussian distribution, the scaling factor will be
00288     # close to one, and the results will not differ from those obtained when it is not used._
00289     #
00290     # +
00291     with torch.no_grad():
00292         with autocast('cuda', enabled=True):
00293             first_batch = first(train_loader)
00294             # Build conditioned input for autoencoder with energy channel if available
00295             images = first_batch["input"].to(self.device)
00296             energies = first_batch.get("energy", None)
00297             if energies is not None:
00298                 energies = energies.to(self.device)
00299                 normalized_energy = energies.float() / 100.0 # match training normalization
00300                 B, C, D, H, W = images.shape
00301                 energy_tensor = normalized_energy.view(B, 1, 1, 1, 1).expand(B, 1, D, H, W)
00302                 conditioned = torch.cat([images, energy_tensor], dim=1)
00303             else:
00304                 conditioned = images
00305             # Encode to latents using conditioned input
00306             z = autoencoder.encode_stage_2_inputs(conditioned)
00307
00308     # **CRITICAL FIX: Handle NaN in scale factor calculation**
00309     z_std = torch.std(z)
00310     logger.info(f"Latent std: {z_std.item():.6f}")
00311
00312     if torch.isnan(z_std) or torch.isinf(z_std) or z_std.item() < 1e-8:
00313         logger.warning(f"Invalid latent std ({z_std.item()}), using fallback scale_factor=1.0")
00314         scale_factor = 1.0
00315     else:
00316         scale_factor = 1 / z_std.item()
00317         # Clamp scale factor to reasonable range
00318         scale_factor = max(min(scale_factor, 10.0), 0.1)
00319
00320     logger.info(f"Scaling factor set to {scale_factor}")
00321
00322     # **Additional validation**
00323     if not np.isfinite(scale_factor):
00324         logger.error(f"Scale factor is not finite: {scale_factor}")
00325         scale_factor = 1.0
00326
00327     inferer = LatentDiffusionInferer(scheduler, scale_factor=scale_factor)
00328
00329     # AGGRESSIVE learning rate for faster convergence (problems identified)
00330     # Current signal ratios 0.03-0.08 too low, need stronger learning
00331     aggressive_lr = self.learning_rate * 1.0 # Use full original LR (1e-5)
00332     logger.info(f"Using AGGRESSIVE learning rate: {aggressive_lr} (original: {self.learning_rate}) for
00333     faster convergence")
00334
00335     opt_g = Adam(autoencoder.parameters(), lr=aggressive_lr, weight_decay=1e-5) # Reduced weight decay
00336     opt_d = Adam(discriminator.parameters(), lr=aggressive_lr, weight_decay=1e-5)
00337     opt_diff = Adam(unet.parameters(), lr=aggressive_lr, weight_decay=1e-5)
00338
00339     # trainers and early stopping
00340     ae_trainer = AutoencoderTrainer(autoencoder, discriminator, opt_g, opt_d, self.energy_min,
00341     self.energy_max, self.device)
00342
00343     # **CRITICAL FIX: Calculate dose normalization parameters before training**
00344     logger.info("Calculating dose normalization parameters from training data...")
00345     try:
00346         ae_trainer.calculate_dose_normalization_params(train_loader, energy=energy)
00347     except Exception as e:
00348         logger.error(f"Failed to calculate dose normalization parameters: {e}")
00349         logger.error("This usually indicates problems with the training data.")
00350         raise
00351
00352     # Validate that normalization parameters are reasonable
00353     if ae_trainer.clip_min is None or ae_trainer.clip_max is None:
00354         raise RuntimeError("clip_min or clip_max is None after calculation")
00355
00356     if ae_trainer.clip_max <= ae_trainer.clip_min:
00357         raise RuntimeError(f"Invalid dose range: clip_max ({ae_trainer.clip_max}) <= clip_min
00358     ({ae_trainer.clip_min})")
00359
00360     # Initialize loss functions after normalization parameters are set

```

```

00356         ae_trainer.setup_loss_functions()
00357
00358         logger.info(f"Starting autoencoder training for resolution={target_resolution}, energy={energy}")
00359         stopper = EarlyStopping(patience=self.patience)
00360         # autoencoder training loop
00361         for epoch in range(self.num_epochs):
00362             if self.stop_training:
00363                 logger.info(f"Autoencoder training aborted by user at epoch {epoch} for
resolution={target_resolution}, energy={energy}")
00364                 break
00365                 train_loss, gen_loss, disc_loss = ae_trainer.train_one_epoch(train_loader, epoch)
00366                 val_loss = ae_trainer.validate(val_loader)
00367                 # record losses
00368                 ae_train_losses.append(train_loss)
00369                 ae_val_losses.append(val_loss)
00370                 gen_losses.append(gen_loss)
00371                 disc_losses.append(disc_loss)
00372                 if stopper.update(val_loss):
00373                     logger.info(f"Early stopping autoencoder at epoch {epoch+1} for resolution={target_resolution},
energy={energy}")
00374                     break
00375
00376         # now diffusion training
00377         logger.info(f"Starting diffusion training for resolution={target_resolution}, energy={energy}")
00378         diff_trainer = DiffusionTrainer(unet, opt_diff, self.energy_min, self.energy_max, self.device)
00379
00380         # diffusion (UNet) training loop
00381         for epoch in range(self.num_epochs):
00382             if self.stop_training:
00383                 logger.info(f"Diffusion training aborted by user at epoch {epoch} for
resolution={target_resolution}, energy={energy}")
00384                 break
00385                 diff_loss = diff_trainer.train_one_epoch(train_loader, epoch, inferer, autoencoder)
00386                 # record diffusion loss
00387                 diff_losses.append(diff_loss)
00388
00389         # plot loss curves for this config
00390         Visualization.plot_loss_curves(
00391             ae_train_losses, ae_val_losses,
00392             gen_losses, disc_losses,
00393             diff_losses,
00394             resolution=target_resolution, energy=energy
00395         )
00396
00397         # **THIS IS THE CRUCIAL ADDITION:**
00398         # Save individual model checkpoint to memory after training each configuration
00399         self.save_models(
00400             autoencoder=autoencoder,
00401             unet=unet,
00402             optimizer_diff=opt_diff,
00403             optimizer_g=opt_g,
00404             optimizer_d=opt_d,
00405             epoch=epoch, # last epoch
00406             res=target_resolution,
00407             energy=energy,
00408             dose_mean=getattr(ae_trainer, 'dose_mean', None),
00409             dose_std=getattr(ae_trainer, 'dose_std', None),
00410             scale_factor=scale_factor,
00411             clip_min=getattr(ae_trainer, 'clip_min', 0.0), # FIX: Get from trainer
00412             clip_max=getattr(ae_trainer, 'clip_max', None), # FIX: Get from trainer
00413             smoothing_kernel=0,
00414         )
00415         logger.info(f" Model checkpoint saved to memory for resolution={target_resolution}, energy={energy}")
00416
00417         # after all individual trainings, save a single combined checkpoint
00418         self.save_combined_checkpoint()
00419         self.training_complete = True
00420         logger.info("All training finished for all configurations.")
00421
00422     def run_inference(self, ct_file_path, model_checkpoint=None):
00423         """
00424         Load a CT scan and run inference to compute dose distribution.
00425         """
00426         logger.info("=" * 60)
00427         logger.info("STARTING INFERENCE WORKFLOW")
00428         logger.info("=" * 60)
00429         logger.info(f"CT file path: {ct_file_path}")
00430         logger.info(f"Model checkpoint: {model_checkpoint}")
00431         logger.info(f"Device: {self.device}")
00432
00433         # determine model sources and load checkpoint dict if needed
00434         ckpt = None
00435         if model_checkpoint is None:
00436             if not self.models_by_energy:
00437                 logger.error("No trained models found and no checkpoint provided")
00438                 raise RuntimeError("No trained models found. Train the models first.")
00439             logger.info("Using pre-trained models from training session")

```



```

00440         elif isinstance(model_checkpoint, str):
00441             logger.info(f"Loading checkpoint from file: {model_checkpoint}")
00442             import torch as _torch
00443             try:
00444                 ckpt = _torch.load(model_checkpoint, map_location=self.device, weights_only=False)
00445                 logger.info("Checkpoint loaded successfully")
00446             except Exception as e:
00447                 logger.warning(f"Direct load failed: {e}")
00448                 logger.info("Attempting to load via file handle...")
00449                 with open(model_checkpoint, 'rb') as f:
00450                     ckpt = _torch.load(f, map_location=self.device, weights_only=False)
00451                 logger.info("Checkpoint loaded via file handle")
00452         elif isinstance(model_checkpoint, dict) and 'autoencoder' in model_checkpoint and 'unet' in
model_checkpoint:
00453             logger.info("Using provided checkpoint dictionary")
00454             ckpt = model_checkpoint
00455         else:
00456             logger.error("Invalid model checkpoint provided")
00457             raise ValueError("Invalid model checkpoint. Provide a path or a dict with 'autoencoder' and 'unet'
keys.")
00458     # if we have a checkpoint dict, rebuild models from state_dict
00459     if ckpt is not None:
00460         # --- Combined checkpoint support ---
00461         models_by_energy_ckpt = ckpt.get("models_by_energy", None)
00462         if models_by_energy_ckpt is not None:
00463             logger.info("Detected combined checkpoint with per-energy models, loading all energies")
00464             self.models_by_energy.clear()
00465             for energy_str, model_dict in models_by_energy_ckpt.items():
00466                 # Extract energy from key format like "res16x16x16_e46.53" or "res16_e46.53"
00467                 if "_e" in energy_str:
00468                     energy_val = float(energy_str.split("_e")[1])
00469                 else:
00470                     # Fallback: try to convert the whole string (for backward compatibility)
00471                     energy_val = float(energy_str)
00472                 logger.warning(f"Using fallback energy extraction for key: {energy_str}")
00473             # instantiate autoencoder
00474             ae = AutoencoderKL(
00475                 spatial_dims=3,
00476                 in_channels=2,
00477                 out_channels=1,
00478                 num_channels=(32, 32, 32),
00479                 latent_channels=2,
00480                 num_res_blocks=1,
00481                 norm_num_groups=8,
00482                 attention_levels=(False, False, True),
00483             ).to(self.device)
00484             ae.load_state_dict(model_dict["autoencoder"])
00485
00486             # instantiate unet
00487             un = DiffusionModelUNet(
00488                 spatial_dims=3,
00489                 in_channels=2,
00490                 out_channels=2,
00491                 with_conditioning=False, # Disabled cross-attention conditioning
00492                 num_res_blocks=1,
00493                 num_channels=(32, 64, 64),
00494                 attention_levels=(False, True, True),
00495                 num_head_channels=(0, 64, 64),
00496             ).to(self.device)
00497             un.load_state_dict(model_dict["unet"], strict=False)
00498             # scheduler
00499             sched = DDPMSScheduler(
00500                 num_train_timesteps=1000,
00501                 schedule="scaled_linear_beta",
00502                 beta_start=0.0015,
00503                 beta_end=0.0195,
00504             )
00505             self.models_by_energy[energy_val] = (ae, un, sched)
00506             logger.info("Loaded combined models for all energies")
00507
00508             # Update quad_energies and quad_weights based on available models
00509             available_energies = sorted(self.models_by_energy.keys())
00510             logger.info(f"Available energies in checkpoint: {available_energies}")
00511
00512             # Validate that all required quad_energies are available in the checkpoint
00513             missing_energies = []
00514             for energy in self.quad_energies:
00515                 if energy not in available_energies:
00516                     missing_energies.append(energy)
00517
00518             if missing_energies:
00519                 logger.warning(f"Missing energies in checkpoint: {missing_energies}")
00520                 logger.warning(f"Available energies: {available_energies}")
00521                 logger.warning(f"Requested energies: {self.quad_energies}")
00522                 # Filter out missing energies and adjust weights
00523                 original_energies = self.quad_energies.copy()
00524                 original_weights = self.quad_weights.copy()

```



```

00525
00526         filtered_energies = []
00527         filtered_weights = []
00528         for i, energy in enumerate(original_energies):
00529             if energy in available_energies:
00530                 filtered_energies.append(energy)
00531                 filtered_weights.append(original_weights[i])
00532
00533         if filtered_energies:
00534             # Renormalize weights
00535             total_weight = sum(filtered_weights)
00536             self.quad_energies = filtered_energies
00537             self.quad_weights = [w/total_weight for w in filtered_weights]
00538             logger.info(f"Filtered quad_energies: {self.quad_energies}")
00539             logger.info(f"Renormalized quad_weights: {self.quad_weights}")
00540         else:
00541             raise ValueError(f"No quadrature energies available in checkpoint! Available:
{available_energies}, Requested: {original_energies}")
00542     else:
00543         logger.info(" All quadrature energies are available in checkpoint")
00544         logger.info(f"Using quad_energies: {self.quad_energies}")
00545         logger.info(f"Using quad_weights: {self.quad_weights}")
00546
00547
00548     # Load common parameters by averaging over all energy checkpoints
00549     if models_by_energy_ckpt:
00550         scale_factors = []
00551         dose_means = []
00552         dose_stds = []
00553         clip_mins = []
00554         clip_maxs = []
00555         smoothing_kernels = []
00556
00557         for energy_key, model_dict in models_by_energy_ckpt.items():
00558             # Only exclude None values, but include 0.0 and other numeric values
00559             sf = model_dict.get("scale_factor")
00560             if sf is not None:
00561                 scale_factors.append(sf)
00562
00563             dm = model_dict.get("dose_mean")
00564             if dm is not None:
00565                 dose_means.append(dm)
00566
00567             ds = model_dict.get("dose_std")
00568             if ds is not None:
00569                 dose_stds.append(ds)
00570
00571             cm = model_dict.get("clip_min")
00572             if cm is not None:
00573                 clip_mins.append(cm)
00574
00575             # clip_max can legitimately be None
00576             clip_maxs.append(model_dict.get("clip_max"))
00577
00578             sk = model_dict.get("smoothing_kernel")
00579             if sk is not None:
00580                 smoothing_kernels.append(sk)
00581
00582     # Calculate averages with fallback defaults if no valid values found
00583     self.scale_factor = sum(scale_factors) / len(scale_factors) if scale_factors else 1.0
00584     self.dose_mean = sum(dose_means) / len(dose_means) if dose_means else 0.0
00585     self.dose_std = sum(dose_stds) / len(dose_stds) if dose_stds else 1.0
00586     self.clip_min = sum(clip_mins) / len(clip_mins) if clip_mins else 0.0
00587     self.smoothing_kernel = sum(smoothing_kernels) / len(smoothing_kernels) if smoothing_kernels
else 0
00588
00589
00590     # For clip_max, use the average of non-None values, or None if all are None
00591     valid_clip_maxs = [x for x in clip_maxs if x is not None]
00592     self.clip_max = sum(valid_clip_maxs) / len(valid_clip_maxs) if valid_clip_maxs else None
00593
00594     logger.info(f"Loaded averaged parameters from {len(models_by_energy_ckpt)} energy models:")
00595     logger.info(f"    scale_factor={self.scale_factor:.6f} (from {len(scale_factors)} values)")
00596     logger.info(f"    dose_mean={self.dose_mean:.6f} (from {len(dose_means)} values)")
00597     logger.info(f"    dose_std={self.dose_std:.6f} (from {len(dose_stds)} values)")
00598     logger.info(f"    clip_min={self.clip_min:.6f} (from {len(clip_mins)} values)")
00599     logger.info(f"    clip_max={self.clip_max} (from {len(valid_clip_maxs)} values)")
00599     logger.info(f"    smoothing_kernel={self.smoothing_kernel:.1f} (from {len(smoothing_kernels)}
values)")
00600
00601     # switch to in-memory inference path
00602     model_checkpoint = None
00603 else:
00604     # --- End combined checkpoint support ---
00605
00606     logger.info("Rebuilding models from checkpoint state_dict...")
00607     ae = AutoencoderKL(spatial_dims=3, in_channels=2, out_channels=1,
00608                       num_channels=(32, 32, 32), latent_channels=2,

```

```

00609             num_res_blocks=1, norm_num_groups=8,
00610             attention_levels=(False, False, True)).to(self.device)
00611         ae.load_state_dict(ckpt['autoencoder'])
00612         logger.info(" Autoencoder loaded from checkpoint")
00613
00614         unet_state = ckpt['unet']
00615
00616         un = DiffusionModelUNet(
00617             spatial_dims=3, in_channels=2, out_channels=2,
00618             with_conditioning=False, # Disabled cross-attention conditioning
00619             num_res_blocks=1, num_channels=(32, 64, 64),
00620             attention_levels=(False, True, True),
00621             num_head_channels=(0, 64, 64)
00622         ).to(self.device)
00623
00624         # Filter checkpoint to only matching shapes before loading
00625         pretrained_dict = ckpt['unet']
00626         model_dict = un.state_dict()
00627         filtered_dict = {
00628             k: v for k, v in pretrained_dict.items()
00629             if k in model_dict and model_dict[k].shape == v.shape
00630         }
00631         missing = set(model_dict.keys()) - set(filtered_dict.keys())
00632         unexpected = set(pretrained_dict.keys()) - set(filtered_dict.keys())
00633         # Load only matching parameters
00634         un.load_state_dict(filtered_dict, strict=False)
00635         if missing or unexpected:
00636             logger.warning(f"UNet checkpoint loaded with missing keys: {sorted(missing)} and unexpected
keys: {sorted(unexpected)}. Mismatched shapes filtered out.")
00637         else:
00638             logger.info(" UNet loaded from checkpoint (all keys matched)")
00639
00640         sched = DDPMSScheduler(num_train_timesteps=1000,
00641                                schedule="scaled_linear_beta",
00642                                beta_start=0.0015, beta_end=0.0195)
00643         self.models_by_energy = {energy: (ae, un, sched) for energy in self.quad_energies}
00644         self.autoencoder, self.unet, self.scheduler = ae, un, sched
00645         logger.info(" Scheduler created and models assigned")
00646
00647         # Retrieve dataset-wide dose statistics for de-normalisation
00648         self.dose_mean = ckpt.get("dose_mean", 0.0)
00649         self.dose_std = ckpt.get("dose_std", 1.0)
00650         logger.info(f"Dose scaling loaded from checkpoint: mean={self.dose_mean}, std={self.dose_std}")
00651         self.scale_factor = ckpt.get("scale_factor", 1.0)
00652         self.clip_min = ckpt.get("clip_min", 0.0)
00653         self.clip_max = ckpt.get("clip_max", None)
00654         self.smoothing_kernel = ckpt.get("smoothing_kernel", 0)
00655
00656         # lazy import to avoid circular
00657         import nibabel as nib
00658         import numpy as np
00659         from corrected_inference import CorrectedInferenceModule
00660
00661         # build tensor from CT file: support both NIfTI (.nii, .nii.gz) and NumPy (.npy)
00662         logger.info("Loading CT data...")
00663         logger.info(f"CT file path: {ct_file_path}")
00664
00665         path_lower = ct_file_path.lower()
00666         if path_lower.endswith('.nii') or path_lower.endswith('.nii.gz'):
00667             logger.info("Loading NIfTI file...")
00668             nifti_img = nib.load(ct_file_path)
00669             arr = np.asarray(nifti_img.dataobj)
00670             logger.info(f" NIfTI loaded: shape={arr.shape}, dtype={arr.dtype}")
00671         elif path_lower.endswith('.npy'):
00672             logger.info("Loading NumPy file...")
00673             arr = np.load(ct_file_path)
00674             logger.info(f" NumPy loaded: shape={arr.shape}, dtype={arr.dtype}")
00675         else:
00676             logger.error(f"Unsupported CT file format: {ct_file_path}")
00677             raise ValueError(f"Unsupported CT file format: {ct_file_path}")
00678
00679         logger.info(f"CT data range: {arr.min():.6f} to {arr.max():.6f}")
00680         ct_tensor = torch.from_numpy(arr).unsqueeze(0).to(self.device) # shape [1, D, H, W]
00681         logger.info(f"CT tensor shape: {ct_tensor.shape}")
00682
00683         # Use corrected inference module
00684         logger.info("Setting up corrected inference module...")
00685         try:
00686             if ckpt is not None and "models_by_energy" in ckpt:
00687                 # Use already loaded models from combined checkpoint
00688                 logger.info("Using already loaded models from combined checkpoint")
00689                 inf_mod = CorrectedInferenceModule(
00690                     models_by_energy=self.models_by_energy,
00691                     device=self.device,
00692                     energies=self.quad_energies,
00693                     scale_factor=self.scale_factor,
00694                     dose_mean=self.dose_mean,

```

```

00695         dose_std=self.dose_std,
00696         clip_min=self.clip_min,
00697         clip_max=self.clip_max,
00698         smoothing_kernel=self.smoothing_kernel
00699     )
00700     elif model_checkpoint is not None:
00701         # Load from checkpoint path
00702         logger.info("Loading models from checkpoint path")
00703         inf_mod = CorrectedInferenceModule(
00704             checkpoint_path=model_checkpoint,
00705             device=self.device,
00706             energies=self.quad_energies
00707         )
00708     else:
00709         # Use pre-trained models from training session
00710         logger.info("Using pre-trained models from training session")
00711         inf_mod = CorrectedInferenceModule(
00712             models_by_energy=self.models_by_energy,
00713             device=self.device,
00714             energies=self.quad_energies,
00715             scale_factor=getattr(self, 'scale_factor', 1.0),
00716             dose_mean=getattr(self, 'dose_mean', 0.0),
00717             dose_std=getattr(self, 'dose_std', 1.0),
00718             clip_min=getattr(self, 'clip_min', 0.0),
00719             clip_max=getattr(self, 'clip_max', None),
00720             smoothing_kernel=getattr(self, 'smoothing_kernel', 0)
00721         )
00722         logger.info(" CorrectedInferenceModule created successfully")
00723     except Exception as e:
00724         logger.error(f"Failed to create CorrectedInferenceModule: {e}")
00725         logger.error("CorrectedInferenceModule creation traceback:", exc_info=True)
00726         raise e
00727
00728     # run quadrature-based inference over all energies
00729     logger.info("Running corrected inference...")
00730     logger.info(f"Quadrature energies: {self.quad_energies}")
00731     logger.info(f"Quadrature weights: {self.quad_weights}")
00732
00733     try:
00734         dose = inf_mod.run_inference(ct_tensor, self.quad_energies, self.quad_weights)
00735         logger.info(" Corrected inference completed successfully")
00736         logger.info(f"Dose tensor shape: {dose.shape}")
00737         logger.info(f"Dose range: {dose.min():.6f} to {dose.max():.6f}")
00738     except Exception as e:
00739         logger.error(f"Corrected inference failed: {e}")
00740         logger.error("Corrected inference traceback:", exc_info=True)
00741         raise e
00742
00743     # convert to numpy and remove batch dim
00744     logger.info("Converting output to NumPy and preparing NifTI...")
00745     dose_np = dose.detach().cpu().numpy()
00746     if dose_np.ndim == 4 and dose_np.shape[0] == 1:
00747         dose_np = dose_np[0]
00748     logger.info(f"Final dose shape: {dose_np.shape}")
00749     logger.info(f"Final dose range: {dose_np.min():.6f} to {dose_np.max():.6f}")
00750
00751     # create affine
00752     import numpy as _np, nibabel as _nib, json as _json, os as _os
00753     from nibabel.nifti1 import Nifti1Extension
00754     affine = _np.eye(4)
00755     img = _nib.Nifti1Image(dose_np, affine)
00756     logger.info(" NifTI image created")
00757
00758     # attach cubes.json manifest if available
00759     manifest_path = _os.path.join(self.root_dir, 'cubes.json')
00760     if _os.path.exists(manifest_path):
00761         logger.info(f"Loading manifest from: {manifest_path}")
00762         with open(manifest_path, 'r') as mf:
00763             manifest = _json.load(mf)
00764         # Encode manifest JSON to bytes for NifTI extension
00765         ext = Nifti1Extension('comment', _json.dumps(manifest).encode('utf-8'))
00766         img.header.extensions.append(ext)
00767         logger.info(" Manifest attached to NifTI header")
00768     else:
00769         logger.info("No cubes.json manifest found, skipping manifest attachment")
00770
00771     # save NifTI file
00772     out_path = _os.path.join(self.root_dir, 'inference_with_manifest.nii.gz')
00773     logger.info(f"Saving inference result to: {out_path}")
00774     _nib.save(img, out_path)
00775     logger.info(" NifTI file saved successfully")
00776
00777     logger.info("=" * 60)
00778     logger.info("INFERENCE WORKFLOW COMPLETED SUCCESSFULLY")
00779     logger.info(f"Output file: {out_path}")
00780     logger.info("=" * 60)
00781     # set result path
00782     self.dose_result_path = out_path

```

```

00782         return out_path
00783
00784     def run_energy_conditioned_training(self):
00785         """
00786         Executes energy-conditioned training pipeline - trains a SINGLE model with ALL energies.
00787
00788         This is the new approach that trains one model with energy conditioning,
00789         instead of separate models for each energy.
00790
00791         Steps:
00792         1. Load dataset with ALL energies (no filtering by energy)
00793         2. Create DataLoaders for training and validation
00794         3. Initialize models with 2-channel input (CT + Energy)
00795         4. Train autoencoder with energy conditioning
00796         5. Train diffusion model with energy conditioning
00797         6. Save the unified model
00798         """
00799         logger.info("=== STARTING ENERGY-CONDITIONED TRAINING ===")
00800         logger.info("Training a SINGLE model with ALL energies instead of separate models")
00801         logger.info("Starting training with hyperparameters:")
00802         logger.info(f"  batch_size={self.batch_size}, num_epochs={self.num_epochs},
learning_rate={self.learning_rate}, patience={self.patience}, cube_size={self.cube_size}")
00803         logger.info(f"  resolutions={self.resolutions}, energies={self.energies}")
00804
00805         # For simplicity, use first resolution only (can be extended later)
00806         res = self.resolutions[0]
00807         logger.info(f"--- Training unified model at resolution={res} with energies={self.energies} ---")
00808
00809         # Setup transforms
00810         logger.info(f" CORRECTED: Pipeline uses consistent resolution: {res}")
00811         self.transforms = Compose([
00812             LoadImage(keys=["input", "target"], reader=NumpyReader),
00813             EnsureChannelFirstd(keys=["input", "target"]),
00814             EnsureTyped(keys=["input", "target"]),
00815             Orientationd(keys=["input", "target"], axcodes="RAS"),
00816             # CRITICAL FIX: Use Resized instead of Spacingd for array dimensions
00817             Resized(keys=["input", "target"], spatial_size=res, mode=("bilinear", "nearest")),
00818             # REMOVED: SpatialPadd and CenterSpatialCropd that destroyed the target resolution
00819             # SpatialPadd(keys=["input", "target"], spatial_size=self.cube_size, method="symmetric"),
00820             # CenterSpatialCropd(keys=["input", "target"], roi_size=self.cube_size),
00821             ScaleIntensityRangePercentilesd(
00822                 keys="input", lower=0, upper=99.5, b_min=0, b_max=1
00823             ),
00824             ToTensord(keys=["input", "target"]),
00825             EnsureTyped(keys=["energy"]),
00826             ToTensord(keys=["energy"])
00827         ])
00828
00829         # Initialize history lists for plotting
00830         ae_train_losses = []
00831         ae_val_losses = []
00832         gen_losses = []
00833         disc_losses = []
00834         diff_losses = []
00835
00836         # Load dataset WITHOUT filtering by energy (this is the key change!)
00837         data_module = DataLoaderModule(
00838             root_dir=self.root_dir,
00839             transforms=self.transforms
00840         )
00841         # Load complete dataset with ALL energies
00842         ds_full = data_module.load_dataset(section=None)
00843         # NO FILTERING BY ENERGY! - This is the crucial difference
00844         logger.info(f"Loaded {len(ds_full)} samples with ALL energies: {set([float(s['energy']).item()) for s in
ds_full])}")
00845
00846         train_ds, val_ds = data_module.split_dataset(ds_full)
00847         train_loader = data_module.create_data_loader(train_ds, self.batch_size, shuffle=True)
00848         val_loader = data_module.create_data_loader(val_ds, self.batch_size, shuffle=False)
00849
00850         logger.info(f"Training set: {len(train_ds)} samples")
00851         logger.info(f"Validation set: {len(val_ds)} samples")
00852
00853         # Instantiate models with 2-channel input for energy conditioning
00854         autoencoder = AutoencoderKL(
00855             spatial_dims=3,
00856             in_channels=2, # CT + Energy channel
00857             out_channels=1,
00858             num_channels=(32, 32, 32),
00859             latent_channels=2,
00860             num_res_blocks=1,
00861             norm_num_groups=8,
00862             attention_levels=(False, False, True),
00863         ).to(self.device)
00864
00865         discriminator = PatchDiscriminator(
00866             spatial_dims=3,

```

```

00867         num_layers_d=3,
00868         num_channels=32,
00869         in_channels=1, # Output is still 1 channel
00870         out_channels=1
00871     ).to(self.device)
00872
00873     unet = DiffusionModelUNet(
00874         spatial_dims=3,
00875         in_channels=2, # Latent space is 2 channels
00876         out_channels=2,
00877         with_conditioning=False, # Disabled cross-attention conditioning
00878         num_res_blocks=1,
00879         num_channels=(32, 64, 64),
00880         attention_levels=(False, True, True),
00881         num_head_channels=(0, 64, 64),
00882     ).to(self.device)
00883
00884     scheduler = DDPMSScheduler(num_train_timesteps=1000, beta_start=0.0015, beta_end=0.0195)
00885
00886     # Calculate latent scaling factor
00887     with torch.no_grad():
00888         # Take a sample from training data to compute scaling factor
00889         sample_batch = first(train_loader)
00890         sample_input = sample_batch["input"].to(self.device)
00891         sample_energy = sample_batch["energy"].to(self.device)
00892
00893         # Apply energy conditioning for scaling factor calculation
00894         B, C, D, H, W = sample_input.shape
00895         normalized_energy = sample_energy.float() / 100.0
00896         energy_tensor = normalized_energy.view(B, 1, 1, 1, 1).expand(B, 1, D, H, W)
00897         conditioned_input = torch.cat([sample_input, energy_tensor], dim=1)
00898
00899         encoded = autoencoder.encode(conditioned_input)
00900         if isinstance(encoded, tuple):
00901             z = encoded[0]
00902         else:
00903             z = encoded.latent_dist.sample()
00904         scale_factor = 1 / torch.std(z)
00905
00906     logger.info(f"Scaling factor set to {scale_factor}")
00907
00908     # Train Autoencoder with energy conditioning
00909     logger.info(f"Starting autoencoder training for unified model")
00910
00911     # Create optimizers with weight decay for stability
00912     optimizer_g = torch.optim.Adam(autoencoder.parameters(), lr=self.learning_rate, weight_decay=1e-5)
00913     optimizer_d = torch.optim.Adam(discriminator.parameters(), lr=self.learning_rate, weight_decay=1e-5)
00914
00915     ae_trainer = AutoencoderTrainer(
00916         autoencoder=autoencoder,
00917         discriminator=discriminator,
00918         optimizer_g=optimizer_g,
00919         optimizer_d=optimizer_d,
00920         device=self.device
00921     )
00922
00923     early_stopping = EarlyStopping(patience=self.patience)
00924
00925     for epoch in range(self.num_epochs):
00926         if self.stop_training:
00927             logger.info("Training aborted by user.")
00928             return
00929
00930         recon_loss, adv_loss, disc_loss = ae_trainer.train_one_epoch(train_loader, epoch)
00931         ae_train_losses.append(recon_loss)
00932         gen_losses.append(adv_loss)
00933         disc_losses.append(disc_loss)
00934
00935         val_loss = ae_trainer.validate(val_loader)
00936         ae_val_losses.append(val_loss)
00937
00938         early_stop = early_stopping.update(val_loss)
00939         if early_stop:
00940             logger.info(f"Early stopping triggered at epoch {epoch}")
00941             break
00942
00943     # Train Diffusion Model with energy conditioning
00944     logger.info(f"Starting diffusion training for unified model")
00945
00946     # Create optimizer for diffusion model
00947     optimizer_diff = torch.optim.Adam(unet.parameters(), lr=self.learning_rate)
00948
00949     diff_trainer = DiffusionTrainer(
00950         diffusion_model=unet,
00951         optimizer_diff=optimizer_diff,
00952         device=self.device
00953     )

```

```

00954
00955     # Train diffusion for fewer epochs (typically 10-20)
00956     from generative.inferers import LatentDiffusionInferer
00957     inferer = LatentDiffusionInferer(scheduler=scheduler, scale_factor=scale_factor)
00958
00959     diff_epochs = max(10, self.num_epochs // 2)
00960     for epoch in range(diff_epochs):
00961         if self.stop_training:
00962             logger.info("Training aborted by user.")
00963             return
00964
00965         diff_loss = diff_trainer.train_one_epoch(train_loader, epoch, inferer=inferer, autoencoder=autoencoder)
00966         diff_losses.append(diff_loss)
00967
00968     # Save the unified model
00969     model_filename = f"unified_energy_conditioned_model_res{res}_energies{len(self.energies)}.ckpt"
00970     model_path = os.path.join(os.getcwd(), model_filename)
00971
00972     torch.save({
00973         'autoencoder': autoencoder.state_dict(),
00974         'unet': unet.state_dict(),
00975         'scheduler': scheduler.state_dict(),
00976         'scale_factor': scale_factor,
00977         'resolutions': [res],
00978         'energies': self.energies,
00979         'training_config': {
00980             'batch_size': self.batch_size,
00981             'num_epochs': self.num_epochs,
00982             'learning_rate': self.learning_rate,
00983             'cube_size': self.cube_size
00984         }
00985     }, model_path)
00986
00987     logger.info(f" Unified energy-conditioned model saved to: {model_path}")
00988     logger.info("=== ENERGY-CONDITIONED TRAINING COMPLETED ===")
00989
00990     return {
00991         'model_path': model_path,
00992         'autoencoder': autoencoder,
00993         'unet': unet,
00994         'scheduler': scheduler,
00995         'scale_factor': scale_factor,
00996         'losses': {
00997             'ae_train': ae_train_losses,
00998             'ae_val': ae_val_losses,
00999             'gen': gen_losses,
01000             'disc': disc_losses,
01001             'diff': diff_losses
01002         }
01003     }

```

8.28 sourcecode/training_pipeline.py File Reference

Classes

- class [training_pipeline.EarlyStopping](#)
- class [training_pipeline.AutoencoderTrainer](#)
- class [training_pipeline.DiffusionTrainer](#)

Namespaces

- namespace [training_pipeline](#)

Variables

- int [training_pipeline.DOSE_EPS](#) = 1e-8
- int [training_pipeline.TV_WEIGHT](#) = 5e-3
- [training_pipeline.CLIP_MIN](#) = None
- [training_pipeline.CLIP_MAX](#) = None
- [training_pipeline.logger](#) = logging.getLogger(__name__)

8.29 training_pipeline.py

[Go to the documentation of this file.](#)

```

00001 #training_pipeline.py
00002
00003 import torch
00004 import torch.nn.functional as F
00005 from torch.nn import L1Loss
00006 from tqdm import tqdm
00007 from monai.losses import PatchAdversarialLoss, PerceptualLoss
00008 import math
00009 from torch.nn.utils import clip_grad_norm_
00010 import log_config
00011 import logging
00012 import numpy as np
00013 from data_management import HotspotPatchDataset
00014
00015
00016
00017
00018 # Constants
00019 #ENERGY_NORMALIZATION_FACTOR = 50.0 # max(energy) in training set (46.53keV)
00020 DOSE_EPS = 1e-8 # small value to avoid division-by-zero in normalisation
00021 TV_WEIGHT = 5e-3 # stronger weight for total-variation smoothness loss
00022 # Min-max limits for dose normalisation (Gy).
00023 # These will be updated dynamically from the dataset statistics
00024 CLIP_MIN = None
00025 CLIP_MAX = None
00026
00027 logger = logging.getLogger(__name__)
00028
00029 class EarlyStopping:
00030     """
00031     Monitors the validation loss and triggers early stopping if the loss
00032     the loss does not improve over a defined number of epochs.
00033     """
00034     def __init__(self, patience=5):
00035         """
00036         Args:
00037             patience (int): Number of epochs permitted without improvement.
00038         """
00039         self.patience = patience
00040         self.best_val_loss = float('inf')
00041         self.epochs_no_improvement = 0
00042
00043     def update(self, val_loss):
00044         """
00045         Compares the current validation loss with the best loss to date.
00046         Args:
00047             val_loss (float): Current loss of validation.
00048         Returns:
00049             bool: If Early Stopping is triggered true otherwise False.
00050         """
00051         if val_loss < self.best_val_loss:
00052             self.best_val_loss = val_loss
00053             self.epochs_no_improvement = 0
00054             return False
00055         else:
00056             self.epochs_no_improvement += 1
00057             return self.epochs_no_improvement >= self.patience
00058
00059     def reset(self):
00060         """
00061         Resets the early stopping parameters.
00062         """
00063         self.best_val_loss = float('inf')
00064         self.epochs_no_improvement = 0
00065
00066 class AutoencoderTrainer:
00067     """
00068     Encapsulates the training process for the autoencoder, including validation.
00069
00070     This implementation has been updated to integrate energy conditioning.
00071     Each sample is expected to contain an "energy" field, which is normalized, expanded to match
00072     the spatial dimensions of the image, and then concatenated as an additional input channel.
00073     """
00074     def __init__(self, autoencoder, discriminator, optimizer_g, optimizer_d, energy_min, energy_max, device,
00075                  kl_weight=1e-6, adv_weight=0.001, perceptual_weight=0.005,
00076                  tv_weight=TV_WEIGHT, warm_up_epochs=2):
00077         """
00078         Args:
00079             autoencoder (torch.nn.Module): The autoencoder model.
00080             discriminator (torch.nn.Module): The discriminator model (for adversarial training).
00081             optimizer_g (torch.optim.Optimizer): Optimizer for the generator.
00082             optimizer_d (torch.optim.Optimizer): Optimizer for the discriminator.
00083             device (str): Device to run the training on ('cpu' or 'cuda').
00084             kl_weight (float): Weight for KL divergence loss.
00085             adv_weight (float): Weight for adversarial loss. Default: 0.001
00086             perceptual_weight (float): Weight for perceptual loss. Default: 0.005
00087             tv_weight (float): Weight for total-variation loss.

```



```

00088         warm_up_epochs (int): Number of epochs to warm up the KL weight.
00089         """
00090         self.autoencoder = autoencoder
00091         self.discriminator = discriminator
00092         self.optimizer_g = optimizer_g
00093         self.optimizer_d = optimizer_d
00094         self.device = device
00095
00096         # Initialize dose normalization parameters (will be calculated from data)
00097         self.clip_min = None
00098         self.clip_max = None
00099         self.energy_min = energy_min
00100         self.energy_max = energy_max
00101         self.kl_weight = kl_weight
00102         self.adv_weight = adv_weight
00103         self.perceptual_weight = perceptual_weight
00104         self.tv_weight = tv_weight
00105         self.warm_up_epochs = warm_up_epochs
00106
00107         # Loss functions will be initialized after normalization parameters are calculated
00108         self.l1_loss = None
00109         self.adv_loss = None
00110         self.perceptual_loss = None
00111
00112     def calculate_dose_normalization_params(self, dataloader, energy=None):
00113         """
00114         Calculate clip_min and clip_max from the training data for dose normalization.
00115         This should be called before training starts.
00116
00117         Args:
00118             dataloader: Training dataloader
00119             energy: Energy level for energy-specific normalization
00120         """
00121         logger.info("Calculating dose normalization parameters from training data...")
00122         if energy is not None:
00123             logger.info(f"Energy-specific normalization for energy: {energy:.2f}")
00124
00125         all_dose_values = []
00126         sample_count = 0
00127
00128         for batch in dataloader:
00129             target_dose = batch["target"] # Assuming target is the dose data
00130             sample_count += target_dose.shape[0]
00131
00132             # Convert to numpy and flatten
00133             dose_values = target_dose.cpu().numpy().flatten()
00134
00135             # Log sample statistics
00136             logger.info(f"Batch {len(all_dose_values)+1}: shape={target_dose.shape}, "
00137                       f"min={dose_values.min():.6f}, max={dose_values.max():.6f}, "
00138                       f"mean={dose_values.mean():.6f}, nonzero_count={np.count_nonzero(dose_values)}")
00139
00140             all_dose_values.append(dose_values)
00141
00142         if not all_dose_values:
00143             raise RuntimeError("No dose data found in dataloader")
00144
00145         all_dose_values = np.concatenate(all_dose_values)
00146         logger.info(f"Total samples processed: {sample_count}, total voxels: {len(all_dose_values)}")
00147         logger.info(f"Overall dose stats: min={all_dose_values.min():.6f}, max={all_dose_values.max():.6f}, "
00148                   f"mean={all_dose_values.mean():.6f}, std={all_dose_values.std():.6f}")
00149
00150         # ENHANCED DEBUGGING: Check if dose values are realistic
00151         nonzero_values = all_dose_values[all_dose_values > 0]
00152         if len(nonzero_values) > 0:
00153             logger.info(f"Non-zero dose analysis:")
00154             logger.info(f"  Count: {len(nonzero_values)}/{len(all_dose_values)} "
00155                       f"({100*len(nonzero_values)/len(all_dose_values):.1f}%)")
00156             logger.info(f"  Min non-zero: {nonzero_values.min():.6f}")
00157             logger.info(f"  Max non-zero: {nonzero_values.max():.6f}")
00158             logger.info(f"  Mean non-zero: {nonzero_values.mean():.6f}")
00159             logger.info(f"  Median non-zero: {np.median(nonzero_values):.6f}")
00160
00161             # Check if values seem suspiciously low (should be in tens, not decimals)
00162             if nonzero_values.max() < 5.0:
00163                 logger.error(f"DOSE VALUES TOO LOW! Max dose {nonzero_values.max():.6f} < 5.0")
00164                 logger.error("Expected dose values in range 10-50+ for radiotherapy!")
00165                 logger.error("This suggests a major pipeline or data scaling issue.")
00166
00167         # Calculate percentiles to avoid outliers, but ensure meaningful range
00168         clip_min_candidate = float(np.percentile(all_dose_values, 0.1))
00169
00170         # **ENERGY-SPECIFIC DOSE NORMALIZATION FIX**
00171         if energy is not None and energy > 40.0:
00172             # **CRITICAL FIX: For very high energies, use actual max to preserve signal strength**
00173             # Don't use percentiles for high energies as they cause severe underestimation
00174             clip_max_candidate = float(np.max(all_dose_values))

```



```

00174         logger.info(f" High energy ({energy:.2f}): Using ACTUAL maximum for clip_max: {clip_max_candidate:.2f}")
00175
00176         # Only cap if extremely unreasonable (>150 Gy)
00177         if clip_max_candidate > 150.0:
00178             clip_max_candidate = 100.0
00179             logger.info(f" Capping clip_max to 100.0 for energy {energy:.2f} (was
float(np.max(all_dose_values):.2f}))")
00180         else:
00181             logger.info(f" Using full range clip_max={clip_max_candidate:.2f} for energy {energy:.2f}")
00182     else:
00183         # Normal case: use actual maximum for better reconstruction
00184         clip_max_candidate = float(np.max(all_dose_values))
00185
00186     # **CRITICAL FIX: Handle edge cases where data is all zeros or very close**
00187     if clip_max_candidate - clip_min_candidate < 1e-6:
00188         logger.warning(f"Percentile dose range too small: {clip_max_candidate - clip_min_candidate:.10f}")
00189         logger.warning("Using actual min/max normalization parameters...")
00190
00191         # Use actual min/max if percentiles fail
00192         actual_min = float(all_dose_values.min())
00193         actual_max = float(all_dose_values.max())
00194
00195         if actual_max - actual_min < 1e-6:
00196             # Data is essentially constant - use safe defaults
00197             logger.error("All dose values are essentially the same! Using emergency defaults.")
00198             self.clip_min = 0.0
00199             self.clip_max = 1.0 # Safe fallback
00200         else:
00201             # Use actual range
00202             self.clip_min = actual_min
00203             self.clip_max = actual_max
00204             logger.info(f"Successfully using actual range: {self.clip_min:.6f} to {self.clip_max:.6f}")
00205     else:
00206         # Normal case - use percentiles
00207         self.clip_min = clip_min_candidate
00208         self.clip_max = clip_max_candidate
00209         logger.info(f"Successfully using percentile range: {self.clip_min:.6f} to {self.clip_max:.6f}")
00210
00211     logger.info(f"Final dose normalization parameters:")
00212     logger.info(f" clip_min: {self.clip_min:.6f}")
00213     logger.info(f" clip_max: {self.clip_max:.6f}")
00214     logger.info(f" dose range: {self.clip_max - self.clip_min:.6f}")
00215
00216     # Validate parameters
00217     if self.clip_max <= self.clip_min:
00218         raise RuntimeError(f"Invalid normalization range: clip_max ({self.clip_max}) <= clip_min
({self.clip_min})")
00219
00220     return self.clip_min, self.clip_max
00221
00222 def setup_loss_functions(self):
00223     """Initialize loss functions after normalization parameters are set."""
00224     # Loss functions
00225     self.l1_loss = L1Loss()
00226     self.adv_loss = PatchAdversarialLoss(criterion="least_squares")
00227     self.perceptual_loss = PerceptualLoss(spatial_dims=3, network_type="alex").to(self.device)
00228
00229     logger.info("Loss functions initialized successfully")
00230
00231 def weighted_dose_loss(self, pred, target, weight_factor=10000.0, dose_threshold=1e-6):
00232     """
00233     Extreme weighted L1 loss for ultra-sparse dose data.
00234
00235     Args:
00236         pred: Predicted dose values
00237         target: Ground truth dose values
00238         weight_factor: Extreme multiplier for non-zero dose voxels (10,000x)
00239         dose_threshold: Minimum value to consider as "real dose"
00240     """
00241     # Create extreme weight mask
00242     # Weights are 10,000x for dose voxels, nearly zero for background
00243     weights = torch.where(target > dose_threshold, 100000.0, 0.01)
00244
00245     # Weighted absolute difference
00246     weighted_diff = torch.abs(pred - target) * weights
00247
00248     # Calculate stats for logging
00249     total_voxels = target.numel()
00250     dose_voxels = (target > dose_threshold).sum().item()
00251
00252     if dose_voxels > 0:
00253         dose_loss = weighted_diff[target > dose_threshold].mean()
00254         background_loss = weighted_diff[target <= dose_threshold].mean()
00255
00256         logger.debug(f"EXTREME Weighted loss - {dose_voxels}/{total_voxels} dose voxels "
00257                     f"({100*dose_voxels/total_voxels:.4f}%), "
00258                     f"dose_loss: {dose_loss:.6f}, bg_loss: {background_loss:.6f}")

```

```

00259         else:
00260             logger.warning("No dose voxels found - using minimal background loss")
00261             return weighted_diff.mean() * 0.001 # Minimal loss when no dose
00262
00263         return weighted_diff.mean()
00264
00265     def masked_dose_loss(self, pred, target, dose_threshold=1e-6):
00266         """
00267         Pure dose-only loss - ignores background completely.
00268
00269         Args:
00270             pred: Predicted dose values
00271             target: Ground truth dose values
00272             dose_threshold: Minimum value to consider as "real dose"
00273         """
00274         # Create mask for non-zero dose regions
00275         dose_mask = target > dose_threshold
00276
00277         if dose_mask.sum() == 0:
00278             # No dose regions - return extremely small loss to avoid stopping training
00279             logger.warning("No dose regions found in batch - returning minimal loss")
00280             return torch.tensor(1e-6, device=target.device, requires_grad=True)
00281
00282         # Apply mask - ONLY train on dose regions
00283         masked_pred = pred[dose_mask]
00284         masked_target = target[dose_mask]
00285
00286         # MSE loss for better gradient signal on sparse data
00287         loss = F.mse_loss(masked_pred, masked_target)
00288
00289         dose_voxels = dose_mask.sum().item()
00290         total_voxels = target.numel()
00291         logger.debug(f"PURE Dose loss - Training ONLY on {dose_voxels}/{total_voxels} voxels "
00292                     f"({100*dose_voxels/total_voxels:.4f}%), MSE loss: {loss:.6f}")
00293
00294         return loss
00295
00296     def extreme_dose_focused_loss(self, pred, target, dose_threshold=1e-6):
00297         """
00298         Combination of extreme weighted and pure masked loss for ultra-sparse data.
00299         Uses 100,000x weights and pure MSE loss for maximum signal.
00300         """
00301         dose_mask = target > dose_threshold
00302         dose_voxels = dose_mask.sum().item()
00303
00304         if dose_voxels == 0:
00305             # No dose - minimal loss to keep training alive
00306             return torch.tensor(1e-6, device=target.device, requires_grad=True)
00307
00308         # Pure dose loss (MSE on dose regions only) - better gradients for sparse data
00309         masked_pred = pred[dose_mask]
00310         masked_target = target[dose_mask]
00311         pure_dose_loss = F.mse_loss(masked_pred, masked_target)
00312
00313         # EXTREME weighted loss for EXTREME sparsity (1,000,000x weight for dose - MASSIVE INCREASE for 0.005%
00314         sparsity)
00315         sparsity_percent = 100 * dose_voxels / target.numel()
00316
00317         # Adaptive weight based on actual sparsity level - MUCH MORE AGGRESSIVE
00318         if sparsity_percent < 0.006: # Less than 0.006% sparsity (like we see: 0.005%)
00319             dose_weight = 1000000.0 # 1M weight for ultra-extreme cases
00320             background_weight = 0.0000001 # Nearly ignore background completely
00321         elif sparsity_percent < 0.01: # Less than 0.01% sparsity
00322             dose_weight = 500000.0 # 500K weight for extreme cases
00323             background_weight = 0.000001 # Nearly ignore background completely
00324         else:
00325             dose_weight = 100000.0 # Original 100K weight
00326             background_weight = 0.00001
00327
00328         weights = torch.where(dose_mask, dose_weight, background_weight)
00329         weighted_loss = (torch.abs(pred - target) * weights).mean()
00330
00331         # Combine: Emphasize pure dose training even more heavily for extreme sparsity
00332         if sparsity_percent < 0.006:
00333             combined_loss = 0.95 * pure_dose_loss + 0.05 * weighted_loss # 95% pure dose focus for ultra-extreme
00334         elif sparsity_percent < 0.01:
00335             combined_loss = 0.9 * pure_dose_loss + 0.1 * weighted_loss # 90% pure dose focus
00336         else:
00337             combined_loss = 0.8 * pure_dose_loss + 0.2 * weighted_loss
00338
00339         total_voxels = target.numel()
00340         logger.debug(f"EXTREME loss - {dose_voxels}/{total_voxels} dose voxels "
00341                     f"({sparsity_percent:.5f}%), dose_weight: {dose_weight:.0f}, "
00342                     f"pure_MSE: {pure_dose_loss:.6f}, weighted_{dose_weight:.0f}: {weighted_loss:.6f}, combined:
00343         {combined_loss:.6f}")
00344
00345         return combined_loss

```

```

00344 # ----- #
00345 # utilities #
00346 # ----- #
00347 def _compute_dataset_dose_stats(self, loader):
00348     """
00349     Go once over the whole loader to obtain global mean / std of the
00350     ground-truth dose. These statistics are required for a stable
00351     normalisation of the target and the network output and will be
00352     cached in `self.dose_mean` and `self.dose_std`.
00353
00354     Args:
00355         loader (DataLoader): training (or validation) loader that yields
00356                             dictionaries with a "target" key.
00357     """
00358     logger.info("[Dose stats] computing dataset-wide mean / std ...")
00359     sum_val, sum_sq, num_vox = 0.0, 0.0, 0
00360     with torch.no_grad():
00361         for b in loader:
00362             tgt = b["target"] # shape [B,1,D,H,W]
00363             sum_val += tgt.sum().item()
00364             sum_sq += (tgt ** 2).sum().item()
00365             num_vox += tgt.numel()
00366
00367     mean = sum_val / num_vox
00368     var = (sum_sq / num_vox) - mean ** 2
00369     std = math.sqrt(max(var, DOSE_EPS))
00370
00371     self.dose_mean = mean
00372     self.dose_std = std
00373     # also capture absolute min / max for min-max normalisation
00374     dataset_min = float('-inf')
00375     dataset_max = float('inf')
00376     for b in loader:
00377         tgt = b["target"]
00378         dataset_min = min(dataset_min, tgt.min().item())
00379         dataset_max = max(dataset_max, tgt.max().item())
00380
00381     self.clip_min = dataset_min
00382     self.clip_max = dataset_max
00383     logger.info(
00384         f"[Dose stats] mean={mean:.5f} std={std:.5f} "
00385         f"min={dataset_min:.5f} max={dataset_max:.5f}"
00386     )
00387
00388 @staticmethod
00389 def _total_variation_loss(img):
00390     """
00391     3-D total-variation loss encouraging spatial smoothness.
00392
00393     Args:
00394         img (torch.Tensor): tensor of shape [B, C, D, H, W]
00395     """
00396     dx = torch.abs(img[..., :-1, :, :] - img[..., 1:, :, :]).mean()
00397     dy = torch.abs(img[..., :, :-1, :] - img[..., :, 1:, :]).mean()
00398     dz = torch.abs(img[..., :, :, :-1] - img[..., :, :, 1:]).mean()
00399     return dx + dy + dz
00400
00401 def train_one_epoch(self, train_loader, epoch):
00402     """
00403     Trains the autoencoder and the discriminator for an epoch.
00404
00405     This method integrates energy conditioning: for each batch, it extracts the "energy" values,
00406     normalizes them, expands them to match the spatial dimensions, and concatenates them as an
00407     additional channel to the input images. The autoencoder must be adapted to accept the increased
00408     number of input channels.
00409
00410     Args:
00411         train_loader (DataLoader): DataLoader for training data.
00412         epoch (int): Current epoch.
00413
00414     Returns:
00415         tuple: Average reconstruction loss, generator loss and discriminator loss.
00416     """
00417     self.autoencoder.train()
00418     self.discriminator.train()
00419     epoch_loss = 0.0
00420     gen_epoch_loss = 0.0
00421     disc_epoch_loss = 0.0
00422
00423     logger.info(f"Starting autoencoder training epoch {epoch}")
00424
00425     # **ENERGY-SPECIFIC LEARNING RATE ADJUSTMENT**
00426     # Extract energy from first batch to determine if adjustment is needed
00427     first_batch = next(iter(train_loader))
00428     if "energy" in first_batch:
00429         current_energy = first_batch["energy"][0].item() # Get first energy value
00430         logger.info(f"Training energy: {current_energy:.2f}")

```

```

00431     # Adaptive Learning Rate for high energies
00432     original_lr_g = None
00433     original_lr_d = None
00434     if current_energy > 40.0:
00435         # Reduce Learning Rate for high energies by factor 3
00436         original_lr_g = self.optimizer_g.param_groups[0]['lr']
00437         original_lr_d = self.optimizer_d.param_groups[0]['lr']
00438         new_lr_g = original_lr_g * 0.33
00439         new_lr_d = original_lr_d * 0.33
00440
00441         for param_group in self.optimizer_g.param_groups:
00442             param_group['lr'] = new_lr_g
00443         for param_group in self.optimizer_d.param_groups:
00444             param_group['lr'] = new_lr_d
00445
00446         logger.info(f" HIGH ENERGY ({current_energy:.2f}): Reduced learning rates")
00447         logger.info(f"     Generator: {original_lr_g:.6f} -> {new_lr_g:.6f}")
00448         logger.info(f"     Discriminator: {original_lr_d:.6f} -> {new_lr_d:.6f}")
00449     else:
00450         current_energy = None
00451         original_lr_g = None
00452         original_lr_d = None
00453
00454     # -----
00455     # one-time initialization of energy normalization range
00456     # -----
00457     if self.energy_min is None:
00458         raise RuntimeError(
00459             "energy_min/max not initialised - run _compute_dataset_energy_stats first"
00460         )
00461     for step, batch in enumerate(tqdm(train_loader, desc=f"Autoencoder Epoch {epoch}")):
00462         # Move data to device
00463         images = batch["input"].to(self.device) # Expected shape: [B, 1, D, H, W]
00464         dose_target = batch["target"].to(self.device) # Ground-truth dose
00465
00466         # **DEBUG: Log dose statistics for first few batches**
00467         if step < 3:
00468             dose_stats = {
00469                 'min': dose_target.min().item(),
00470                 'max': dose_target.max().item(),
00471                 'mean': dose_target.mean().item(),
00472                 'nonzero_count': (dose_target > 1e-6).sum().item(),
00473                 'total_voxels': dose_target.numel()
00474             }
00475             logger.info(f"Batch {step+1} dose stats: {dose_stats}")
00476
00477         # **CRITICAL FIX: Apply consistent dose normalization during training**
00478         # Normalize dose to [0,1] range using clip_min/clip_max with safety checks
00479         if self.clip_min is not None and self.clip_max is not None and (self.clip_max - self.clip_min) > 1e-6:
00480             dose_range = self.clip_max - self.clip_min
00481             dose_target_norm = (dose_target - self.clip_min) / dose_range
00482             # Clamp to [0,1] range to handle outliers
00483             dose_target_norm = torch.clamp(dose_target_norm, 0.0, 1.0)
00484             logger.debug(f"Normalized dose: orig_range={dose_target.min().item():.6f},
00485 {dose_target.max().item():.6f} "
00486                        f"-> norm_range=[{dose_target_norm.min().item():.6f},
00487 {dose_target_norm.max().item():.6f}]"
00488             )
00489         else:
00490             # Fallback: use raw dose values if normalization parameters not available
00491             logger.warning("Skipping dose normalization - using raw dose values")
00492             dose_target_norm = dose_target
00493
00494         # Apply energy conditioning if energy data is available
00495         if "energy" in batch:
00496             energies = batch["energy"].to(self.device)
00497             # Min-Max normalize energy using dataset range with zero-division guard
00498             eps = 1e-8
00499             normalized_energy = energies.float() / 100.00
00500             print(f"xNormalized energy: {normalized_energy}")
00501             B, C, D, H, W = images.shape
00502             energy_tensor = normalized_energy.view(B, 1, 1, 1, 1).expand(B, 1, D, H, W)
00503             conditioned_input = torch.cat([images, energy_tensor], dim=1)
00504         else:
00505             # Throw an error if energy conditioning is expected but not provided
00506             raise ValueError(
00507                 "Energy conditioning is expected but 'energy' field is missing in the batch."
00508             )
00509
00510         # Verify input channels match autoencoder expectations
00511         first_conv = next((m for m in self.autoencoder.modules() if isinstance(m, torch.nn.Conv3d)), None)
00512         expected_in_channels = first_conv.in_channels if first_conv else conditioned_input.shape[1]
00513
00514         if conditioned_input.shape[1] != expected_in_channels:
00515             raise ValueError(
00516                 f"Input channel mismatch: model expects {expected_in_channels} channels, "
00517                 f"but got {conditioned_input.shape[1]} channels. "
00518                 f"Energy conditioning: {'enabled' if 'energy' in batch else 'disabled'}"
00519             )

```

```

00516         )
00517
00518     #Zero the gradients for the generator
00519     self.optimizer_g.zero_grad(set_to_none=True)
00520     # Pass the conditioned input through the autoencoder
00521     # (Note: autoencoder's in_channels should be updated to handle conditioned input)
00522     reconstruction, z_mu, z_sigma = self.autoencoder(conditioned_input)
00523
00524     # **CRITICAL FIX: Apply same normalization to reconstruction as target**
00525     if self.clip_min is not None and self.clip_max is not None and (self.clip_max - self.clip_min) > 1e-6:
00526         dose_range = self.clip_max - self.clip_min
00527         reconstruction_norm = (reconstruction - self.clip_min) / dose_range
00528         reconstruction_norm = torch.clamp(reconstruction_norm, 0.0, 1.0)
00529     else:
00530         reconstruction_norm = reconstruction
00531
00532     #calculate KL loss
00533     kl_loss = 0.5 * torch.sum(
00534         z_mu.pow(2) + z_sigma.pow(2) - torch.log(z_sigma.pow(2)) - 1
00535     ) / images.size(0)
00536     # **CRITICAL FIX: Use extreme dose-focused loss for ultra-sparse data**
00537
00538     # Check dose statistics first
00539     dose_mask = dose_target_norm > 1e-6
00540     dose_voxels = dose_mask.sum().item()
00541     total_voxels = dose_target_norm.numel()
00542     dose_percentage = 100 * dose_voxels / total_voxels
00543
00544     if dose_voxels == 0:
00545         logger.warning("Batch has no dose voxels - skipping")
00546         continue
00547
00548     if dose_voxels < 10000: # Schwelle für MEGA-ULTRA-EXTREME deutlich erhöht
00549         # Ultra-sparse batch - use extreme loss
00550         recons_loss = self.extreme_dose_focused_loss(reconstruction_norm.float(), dose_target_norm.float())
00551         loss_type = "EXTREME (1,000,000x weights + MSE)"
00552     else:
00553         # Normal sparse batch - use combined weighted/masked approach
00554         weighted_loss = self.weighted_dose_loss(reconstruction_norm.float(), dose_target_norm.float())
00555         masked_loss = self.masked_dose_loss(reconstruction_norm.float(), dose_target_norm.float())
00556         recons_loss = 0.6 * weighted_loss + 0.4 * masked_loss
00557         loss_type = "COMBINED (100,000x weighted + pure masked)"
00558
00559     # **ENERGY-SPECIFIC LOSS AMPLIFICATION FOR HIGH ENERGIES**
00560     if current_energy is not None and current_energy > 40.0:
00561         recons_loss = recons_loss * 1.5 # 50% stronger weighting
00562         loss_type += " + HIGH_ENERGY_BOOST"
00563         if step < 3: # Log only for first few batches
00564             logger.info(f" Applied 1.5x loss boost for high energy {current_energy:.2f}")
00565
00566     # **ENHANCED: Comprehensive sparse data analysis**
00567     dose_ratio = dose_voxels / total_voxels
00568     sparsity_level = f"{dose_ratio*100:.6f}%" if dose_ratio > 0 else "NO_DOSE"
00569
00570     logger.debug(f"{loss_type} loss on {dose_voxels}/{total_voxels} dose voxels "
00571                 f"({dose_percentage:.4f}%), loss: {recons_loss:.6f}")
00572     logger.debug(f"    -> Sparsity: {sparsity_level}, recon_range: [{reconstruction_norm.min().item():.6f},
00573 {reconstruction_norm.max().item():.6f}]")
00574
00575     # Track dose-only reconstruction quality with enhanced metrics
00576     if dose_voxels > 0:
00577         dose_mask_bool = dose_target_norm > 1e-6
00578         dose_mse = F.mse_loss(reconstruction_norm[dose_mask_bool], dose_target_norm[dose_mask_bool])
00579         dose_mae = F.l1_loss(reconstruction_norm[dose_mask_bool], dose_target_norm[dose_mask_bool])
00580
00581         # Signal strength analysis
00582         target_signal = dose_target_norm[dose_mask_bool].mean().item()
00583         recon_signal = reconstruction_norm[dose_mask_bool].mean().item()
00584         signal_ratio = (recon_signal / target_signal) if target_signal > 0 else 0.0
00585
00586         logger.debug(f"    -> Dose-only MSE: {dose_mse:.6f}, MAE: {dose_mae:.6f}")
00587         logger.debug(f"    -> Signal ratio: {signal_ratio:.6f} (target: {target_signal:.6f}, recon:
00588 {recon_signal:.6f})")
00589
00590     # Critical thresholds
00591     # Nur jede 20. Warnung loggen, um Spam zu vermeiden
00592     if step % 20 == 0:
00593         if signal_ratio < 0.01:
00594             logger.warning(f" SIGNAL COLLAPSE: ratio={signal_ratio:.6f} - model barely responding!")
00595         elif signal_ratio < 0.1:
00596             logger.warning(f" WEAK SIGNAL: ratio={signal_ratio:.6f} - consider higher weights!")
00597
00598     # **CRITICAL FIX: Check for NaN in loss**
00599     if torch.isnan(recons_loss) or torch.isinf(recons_loss):
00600         logger.error(f"NaN/Inf detected in reconstruction loss: {recons_loss.item()}")
00601         logger.error(f"reconstruction_norm stats: min={reconstruction_norm.min().item():.6f}, "
00602                     f"max={reconstruction_norm.max().item():.6f},

```

```

mean={reconstruction_norm.mean().item():.6f}")
00601         logger.error(f"dose_target_norm stats: min={dose_target_norm.min().item():.6f}, "
00602                       f"max={dose_target_norm.max().item():.6f}, mean={dose_target_norm.mean().item():.6f}")
00603         # Skip this batch
00604         continue
00605
00606     # **ENHANCED: Model collapse detection and automatic intervention**
00607     recon_max = reconstruction_norm.max().item()
00608     target_max = dose_target_norm.max().item()
00609
00610     # **ENERGY-SPECIFIC DEGRADATION THRESHOLDS**
00611     if current_energy is not None and current_energy > 40.0:
00612         # Stricter threshold for high energies
00613         degradation_threshold = 0.05
00614         collapse_threshold = 0.001
00615     else:
00616         # Normal thresholds
00617         degradation_threshold = 0.02
00618         collapse_threshold = 0.0001
00619
00620     # Nur jede 20. Warnung loggen, um Spam zu vermeiden
00621     if step % 20 == 0:
00622         ratio = recon_max / max(target_max, 1e-6)
00623         if recon_max < collapse_threshold and target_max > 0.01:
00624             logger.warning(f" CRITICAL MODEL COLLAPSE: recon_max={recon_max:.6f},
target_max={target_max:.6f}")
00625             logger.warning(f"    Signal ratio: {ratio:.8f} (should be >{degradation_threshold})")
00626             if current_energy is not None and current_energy > 40.0:
00627                 logger.warning(f" HIGH ENERGY COLLAPSE: energy={current_energy:.2f} - Consider reducing
clip_max")
00628                 logger.warning(f"    URGENT: Consider 500K+ weights or alternative loss function")
00629             elif ratio < degradation_threshold and target_max > 0.1:
00630                 logger.warning(f"    MODEL DEGRADATION: recon_max={recon_max:.6f}, target_max={target_max:.6f}")
00631                 logger.warning(f"    Model producing weak signals - monitor closely")
00632             if current_energy is not None and current_energy > 40.0:
00633                 logger.warning(f" HIGH ENERGY DEGRADATION: energy={current_energy:.2f}, ratio={ratio:.6f}")
00634                 logger.warning(f"    Suggested: Reduce dose normalization clip_max")
00635
00636     # compute adversarial loss if using discriminator
00637     adv = 0.0
00638     if epoch > self.warm_up_epochs and self.discriminator is not None:
00639         logits_fake = self.discriminator(reconstruction.contiguous().float())[-1]
00640         adv = self.adv_loss(logits_fake, target_is_real=True, for_discriminator=False)
00641     # compute perceptual loss (use dose target as reference, not CT input)
00642     perc = self.perceptual_loss(reconstruction_norm, dose_target_norm)
00643     # compute TV in physical Gy units (before normalisation)
00644     tv = self._total_variation_loss(reconstruction)
00645     # total generator loss
00646     loss_g = (recons_loss
00647              + self.kl_weight * kl_loss
00648              + self.adv_weight * adv
00649              + self.perceptual_weight * perc
00650              + self.tv_weight * tv)
00651
00652     # Add adversarial loss if past warm-up phase and discriminator is used.
00653     if epoch > self.warm_up_epochs and self.discriminator is not None:
00654         gen_epoch_loss += adv.item()
00655     # keep track of TV loss for logging
00656     gen_epoch_loss += self.tv_weight * tv.item()
00657     loss_g.backward()
00658     # ULTRA-conservative gradient clipping for extreme stability
00659     torch.nn.utils.clip_grad_norm_(self.autoencoder.parameters(), max_norm=0.1) # Much more conservative
00660     torch.nn.utils.clip_grad_value_(self.autoencoder.parameters(), clip_value=0.01) # Much more
conservative
00661     self.optimizer_g.step()
00662
00663     # **CRITICAL FIX: Check for NaN in generator loss**
00664     if epoch > self.warm_up_epochs and self.discriminator is not None:
00665         self.optimizer_d.zero_grad(set_to_none=True)
00666
00667         # Discriminator loss: real vs fake
00668         logits_fake = self.discriminator(reconstruction.contiguous().detach())[-1]
00669         loss_d_fake = F.mse_loss(logits_fake, torch.zeros_like(logits_fake))
00670
00671         # Discriminator loss: real vs real
00672         logits_real = self.discriminator(dose_target.contiguous().detach())[-1]
00673         loss_d_real = F.mse_loss(logits_real, torch.ones_like(logits_real))
00674
00675         discriminator_loss = (loss_d_fake + loss_d_real) / 2
00676         discriminator_loss.backward()
00677         self.optimizer_d.step()
00678         disc_epoch_loss += discriminator_loss.item()
00679
00680     epoch_loss += recons_loss.item()
00681
00682     avg_loss = epoch_loss / (step + 1)
00683     avg_gen_loss = gen_epoch_loss / (step + 1) if self.discriminator is not None else 0

```



```

00684         avg_disc_loss = disc_epoch_loss / (step + 1) if self.discriminator is not None else 0
00685         logger.info(
00686             f"Epoch {epoch} completed: recon_loss={avg_loss:.6f}, adv_loss={avg_gen_loss:.4f}, "
00687             f"disc_loss={avg_disc_loss:.4f}, tv={tv.item():.4f}"
00688         )
00689         # Dynamic message based on actual loss types used
00690         logger.info(f" -> Training with ADAPTIVE loss selection: MEGA-ULTRA-EXTREME (<300 dose voxels, 1M weights
for <0.006% sparsity) + COMBINED (300 dose voxels)")
00691
00692         # **RESTORE ORIGINAL LEARNING RATES**
00693         if original_lr_g is not None and original_lr_d is not None:
00694             for param_group in self.optimizer_g.param_groups:
00695                 param_group['lr'] = original_lr_g
00696             for param_group in self.optimizer_d.param_groups:
00697                 param_group['lr'] = original_lr_d
00698             logger.info(f" Restored original learning rates: G={original_lr_g:.6f}, D={original_lr_d:.6f}")
00699
00700         return avg_loss, avg_gen_loss, avg_disc_loss
00701
00702     def validate(self, val_loader):
00703         """
00704         Performs the validation of the autoencoder and returns the average loss.
00705         Args:
00706             val_loader (DataLoader): DataLoader for validation data.
00707         Returns:
00708             float: Average validation loss.
00709         """
00710         self.autoencoder.eval()
00711         val_loss = 0.0
00712         batch_losses = []
00713         extreme_count = 0
00714         combined_count = 0
00715
00716         logger.info("Starting validation")
00717         logger.info(f"Validation dataset: {len(val_loader)} batches")
00718
00719         with torch.no_grad():
00720             for i, batch in enumerate(val_loader):
00721                 images = batch["input"].to(self.device)
00722                 dose_target = batch["target"].to(self.device) # ground-truth dose
00723
00724                 # **CRITICAL FIX: Apply consistent dose normalization during validation**
00725                 if self.clip_min is not None and self.clip_max is not None and (self.clip_max - self.clip_min) >
1e-6:
00726                     dose_range = self.clip_max - self.clip_min
00727                     dose_target_norm = (dose_target - self.clip_min) / dose_range
00728                     dose_target_norm = torch.clamp(dose_target_norm, 0.0, 1.0)
00729                 else:
00730                     dose_target_norm = dose_target
00731                 # Apply energy conditioning if energy data is available
00732                 if "energy" in batch:
00733                     energies = batch["energy"].to(self.device)
00734                     # Min-Max normalize energy using dataset range with zero-division guard
00735                     eps = 1e-8
00736                     normalized_energy = energies.float() / 100.00
00737                     if i == 0: # Log energy only for first batch
00738                         energy_val = energies[0].item() if energies.numel() > 1 else energies.item()
00739                         norm_val = normalized_energy[0].item() if normalized_energy.numel() > 1 else
normalized_energy.item()
00740                         logger.info(f"Val energy normalization: {energy_val:.2f} -> {norm_val:.6f}")
00741                         B, C, D, H, W = images.shape
00742                         energy_tensor = normalized_energy.view(B, 1, 1, 1, 1).expand(B, 1, D, H, W)
00743                         conditioned_input = torch.cat([images, energy_tensor], dim=1)
00744                 else:
00745                     raise ValueError(
00746                         "Energy conditioning is expected but 'energy' field is missing in the batch."
00747                     )
00748
00749                 # Verify input channels match autoencoder expectations
00750                 first_conv = next((m for m in self.autoencoder.modules() if isinstance(m, torch.nn.Conv3d)), None)
00751                 expected_in_channels = first_conv.in_channels if first_conv else conditioned_input.shape[1]
00752
00753                 if conditioned_input.shape[1] != expected_in_channels:
00754                     raise ValueError(
00755                         f"Validation input channel mismatch: model expects {expected_in_channels} channels, "
00756                         f"but got {conditioned_input.shape[1]} channels."
00757                     )
00758
00759                 reconstruction, z_mu, z_sigma = self.autoencoder(conditioned_input)
00760
00761                 # **CRITICAL FIX: Apply same normalization to reconstruction during validation**
00762                 if self.clip_min is not None and self.clip_max is not None and (self.clip_max - self.clip_min) >
1e-6:
00763                     dose_range = self.clip_max - self.clip_min
00764                     reconstruction_norm = (reconstruction - self.clip_min) / dose_range
00765                     reconstruction_norm = torch.clamp(reconstruction_norm, 0.0, 1.0)
00766                 else:

```

```

00767         reconstruction_norm = reconstruction
00768
00769         # **CRITICAL FIX: Use SAME adaptive loss selection as training**
00770         dose_mask = dose_target_norm > 1e-6
00771         dose_voxels = dose_mask.sum().item()
00772
00773         if dose_voxels < 300: # CRITICAL FIX: Same threshold as training (300)
00774             # Ultra-sparse batch - use extreme loss (same as training)
00775             loss = self.extreme_dose_focused_loss(reconstruction_norm, dose_target_norm)
00776             loss_type = "MEGA-ULTRA-EXTREME"
00777             extreme_count += 1
00778         else:
00779             # Normal sparse batch - use combined weighted/masked approach (same as training)
00780             weighted_loss = self.weighted_dose_loss(reconstruction_norm, dose_target_norm)
00781             masked_loss = self.masked_dose_loss(reconstruction_norm, dose_target_norm)
00782             loss = 0.6 * weighted_loss + 0.4 * masked_loss
00783             loss_type = "COMBINED"
00784             combined_count += 1
00785
00786         val_loss += loss.item()
00787         batch_losses.append(loss.item())
00788
00789         # Debug logging for first few batches
00790         if i < 3:
00791             total_voxels = dose_target_norm.numel()
00792             energy_val = energies[0].item() if energies.numel() > 1 else energies.item()
00793             logger.info(f"Val batch {i}: energy={energy_val:.1f}, {loss_type}, "
00794                       f"{dose_voxels}/{total_voxels} dose voxels "
00795                       f"({100*dose_voxels/total_voxels:.4f}%), loss={loss.item():.6f}")
00796             logger.info(f" -> target_range=[{dose_target_norm.min():.6f}, {dose_target_norm.max():.6f}], "
00797                       f"recon_range=[{reconstruction_norm.min():.6f}, {reconstruction_norm.max():.6f}]")
00798
00799         avg_val_loss = val_loss / len(val_loader)
00800         loss_std = np.std(batch_losses) if len(batch_losses) > 1 else 0.0
00801
00802         logger.info(f"Validation completed: avg_val_loss={avg_val_loss:.6f} ± {loss_std:.6f}")
00803         logger.info(f" -> Loss distribution: {extreme_count} MEGA-ULTRA-EXTREME batches, {combined_count} COMBINED
00804 batches")
00805         logger.info(f" -> Loss range: min={min(batch_losses):.6f}, max={max(batch_losses):.6f}")
00806
00807         return avg_val_loss
00808
00808 class DiffusionTrainer:
00809     """
00810     Encapsulates the training process for the diffusion model.
00811
00812     """
00813     Trains diffusion model for one epoch.
00814
00815     Args:
00816         train_loader (DataLoader): DataLoader for training data.
00817         epoch (int): Current epoch index.
00818         inferer (callable): A callable with signature
00819             inferer(inputs, autoencoder_model, diffusion_model, noise, timesteps)
00820             that returns predicted noise.
00821     """
00822     def __init__(self, diffusion_model, optimizer_diff, energy_min, energy_max, device):
00823         """
00824         Args:
00825             diffusion_model (torch.nn.Module): Das Diffusionsmodell (z. B. ein UNet).
00826             optimizer_diff (torch.optim.Optimizer): Optimierer für das Diffusionsmodell.
00827             device (torch.device): CPU oder GPU.
00828         """
00829         self.diffusion_model = diffusion_model
00830         self.optimizer_diff = optimizer_diff
00831         # Min-Max energy normalization parameters
00832         self.energy_min = energy_min
00833         self.energy_max = energy_max
00834         self.device = device
00835         # -----
00836         # Determine the cross-attention dimension expected by the UNet.
00837         #
00838         # Many UNet variants have several cross-attention blocks operating
00839         # at different channel widths. Each block owns a projection layer
00840         # called "to_k"; its *input* dimension equals the context-vector
00841         # length expected by this block. All blocks that consume the same
00842         # context embedding should therefore agree on that dimension -
00843         # but some architectures prepend an additional projection layer,
00844         # so we might see values like 2, 64, 128, ...
00845         #
00846         # Empirically, the *smallest* of these 'in_features' values
00847         # corresponds to the original conditioning size (in our case
00848         # '(energy, bias) = 2'). We therefore collect *all* 'to_k'
00849         # modules and pick the minimum 'in_features'.
00850         # -----
00851         cand_dims = set()
00852         for name, mod in self.diffusion_model.named_modules():

```



```

00853         if isinstance(mod, torch.nn.Linear) and (name.endswith("to_k") or ".to_k" in name):
00854             cand_dims.add(mod.in_features)
00855
00856     if cand_dims:
00857         # choose the smallest dimension - typically 2
00858         self.cross_attn_dim = min(cand_dims)
00859     else:
00860         raise RuntimeError(
00861             "Could not find any cross-attention projection layer in the diffusion model. "
00862             "Ensure that the model is correctly configured for energy conditioning."
00863         )
00864
00865     if self.cross_attn_dim is None:
00866         raise RuntimeError("Could not determine cross-attention dimension from diffusion model.")
00867
00868     # Note: Context projection disabled since running in unconditional mode
00869     # Small MLP that maps (energy, bias) → vector of length `cross_attn_dim`
00870     # self.context_proj = torch.nn.Linear(2, self.cross_attn_dim).to(self.device)
00871
00872 def train_one_epoch(self, train_loader, epoch, inferer=None, autoencoder=None):
00873     """
00874     Args:
00875         diffusion_model (torch.nn.Module): The diffusion model (e.g. a UNet).
00876         optimizer_diff (torch.optim.Optimizer): Optimizer for the diffusion model.
00877         device (torch.device): CPU or GPU.
00878     """
00879     # if no inferer or autoencoder provided, skip diffusion training
00880     if inferer is None or autoencoder is None:
00881         return 0.0
00882
00883     self.diffusion_model.train()
00884     epoch_loss = 0.0
00885
00886     for step, batch in enumerate(tqdm(train_loader, desc=f"Diffusion Epoch {epoch}")):
00887         images = batch["input"].to(self.device)
00888
00889         # Apply energy conditioning consistently
00890         if "energy" in batch:
00891             energies = batch["energy"].to(self.device)
00892             # Min-Max normalize energy using dataset range with zero-division guard
00893             eps = 1e-8
00894             #normalized_energy = (energies.float() - self.energy_min) / (self.energy_max - self.energy_min +
00895
00896         eps)
00897             normalized_energy = energies.float() / 100.00
00898             print(f"Normalized energy: {normalized_energy}")
00899             B, C, D, H, W = images.shape
00900             energy_tensor = normalized_energy.view(B, 1, 1, 1, 1).expand(B, 1, D, H, W)
00901             conditioned_input = torch.cat([images, energy_tensor], dim=1)
00902         else:
00903             # If energy conditioning is expected but not provided, raise an error
00904             raise ValueError(
00905                 "Energy conditioning is expected but 'energy' field is missing in the batch."
00906             )
00907
00908         # Zero gradients for diffusion optimizer
00909         self.optimizer_diff.zero_grad(set_to_none=True)
00910
00911         # Encode conditioned input to latents using the autoencoder
00912         with torch.no_grad():
00913             # MONAI's AutoencoderKL.encode() returns a tuple (mu, logvar)
00914             # For consistent handling, always expect this format
00915             mu, logvar = autoencoder.encode(conditioned_input)
00916             # Sample from the latent distribution
00917             std = torch.exp(0.5 * logvar)
00918             eps = torch.randn_like(std)
00919             latents = mu + eps * std
00920             latents = latents.to(self.device)
00921
00922         # Crop / pad latents symmetrically so that spatial dims are multiples
00923         # of the UNet down-sampling factor
00924         down_blocks = [m for m in self.diffusion_model.modules()
00925                        if isinstance(m, torch.nn.Conv3d) and any(s > 1 for s in m.stride)]
00926         downsample_factor = 2 ** len(down_blocks) if down_blocks else 1
00927
00928         B, C, D, H, W = latents.shape
00929         new_D = (D // downsample_factor) * downsample_factor
00930         new_H = (H // downsample_factor) * downsample_factor
00931         new_W = (W // downsample_factor) * downsample_factor
00932
00933         if (new_D, new_H, new_W) != (D, H, W):
00934             d0 = (D - new_D) // 2
00935             h0 = (H - new_H) // 2
00936             w0 = (W - new_W) // 2
00937             latents = latents[:, :, d0:d0 + new_D, h0:h0 + new_H, w0:w0 + new_W]
00938
00939         # Verify latent channels match diffusion model expectations
00940         first_conv_diff = next((m for m in self.diffusion_model.modules() if isinstance(m,

```

```

torch.nn.Conv3d)), None)
00939         expected_latent_ch = first_conv_diff.in_channels if first_conv_diff else latents.shape[1]
00940
00941         if latents.shape[1] != expected_latent_ch:
00942             raise ValueError(
00943                 f"Latent channel mismatch: diffusion model expects {expected_latent_ch} channels, "
00944                 f"but autoencoder produces {latents.shape[1]} channels"
00945             )
00946
00947         # generates random noise
00948         noise = torch.randn_like(latents)
00949
00950         # generates random Timesteps
00951         timesteps = torch.randint(0, inferer.scheduler.num_train_timesteps, (images.shape[0],),
device=self.device).long()
00952
00953         # Add noise to the latents to create x_t
00954         noisy_latents = inferer.scheduler.add_noise(latents, noise, timesteps)
00955
00956         # -----
00957         # Note: Running in unconditional mode (with_conditioning=False)
00958         # Context/conditioning has been disabled in system_manager.py
00959         # -----
00960         energy_feat = normalized_energy.view(B, 1) # [B,1]
00961         bias_feat = torch.ones_like(energy_feat) # [B,1]
00962         base_ctx = torch.cat([energy_feat, bias_feat], dim=-1) # [B,2]
00963         proj_ctx = self.context_proj(base_ctx) # [B,cross_attn_dim]
00964         context_tensor = proj_ctx.unsqueeze(1) # [B,1,cross_attn_dim]
00965
00966         # --- classifier-free guidance dropout -----
00967         # if torch.rand(1, device=self.device).item() < 0.1:
00968         #     context_tensor = None # unconditional pass
00969
00970         # Predict the noise component and optimise (unconditional)
00971         noise_pred = self.diffusion_model(
00972             noisy_latents,
00973             timesteps=timesteps
00974         )
00975
00976         # ----- optimisation step -----
00977         loss = F.mse_loss(noise_pred.float(), noise.float())
00978         loss.backward()
00979
00980         # gradient-clipping stabilises very small batches
00981         clip_grad_norm_(self.diffusion_model.parameters(), max_norm=1.0)
00982
00983         self.optimizer_diff.step()
00984         # Crop original noise to match predicted noise spatial dimensions (symmetric)
00985         if noise.ndim == noise_pred.ndim and noise.shape[2:] != noise_pred.shape[2:]:
00986             _, _, Dn, Hn, Wn = noise_pred.shape
00987             d0 = (noise.shape[2] - Dn) // 2
00988             h0 = (noise.shape[3] - Hn) // 2
00989             w0 = (noise.shape[4] - Wn) // 2
00990             noise = noise[:, :, d0:d0 + Dn, h0:h0 + Hn, w0:w0 + Wn]
00991         epoch_loss += loss.item()
00992
00993         avg_loss = epoch_loss / (step + 1)
00994         return avg_loss

```

8.30 sourcecode/visualization.py File Reference

Classes

- class [visualization.Visualization](#)
- class [visualization.MplCanvas](#)

Namespaces

- namespace [visualization](#)

Functions

- [visualization.load_and_visualize](#) (nifti_path, ct_scan=None, title="Dose distribution - Axial View")
- [visualization.get_visualization_widget](#) (figure)
- [visualization.interactive_slice_viewer](#) (cube, ct_volume=None, title="Interactive Slice Viewer", show=True)
- [visualization.volume_rendering](#) (cube, opacity=0.2, surface_count=20)

8.31 visualization.py

[Go to the documentation of this file.](#)

```

00001 # visualization.py
00002
00003 import matplotlib
00004 import matplotlib.pyplot as plt
00005 from matplotlib.backends.backend_qtagg import FigureCanvasQTAgg
00006 import nibabel as nib
00007 import numpy as np
00008
00009 class Visualization:
00010     """
00011     The Visualization module provides methods to plot training curves and display sample images.
00012     This is useful for monitoring the training process and visualizing inference results.
00013     """
00014
00015     def __init__(self):
00016         """
00017         Initializes the Visualization module.
00018         """
00019         pass
00020
00021     def plot_learning_curve(self, epochs, loss_list, title="Learning Curve", filename=None):
00022         """
00023         Plots the learning curve over epochs.
00024
00025         Args:
00026             epochs (list or array): Epoch numbers.
00027             loss_list (list or array): Loss values corresponding to each epoch.
00028             title (str): Title of the plot.
00029             filename (str, optional): If provided, the plot is saved to this file.
00030         """
00031         plt.figure(figsize=(10, 6))
00032         plt.plot(epochs, loss_list, marker='o', label='Training Loss')
00033         plt.title(title, fontsize=20)
00034         plt.xlabel("Epochs", fontsize=16)
00035         plt.ylabel("Loss", fontsize=16)
00036         plt.legend(prop={"size": 14})
00037         plt.grid(True)
00038         if filename:
00039             plt.savefig(filename)
00040         else:
00041             # save to default file
00042             plt.savefig('learning_curve.png')
00043         plt.close()
00044
00045     def plot_adversarial_curves(self, epochs, gen_loss, disc_loss, title="Adversarial Training Curves",
00046                               filename=None):
00047         """
00048         Plots the adversarial training curves for generator and discriminator.
00049
00050         Args:
00051             epochs (list or array): Epoch numbers.
00052             gen_loss (list or array): Generator loss values.
00053             disc_loss (list or array): Discriminator loss values.
00054             title (str): Title of the plot.
00055             filename (str, optional): If provided, the plot is saved to this file.
00056         """
00057         plt.figure(figsize=(10, 6))
00058         plt.plot(epochs, gen_loss, marker='o', linewidth=2, label='Generator Loss')
00059         plt.plot(epochs, disc_loss, marker='o', linewidth=2, label='Discriminator Loss')
00060         plt.title(title, fontsize=20)
00061         plt.xlabel("Epochs", fontsize=16)
00062         plt.ylabel("Loss", fontsize=16)
00063         plt.legend(prop={"size": 14})
00064         plt.grid(True)
00065         if filename:
00066             plt.savefig(filename)
00067         else:
00068             plt.savefig('adv_curves.png')
00069         plt.close()
00070
00071     def display_sample_images(self, images, title="Sample Images", filename=None):
00072         """
00073         Displays sample images from the given batch. For 3D images, it shows three orthogonal slices.
00074
00075         Args:
00076             images (torch.Tensor or numpy.ndarray): Images to display. Expected shape is either
00077                 [batch, channels, depth, height, width] for 3D images or [batch, channels, height, width] for 2D.
00078             title (str): Title for the plot.
00079             filename (str, optional): If provided, the figure is saved to this file.
00080         """
00081         # Convert to numpy if input is a tensor
00082         if hasattr(images, 'cpu'):
00083             images = images.cpu().numpy()

```

```

00083
00084     # For simplicity, take the first image from the batch and its first channel
00085     sample = images[0, 0]
00086
00087     # Check dimensions for 3D image
00088     if sample.ndim == 3:
00089         depth, height, width = sample.shape
00090         # Select slices: middle slice along each axis
00091         slice_depth = sample[depth // 2, :, :]
00092         slice_height = sample[:, height // 2, :]
00093         slice_width = sample[:, :, width // 2]
00094
00095         fig, axs = plt.subplots(1, 3, figsize=(15, 5))
00096         axs[0].imshow(slice_depth, cmap='gray')
00097         axs[0].set_title("Depth Slice")
00098         axs[0].axis("off")
00099         axs[1].imshow(slice_height, cmap='gray')
00100         axs[1].set_title("Height Slice")
00101         axs[1].axis("off")
00102         axs[2].imshow(slice_width, cmap='gray')
00103         axs[2].set_title("Width Slice")
00104         axs[2].axis("off")
00105         plt.suptitle(title, fontsize=20)
00106     else:
00107         # For 2D images, simply show the first image
00108         plt.figure(figsize=(6, 6))
00109         plt.imshow(sample, cmap='gray')
00110         plt.title(title, fontsize=20)
00111         plt.axis("off")
00112
00113     if filename:
00114         plt.savefig(filename)
00115     else:
00116         plt.savefig('sample_images.png')
00117     plt.close()
00118
00119     @staticmethod
00120     def plot_loss_curves(ae_train, ae_val, gen, disc, diff, resolution, energy):
00121         """
00122         Plot training and validation losses for autoencoder and diffusion model.
00123
00124         Args:
00125             ae_train (list of float): Autoencoder training losses per epoch.
00126             ae_val (list of float): Autoencoder validation losses per epoch.
00127             gen (list of float): Generator losses per epoch.
00128             disc (list of float): Discriminator losses per epoch.
00129             diff (list of float): Diffusion model losses per epoch.
00130             resolution (tuple): Spatial resolution used for this run.
00131             energy (float): Energy level used for this run.
00132         """
00133         import matplotlib.pyplot as plt
00134         plt.switch_backend('Agg')
00135         plt.figure(figsize=(10, 6))
00136         plt.plot(ae_train, marker='o', label='AE Train')
00137         plt.plot(ae_val, marker='o', label='AE Val')
00138         plt.plot(gen, marker='o', label='Generator Loss')
00139         plt.plot(disc, marker='o', label='Discriminator Loss')
00140         plt.plot(diff, marker='o', label='Diffusion Loss')
00141         plt.title(f'Loss Curves (res={resolution}, energy={energy} eV)', fontsize=18)
00142         plt.xlabel('Epoch', fontsize=14)
00143         plt.ylabel('Loss', fontsize=14)
00144         plt.legend(prop={"size": 12})
00145         plt.grid(True)
00146         # save loss curves to file
00147         fname = f'loss_curves_res{resolution}_energy{energy}.png'
00148         plt.savefig(fname)
00149         plt.close()
00150
00151     class MplCanvas(FigureCanvasQTAgg):
00152         """
00153         A custom widget that converts a matplotlib figure to a Qt widget.
00154         This enables embedding matplotlib visualizations directly in the GUI.
00155         """
00156         def __init__(self, fig):
00157             super().__init__(fig)
00158             self.setMinimumSize(400, 300)
00159
00160     def load_and_visualize(nifti_path, ct_scan=None, title="Dose distribution - Axial View"):
00161         """
00162         Loads a Nifti dose distribution file and visualizes its middle axial slice.
00163
00164         Args:
00165             nifti_path: Path to the NIfTI file
00166             title: Title for the visualization
00167
00168         Returns:

```

```

00170         matplotlib.figure.Figure: A figure object that can be displayed
00171     """
00172     if nifti_path.lower().endswith((''.nii', '.nii.gz')):
00173         img = nib.load(nifti_path)
00174         cube = np.asarray(img.dataobj)
00175     else:
00176         cube = np.load(nifti_path, allow_pickle=True)
00177         # Aggregate over energy dimension and ensure 3D volume
00178     if cube.ndim == 5 and cube.shape[1] == 1:
00179         cube = cube.squeeze(1)
00180     if cube.ndim == 4:
00181         cube = np.sum(cube, axis=0)
00182     cube = np.squeeze(cube)
00183     # Call visualization module for interactive slice viewer and volume rendering
00184     interactive_slice_viewer(cube, ct_volume=ct_scan)
00185     volume_rendering(cube)
00186
00187
00188 def get_visualization_widget(figure):
00189     """
00190     Converts a matplotlib figure to a Qt widget for embedding in the GUI.
00191
00192     Args:
00193         figure: matplotlib.figure.Figure object
00194
00195     Returns:
00196         MplCanvas: A Qt widget containing the visualization
00197     """
00198     return MplCanvas(figure)
00199
00200
00201 def interactive_slice_viewer(cube, ct_volume=None, title="Interactive Slice Viewer", show=True):
00202     """
00203     Launches an interactive Matplotlib window with a slider to browse through axial slices of a 3D volume.
00204     """
00205     if cube.ndim != 3:
00206         raise ValueError("Volume must be a 3D array for interactive slice viewing.")
00207
00208     # Ensure interactive backend is set properly
00209     import matplotlib
00210     matplotlib.use('QtAgg')
00211     import matplotlib.pyplot as plt
00212     from matplotlib.widgets import Slider
00213
00214     # Clear any existing figures to prevent conflicts
00215     plt.close('all')
00216     plt.ion()
00217     plt.switch_backend('QtAgg')
00218     # Initial slice
00219     z0 = cube.shape[2] // 2
00220     fig, ax = plt.subplots(figsize=(8, 7))
00221
00222     # Adjust layout for sliders
00223     if ct_volume is not None:
00224         fig.subplots_adjust(bottom=0.25)
00225     else:
00226         fig.subplots_adjust(bottom=0.15)
00227
00228     # Initialize images
00229     if ct_volume is not None:
00230         base_im = ax.imshow(ct_volume[:, :, z0], cmap='gray')
00231         overlay_im = ax.imshow(cube[:, :, z0], cmap='jet', alpha=0.5)
00232     else:
00233         overlay_im = ax.imshow(cube[:, :, z0], cmap='gray')
00234
00235     ax.set_title(f"[title] - Slice {z0}")
00236     ax.axis('off')
00237
00238     # Create slice slider
00239     slider_ax = fig.add_axes([0.15, 0.02, 0.7, 0.03])
00240     slider = Slider(slider_ax, 'Slice', 0, cube.shape[2] - 1, valinit=z0, valstep=1)
00241
00242     # Create alpha slider if CT volume is provided
00243     alpha_slider = None
00244     if ct_volume is not None:
00245         alpha_ax = fig.add_axes([0.15, 0.10, 0.7, 0.03])
00246         alpha_slider = Slider(alpha_ax, 'Alpha', 0.0, 1.0, valinit=0.5, valstep=0.01)
00247
00248     # Keep references to prevent garbage collection
00249     slider._fig = fig
00250     if alpha_slider is not None:
00251         alpha_slider._fig = fig
00252
00253     # Define callback functions with error handling
00254     def update_slice(val):
00255         try:
00256             z = int(slider.val)

```

```

00257         if z < 0 or z >= cube.shape[2]:
00258             return
00259
00260         if ct_volume is not None:
00261             base_im.set_data(ct_volume[:, :, z])
00262             overlay_im.set_data(cube[:, :, z])
00263         else:
00264             overlay_im.set_data(cube[:, :, z])
00265             ax.set_title(f"{title} - Slice {z}")
00266             fig.canvas.draw_idle()
00267     except Exception as e:
00268         print(f"Error in update_slice: {e}")
00269
00270 def update_alpha(val):
00271     try:
00272         if ct_volume is not None and alpha_slider is not None:
00273             alpha_val = float(alpha_slider.val)
00274             if 0.0 <= alpha_val <= 1.0:
00275                 overlay_im.set_alpha(alpha_val)
00276                 fig.canvas.draw_idle()
00277     except Exception as e:
00278         print(f"Error in update_alpha: {e}")
00279
00280 # Connect callbacks
00281 slider.on_changed(update_slice)
00282 if alpha_slider is not None:
00283     alpha_slider.on_changed(update_alpha)
00284
00285 # Ensure widgets are properly initialized
00286 fig.canvas.draw()
00287
00288 # Return objects for testing when show=False
00289 if not show:
00290     if ct_volume is not None:
00291         return fig, slider, alpha_slider, overlay_im
00292     return fig, slider, overlay_im
00293
00294 if show:
00295     try:
00296         plt.show(block=True)
00297     except KeyboardInterrupt:
00298         plt.close(fig)
00299
00300
00301 def volume_rendering(cube, opacity=0.2, surface_count=20):
00302     """
00303     Performs a 3D volume rendering of a normalized 3D array using Plotly.
00304     """
00305     if cube.ndim != 3:
00306         raise ValueError("Volume must be a 3D array for volume rendering.")
00307     import numpy as _np
00308     import plotly.graph_objects as go
00309
00310     # normalize
00311     cube_norm = (cube - _np.min(cube)) / (_np.max(cube) - _np.min(cube) + 1e-8)
00312     # Downsample volume for performance if too large
00313     max_points = 200_000 # limit total points to ~200k
00314     n_voxels = cube_norm.size
00315     if n_voxels > max_points:
00316         factor = int(_np.ceil((n_voxels / max_points) ** (1/3)))
00317         cube_norm = cube_norm[::factor, ::factor, ::factor]
00318     # thresholds
00319     p_low, p_high = _np.percentile(cube_norm, [1, 99])
00320     # coordinate grid
00321     x, y, z = _np.mgrid[0:cube_norm.shape[0],
00322                        0:cube_norm.shape[1],
00323                        0:cube_norm.shape[2]]
00324     fig = go.Figure(data=go.Volume(
00325         x=x.flatten(), y=y.flatten(), z=z.flatten(),
00326         value=cube_norm.flatten(),
00327         opacity=opacity,
00328         surface_count=surface_count,
00329         isomin=p_low, isomax=p_high,
00330         colorscale='Viridis'
00331     ))
00332     fig.update_layout(scene=dict(aspectmode='data'))
00333     fig.show(renderer="browser")

```

8.32 tests/analyze_traincubes.py File Reference

Namespaces

- namespace [analyze_traincubes](#)

8.33 analyze_traincubes.py

[Go to the documentation of this file.](#)

00001

8.34 tests/debug_inference_problem.py File Reference

Namespaces

- namespace [debug_inference_problem](#)

Variables

- [debug_inference_problem.level](#)
- [debug_inference_problem.INFO](#)
- [debug_inference_problem.format](#)
- str [debug_inference_problem.input_file](#) = 'traindata/11_5/inputcube/235101017859661465075472232303048949736_0.npy'
- str [debug_inference_problem.output_file](#) = 'traindata/11_5/outputcube/235101017859661465075472232303048949736_0.npy'
- [debug_inference_problem.input_data](#) = np.load([input_file](#))
- [debug_inference_problem.ground_truth](#) = np.load([output_file](#))
- str [debug_inference_problem.model_path](#) = '/Users/giannigagliardi/Documents/Git/RadioTherapy/unified_energy_conditioned_model_res16.0_energies2.ckpt'
- [debug_inference_problem.device](#) = torch.device("cpu")
- [debug_inference_problem.checkpoint](#) = torch.load([model_path](#), map_location=[device](#))
- [debug_inference_problem.autoencoder](#)
- float [debug_inference_problem.energy_value](#) = 11.5
- float [debug_inference_problem.energy_normalized](#) = [energy_value](#) / 40.0
- [debug_inference_problem.input_2channel](#) = np.zeros((2, *[input_data](#).shape))
- [debug_inference_problem.input_tensor](#) = torch.tensor([input_2channel](#), dtype=torch.float32).unsqueeze(0).to([device](#))
- [debug_inference_problem.encoded](#) = autoencoder.encode([input_tensor](#))
- [debug_inference_problem.latent](#) = [encoded](#)[0]
- [debug_inference_problem.decoded](#) = autoencoder.decode([latent](#))
- [debug_inference_problem.reconstruction](#) = [decoded](#)[0]
- [debug_inference_problem.reconstruction_np](#) = [reconstruction](#).cpu().numpy().squeeze()
- [debug_inference_problem.mse](#) = np.mean(([reconstruction_np](#) - [ground_truth](#)) ** 2)
- [debug_inference_problem.mae](#) = np.mean(np.abs([reconstruction_np](#) - [ground_truth](#)))
- [debug_inference_problem.correlation](#) = np.corrcorcoef([reconstruction_np](#).flatten(), [ground_truth](#).flatten())[0, 1]

8.35 debug_inference_problem.py

[Go to the documentation of this file.](#)

```
00001 #!/usr/bin/env python3
00002 """
00003 Debug-Script um das Inference-Problem zu identifizieren
00004 """
00005
00006 import os
00007 import sys
00008 import torch
00009 import numpy as np
00010 import logging
00011
00012 # Add paths
00013 sys.path.append('sourcecode')
00014 os.environ['PYTHONPATH'] = '/Users/giannigagliardi/Documents/Git/RadioTherapy'
00015
00016 # Setup logging
00017 logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(name)s - %(levelname)s - %(message)s')
00018
```



```

00019 print(" DEBUG: Inference Problem Analysis")
00020 print("=" * 60)
00021
00022 # Test: Lade den gleichen Input/Output wie in visualizationTEST
00023 input_file = 'traindata/11_5/inputcube/235101017859661465075472232303048949736_0.npy'
00024 output_file = 'traindata/11_5/outputcube/235101017859661465075472232303048949736_0.npy'
00025
00026 print(f"Loading test files:")
00027 print(f"  Input: {input_file}")
00028 print(f"  Output: {output_file}")
00029
00030 # Lade die Daten
00031 if os.path.exists(input_file) and os.path.exists(output_file):
00032     input_data = np.load(input_file)
00033     ground_truth = np.load(output_file)
00034
00035     print(f"\n Ground Truth Analysis:")
00036     print(f"  Input shape: {input_data.shape}")
00037     print(f"  Output shape: {ground_truth.shape}")
00038     print(f"  Output min/max: {ground_truth.min():.6f} / {ground_truth.max():.6f}")
00039     print(f"  Output mean/std: {ground_truth.mean():.6f} / {ground_truth.std():.6f}")
00040     print(f"  Output non-zero elements: {np.count_nonzero(ground_truth)}")
00041
00042     # Test 1: Direkte Validation mit unserem Modell
00043     print(f"\n Test 1: Direct Model Validation")
00044
00045     # Lade das trainierte Modell
00046     model_path =
'/Users/giannigagliardi/Documents/Git/RadioTherapy/unified_energy_conditioned_model_res16.0_energies2.ckpt'
00047
00048     if os.path.exists(model_path):
00049         print(f"Loading model: {model_path}")
00050
00051         device = torch.device("cpu")
00052         checkpoint = torch.load(model_path, map_location=device)
00053
00054         # Lade Autoencoder
00055         from generative.networks.nets import AutoencoderKL
00056         autoencoder = AutoencoderKL(
00057             spatial_dims=3,
00058             in_channels=2,
00059             out_channels=1,
00060             num_channels=(32, 32, 32),
00061             latent_channels=2,
00062             num_res_blocks=1,
00063             norm_num_groups=8,
00064             attention_levels=(False, False, True),
00065         ).to(device)
00066
00067         autoencoder.load_state_dict(checkpoint['autoencoder'])
00068         autoencoder.eval()
00069
00070         print(" Model loaded successfully")
00071
00072         # Test: Verwende die gleiche Energie-Konditionierung wie im Training
00073         print(f"\n Test 2: Energy Conditioning")
00074
00075         # Erstelle Input mit Energy-Kanal (wie im Training)
00076         energy_value = 11.5
00077         energy_normalized = energy_value / 40.0 # Normalisierung wie in validate_energy_conditioning.py
00078
00079         # Create 2-channel input
00080         input_2channel = np.zeros((2, *input_data.shape))
00081         input_2channel[0] = input_data # Physical dose
00082         input_2channel[1] = energy_normalized # Energy channel
00083
00084         # Convert to tensor
00085         input_tensor = torch.tensor(input_2channel, dtype=torch.float32).unsqueeze(0).to(device)
00086
00087         print(f"  Input tensor shape: {input_tensor.shape}")
00088         print(f"  Energy channel value: {energy_normalized:.4f}")
00089         print(f"  Input channel 0 (CT) min/max: {input_tensor[0,0].min():.6f} / {input_tensor[0,0].max():.6f}")
00090         print(f"  Input channel 1 (Energy) min/max: {input_tensor[0,1].min():.6f} / {input_tensor[0,1].max():.6f}")
00091
00092         # Test direkte Autoencoder-Rekonstruktion
00093         print(f"\n Test 3: Direct Autoencoder Reconstruction")
00094         with torch.no_grad():
00095             # Encode
00096             encoded = autoencoder.encode(input_tensor)
00097             if isinstance(encoded, tuple):
00098                 latent = encoded[0]
00099             else:
00100                 latent = encoded.latent_dist.sample()
00101
00102         print(f"  Latent shape: {latent.shape}")
00103         print(f"  Latent min/max: {latent.min():.6f} / {latent.max():.6f}")
00104

```



```

00105         # Decode
00106         decoded = autoencoder.decode(latent)
00107         if isinstance(decoded, tuple):
00108             reconstruction = decoded[0]
00109         else:
00110             reconstruction = decoded
00111
00112         reconstruction_np = reconstruction.cpu().numpy().squeeze()
00113
00114         print(f" Reconstruction shape: {reconstruction_np.shape}")
00115         print(f" Reconstruction min/max: {reconstruction_np.min():.6f} / {reconstruction_np.max():.6f}")
00116         print(f" Reconstruction mean/std: {reconstruction_np.mean():.6f} / {reconstruction_np.std():.6f}")
00117
00118         # Vergleiche mit Ground Truth
00119         print(f"\n Comparison with Ground Truth:")
00120         mse = np.mean((reconstruction_np - ground_truth) ** 2)
00121         mae = np.mean(np.abs(reconstruction_np - ground_truth))
00122         correlation = np.corrcoef(reconstruction_np.flatten(), ground_truth.flatten())[0, 1]
00123
00124         print(f" MSE: {mse:.8f}")
00125         print(f" MAE: {mae:.8f}")
00126         print(f" Correlation: {correlation:.6f}")
00127
00128         if correlation > 0.5:
00129             print(" GOOD: Reconstruction correlates with ground truth")
00130         else:
00131             print(" BAD: Poor correlation with ground truth")
00132
00133         # Speichere zur Visualisierung
00134         np.save('debug_reconstruction.npy', reconstruction_np)
00135         np.save('debug_ground_truth.npy', ground_truth)
00136         np.save('debug_input.npy', input_data)
00137
00138         print(f"\n Saved debug files:")
00139         print(f" debug_reconstruction.npy - Model reconstruction")
00140         print(f" debug_ground_truth.npy - Ground truth")
00141         print(f" debug_input.npy - Input data")
00142
00143     else:
00144         print(f" Model not found: {model_path}")
00145
00146     else:
00147         print(f" Test files not found!")
00148         print(f" Looking for: {input_file}")
00149         print(f" Looking for: {output_file}")
00150
00151     print("\n" + "=" * 60)
00152     print("Debug analysis complete!")

```

8.36 tests/debug_inference_systematic.py File Reference

Namespaces

- namespace [debug_inference_systematic](#)

Variables

- [debug_inference_systematic.level](#)
- [debug_inference_systematic.INFO](#)
- [debug_inference_systematic.format](#)
- str [debug_inference_systematic.input_file](#) = 'traindata/11_5/inputcube/235101017859661465075472232303048949736_0.npy'
- str [debug_inference_systematic.output_file](#) = 'traindata/11_5/outputcube/235101017859661465075472232303048949736_0.npy'
- [debug_inference_systematic.input_data](#) = np.load([input_file](#))
- [debug_inference_systematic.ground_truth](#) = np.load([output_file](#))
- str [debug_inference_systematic.model_path](#) = '/Users/giannigagliardi/Documents/Git/RadioTherapy/unified_energy_conditioned_model_res16.0_energies2.ckpt'
- [debug_inference_systematic.device](#) = torch.device("cpu")
- [debug_inference_systematic.checkpoint](#) = torch.load([model_path](#), map_location=[device](#))
- [debug_inference_systematic.config](#) = [checkpoint](#)['training_config']
- [debug_inference_systematic.training_cube_size](#) = [config](#)['cube_size']
- [debug_inference_systematic.autoencoder](#)

- `debug_inference_systematic.input_tensor` = `torch.tensor(input_data, dtype=torch.float32).unsqueeze(0).unsqueeze(0).to(device)`
- `debug_inference_systematic.ground_truth_tensor` = `torch.tensor(ground_truth, dtype=torch.float32).unsqueeze(0).unsqueeze(0).to(device)`
- `debug_inference_systematic.input_resized` = `interpolate(input_tensor, size=training_cube_size, mode='trilinear', align_corners=True)`
- `debug_inference_systematic.ground_truth_resized` = `interpolate(ground_truth_tensor, size=training_cube_size, mode='trilinear', align_corners=True)`
- `float debug_inference_systematic.energy_value` = 11.5
- `list debug_inference_systematic.energy_normalizations`
- `int debug_inference_systematic.best_correlation` = -1
- `debug_inference_systematic.best_method` = None
- `debug_inference_systematic.best_reconstruction` = None
- `debug_inference_systematic.input_2channel` = `torch.zeros((1, 2, *training_cube_size), device=device)`
- `debug_inference_systematic.encoded` = `autoencoder.encode(input_2channel)`
- `debug_inference_systematic.latent` = `encoded[0]`
- `debug_inference_systematic.decoded` = `autoencoder.decode(latent)`
- `debug_inference_systematic.reconstruction` = `decoded[0]`
- `debug_inference_systematic.reconstruction_full` = `interpolate(reconstruction, size=(100, 100, 100), mode='trilinear', align_corners=True)`
- `debug_inference_systematic.reconstruction_np` = `reconstruction_full.cpu().numpy().squeeze()`
- `debug_inference_systematic.mse` = `np.mean((reconstruction_np - ground_truth) ** 2)`
- `debug_inference_systematic.mae` = `np.mean(np.abs(reconstruction_np - ground_truth))`
- `debug_inference_systematic.correlation` = `np.corrcoef(reconstruction_np.flatten(), ground_truth.flatten())[0, 1]`

8.37 debug_inference_systematic.py

[Go to the documentation of this file.](#)

```

00001 #!/usr/bin/env python3
00002 """
00003 Systematische Analyse der Inference-Pipeline Probleme
00004 """
00005
00006 import os
00007 import sys
00008 import torch
00009 import numpy as np
00010 import logging
00011
00012 # Add paths
00013 sys.path.append('sourcecode')
00014 os.environ['PYTHONPATH'] = '/Users/giannigagliardi/Documents/Git/RadioTherapy'
00015
00016 # Setup logging
00017 logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(name)s - %(levelname)s - %(message)s')
00018
00019 print(" SYSTEMATIC INFERENCE ANALYSIS")
00020 print("=" * 60)
00021
00022 # Test verschiedene Cube-Größen und Normalisierungen
00023 input_file = 'traindata/11_5/inputcube/235101017859661465075472232303048949736_0.npy'
00024 output_file = 'traindata/11_5/outputcube/235101017859661465075472232303048949736_0.npy'
00025
00026 if os.path.exists(input_file) and os.path.exists(output_file):
00027     input_data = np.load(input_file)
00028     ground_truth = np.load(output_file)
00029
00030     print(f"Original data shapes: {input_data.shape} -> {ground_truth.shape}")
00031
00032     # Lade das trainierte Modell
00033     model_path =
00034     '/Users/giannigagliardi/Documents/Git/RadioTherapy/unified_energy_conditioned_model_res16.0_energies2.ckpt'
00035     device = torch.device("cpu")
00036     checkpoint = torch.load(model_path, map_location=device)
00037
00038     # Schau dir die Training-Konfiguration an
00039     print(f"\n Training Configuration:")
00040     if 'training_config' in checkpoint:
00041         config = checkpoint['training_config']
00042         print(f" Cube size: {config.get('cube_size', 'Not found')}")

```

```

00042     print(f" Batch size: {config.get('batch_size', 'Not found')}")
00043     print(f" Epochs: {config.get('num_epochs', 'Not found')}")
00044     print(f" Learning rate: {config.get('learning_rate', 'Not found')}")
00045
00046     # Verwende die TRAINING cube_size!
00047     if 'cube_size' in config:
00048         training_cube_size = config['cube_size']
00049         print(f" Using training cube size: {training_cube_size}")
00050     else:
00051         training_cube_size = (32, 32, 32) # Default
00052         print(f" Using default cube size: {training_cube_size}")
00053     else:
00054         training_cube_size = (32, 32, 32)
00055         print(f" No training config found, using default: {training_cube_size}")
00056
00057     # Lade Autoencoder
00058     from generative.networks.nets import AutoencoderKL
00059     autoencoder = AutoencoderKL(
00060         spatial_dims=3,
00061         in_channels=2,
00062         out_channels=1,
00063         num_channels=(32, 32, 32),
00064         latent_channels=2,
00065         num_res_blocks=1,
00066         norm_num_groups=8,
00067         attention_levels=(False, False, True),
00068     ).to(device)
00069
00070     autoencoder.load_state_dict(checkpoint['autoencoder'])
00071     autoencoder.eval()
00072
00073     print(f"\n CORRECTED TEST: Using training cube size")
00074
00075     # Resize input auf training cube size
00076     from torch.nn.functional import interpolate
00077
00078     # Konvertiere zu Tensor und füge Batch-Dimension hinzu
00079     input_tensor = torch.tensor(input_data, dtype=torch.float32).unsqueeze(0).unsqueeze(0).to(device)
00080     ground_truth_tensor = torch.tensor(ground_truth, dtype=torch.float32).unsqueeze(0).unsqueeze(0).to(device)
00081
00082     # Resize auf Training-Größe
00083     input_resized = interpolate(input_tensor, size=training_cube_size, mode='trilinear', align_corners=True)
00084     ground_truth_resized = interpolate(ground_truth_tensor, size=training_cube_size, mode='trilinear',
align_corners=True)
00085
00086     print(f" Resized input: {input_tensor.shape} -> {input_resized.shape}")
00087     print(f" Resized ground truth: {ground_truth_tensor.shape} -> {ground_truth_resized.shape}")
00088
00089     # Teste verschiedene Energie-Normalisierungen
00090     energy_value = 11.5
00091     energy_normalizations = [
00092         ("div_40", energy_value / 40.0),
00093         ("div_100", energy_value / 100.0),
00094         ("div_50", energy_value / 50.0),
00095         ("raw", energy_value),
00096         ("normalized_0_1", (energy_value - 11.5) / (34.25 - 11.5)) # Min-Max normalization
00097     ]
00098
00099     best_correlation = -1
00100     best_method = None
00101     best_reconstruction = None
00102
00103     for method_name, energy_normalized in energy_normalizations:
00104         print(f"\n Testing energy normalization: {method_name} (value: {energy_normalized:.6f})")
00105
00106         # Erstelle 2-channel input
00107         input_2channel = torch.zeros((1, 2, *training_cube_size), device=device)
00108         input_2channel[0, 0] = input_resized.squeeze() # CT data
00109         input_2channel[0, 1] = energy_normalized # Energy channel
00110
00111         with torch.no_grad():
00112             # Encode
00113             encoded = autoencoder.encode(input_2channel)
00114             if isinstance(encoded, tuple):
00115                 latent = encoded[0]
00116             else:
00117                 latent = encoded.latent_dist.sample()
00118
00119             # Decode
00120             decoded = autoencoder.decode(latent)
00121             if isinstance(decoded, tuple):
00122                 reconstruction = decoded[0]
00123             else:
00124                 reconstruction = decoded
00125
00126             # Resize zurück auf Original-Größe für Vergleich
00127             reconstruction_full = interpolate(reconstruction, size=(100, 100, 100), mode='trilinear',

```

```

align_corners=True)
00128
00129     reconstruction_np = reconstruction_full.cpu().numpy().squeeze()
00130
00131     # Vergleiche mit Ground Truth
00132     mse = np.mean((reconstruction_np - ground_truth) ** 2)
00133     mae = np.mean(np.abs(reconstruction_np - ground_truth))
00134     correlation = np.corrcoef(reconstruction_np.flatten(), ground_truth.flatten())[0, 1]
00135
00136     print(f"    MSE: {mse:.8f}")
00137     print(f"    MAE: {mae:.8f}")
00138     print(f"    Correlation: {correlation:.6f}")
00139     print(f"    Reconstruction min/max: {reconstruction_np.min():.6f} / {reconstruction_np.max():.6f}")
00140
00141     if correlation > best_correlation:
00142         best_correlation = correlation
00143         best_method = method_name
00144         best_reconstruction = reconstruction_np.copy()
00145
00146     print(f"    NEW BEST METHOD!")
00147
00148     print(f"\n BEST RESULT:")
00149     print(f"    Method: {best_method}")
00150     print(f"    Correlation: {best_correlation:.6f}")
00151
00152     if best_correlation > 0.3:
00153         print(f"    GOOD: Reasonable correlation found!")
00154     else:
00155         print(f"    STILL BAD: Poor correlation even with corrections")
00156
00157     # Save best reconstruction
00158     if best_reconstruction is not None:
00159         np.save('debug_best_reconstruction.npy', best_reconstruction)
00160         print(f"    Saved best reconstruction to: debug_best_reconstruction.npy")
00161
00162     print(f"\n RECOMMENDATION:")
00163     if best_correlation > 0.3:
00164         print(f"    Use energy normalization method: {best_method}")
00165         print(f"    Ensure cube size matches training: {training_cube_size}")
00166         print(f"    Fix inference pipeline to use these parameters!")
00167     else:
00168         print(f"    The model may not be properly trained for energy conditioning")
00169         print(f"    Consider retraining with more epochs or different parameters")
00170
00171 else:
00172     print(f" Test files not found!")
00173
00174 print("\n" + "=" * 60)
00175 print("Systematic analysis complete!")

```

8.38 tests/debug_training_data.py File Reference

Namespaces

- namespace [debug_training_data](#)

Variables

- list [debug_training_data.test_files](#)
- list [debug_training_data.output_files](#)
- list [debug_training_data.energies](#) = [11.5, 15.75, 34.25]
- [debug_training_data.input_data](#) = np.load(input_file)
- [debug_training_data.output_data](#) = np.load(output_file)
- [debug_training_data.correlation](#) = np.corrcoef(input_data.flatten(), output_data.flatten())[0, 1]
- [debug_training_data.data_loader](#)
- [debug_training_data.train_loader](#)
- [debug_training_data.val_loader](#)
- [debug_training_data._](#)
- [debug_training_data.input_batch](#) = batch['inputcube']
- [debug_training_data.output_batch](#) = batch['outputcube']
- [debug_training_data.energy_batch](#) = batch['energy']
- [debug_training_data.device](#) = torch.device("cpu")
- [debug_training_data.test_input](#) = torch.randn(1, 2, 32, 32, 32)

- `debug_training_data.test_output = test_input[:, 0:1, :, :, :]`
- `str debug_training_data.model_path = '/Users/giannigagliardi/Documents/Git/RadioTherapy/unified_←_energy_conditioned_model_res16.0_energies2.ckpt'`
- `debug_training_data.checkpoint = torch.load(model_path, map_location=device)`
- `debug_training_data.autoencoder`
- `debug_training_data.encoded = autoencoder.encode(test_input)`
- `debug_training_data.latent = encoded[0]`
- `debug_training_data.decoded = autoencoder.decode(latent)`
- `debug_training_data.reconstruction = decoded[0]`

8.39 debug_training_data.py

Go to the documentation of this file.

```
00001 #!/usr/bin/env python3
00002 """
00003 Überprüfe die Trainings-Daten auf Konsistenz
00004 """
00005
00006 import os
00007 import sys
00008 import torch
00009 import numpy as np
00010 import matplotlib.pyplot as plt
00011
00012 # Add paths
00013 sys.path.append('sourcecode')
00014 os.environ['PYTHONPATH'] = '/Users/giannigagliardi/Documents/Git/RadioTherapy'
00015
00016 print(" TRAINING DATA VALIDATION")
00017 print("=" * 50)
00018
00019 # Teste mehrere Input-Output Paare
00020 test_files = [
00021     'traindata/11_5/inputcube/235101017859661465075472232303048949736_0.npy',
00022     'traindata/15_75/inputcube/235101017859661465075472232303048949736_0.npy',
00023     'traindata/34_25/inputcube/235101017859661465075472232303048949736_0.npy'
00024 ]
00025
00026 output_files = [
00027     'traindata/11_5/outputcube/235101017859661465075472232303048949736_0.npy',
00028     'traindata/15_75/outputcube/235101017859661465075472232303048949736_0.npy',
00029     'traindata/34_25/outputcube/235101017859661465075472232303048949736_0.npy'
00030 ]
00031
00032 energies = [11.5, 15.75, 34.25]
00033
00034 for i, (input_file, output_file, energy) in enumerate(zip(test_files, output_files, energies)):
00035     print(f"\n Testing Energy {energy} MeV:")
00036     print(f" Input: {input_file}")
00037     print(f" Output: {output_file}")
00038
00039     if os.path.exists(input_file) and os.path.exists(output_file):
00040         input_data = np.load(input_file)
00041         output_data = np.load(output_file)
00042
00043         print(f" Files exist")
00044         print(f" Input shape: {input_data.shape}")
00045         print(f" Output shape: {output_data.shape}")
00046         print(f" Input min/max: {input_data.min():.6f} / {input_data.max():.6f}")
00047         print(f" Output min/max: {output_data.min():.6f} / {output_data.max():.6f}")
00048         print(f" Output mean/std: {output_data.mean():.6f} / {output_data.std():.6f}")
00049         print(f" Output non-zero: {np.count_nonzero(output_data)}")
00050
00051         # Check if output is all zeros or constant
00052         if np.all(output_data == 0):
00053             print(f" OUTPUT IS ALL ZEROS!")
00054         elif np.all(output_data == output_data.flat[0]):
00055             print(f" OUTPUT IS CONSTANT!")
00056         else:
00057             print(f" Output has variation")
00058
00059         # Quick correlation check between input and output
00060         correlation = np.corrcoef(input_data.flatten(), output_data.flatten())[0, 1]
00061         print(f" Input-Output correlation: {correlation:.6f}")
00062
00063     else:
00064         print(f" Files missing")
00065
00066 # Teste das Data Loading System
```

```

00067 print(f"\n Testing Data Loading System:")
00068
00069 try:
00070     from sourcecode.data_management import DataLoaderModule
00071     from monai.transforms import Compose
00072
00073     # Test data loading
00074     data_loader = DataLoaderModule(
00075         root_dir='traindata',
00076         transforms=Compose([]),
00077         resolutions=[16.0],
00078         energies=[11.5, 15.75, 34.25],
00079         batch_size=1,
00080         cube_size=(32, 32, 32),
00081         seed=42
00082     )
00083
00084     print(f"    DataLoaderModule imported successfully")
00085
00086     # Get a sample
00087     train_loader, val_loader, _ = data_loader.get_data_loaders()
00088     print(f"    Train batches: {len(train_loader)}")
00089     print(f"    Val batches: {len(val_loader)}")
00090
00091     # Test one batch
00092     for batch in train_loader:
00093         input_batch = batch['inputcube']
00094         output_batch = batch['outputcube']
00095         energy_batch = batch['energy']
00096
00097         print(f"    Sample batch:")
00098         print(f"        Input shape: {input_batch.shape}")
00099         print(f"        Output shape: {output_batch.shape}")
00100         print(f"        Energy values: {energy_batch}")
00101         print(f"        Input min/max: {input_batch.min():.6f} / {input_batch.max():.6f}")
00102         print(f"        Output min/max: {output_batch.min():.6f} / {output_batch.max():.6f}")
00103         print(f"        Output mean/std: {output_batch.mean():.6f} / {output_batch.std():.6f}")
00104
00105         # Check for problems
00106         if torch.all(output_batch == 0):
00107             print(f"            BATCH OUTPUT IS ALL ZEROS!")
00108         elif torch.all(output_batch == output_batch.flat[0]):
00109             print(f"            BATCH OUTPUT IS CONSTANT!")
00110         else:
00111             print(f"            Batch output has variation")
00112
00113         break # Only test first batch
00114
00115 except Exception as e:
00116     print(f"    Error testing data loading: {e}")
00117
00118 # Test if the model can even learn identity mapping
00119 print(f"\n Testing if model can learn identity mapping:")
00120
00121 # Create simple test: input = output
00122 device = torch.device("cpu")
00123 test_input = torch.randn(1, 2, 32, 32, 32) # 2 channels for energy conditioning
00124 test_output = test_input[:, 0:1, :, :, :] # Take only first channel as output
00125
00126 print(f"    Test input shape: {test_input.shape}")
00127 print(f"    Test output shape: {test_output.shape}")
00128 print(f"    Input min/max: {test_input.min():.6f} / {test_input.max():.6f}")
00129 print(f"    Output min/max: {test_output.min():.6f} / {test_output.max():.6f}")
00130
00131 # Load model and test
00132 model_path =
00133     '/Users/giannigagliardi/Documents/Git/RadioTherapy/unified_energy_conditioned_model_res16.0_energies2.ckpt'
00134 if os.path.exists(model_path):
00135     checkpoint = torch.load(model_path, map_location=device)
00136
00137     from generative.networks.nets import AutoencoderKL
00138     autoencoder = AutoencoderKL(
00139         spatial_dims=3,
00140         in_channels=2,
00141         out_channels=1,
00142         num_channels=(32, 32, 32),
00143         latent_channels=2,
00144         num_res_blocks=1,
00145         norm_num_groups=8,
00146         attention_levels=(False, False, True),
00147     ).to(device)
00148
00149     autoencoder.load_state_dict(checkpoint['autoencoder'])
00150     autoencoder.eval()
00151
00152     with torch.no_grad():
00153         encoded = autoencoder.encode(test_input)

```

```

00153         if isinstance(encoded, tuple):
00154             latent = encoded[0]
00155         else:
00156             latent = encoded.latent_dist.sample()
00157
00158         decoded = autoencoder.decode(latent)
00159         if isinstance(decoded, tuple):
00160             reconstruction = decoded[0]
00161         else:
00162             reconstruction = decoded
00163
00164         print(f" Model reconstruction shape: {reconstruction.shape}")
00165         print(f" Model reconstruction min/max: {reconstruction.min():.6f} / {reconstruction.max():.6f}")
00166
00167         # Test correlation
00168         correlation = np.corrcoef(test_input[:, 0].flatten(), reconstruction.flatten())[0, 1]
00169         print(f" Correlation with first input channel: {correlation:.6f}")
00170
00171         if correlation > 0.5:
00172             print(f" Model can reconstruct reasonably well")
00173         else:
00174             print(f" Model cannot even reconstruct simple inputs")
00175
00176     print("\n" + "=" * 50)
00177     print("Training data validation complete!")

```

8.40 tests/debug_visualization.py File Reference

Namespaces

- namespace [debug_visualization](#)

Functions

- [debug_visualization.visualize_dose_distribution](#) (cube_path, smooth_sigma=0.5, title="Dose Distribution")
- [debug_visualization.create_slice_comparison](#) (cube_path1, cube_path2, slice_axis=2)

Variables

- str [debug_visualization.cube_path](#) = "/Users/giannigagliardi/Documents/Git/RadioTherapy/traindata/11↵_5/outputcube/235101017859661465075472232303048949736_10.npy"
- [debug_visualization.fig](#) = [visualize_dose_distribution](#)(cube_path, smooth_sigma=0.5)

8.41 debug_visualization.py

[Go to the documentation of this file.](#)

```

00001 import matplotlib.pyplot as plt
00002 import numpy as np
00003 import plotly.graph_objects as go
00004 from scipy import ndimage
00005
00006 def visualize_dose_distribution(cube_path, smooth_sigma=0.5, title="Dose Distribution"):
00007     """
00008     Verbesserte Visualisierung für Dosisverteilungen.
00009
00010     Args:
00011         cube_path: Pfad zur .npy Datei
00012         smooth_sigma: Sigma für Gaussian-Filter (0 = kein Smoothing)
00013         title: Titel für die Visualisierung
00014     """
00015     # Lade die Daten
00016     cube = np.load(cube_path)
00017
00018     # Handle verschiedene Dimensionen
00019     if cube.ndim == 4:
00020         cube = cube[0]
00021     elif cube.ndim == 5:
00022         cube = cube[0, 0] # Entferne Batch und Channel-Dimensionen
00023
00024     print(f"Cube shape: {cube.shape}")
00025     print(f"Cube min/max: {cube.min():.4f}/{cube.max():.4f}")
00026     print(f"Cube mean/std: {cube.mean():.4f}/{cube.std():.4f}")
00027
00028     # Normalisierung

```

```

00029     cube_norm = (cube - cube.min()) / (cube.max() - cube.min() + 1e-8)
00030
00031     # Optional: Smoothing anwenden
00032     if smooth_sigma > 0:
00033         cube_smoothed = ndimage.gaussian_filter(cube_norm, sigma=smooth_sigma)
00034     else:
00035         cube_smoothed = cube_norm
00036
00037     # Berechne Percentile für bessere Threshold-Wahl
00038     percentiles = np.percentile(cube_smoothed, [5, 10, 25, 50, 75, 90, 95, 99])
00039     print(f"Percentiles: {dict(zip([5, 10, 25, 50, 75, 90, 95, 99], percentiles))}")
00040
00041     # 3D Koordinaten
00042     x, y, z = np.mgrid[0:cube.shape[0], 0:cube.shape[1], 0:cube.shape[2]]
00043
00044     # Erstelle Volume Rendering
00045     fig = go.Figure(
00046         data=go.Volume(
00047             x=x.flatten(),
00048             y=y.flatten(),
00049             z=z.flatten(),
00050             value=cube_smoothed.flatten(),
00051             opacity=0.15,
00052             surface_count=12,
00053             isomin=percentiles[2], # 25th percentile
00054             isomax=percentiles[6], # 95th percentile
00055             colorscale="Hot",
00056             caps=dict(x_show=False, y_show=False, z_show=False),
00057             colorbar=dict(title="Normalized Dose")
00058         )
00059     )
00060
00061     fig.update_layout(
00062         title=title,
00063         scene=dict(
00064             aspectmode="data",
00065             xaxis=dict(title="X", range=[0, cube.shape[0]]),
00066             yaxis=dict(title="Y", range=[0, cube.shape[1]]),
00067             zaxis=dict(title="Z", range=[0, cube.shape[2]])
00068         )
00069     )
00070
00071     return fig
00072
00073 def create_slice_comparison(cube_path1, cube_path2, slice_axis=2):
00074     """
00075     Vergleiche zwei Dosisverteilungen slice-weise.
00076     """
00077     cube1 = np.load(cube_path1)
00078     cube2 = np.load(cube_path2)
00079
00080     if cube1.ndim == 4:
00081         cube1 = cube1[0]
00082     if cube2.ndim == 4:
00083         cube2 = cube2[0]
00084
00085     mid_slice = cube1.shape[slice_axis] // 2
00086
00087     if slice_axis == 2:
00088         slice1 = cube1[:, :, mid_slice]
00089         slice2 = cube2[:, :, mid_slice]
00090     elif slice_axis == 1:
00091         slice1 = cube1[:, mid_slice, :]
00092         slice2 = cube2[:, mid_slice, :]
00093     else:
00094         slice1 = cube1[mid_slice, :, :]
00095         slice2 = cube2[mid_slice, :, :]
00096
00097     fig, axes = plt.subplots(1, 3, figsize=(15, 5))
00098
00099     im1 = axes[0].imshow(slice1, cmap='hot')
00100     axes[0].set_title('Expected')
00101     axes[0].axis('off')
00102     plt.colorbar(im1, ax=axes[0])
00103
00104     im2 = axes[1].imshow(slice2, cmap='hot')
00105     axes[1].set_title('Actual')
00106     axes[1].axis('off')
00107     plt.colorbar(im2, ax=axes[1])
00108
00109     # Differenz
00110     diff = slice1 - slice2
00111     im3 = axes[2].imshow(diff, cmap='RdBu_r')
00112     axes[2].set_title('Difference')
00113     axes[2].axis('off')
00114     plt.colorbar(im3, ax=axes[2])
00115

```



```

00116     plt.tight_layout()
00117     plt.show()
00118
00119 if __name__ == "__main__":
00120     # Beispiel-Nutzung
00121     cube_path =
00122     "/Users/giannigagliardi/Documents/Git/RadioTherapy/traindata/11_5/outputcube/235101017859661465075472232303048949736_10.npy"
00123
00123     fig = visualize_dose_distribution(cube_path, smooth_sigma=0.5)
00124     fig.show()

```

8.42 tests/simple_inference_test.py File Reference

Namespaces

- namespace `simple_inference_test`

Functions

- `simple_inference_test.main()`

Variables

- `simple_inference_test.level`
- `simple_inference_test.logger = logging.getLogger(__name__)`

8.43 simple_inference_test.py

[Go to the documentation of this file.](#)

```

00001 #!/usr/bin/env python3
00002 """
00003 Simple test to compare inference output with ground truth data.
00004 """
00005
00006 import numpy as np
00007 import torch
00008 import torch.nn.functional as F
00009 import matplotlib.pyplot as plt
00010 from pathlib import Path
00011 import logging
00012
00013 # Setup logging
00014 logging.basicConfig(level=logging.INFO)
00015 logger = logging.getLogger(__name__)
00016
00017 def main():
00018     """
00019     Simple comparison between inference and ground truth.
00020     """
00021     logger.info("=== SIMPLE INFERENCE TEST ===")
00022
00023     # 1. Load ground truth data
00024     logger.info("Loading ground truth data...")
00025
00026     # Load input and output from 11.5 keV energy
00027     input_file = Path("traindata/11_5/inputcube/235101017859661465075472232303048949736_0.npy")
00028     output_file = Path("traindata/11_5/outputcube/235101017859661465075472232303048949736_0.npy")
00029
00030     if not input_file.exists() or not output_file.exists():
00031         logger.error(f"Files not found: {input_file}, {output_file}")
00032         return
00033
00034     input_data = np.load(input_file)
00035     output_data = np.load(output_file)
00036
00037     logger.info(f"Input shape: {input_data.shape}, range: {input_data.min():.3f} to {input_data.max():.3f}")
00038     logger.info(f"Output shape: {output_data.shape}, range: {output_data.min():.6f} to {output_data.max():.6f}")
00039
00040     # 2. Load the trained model
00041     logger.info("Loading model...")
00042
00043     model_path = "unified_energy_conditioned_model_res16.0_energies3.ckpt"
00044     if not Path(model_path).exists():
00045         logger.error(f"Model not found: {model_path}")
00046         return
00047
00048     try:

```

```

00049     checkpoint = torch.load(model_path, map_location='cpu')
00050     logger.info(f"Checkpoint keys: {list(checkpoint.keys())}")
00051
00052     # Try to understand the model structure
00053     if 'unet' in checkpoint:
00054         unet_state = checkpoint['unet']
00055         autoencoder_state = checkpoint.get('autoencoder', {})
00056         scheduler_state = checkpoint.get('scheduler', {})
00057
00058         logger.info(f"Found separate components: unet, autoencoder, scheduler")
00059         logger.info(f"UNet state keys (first 10): {list(unet_state.keys())[:10]}")
00060
00061         # Check UNet architecture
00062         input_layer_found = False
00063         for key in unet_state.keys():
00064             if 'conv' in key.lower() and 'weight' in key:
00065                 weight_shape = unet_state[key].shape
00066                 logger.info(f"UNet layer {key}: weight shape {weight_shape}")
00067                 if not input_layer_found:
00068                     logger.info(f"First conv layer appears to expect {weight_shape[1]} input channels")
00069                     input_layer_found = True
00070                 break
00071
00072         model_state = unet_state
00073
00074     elif 'model_state_dict' in checkpoint:
00075         model_state = checkpoint['model_state_dict']
00076     else:
00077         model_state = checkpoint
00078
00079     # Print some keys to understand the model
00080     state_keys = list(model_state.keys())[:10] # First 10 keys
00081     logger.info(f"Model state keys (first 10): {state_keys}")
00082
00083 except Exception as e:
00084     logger.error(f"Error loading model: {e}")
00085     return
00086
00087 # 3. Simple direct comparison - can we even load the model?
00088 logger.info("Attempting to create and load model...")
00089
00090 try:
00091     # Try to create a simple model that matches the architecture
00092     from generative.networks.nets import DiffusionModelUNet
00093
00094     # Use the EXACT configuration from the checkpoint
00095     configs = [
00096         {
00097             "in_channels": 2,
00098             "out_channels": 2,
00099             "cross_attention_dim": 2,
00100             "num_channels": (32, 64, 64),
00101             "name": "EXACT match from checkpoint"
00102         },
00103     ]
00104
00105     for config in configs:
00106         try:
00107             logger.info(f"Testing {config['name']} configuration...")
00108
00109             model = DiffusionModelUNet(
00110                 spatial_dims=3,
00111                 in_channels=config["in_channels"],
00112                 out_channels=config["out_channels"],
00113                 num_res_blocks=1, # From SystemManager
00114                 num_channels=config["num_channels"],
00115                 attention_levels=(False, True, True),
00116                 num_head_channels=(0, 32, 32), # From SystemManager
00117                 with_conditioning=True,
00118                 transformer_num_layers=1,
00119                 cross_attention_dim=config["cross_attention_dim"],
00120                 upcast_attention=True,
00121             )
00122
00123             # Try to load state dict
00124             missing_keys, unexpected_keys = model.load_state_dict(model_state, strict=False)
00125
00126             logger.info(f"Missing keys: {len(missing_keys)}")
00127             logger.info(f"Unexpected keys: {len(unexpected_keys)}")
00128
00129             if len(missing_keys) == 0:
00130                 logger.info(f"{config['name']} configuration loaded successfully!")
00131
00132             # Test inference
00133             model.eval()
00134
00135             # Prepare input

```

```

00136         input_tensor = torch.from_numpy(input_data).float().unsqueeze(0).unsqueeze(0) # [1, 1, D, H, W]
00137
00138         # Resize to model expected size (64x64x64)
00139         if input_tensor.shape[2:] != (64, 64, 64):
00140             input_tensor = F.interpolate(input_tensor, size=(64, 64, 64), mode='trilinear',
align_corners=False)
00141
00142         if config["in_channels"] == 2:
00143             # Add energy channel
00144             energy_value = 11.5
00145             energy_normalized = energy_value / 100.0 # Normalize
00146             energy_channel = torch.full_like(input_tensor, energy_normalized)
00147             input_tensor = torch.cat([input_tensor, energy_channel], dim=1)
00148
00149         logger.info(f" Input tensor shape: {input_tensor.shape}")
00150
00151         # Create context for cross-attention (2D context)
00152         energy_tensor = torch.tensor([[energy_value / 100.0, 1.0]], dtype=torch.float32) # [1, 2]
00153         context = energy_tensor.unsqueeze(1) # [1, 1, 2] - Add sequence dimension
00154
00155         # Create timestep
00156         timestep = torch.tensor([0], dtype=torch.long)
00157
00158         # Run inference
00159         with torch.no_grad():
00160             try:
00161                 output = model(
00162                     x=input_tensor,
00163                     timesteps=timestep,
00164                     context=context
00165                 )
00166
00167                 logger.info(f" Inference successful! Output shape: {output.shape}")
00168                 logger.info(f" Output range: {output.min():.6f} to {output.max():.6f}")
00169
00170                 # If model outputs 2 channels, take the first one
00171                 if output.shape[1] == 2:
00172                     output = output[:, :1] # Take first channel
00173                     logger.info(f" Took first channel, new shape: {output.shape}")
00174
00175                 # Resize back to original size
00176                 if output.shape[2:] != output_data.shape:
00177                     output = F.interpolate(output, size=output_data.shape, mode='trilinear',
align_corners=False)
00178
00179                 # Compare with ground truth
00180                 output_np = output[0, 0].cpu().numpy()
00181
00182                 # Statistical comparison
00183                 logger.info(" === COMPARISON WITH GROUND TRUTH ===")
00184                 logger.info(f" Ground truth range: {output_data.min():.6f} to {output_data.max():.6f}")
00185                 logger.info(f" Inference range: {output_np.min():.6f} to {output_np.max():.6f}")
00186
00187                 # Check if ranges are reasonable
00188                 gt_max = output_data.max()
00189                 inf_max = output_np.max()
00190
00191                 if gt_max > 0:
00192                     scale_ratio = inf_max / gt_max
00193                     logger.info(f" Scale ratio (inference/ground_truth): {scale_ratio:.6f}")
00194
00195                 # Correlation
00196                 try:
00197                     correlation = np.corrcoef(output_data.flatten(), output_np.flatten())[0, 1]
00198                     logger.info(f" Correlation: {correlation:.6f}")
00199
00200                     if abs(correlation) < 0.1:
00201                         logger.info(" VERY LOW CORRELATION!")
00202                     elif abs(correlation) < 0.3:
00203                         logger.info(" LOW CORRELATION")
00204                     else:
00205                         logger.info(" Reasonable correlation")
00206                 except:
00207                     logger.info(" Could not compute correlation")
00208
00209                 # Save visualization
00210                 fig, axes = plt.subplots(2, 3, figsize=(15, 10))
00211
00212                 # Take middle slices
00213                 slice_idx = output_data.shape[2] // 2
00214
00215                 # Ground truth
00216                 im1 = axes[0, 0].imshow(output_data[:, :, slice_idx], cmap='hot')
00217                 axes[0, 0].set_title(f'Ground Truth\nMax: {output_data.max():.6f}')
00218                 axes[0, 0].axis('off')
00219                 plt.colorbar(im1, ax=axes[0, 0])
00220

```

```

00221         # Inference
00222         im2 = axes[0, 1].imshow(output_np[:, :, slice_idx], cmap='hot')
00223         axes[0, 1].set_title(f'Inference\nMax: {output_np.max():.6f}')
00224         axes[0, 1].axis('off')
00225         plt.colorbar(im2, ax=axes[0, 1])
00226
00227         # Difference
00228         diff = output_np - output_data
00229         im3 = axes[0, 2].imshow(diff[:, :, slice_idx], cmap='RdBu_r')
00230         axes[0, 2].set_title(f'Difference\nMax: {abs(diff).max():.6f}')
00231         axes[0, 2].axis('off')
00232         plt.colorbar(im3, ax=axes[0, 2])
00233
00234         # Show different slice (axial)
00235         slice_idx_y = output_data.shape[1] // 2
00236
00237         # Ground truth (axial)
00238         im4 = axes[1, 0].imshow(output_data[:, slice_idx_y, :], cmap='hot')
00239         axes[1, 0].set_title('Ground Truth (Axial)')
00240         axes[1, 0].axis('off')
00241         plt.colorbar(im4, ax=axes[1, 0])
00242
00243         # Inference (axial)
00244         im5 = axes[1, 1].imshow(output_np[:, slice_idx_y, :], cmap='hot')
00245         axes[1, 1].set_title('Inference (Axial)')
00246         axes[1, 1].axis('off')
00247         plt.colorbar(im5, ax=axes[1, 1])
00248
00249         # Difference (axial)
00250         im6 = axes[1, 2].imshow(diff[:, slice_idx_y, :], cmap='RdBu_r')
00251         axes[1, 2].set_title('Difference (Axial)')
00252         axes[1, 2].axis('off')
00253         plt.colorbar(im6, ax=axes[1, 2])
00254
00255         plt.tight_layout()
00256         plt.savefig(f'comparison_correct_model.png', dpi=150, bbox_inches='tight')
00257         logger.info(f" Saved comparison as comparison_correct_model.png")
00258
00259         return True # Success
00260
00261     except Exception as e:
00262         logger.info(f" Inference failed: {e}")
00263         import traceback
00264         traceback.print_exc()
00265
00266     else:
00267         logger.info(f" {config['name']} configuration has missing keys")
00268
00269     except Exception as e:
00270         logger.info(f" Error with {config['name']} configuration: {e}")
00271
00272     except Exception as e:
00273         logger.error(f"Error creating model: {e}")
00274         return
00275
00276     logger.info("=== TEST COMPLETED ===")
00277
00278 if __name__ == "__main__":
00279     main()

```

8.44 tests/systematic_dose_diagnosis.py File Reference

Classes

- class [systematic_dose_diagnosis.DoseNormalizationDiagnostic](#)

Namespaces

- namespace [systematic_dose_diagnosis](#)

Functions

- [systematic_dose_diagnosis.main](#) ()

Variables

- [systematic_dose_diagnosis.level](#)
- [systematic_dose_diagnosis.INFO](#)

- `systematic_dose_diagnosis.format`
- `systematic_dose_diagnosis.logger = logging.getLogger(__name__)`
- `bool systematic_dose_diagnosis.has_nibabel = True`

8.45 systematic_dose_diagnosis.py

[Go to the documentation of this file.](#)

```

00001 #!/usr/bin/env python3
00002 """
00003 Systematic Dose Normalization Diagnosis Script
00004 Phase 1: Root Cause Investigation
00005
00006 This script performs comprehensive analysis of the training data pipeline
00007 to identify the exact source of the 21x dose scaling discrepancy.
00008 """
00009
00010 import logging
00011 import numpy as np
00012 import torch
00013 from pathlib import Path
00014 import json
00015 import sys
00016 import matplotlib.pyplot as plt
00017 from collections import defaultdict
00018
00019 # Setup logging
00020 logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')
00021 logger = logging.getLogger(__name__)
00022
00023 # Add sourcecode to path
00024 sys.path.append('sourcecode')
00025
00026 try:
00027     import nibabel as nib
00028     has_nibabel = True
00029 except ImportError:
00030     has_nibabel = False
00031
00032
00033 class DoseNormalizationDiagnostic:
00034     """Comprehensive diagnosis of dose normalization pipeline."""
00035
00036     def __init__(self, traindata_dir="traindata", checkpoint_path=None):
00037         self.traindata_dir = Path(traindata_dir)
00038         self.checkpoint_path = checkpoint_path
00039         self.results = {
00040             'raw_dose_statistics': {},
00041             'training_normalization_params': {},
00042             'checkpoint_normalization_params': {},
00043             'inference_parameters': {},
00044             'medical_plausibility': {},
00045             'data_flow_analysis': {},
00046             'recommendations': []
00047         }
00048
00049     def analyze_raw_dose_data(self):
00050         """Phase 1.1: Analyze raw dose data in traindata directory."""
00051         logger.info("=" * 60)
00052         logger.info("PHASE 1.1: RAW DOSE DATA ANALYSIS")
00053         logger.info("=" * 60)
00054
00055         if not self.traindata_dir.exists():
00056             logger.error(f"Training data directory not found: {self.traindata_dir}")
00057             return False
00058
00059         energy_stats = {}
00060         global_min, global_max = float('inf'), float('-inf')
00061         total_files = 0
00062
00063         # Search all energy subdirectories
00064         for energy_dir in self.traindata_dir.iterdir():
00065             if not energy_dir.is_dir():
00066                 continue
00067
00068             outputcube_dir = energy_dir / "outputcube"
00069             if not outputcube_dir.exists():
00070                 logger.warning(f"No outputcube directory in {energy_dir}")
00071                 continue
00072
00073             logger.info(f"\nAnalyzing energy directory: {energy_dir.name}")
00074
00075             dose_files = list(outputcube_dir.glob("*.npy")) + list(outputcube_dir.glob("*.nii*"))
00076             if not dose_files:

```

```

00077         logger.warning(f"No dose files found in {outputcube_dir}")
00078         continue
00079
00080     energy_doses = []
00081     for dose_file in dose_files[:10]: # Sample first 10 files for speed
00082         try:
00083             if dose_file.suffix.lower() in ['.nii', '.gz']:
00084                 if has_nibabel:
00085                     dose_data = nib.load(str(dose_file)).get_fdata()
00086                 else:
00087                     logger.warning(f"Cannot load {dose_file} - nibabel not available")
00088                     continue
00089             else:
00090                 dose_data = np.load(str(dose_file))
00091
00092             # Handle different array shapes
00093             if dose_data.ndim > 3:
00094                 dose_data = dose_data.squeeze()
00095
00096             energy_doses.append(dose_data)
00097             global_min = min(global_min, dose_data.min())
00098             global_max = max(global_max, dose_data.max())
00099             total_files += 1
00100
00101         except Exception as e:
00102             logger.warning(f"Failed to load {dose_file}: {e}")
00103
00104     if energy_doses:
00105         all_values = np.concatenate([d.flatten() for d in energy_doses])
00106         energy_stats[energy_dir.name] = {
00107             'file_count': len(energy_doses),
00108             'min': float(all_values.min()),
00109             'max': float(all_values.max()),
00110             'mean': float(all_values.mean()),
00111             'std': float(all_values.std()),
00112             'median': float(np.median(all_values)),
00113             'percentile_95': float(np.percentile(all_values, 95)),
00114             'percentile_99': float(np.percentile(all_values, 99)),
00115             'shape': energy_doses[0].shape
00116         }
00117
00118         logger.info(f" Files analyzed: {len(energy_doses)}")
00119         logger.info(f" Value range: {all_values.min():.6f} to {all_values.max():.6f}")
00120         logger.info(f" Mean: {all_values.mean():.6f}, Std: {all_values.std():.6f}")
00121         logger.info(f" 95th percentile: {np.percentile(all_values, 95):.6f}")
00122         logger.info(f" 99th percentile: {np.percentile(all_values, 99):.6f}")
00123
00124     self.results['raw_dose_statistics'] = {
00125         'global_min': float(global_min) if global_min != float('inf') else None,
00126         'global_max': float(global_max) if global_max != float('-inf') else None,
00127         'total_files_analyzed': total_files,
00128         'energy_statistics': energy_stats
00129     }
00130
00131     # Medical plausibility check
00132     if global_max != float('-inf'):
00133         logger.info(f"\n MEDICAL PLAUSIBILITY CHECK:")
00134         logger.info(f" Global maximum dose: {global_max:.6f}")
00135         if global_max < 10:
00136             logger.warning(" Maximum dose < 10 - suspiciously low for radiotherapy!")
00137         elif global_max > 100:
00138             logger.warning(" Maximum dose > 100 - suspiciously high!")
00139         else:
00140             logger.info(" Dose range appears medically plausible")
00141
00142     return True
00143
00144 def analyze_training_normalization(self):
00145     """Phase 1.2: Analyze training normalization logic."""
00146     logger.info("=" * 60)
00147     logger.info("PHASE 1.2: TRAINING NORMALIZATION ANALYSIS")
00148     logger.info("=" * 60)
00149
00150     # Try to import training pipeline to understand normalization
00151     try:
00152         # Check if training pipeline files exist
00153         training_files = ['training_pipeline.py', 'sourcecode/training_pipeline.py', 'system_manager.py']
00154         found_file = None
00155
00156         for tf in training_files:
00157             if Path(tf).exists():
00158                 found_file = tf
00159                 break
00160
00161         if found_file:
00162             logger.info(f"Found training configuration in: {found_file}")
00163

```

```

00164         # Read the file to analyze normalization logic
00165         with open(found_file, 'r') as f:
00166             content = f.read()
00167
00168         # Look for dose normalization patterns
00169         norm_patterns = ['clip_max', 'clip_min', 'dose_normalization', 'ScaleIntensityRanged',
'NormalizeIntensity']
00170         found_patterns = {}
00171
00172         for pattern in norm_patterns:
00173             occurrences = content.count(pattern)
00174             if occurrences > 0:
00175                 found_patterns[pattern] = occurrences
00176                 logger.info(f" Found '{pattern}': {occurrences} occurrences")
00177
00178         # Look for specific normalization values
00179         import re
00180         clip_max_matches = re.findall(r'clip_max["\s]*[=:]["\s]*([0-9.]+)', content)
00181         clip_min_matches = re.findall(r'clip_min["\s]*[=:]["\s]*([0-9.]+)', content)
00182
00183         if clip_max_matches:
00184             logger.info(f" Found clip_max values: {clip_max_matches}")
00185         if clip_min_matches:
00186             logger.info(f" Found clip_min values: {clip_min_matches}")
00187
00188         self.results['training_normalization_params'] = {
00189             'source_file': found_file,
00190             'normalization_patterns': found_patterns,
00191             'clip_max_values': clip_max_matches,
00192             'clip_min_values': clip_min_matches
00193         }
00194     else:
00195         logger.warning("No training pipeline files found")
00196         self.results['training_normalization_params'] = None
00197
00198     except Exception as e:
00199         logger.warning(f"Error analyzing training normalization: {e}")
00200         self.results['training_normalization_params'] = None
00201
00202     def analyze_checkpoint_parameters(self):
00203         """Phase 1.3: Analyze checkpoint normalization parameters."""
00204         logger.info("=" * 60)
00205         logger.info("PHASE 1.3: CHECKPOINT PARAMETER ANALYSIS")
00206         logger.info("=" * 60)
00207
00208         if not self.checkpoint_path or not Path(self.checkpoint_path).exists():
00209             logger.warning("No checkpoint path provided or file not found")
00210             return False
00211
00212         try:
00213             checkpoint = torch.load(self.checkpoint_path, map_location='cpu', weights_only=False)
00214
00215             # Extract dose normalization parameters
00216             dose_params = checkpoint.get("dose_normalization_params", {})
00217             models_by_energy = checkpoint.get("models_by_energy", {})
00218
00219             logger.info(f"Checkpoint contains {len(models_by_energy)} energy-specific models")
00220             logger.info(f"Available model keys: {list(models_by_energy.keys())}")
00221
00222             if dose_params:
00223                 logger.info(f"\nDose normalization parameters in checkpoint:")
00224                 for key, params in dose_params.items():
00225                     clip_min = params.get('clip_min', 'N/A')
00226                     clip_max = params.get('clip_max', 'N/A')
00227                     logger.info(f" {key}:")
00228                     logger.info(f"     clip_min: {clip_min}")
00229                     logger.info(f"     clip_max: {clip_max}")
00230
00231                 # Medical plausibility check for clip_max
00232                 if isinstance(clip_max, (int, float)) and clip_max < 10:
00233                     logger.warning(f"     clip_max={clip_max} is suspiciously low for radiotherapy!")
00234             else:
00235                 logger.warning("No dose_normalization_params found in checkpoint!")
00236
00237             # Check individual model parameters
00238             model_params = {}
00239             for energy_key, model_dict in models_by_energy.items():
00240                 if isinstance(model_dict, dict):
00241                     model_params[energy_key] = {
00242                         'clip_min': model_dict.get('clip_min'),
00243                         'clip_max': model_dict.get('clip_max'),
00244                         'scale_factor': model_dict.get('scale_factor')
00245                     }
00246
00247             logger.info(f"\nIndividual model parameters:")
00248             for energy_key, params in model_params.items():
00249                 logger.info(f" {energy_key}:")

```

```

00250         for param, value in params.items():
00251             logger.info(f"    {param}: {value}")
00252
00253     self.results['checkpoint_normalization_params'] = {
00254         'dose_normalization_params': dose_params,
00255         'model_parameters': model_params,
00256         'available_energies': list(models_by_energy.keys())
00257     }
00258
00259     return True
00260
00261 except Exception as e:
00262     logger.error(f"Failed to analyze checkpoint: {e}")
00263     return False
00264
00265 def analyze_inference_flow(self):
00266     """Phase 1.4: Analyze inference denormalization flow."""
00267     logger.info("=" * 60)
00268     logger.info("PHASE 1.4: INFERENCE FLOW ANALYSIS")
00269     logger.info("=" * 60)
00270
00271     try:
00272         from inference_module import InferenceModule
00273
00274         # Analyze InferenceModule's denormalization logic
00275         logger.info("Analyzing InferenceModule denormalization logic...")
00276
00277         # Check source code for denormalization methods
00278         import inspect
00279
00280         # Get source of InferenceModule
00281         source = inspect.getsource(InferenceModule)
00282
00283         # Look for denormalization-related methods
00284         if 'denormalize_dose' in source:
00285             logger.info(" Found denormalize_dose method in InferenceModule")
00286         else:
00287             logger.warning(" No denormalize_dose method found in InferenceModule")
00288
00289         if 'clip_max' in source:
00290             logger.info(" InferenceModule references clip_max parameters")
00291         else:
00292             logger.warning(" InferenceModule doesn't reference clip_max parameters")
00293
00294         # Count occurrences of normalization-related terms
00295         norm_terms = ['normalize', 'denormalize', 'clip_min', 'clip_max', 'scale_factor']
00296         for term in norm_terms:
00297             count = source.count(term)
00298             logger.info(f"    '{term}' appears {count} times in InferenceModule")
00299
00300         self.results['inference_parameters'] = {
00301             'has_denormalize_dose': 'denormalize_dose' in source,
00302             'references_clip_max': 'clip_max' in source,
00303             'normalization_term_counts': {term: source.count(term) for term in norm_terms}
00304         }
00305
00306     except Exception as e:
00307         logger.error(f"Failed to analyze inference module: {e}")
00308         self.results['inference_parameters'] = None
00309
00310 def perform_data_flow_analysis(self):
00311     """Phase 1.5: Comprehensive data flow analysis."""
00312     logger.info("=" * 60)
00313     logger.info("PHASE 1.5: DATA FLOW ANALYSIS")
00314     logger.info("=" * 60)
00315
00316     # Compare raw data statistics with checkpoint parameters
00317     raw_stats = self.results.get('raw_dose_statistics', {})
00318     checkpoint_params = self.results.get('checkpoint_normalization_params', {})
00319
00320     if raw_stats.get('global_max') and checkpoint_params:
00321         raw_max = raw_stats['global_max']
00322
00323         # Compare with checkpoint clip_max values
00324         dose_params = checkpoint_params.get('dose_normalization_params', {})
00325         if dose_params:
00326             logger.info("Comparing raw data with checkpoint normalization:")
00327             for key, params in dose_params.items():
00328                 clip_max = params.get('clip_max')
00329                 if isinstance(clip_max, (int, float)):
00330                     ratio = raw_max / clip_max
00331                     logger.info(f"    {key}: raw_max={raw_max:.6f}, clip_max={clip_max:.6f}, ratio={ratio:.2f}x")
00332
00333                 if ratio > 15:
00334                     logger.warning(f"    Raw data is {ratio:.1f}x larger than clip_max - major
00335 discrepancy!")
00336
00337         self.results['recommendations'].append(

```



```

00336             f"CRITICAL: {key} clip_max={clip_max:.6f} is {ratio:.1f}x too small. "
00337             f"Should be close to {raw_max:.6f}"
00338         )
00339
00340     # Analyze training vs inference energy coverage
00341     raw_energies = set(self.results['raw_dose_statistics'].get('energy_statistics', {}).keys())
00342     checkpoint_energies = set()
00343
00344     if checkpoint_params:
00345         available_energies = checkpoint_params.get('available_energies', [])
00346         for energy_key in available_energies:
00347             # Extract energy value from key like "res64x64x64_e11.5"
00348             if '_e' in energy_key:
00349                 energy_val = energy_key.split('_e')[1]
00350                 checkpoint_energies.add(f"energy_{energy_val}")
00351
00352     logger.info(f"Raw data energies: {raw_energies}")
00353     logger.info(f"Checkpoint energies: {checkpoint_energies}")
00354
00355     missing_in_checkpoint = raw_energies - checkpoint_energies
00356     if missing_in_checkpoint:
00357         logger.warning(f"Energies in raw data but missing in checkpoint: {missing_in_checkpoint}")
00358
00359     self.results['data_flow_analysis'] = {
00360         'raw_energies': list(raw_energies),
00361         'checkpoint_energies': list(checkpoint_energies),
00362         'missing_energies': list(missing_in_checkpoint)
00363     }
00364
00365     def generate_recommendations(self):
00366         """Phase 1.6: Generate specific recommendations."""
00367         logger.info("=" * 60)
00368         logger.info("PHASE 1.6: RECOMMENDATIONS")
00369         logger.info("=" * 60)
00370
00371         # Analyze all findings and generate actionable recommendations
00372         recommendations = []
00373
00374         raw_stats = self.results.get('raw_dose_statistics', {})
00375         checkpoint_params = self.results.get('checkpoint_normalization_params', {})
00376
00377         # Check for major dose scaling issues
00378         if raw_stats.get('global_max'):
00379             raw_max = raw_stats['global_max']
00380
00381             if checkpoint_params:
00382                 dose_params = checkpoint_params.get('dose_normalization_params', {})
00383                 if dose_params:
00384                     max_clip_max = max([p.get('clip_max', 0) for p in dose_params.values() if
00385 isinstance(p.get('clip_max'), (int, float))])
00386                     if max_clip_max > 0:
00387                         ratio = raw_max / max_clip_max
00388                         if ratio > 10:
00389                             recommendations.append({
00390                                 'priority': 'CRITICAL',
00391                                 'issue': 'Dose normalization parameters severely wrong',
00392                                 'description': f'Raw dose maximum ({raw_max:.6f}) is {ratio:.1f}x larger than
00393 largest clip_max ({max_clip_max:.6f})',
00394                                 'solution': 'Recalculate dose normalization parameters from raw training data',
00395                                 'implementation': 'Run fix_dose_normalization_parameters() function'
00396                             })
00397
00398         # Check for missing normalization parameters
00399         if not checkpoint_params or not checkpoint_params.get('dose_normalization_params'):
00400             recommendations.append({
00401                 'priority': 'HIGH',
00402                 'issue': 'Missing dose normalization parameters in checkpoint',
00403                 'description': 'Checkpoint does not contain dose_normalization_params',
00404                 'solution': 'Regenerate checkpoint with proper dose normalization parameters',
00405                 'implementation': 'Retrain model with corrected dose normalization pipeline'
00406             })
00407
00408         # Check for inference module issues
00409         inference_params = self.results.get('inference_parameters')
00410         if inference_params and not inference_params.get('has_denormalize_dose'):
00411             recommendations.append({
00412                 'priority': 'MEDIUM',
00413                 'issue': 'Missing denormalize_dose method in InferenceModule',
00414                 'description': 'InferenceModule may not properly denormalize dose outputs',
00415                 'solution': 'Add proper dose denormalization logic to InferenceModule',
00416                 'implementation': 'Implement denormalize_dose() method with energy-specific parameters'
00417             })
00418
00419         self.results['recommendations'].extend(recommendations)
00420
00421         # Print all recommendations
00422         for i, rec in enumerate(recommendations, 1):

```

```

00421         logger.info(f"\n RECOMMENDATION {i} [{rec['priority']}]:")
00422         logger.info(f"     Issue: {rec['issue']}")
00423         logger.info(f"     Description: {rec['description']}")
00424         logger.info(f"     Solution: {rec['solution']}")
00425         logger.info(f"     Implementation: {rec['implementation']}")
00426
00427     def save_results(self, output_path="dose_diagnosis_results.json"):
00428         """Save all analysis results to JSON file."""
00429         logger.info(f"\nSaving diagnosis results to {output_path}...")
00430
00431         with open(output_path, 'w') as f:
00432             json.dump(self.results, f, indent=2, ensure_ascii=False)
00433
00434         logger.info(" Results saved")
00435
00436     def create_visualization(self, output_path="dose_diagnosis_visualization.png"):
00437         """Create visualization of key findings."""
00438         logger.info(f"\nCreating diagnosis visualization...")
00439
00440         try:
00441             fig, axes = plt.subplots(2, 2, figsize=(15, 10))
00442             fig.suptitle('Dose Normalization Diagnosis Results', fontsize=16)
00443
00444             # Plot 1: Raw dose statistics by energy
00445             raw_stats = self.results.get('raw_dose_statistics', {})
00446             energy_stats = raw_stats.get('energy_statistics', {})
00447
00448             if energy_stats:
00449                 energies = list(energy_stats.keys())
00450                 max_values = [stats['max'] for stats in energy_stats.values()]
00451                 mean_values = [stats['mean'] for stats in energy_stats.values()]
00452
00453                 axes[0, 0].bar(range(len(energies)), max_values, alpha=0.7, label='Maximum')
00454                 axes[0, 0].bar(range(len(energies)), mean_values, alpha=0.7, label='Mean')
00455                 axes[0, 0].set_xlabel('Energy')
00456                 axes[0, 0].set_ylabel('Dose Value')
00457                 axes[0, 0].set_title('Raw Dose Statistics by Energy')
00458                 axes[0, 0].set_xticks(range(len(energies)))
00459                 axes[0, 0].set_xticklabels(energies, rotation=45)
00460                 axes[0, 0].legend()
00461
00462             # Plot 2: Checkpoint clip_max vs raw data
00463             checkpoint_params = self.results.get('checkpoint_normalization_params', {})
00464             dose_params = checkpoint_params.get('dose_normalization_params', {})
00465
00466             if dose_params and energy_stats:
00467                 clip_maxes = []
00468                 raw_maxes = []
00469                 labels = []
00470
00471                 for key, params in dose_params.items():
00472                     clip_max = params.get('clip_max')
00473                     if isinstance(clip_max, (int, float)):
00474                         clip_maxes.append(clip_max)
00475                         # Try to match with raw data
00476                         raw_max = raw_stats.get('global_max', 0)
00477                         raw_maxes.append(raw_max)
00478                         labels.append(key.split('_')[-1] if '_' in key else key)
00479
00480                 if clip_maxes:
00481                     x = np.arange(len(labels))
00482                     width = 0.35
00483
00484                     axes[0, 1].bar(x - width/2, clip_maxes, width, label='Checkpoint clip_max', alpha=0.7)
00485                     axes[0, 1].bar(x + width/2, raw_maxes, width, label='Raw data max', alpha=0.7)
00486                     axes[0, 1].set_xlabel('Energy/Resolution')
00487                     axes[0, 1].set_ylabel('Dose Value')
00488                     axes[0, 1].set_title('Checkpoint vs Raw Data Comparison')
00489                     axes[0, 1].set_xticks(x)
00490                     axes[0, 1].set_xticklabels(labels, rotation=45)
00491                     axes[0, 1].legend()
00492                     axes[0, 1].set_yscale('log') # Log scale to show differences
00493
00494             # Plot 3: Recommendations priority
00495             recommendations = self.results.get('recommendations', [])
00496             if recommendations:
00497                 priorities = [rec['priority'] for rec in recommendations]
00498                 priority_counts = {p: priorities.count(p) for p in set(priorities)}
00499
00500                 axes[1, 0].pie(priority_counts.values(), labels=priority_counts.keys(), autopct='%1.1f%%')
00501                 axes[1, 0].set_title('Recommendations by Priority')
00502
00503             # Plot 4: Summary statistics
00504             summary_text = f"""\nDiagnosis Summary:
00505
00506 Raw Data:
00507 • Total files analyzed: {raw_stats.get('total_files_analyzed', 'N/A')}

```

```

00508 • Global max dose: {raw_stats.get('global_max', 'N/A'):.6f}
00509 • Energy directories: {len(energy_stats)}
00510
00511 Checkpoint:
00512 • Available models: {len(checkpoint_params.get('available_energies', []))}
00513 • Has dose params: {'Yes' if dose_params else 'No'}
00514
00515 Critical Issues Found: {len([r for r in recommendations if r.get('priority') == 'CRITICAL'])}
00516 Total Recommendations: {len(recommendations)}"""
00517
00518         axes[1, 1].text(0.1, 0.5, summary_text, fontsize=10, verticalalignment='center',
00519                        bbox=dict(boxstyle='round', facecolor='lightblue', alpha=0.8))
00520         axes[1, 1].set_xlim(0, 1)
00521         axes[1, 1].set_ylim(0, 1)
00522         axes[1, 1].axis('off')
00523         axes[1, 1].set_title('Diagnosis Summary')
00524
00525         plt.tight_layout()
00526         plt.savefig(output_path, dpi=150, bbox_inches='tight')
00527         plt.close()
00528
00529         logger.info(f" Visualization saved: {output_path}")
00530
00531     except Exception as e:
00532         logger.warning(f"Failed to create visualization: {e}")
00533
00534 def run_full_diagnosis(self, checkpoint_path=None):
00535     """Run complete diagnosis pipeline."""
00536     logger.info(" STARTING SYSTEMATIC DOSE NORMALIZATION DIAGNOSIS")
00537     logger.info("=" * 80)
00538
00539     if checkpoint_path:
00540         self.checkpoint_path = checkpoint_path
00541
00542     # Execute all analysis phases
00543     success = True
00544     success &= self.analyze_raw_dose_data()
00545     self.analyze_training_normalization()
00546     if self.checkpoint_path:
00547         success &= self.analyze_checkpoint_parameters()
00548     self.analyze_inference_flow()
00549     self.perform_data_flow_analysis()
00550     self.generate_recommendations()
00551
00552     # Save results and create visualization
00553     self.save_results()
00554     self.create_visualization()
00555
00556     logger.info("=" * 80)
00557     logger.info(" DIAGNOSIS COMPLETE")
00558     logger.info("=" * 80)
00559
00560     return success
00561
00562
00563 def main():
00564     """Main function for running systematic diagnosis."""
00565     import argparse
00566
00567     parser = argparse.ArgumentParser(description="Systematic Dose Normalization Diagnosis")
00568     parser.add_argument('--traindata-dir', default='traindata', help='Training data directory')
00569     parser.add_argument('--checkpoint', help='Path to model checkpoint for analysis')
00570     parser.add_argument('--output-dir', default='.', help='Output directory for results')
00571
00572     args = parser.parse_args()
00573
00574     # Create diagnostic instance
00575     diagnostic = DoseNormalizationDiagnostic(
00576         traindata_dir=args.traindata_dir,
00577         checkpoint_path=args.checkpoint
00578     )
00579
00580     # Run full diagnosis
00581     success = diagnostic.run_full_diagnosis()
00582
00583     if success:
00584         logger.info(" Diagnosis completed successfully")
00585         logger.info(" Check dose_diagnosis_results.json for detailed findings")
00586         logger.info(" Check dose_diagnosis_visualization.png for visual summary")
00587         return 0
00588     else:
00589         logger.error(" Diagnosis failed or incomplete")
00590         return 1
00591
00592
00593 if __name__ == '__main__':
00594     exit(main())

```

8.46 tests/test_critical_fixes.py File Reference

Namespaces

- namespace `test_critical_fixes`

Functions

- `test_critical_fixes.test_dose_focused_preprocessing()`
- `test_critical_fixes.test_extreme_loss_functions()`
- `test_critical_fixes.test_sample_filtering()`
- `test_critical_fixes.test_realistic_training_scenario()`
- `test_critical_fixes.main()`

Variables

- `test_critical_fixes.logger = logging.getLogger(__name__)`

8.47 test_critical_fixes.py

[Go to the documentation of this file.](#)

```
00001 #!/usr/bin/env python3
00002 """
00003 Quick test script to validate the critical fixes for ultra-sparse dose training.
00004 Tests dose-focused preprocessing, extreme weighted loss, and sample filtering.
00005 """
00006
00007 import sys
00008 import os
00009 sys.path.append('/home/mpalm/RadioTherapy/sourcecode')
00010
00011 import torch
00012 import numpy as np
00013 import logging
00014 import log_config
00015
00016 # Import our modules
00017 from training_pipeline import AutoencoderTrainer
00018 from data_management import DataLoaderModule, DoseNpyDataset
00019
00020 logger = logging.getLogger(__name__)
00021
00022 def test_dose_focused_preprocessing():
00023     """Test the dose-focused preprocessing functions"""
00024     print("=== Testing Dose-Focused Preprocessing ===")
00025
00026     # Import the preprocessing functions
00027     sys.path.append('/home/mpalm/RadioTherapy')
00028     from dose_focused_preprocessing import validate_sample_quality, crop_to_dose_region
00029
00030     # Create test data with ultra-sparse dose
00031     test_ct = np.random.randn(100, 100, 100).astype(np.float32)
00032     test_dose = np.zeros((100, 100, 100), dtype=np.float32)
00033
00034     # Add small dose region (simulating ultra-sparse data)
00035     test_dose[45:55, 45:55, 45:55] = np.random.uniform(0.1, 2.0, (10, 10, 10))
00036
00037     print(f"Test dose stats: max={test_dose.max():.6f}, "
00038           f"dose_voxels={np.sum(test_dose > 1e-6)}, "
00039           f"total_voxels={test_dose.size}")
00040
00041     # Test validation
00042     validation = validate_sample_quality(test_dose)
00043     print(f"Sample validation: {validation}")
00044
00045     # Test cropping
00046     ct_cropped, dose_cropped, crop_info = crop_to_dose_region(test_ct, test_dose)
00047
00048     if ct_cropped is not None:
00049         print(f"Cropping successful: {test_dose.shape} → {dose_cropped.shape}")
00050         print(f"Crop info: {crop_info}")
00051     else:
00052         print("Cropping failed")
00053
00054     print(" Dose-focused preprocessing test completed\n")
00055
00056 def test_extreme_loss_functions():
```

```

00057     """Test the extreme loss functions on sparse data"""
00058     print("=== Testing Extreme Loss Functions ===")
00059
00060     # Create AutoencoderTrainer instance
00061     trainer = AutoencoderTrainer(
00062         autoencoder=None, discriminator=None, unet=None,
00063         optimizer_g=None, optimizer_d=None, optimizer_diff=None,
00064         device='cpu'
00065     )
00066
00067     # Create ultra-sparse test data
00068     batch_size = 2
00069     spatial_size = 64
00070
00071     # Prediction (random values)
00072     pred = torch.randn(batch_size, 1, spatial_size, spatial_size, spatial_size) * 0.1
00073
00074     # Target with ultra-sparse dose (only 16 voxels have dose)
00075     target = torch.zeros(batch_size, 1, spatial_size, spatial_size, spatial_size)
00076
00077     # Add tiny dose regions
00078     for b in range(batch_size):
00079         # 8 voxels per batch = 16 total (ultra-sparse like in logs)
00080         coords = torch.randint(0, spatial_size, (8, 3))
00081         for i in range(8):
00082             x, y, z = coords[i]
00083             target[b, 0, x, y, z] = torch.rand(1) * 0.2 # Max dose -0.2 (like logs)
00084
00085     dose_voxels = (target > 1e-6).sum().item()
00086     total_voxels = target.numel()
00087     print(f"Test data: {dose_voxels}/{total_voxels} dose voxels ({100*dose_voxels/total_voxels:.4f}%)")
00088
00089     # Test different loss functions
00090     try:
00091         extreme_loss = trainer.extreme_dose_focused_loss(pred, target)
00092         print(f" Extreme dose-focused loss: {extreme_loss:.6f}")
00093     except Exception as e:
00094         print(f" Extreme loss failed: {e}")
00095
00096     try:
00097         weighted_loss = trainer.weighted_dose_loss(pred, target)
00098         print(f" Extreme weighted loss: {weighted_loss:.6f}")
00099     except Exception as e:
00100         print(f" Weighted loss failed: {e}")
00101
00102     try:
00103         masked_loss = trainer.masked_dose_loss(pred, target)
00104         print(f" Pure dose loss: {masked_loss:.6f}")
00105     except Exception as e:
00106         print(f" Masked loss failed: {e}")
00107
00108     print(" Extreme loss functions test completed\n")
00109
00110 def test_sample_filtering():
00111     """Test the sample filtering in DataLoaderModule"""
00112     print("=== Testing Sample Filtering ===")
00113
00114     # Create test dataset instance
00115     test_dataset = DoseNpyDataset(root_dir="/home/mpalm/RadioTherapy/traindata")
00116
00117     # Check if validation method exists
00118     if hasattr(test_dataset, 'validate_sample_quality'):
00119         print(" Sample validation method found")
00120
00121         # Test with dummy dose files
00122         test_dose_good = np.zeros((100, 100, 100))
00123         test_dose_good[40:60, 40:60, 40:60] = np.random.uniform(0.1, 1.0, (20, 20, 20))
00124
00125         test_dose_bad = np.zeros((100, 100, 100))
00126         test_dose_bad[50:55, 50:55, 50:55] = np.random.uniform(0.001, 0.01, (5, 5, 5))
00127
00128         good_result = test_dataset.validate_sample_quality('/tmp/dummy_good.npy')
00129         bad_result = test_dataset.validate_sample_quality('/tmp/dummy_bad.npy')
00130
00131         print(f"Sample filtering working: good={good_result}, bad={bad_result}")
00132     else:
00133         print(" Sample validation method not found")
00134
00135     print(" Sample filtering test completed\n")
00136
00137 def test_realistic_training_scenario():
00138     """Simulate realistic training with ultra-sparse data"""
00139     print("=== Testing Realistic Training Scenario ===")
00140
00141     # Simulate batch with statistics from logs
00142     batch_size = 2
00143     spatial_size = 50 # Smaller for faster testing

```

```

00144
00145 # Create batch similar to problematic log entries
00146 ct_batch = torch.randn(batch_size, 1, spatial_size, spatial_size)
00147 dose_batch = torch.zeros(batch_size, 1, spatial_size, spatial_size, spatial_size)
00148
00149 # Add ultra-sparse dose (16 voxels total like in logs)
00150 dose_coords = [
00151     (25, 25, 25), (26, 25, 25), (25, 26, 25), (25, 25, 26),
00152     (27, 25, 25), (25, 27, 25), (25, 25, 27), (26, 26, 25),
00153     (30, 30, 30), (31, 30, 30), (30, 31, 30), (30, 30, 31),
00154     (32, 30, 30), (30, 32, 30), (30, 30, 32), (31, 31, 30)
00155 ]
00156
00157 for i, (x, y, z) in enumerate(dose_coords):
00158     batch_idx = i % batch_size
00159     dose_batch[batch_idx, 0, x, y, z] = np.random.uniform(0.1, 0.25)
00160
00161 dose_voxels = (dose_batch > 1e-6).sum().item()
00162 total_voxels = dose_batch.numel()
00163 max_dose = dose_batch.max().item()
00164 mean_dose = dose_batch[dose_batch > 1e-6].mean().item()
00165
00166 print(f"Realistic scenario stats:")
00167 print(f" Dose voxels: {dose_voxels}/{total_voxels} ({100*dose_voxels/total_voxels:.4f}%)")
00168 print(f" Max dose: {max_dose:.6f}")
00169 print(f" Mean dose: {mean_dose:.6f}")
00170 print(f" (Similar to logs: 16/2M voxels, max=0.25)")
00171
00172 # Test if extreme loss can handle this
00173 trainer = AutoencoderTrainer(
00174     autoencoder=None, discriminator=None, unet=None,
00175     optimizer_g=None, optimizer_d=None, optimizer_diff=None,
00176     device='cpu'
00177 )
00178
00179 # Random prediction
00180 pred_batch = torch.randn_like(dose_batch) * 0.1
00181
00182 try:
00183     loss = trainer.extreme_dose_focused_loss(pred_batch, dose_batch)
00184     print(f" Extreme loss on realistic data: {loss:.6f}")
00185
00186     if loss.item() > 1e-6: # Should be meaningful, not zero
00187         print(" Loss is meaningful (not collapsed to zero)")
00188     else:
00189         print(" Loss is very small - might indicate problems")
00190
00191 except Exception as e:
00192     print(f" Extreme loss failed on realistic data: {e}")
00193
00194 print(" Realistic training scenario test completed\n")
00195
00196 def main():
00197     """Run all critical tests"""
00198     print("=== CRITICAL FIXES VALIDATION ===")
00199     print("Testing fixes for ultra-sparse dose training problems\n")
00200
00201     try:
00202         test_dose_focused_preprocessing()
00203         test_extreme_loss_functions()
00204         test_sample_filtering()
00205         test_realistic_training_scenario()
00206
00207         print("=== ALL TESTS COMPLETED ===")
00208         print(" Critical fixes appear to be working")
00209         print("Ready for training test with real data")
00210
00211     except Exception as e:
00212         print(f" Test failed: {e}")
00213         import traceback
00214         traceback.print_exc()
00215
00216 if __name__ == "__main__":
00217     main()

```

8.48 tests/test_error_logging.py File Reference

Namespaces

- namespace [test_error_logging](#)

Functions

- `test_error_logging.test_error_logging ()`

Variables

- `test_error_logging.console_handler = logging.StreamHandler()`
- `test_error_logging.formatter = logging.Formatter('%(asctime)s %(levelname)s: %(name)s: %(message)s')`
- `test_error_logging.root_logger = logging.getLogger()`
- `test_error_logging.logger = logging.getLogger(__name__)`

8.49 test_error_logging.py

[Go to the documentation of this file.](#)

```
00001 #!/usr/bin/env python3
00002 """
00003 Test error logging in the corrected inference pipeline
00004 """
00005
00006 import sys
00007 import os
00008 sys.path.insert(0, os.path.abspath(os.path.join(os.path.dirname(__file__), 'sourcecode')))
00009
00010 # Initialize logging first
00011 import log_config
00012 import logging
00013
00014 # Set up logging to also output to console
00015 console_handler = logging.StreamHandler()
00016 console_handler.setLevel(logging.INFO)
00017 formatter = logging.Formatter('%(asctime)s %(levelname)s: %(name)s: %(message)s')
00018 console_handler.setFormatter(formatter)
00019
00020 # Get root logger and add console handler
00021 root_logger = logging.getLogger()
00022 root_logger.addHandler(console_handler)
00023 root_logger.setLevel(logging.INFO)
00024
00025 logger = logging.getLogger(__name__)
00026
00027 import torch
00028 import numpy as np
00029 from system_manager import SystemManager
00030
00031 def test_error_logging():
00032     """Test error logging by providing an invalid model checkpoint"""
00033     logger.info("=" * 60)
00034     logger.info("TESTING ERROR LOGGING")
00035     logger.info("=" * 60)
00036
00037     # Create a simple test CT file
00038     logger.info("Creating test CT data...")
00039     test_ct = np.random.rand(100, 100, 100).astype(np.float32)
00040     test_ct_path = "test_ct.npy"
00041     np.save(test_ct_path, test_ct)
00042     logger.info(f" Test CT saved to: {test_ct_path}")
00043
00044     # Create SystemManager with minimal setup
00045     logger.info("Creating SystemManager...")
00046     system_manager = SystemManager(
00047         root_dir=".",
00048         transforms=None,
00049         resolutions=[(64, 64, 64)],
00050         energies=[11.5],
00051         quad_energies=[11.5],
00052         quad_weights=[1.0],
00053         batch_size=1,
00054         device=torch.device('cpu'),
00055         num_epochs=1,
00056         learning_rate=0.001,
00057         patience=10,
00058         cube_size=64
00059     )
00060     logger.info(" SystemManager created successfully")
00061
00062     # Test inference with invalid checkpoint (should trigger error logging)
00063     logger.info("Starting inference test with invalid checkpoint...")
00064     try:
```

```

00065         output_path = system_manager.run_inference(
00066             test_ct_path,
00067             model_checkpoint="nonexistent_model.ckpt" # This will cause an error
00068         )
00069
00070         logger.error("This should not happen - expected an error!")
00071
00072     except Exception as e:
00073         logger.info(f" Expected error caught: {e}")
00074         logger.info(" Error logging test completed successfully")
00075
00076     finally:
00077         # Clean up
00078         logger.info("Cleaning up test files...")
00079         if os.path.exists(test_ct_path):
00080             os.remove(test_ct_path)
00081             logger.info(f" Removed test file: {test_ct_path}")
00082
00083         logger.info("=" * 60)
00084         logger.info("ERROR LOGGING TEST COMPLETED")
00085         logger.info("=" * 60)
00086
00087 if __name__ == "__main__":
00088     test_error_logging()

```

8.50 tests/test_gui.py File Reference

Namespaces

- namespace [test_gui](#)

Functions

- [test_gui.qapp](#) ()
- [test_gui.window](#) (qapp, qtbot, tmp_path, monkeypatch)
- [test_gui.test_initial_button_states](#) (window)
- [test_gui.test_energy_range_validation](#) (window, monkeypatch)
- [test_gui.test_select_input_folder_enables_train](#) (window)

8.51 test_gui.py

[Go to the documentation of this file.](#)

```

00001 import sys
00002 import os
00003 import numpy as np
00004 import pytest
00005 from PySide6.QtWidgets import QApplication
00006 from gui import MainWindow
00007
00008 @pytest.fixture(scope="session")
00009 def qapp():
00010     """Provides a single QApplication instance for all tests."""
00011     app = QApplication.instance() or QApplication(sys.argv)
00012     return app
00013
00014 @pytest.fixture
00015 def window(qapp, qtbot, tmp_path, monkeypatch):
00016     """
00017     Creates a MainWindow, constructs a dummy dataset in tmp_path,
00018     patches QFileDialog.select_input_folder, and returns the window.
00019     """
00020     # Create dummy dataset folder structure
00021     root = tmp_path / "dataset"
00022     e1 = root / "10_0"
00023     inputcube = e1 / "inputcube"
00024     outputcube = e1 / "outputcube"
00025     inputcube.mkdir(parents=True)
00026     outputcube.mkdir(parents=True)
00027     # Save dummy .npy file
00028     arr = np.zeros((4, 4, 4))
00029     np.save(inputcube / "dummy.npy", arr)
00030
00031     # Instantiate MainWindow
00032     win = MainWindow()
00033     qtbot.addWidget(win)
00034

```



```

00035     # Patch QFileDialog.select_input_folder to return our tmp folder
00036     def fake_select():
00037         win.input_dir = str(inputcube)
00038         win.output_dir = str(outputcube)
00039         # energies and transforms are also updated internally
00040         win.update_train_button_state()
00041     monkeypatch.setattr(win, "select_input_folder", fake_select)
00042
00043     return win
00044
00045 def test_initial_button_states(window):
00046     """Dose, Train, and Visualize buttons are initially disabled."""
00047     assert not window.dose_button.isEnabled()
00048     assert not window.train_button.isEnabled()
00049     assert not window.visualize_button.isEnabled()
00050
00051 def test_energy_range_validation(window, monkeypatch):
00052     """calculate_dose raises an exception when min > max."""
00053     # Set conditions so that dose_button can't be enabled
00054     window.ct_file = "dummy.nii" # Dummy-Pfad
00055     window.model_file_bool = True
00056     window.dose_button.setEnabled(True)
00057
00058     # invalid range
00059     window.energy_min_spin.setValue(50.0)
00060     window.energy_max_spin.setValue(10.0)
00061
00062     # Intercept warning dialog instead of exception
00063     warnings = []
00064     from PySide6.QtWidgets import QMessageBox
00065     monkeypatch.setattr(QMessageBox, "warning", lambda *args, **kwargs: warnings.append(True))
00066     window.calculate_dose()
00067     assert warnings, "Expected a warning dialog for invalid energy range"
00068
00069 def test_select_input_folder_enables_train(window):
00070     """select_input_folder enables the train_button when structure is correct."""
00071     # simulate user action
00072     window.select_input_folder()
00073     assert window.train_button.isEnabled()

```

8.52 tests/test_gui_inference.py File Reference

Namespaces

- namespace `test_gui_inference`

Functions

- `test_gui_inference.test_gui_inference()`

8.53 test_gui_inference.py

[Go to the documentation of this file.](#)

```

00001 #!/usr/bin/env python3
00002 """
00003 Test that the GUI would produce corrected inference output
00004 """
00005
00006 import sys
00007 import os
00008 sys.path.insert(0, os.path.abspath(os.path.join(os.path.dirname(__file__), 'sourcecode')))
00009
00010 import torch
00011 import numpy as np
00012 from system_manager import SystemManager
00013
00014 def test_gui_inference():
00015     """Simulate what the GUI would do for inference"""
00016     print("Testing GUI inference path...")
00017
00018     # Use the same real training data
00019     input_path = "traindata/11_5/inputcube/235101017859661465075472232303048949736_0.npy"
00020     ground_truth_path = "traindata/11_5/outputcube/235101017859661465075472232303048949736_0.npy"
00021
00022     if not os.path.exists(input_path):
00023         print(f"Input file not found: {input_path}")
00024         return
00025
00026     if not os.path.exists(ground_truth_path):

```

```

00027         print(f"Ground truth file not found: {ground_truth_path}")
00028         return
00029
00030     # Load ground truth for comparison
00031     ground_truth = np.load(ground_truth_path)
00032     print(f"Ground truth shape: {ground_truth.shape}")
00033     print(f"Ground truth range: {ground_truth.min():.6f} to {ground_truth.max():.6f}")
00034
00035     # Create SystemManager with GUI-like configuration
00036     system_manager = SystemManager(
00037         root_dir=".",
00038         transforms=None,
00039         resolutions=[(64, 64, 64)],
00040         energies=[11.5],
00041         quad_energies=[11.5],
00042         quad_weights=[1.0],
00043         batch_size=1,
00044         device=torch.device('cpu'),
00045         num_epochs=1,
00046         learning_rate=0.001,
00047         patience=10,
00048         cube_size=64
00049     )
00050
00051     # Test inference exactly as the GUI would call it
00052     try:
00053         print("Running inference (as GUI would)...")
00054         output_path = system_manager.run_inference(
00055             input_path,
00056             model_checkpoint="unified_energy_conditioned_model_res16.0_energies3.ckpt"
00057         )
00058
00059         print(f"Inference completed successfully!")
00060         print(f"Output saved to: {output_path}")
00061
00062     # Load and analyze the result
00063     import nibabel as nib
00064     img = nib.load(output_path)
00065     data = img.get_fdata()
00066
00067     print(f"Output shape: {data.shape}")
00068     print(f"Output range: {data.min():.6f} to {data.max():.6f}")
00069
00070     # Calculate correlation with ground truth
00071     if data.shape == ground_truth.shape:
00072         correlation = np.corrcoef(data.flatten(), ground_truth.flatten())[0, 1]
00073         print(f"Correlation with ground truth: {correlation:.6f}")
00074
00075     # Check if the correlation is good (should be ~0.23)
00076     if correlation > 0.2:
00077         print(" SUCCESS: GUI inference produces meaningful correlation!")
00078     else:
00079         print(" FAILURE: GUI inference correlation too low")
00080
00081     else:
00082         print(f" FAILURE: Shape mismatch - output {data.shape} vs ground truth {ground_truth.shape}")
00083
00084     except Exception as e:
00085         print(f" ERROR during inference: {e}")
00086         import traceback
00087         traceback.print_exc()
00088
00089     if __name__ == "__main__":
00090         test_gui_inference()

```

8.54 tests/test_gui_training.py File Reference

Namespaces

- namespace `test_gui_training`

Functions

- `test_gui_training.test_gui_training()`

Variables

- `test_gui_training.logger` = `logging.getLogger(__name__)`
- `test_gui_training.success` = `test_gui_training()`

8.55 test_gui_training.py

[Go to the documentation of this file.](#)

```

00001 #!/usr/bin/env python3
00002 """
00003 Test GUI Training (nicht das vollständige GUI, nur die Training-Logik)
00004 """
00005
00006 import sys
00007 import os
00008 sys.path.insert(0, os.path.abspath(os.path.join(os.path.dirname(__file__), 'sourcecode')))
00009
00010 # Initialize logging first
00011 import log_config
00012 import logging
00013
00014 logger = logging.getLogger(__name__)
00015
00016 import torch
00017 from system_manager import SystemManager
00018 from monai.transforms import Compose
00019
00020 def test_gui_training():
00021     """Test the GUI training logic without actually running the GUI"""
00022
00023     logger.info("=" * 60)
00024     logger.info("TESTING GUI TRAINING LOGIC")
00025     logger.info("=" * 60)
00026
00027     # Create SystemManager similar to GUI
00028     logger.info("Creating SystemManager for GUI training test...")
00029
00030     system_manager = SystemManager(
00031         root_dir="traindata",
00032         transforms=Compose([]),
00033         resolutions=[16.0],
00034         energies=[11.5, 15.75, 34.25], # All energies for corrected training
00035         quad_energies=[11.5, 15.75, 34.25],
00036         quad_weights=[0.33, 0.33, 0.34],
00037         batch_size=1,
00038         device=torch.device('cpu'),
00039         num_epochs=2, # Short test
00040         learning_rate=0.0001,
00041         patience=3,
00042         cube_size=32
00043     )
00044
00045     logger.info(" SystemManager created")
00046     logger.info(f"Training with energies: {system_manager.energies}")
00047
00048     # Test corrected training (like GUI would do)
00049     logger.info("Testing energy-conditioned training...")
00050     try:
00051         result = system_manager.run_energy_conditioned_training()
00052         logger.info(" Energy-conditioned training completed successfully!")
00053         logger.info(f"Model saved to: {result['model_path']}")
00054     except Exception as e:
00055         logger.error(f"Energy-conditioned training failed: {e}")
00056         logger.error("Training traceback:", exc_info=True)
00057         return False
00058
00059     logger.info("=" * 60)
00060     logger.info("GUI TRAINING TEST COMPLETED SUCCESSFULLY")
00061     logger.info("=" * 60)
00062     return True
00063
00064
00065 if __name__ == "__main__":
00066     success = test_gui_training()
00067     if success:
00068         print(" GUI training test passed!")
00069     else:
00070         print(" GUI training test failed!")

```

8.56 tests/test_pipeline_fixes.py File Reference

Namespaces

- namespace `test_pipeline_fixes`

Functions

- `test_pipeline_fixes.test_transforms_consistency()`
- `test_pipeline_fixes.test_dose_range_detection()`
- `test_pipeline_fixes.test_learning_rate_adjustment()`
- `test_pipeline_fixes.run_comprehensive_test()`

Variables

- `test_pipeline_fixes.level`
- `test_pipeline_fixes.INFO`
- `test_pipeline_fixes.format`
- `test_pipeline_fixes.logger = logging.getLogger(__name__)`
- `test_pipeline_fixes.success = run_comprehensive_test()`

8.57 test_pipeline_fixes.py

[Go to the documentation of this file.](#)

```

00001 #!/usr/bin/env python3
00002 """
00003 Test script to validate pipeline fixes for RadioTherapy dose prediction.
00004 This script tests the key fixes applied to resolve the 30-50x too low dose values.
00005 """
00006
00007 import logging
00008 import torch
00009 import numpy as np
00010 from pathlib import Path
00011
00012 # Setup logging
00013 logging.basicConfig(level=logging.INFO, format='%(asctime)s %(levelname)s: %(name)s: %(message)s')
00014 logger = logging.getLogger(__name__)
00015
00016 def test_transforms_consistency():
00017     """Test that transforms produce consistent 64x64x64 outputs."""
00018     logger.info(" Testing transform consistency...")
00019
00020     try:
00021         from sourcecode.system_manager import SystemManager
00022
00023         # Initialize minimal SystemManager to test transforms
00024         system = SystemManager(
00025             root_dir="/home/mpalm/RadioTherapy/traindata",
00026             transforms=None,
00027             resolutions=[(64, 64, 64)],
00028             energies=[11.5],
00029             energy_min=3.47,
00030             energy_max=46.53,
00031             quad_energies=[11.5],
00032             quad_weights=[1.0],
00033             batch_size=2,
00034             device="cpu",
00035             num_epochs=1,
00036             learning_rate=1e-5,
00037             patience=5,
00038             cube_size=100,
00039             seed=42
00040         )
00041
00042         # Initialize transforms for resolution 64x64x64
00043         system._prepare_training(target_resolution=(64, 64, 64), energy=11.5)
00044
00045         logger.info(" Transform initialization successful")
00046         logger.info(f" Target resolution: (64, 64, 64)")
00047         logger.info(f" Cube size: {system.cube_size}")
00048
00049         # Check if the problematic SpatialPadd is removed
00050         transform_str = str(system.transforms)
00051         has_spatial_padd = "SpatialPadd" in transform_str
00052         has_resized = "Resized" in transform_str
00053
00054         logger.info(f" Contains Resized: {has_resized}")
00055         logger.info(f" Contains SpatialPadd: {has_spatial_padd}")
00056
00057         if has_spatial_padd:
00058             logger.error(" SpatialPadd still present! This will cause resolution mismatch.")
00059             return False
00060         elif has_resized:

```

```

00061         logger.info(" Pipeline uses Resized without SpatialPadd - correct!")
00062         return True
00063     else:
00064         logger.warning(" No Resized found - check pipeline configuration")
00065         return False
00066
00067 except Exception as e:
00068     logger.error(f" Transform test failed: {e}")
00069     return False
00070
00071 def test_dose_range_detection():
00072     """Test dose range detection logic."""
00073     logger.info("Testing dose range detection...")
00074
00075     try:
00076         from sourcecode.training_pipeline import AutoencoderTrainer
00077
00078         # Create synthetic dose data with realistic ranges
00079         realistic_dose = np.random.exponential(scale=15.0, size=(1000, 64, 64, 64))
00080         realistic_dose[realistic_dose > 60] = 60 # Cap at reasonable max
00081
00082         # Create problematically low dose data (like what we observed)
00083         low_dose = realistic_dose * 0.02 # Scale down by ~50x
00084
00085         logger.info("Realistic dose stats:")
00086         logger.info(f" Min: {realistic_dose.min():.6f}")
00087         logger.info(f" Max: {realistic_dose.max():.6f}")
00088         logger.info(f" Mean: {realistic_dose.mean():.6f}")
00089
00090         logger.info("Problematic low dose stats:")
00091         logger.info(f" Min: {low_dose.min():.6f}")
00092         logger.info(f" Max: {low_dose.max():.6f}")
00093         logger.info(f" Mean: {low_dose.mean():.6f}")
00094
00095         # The enhanced detection should flag low_dose as problematic
00096         if low_dose.max() < 5.0:
00097             logger.info(" Enhanced detection would flag this as problematic (max < 5.0)")
00098             return True
00099         else:
00100             logger.error(" Detection logic may not catch this issue")
00101             return False
00102
00103 except Exception as e:
00104     logger.error(f" Dose range test failed: {e}")
00105     return False
00106
00107 def test_learning_rate_adjustment():
00108     """Test learning rate is properly adjusted after fixes."""
00109     logger.info(" Testing learning rate adjustment...")
00110
00111     original_lr = 1e-5
00112     ultra_conservative_lr = original_lr * 0.1 # Old: 1e-6
00113     balanced_lr = original_lr * 0.5 # New: 5e-6
00114
00115     logger.info(f"Original LR: {original_lr}")
00116     logger.info(f"Old ultra-conservative LR: {ultra_conservative_lr}")
00117     logger.info(f"New balanced LR: {balanced_lr}")
00118
00119     # Check if the new LR is reasonable (between old and original)
00120     if ultra_conservative_lr < balanced_lr < original_lr:
00121         logger.info(" Learning rate properly balanced")
00122         return True
00123     else:
00124         logger.error(" Learning rate not properly adjusted")
00125         return False
00126
00127 def run_comprehensive_test():
00128     """Run all tests and summarize results."""
00129     logger.info(" Starting comprehensive pipeline fix validation...")
00130
00131     tests = [
00132         ("Transform Consistency", test_transforms_consistency),
00133         ("Dose Range Detection", test_dose_range_detection),
00134         ("Learning Rate Adjustment", test_learning_rate_adjustment),
00135     ]
00136
00137     results = []
00138     for test_name, test_func in tests:
00139         logger.info(f"\n Running {test_name}...")
00140         try:
00141             result = test_func()
00142             results.append((test_name, result))
00143             logger.info(f" Result: {' PASS' if result else ' FAIL'}")
00144         except Exception as e:
00145             logger.error(f" Result: ERROR - {e}")
00146             results.append((test_name, False))
00147

```

```

00148     # Summary
00149     logger.info("\n" + "="*60)
00150     logger.info(" TEST SUMMARY")
00151     logger.info("="*60)
00152
00153     passed = sum(1 for _, result in results if result)
00154     total = len(results)
00155
00156     for test_name, result in results:
00157         status = " PASS" if result else " FAIL"
00158         logger.info(f" {test_name:<25} {status}")
00159
00160     logger.info(f"\nOverall: {passed}/{total} tests passed")
00161
00162     if passed == total:
00163         logger.info(" All tests passed! Pipeline fixes should resolve the dose scaling issue.")
00164     else:
00165         logger.warning(f" {total-passed} test(s) failed. Additional fixes may be needed.")
00166
00167     return passed == total
00168
00169 if __name__ == "__main__":
00170     success = run_comprehensive_test()
00171     exit(0 if success else 1)

```

8.58 tests/test_system_manager_inference.py File Reference

Namespaces

- namespace `test_system_manager_inference`

Functions

- `test_system_manager_inference.test_system_manager_inference()`

Variables

- `test_system_manager_inference.console_handler` = `logging.StreamHandler()`
- `test_system_manager_inference.formatter` = `logging.Formatter('%(asctime)s %(levelname)s: %(name)s: %(message)s')`
- `test_system_manager_inference.root_logger` = `logging.getLogger()`
- `test_system_manager_inference.logger` = `logging.getLogger(__name__)`

8.59 test_system_manager_inference.py

[Go to the documentation of this file.](#)

```

00001 #!/usr/bin/env python3
00002 """
00003 Test the corrected inference through SystemManager
00004 """
00005
00006 import sys
00007 import os
00008 sys.path.insert(0, os.path.abspath(os.path.join(os.path.dirname(__file__), 'sourcecode')))
00009
00010 # Initialize logging first
00011 import log_config
00012 import logging
00013
00014 # Set up logging to also output to console
00015 console_handler = logging.StreamHandler()
00016 console_handler.setLevel(logging.INFO)
00017 formatter = logging.Formatter('%(asctime)s %(levelname)s: %(name)s: %(message)s')
00018 console_handler.setFormatter(formatter)
00019
00020 # Get root logger and add console handler
00021 root_logger = logging.getLogger()
00022 root_logger.addHandler(console_handler)
00023 root_logger.setLevel(logging.INFO)
00024
00025 logger = logging.getLogger(__name__)
00026
00027 import torch
00028 import numpy as np
00029 from system_manager import SystemManager

```

```

00030
00031 def test_system_manager_inference():
00032     """Test that the SystemManager uses the corrected inference"""
00033     logger.info("=" * 60)
00034     logger.info("TESTING SYSTEM MANAGER WITH CORRECTED INFERENCE")
00035     logger.info("=" * 60)
00036
00037     # Create a simple test CT file
00038     logger.info("Creating test CT data...")
00039     test_ct = np.random.rand(100, 100, 100).astype(np.float32)
00040     test_ct_path = "test_ct.npy"
00041     np.save(test_ct_path, test_ct)
00042     logger.info(f" Test CT saved to: {test_ct_path}")
00043     logger.info(f"Test CT shape: {test_ct.shape}")
00044     logger.info(f"Test CT range: {test_ct.min():.6f} to {test_ct.max():.6f}")
00045
00046     # Create SystemManager with minimal setup
00047     logger.info("Creating SystemManager...")
00048     system_manager = SystemManager(
00049         root_dir=".",
00050         transforms=None,
00051         resolutions=[[64, 64, 64]],
00052         energies=[11.5],
00053         quad_energies=[11.5],
00054         quad_weights=[1.0],
00055         batch_size=1,
00056         device=torch.device('cpu'),
00057         num_epochs=1,
00058         learning_rate=0.001,
00059         patience=10,
00060         cube_size=64
00061     )
00062     logger.info(" SystemManager created successfully")
00063
00064     # Test inference
00065     logger.info("Starting inference test...")
00066     try:
00067         output_path = system_manager.run_inference(
00068             test_ct_path,
00069             model_checkpoint="unified_energy_conditioned_model_res16.0_energies3.ckpt"
00070         )
00071
00072         logger.info(f" Inference completed successfully!")
00073         logger.info(f"Output saved to: {output_path}")
00074
00075         # Check if the output file exists
00076         if os.path.exists(output_path):
00077             logger.info(" Output file created successfully")
00078
00079             # Load and check the output
00080             import nibabel as nib
00081             img = nib.load(output_path)
00082             data = img.get_fdata()
00083
00084             logger.info(f"Output shape: {data.shape}")
00085             logger.info(f"Output range: {data.min():.6f} to {data.max():.6f}")
00086             logger.info(" Output file loaded and validated successfully")
00087
00088         else:
00089             logger.error(" Output file not found")
00090
00091     except Exception as e:
00092         logger.error(f" Error during inference: {e}")
00093         logger.error("Inference test traceback:", exc_info=True)
00094         import traceback
00095         traceback.print_exc()
00096
00097     finally:
00098         # Clean up
00099         logger.info("Cleaning up test files...")
00100         if os.path.exists(test_ct_path):
00101             os.remove(test_ct_path)
00102             logger.info(f" Removed test file: {test_ct_path}")
00103
00104         logger.info("=" * 60)
00105         logger.info("SYSTEM MANAGER INFERENCE TEST COMPLETED")
00106         logger.info("=" * 60)
00107
00108 if __name__ == "__main__":
00109     test_system_manager_inference()

```

8.60 tests/test_system_manager_real_data.py File Reference

Namespaces

- namespace `test_system_manager_real_data`

Functions

- `test_system_manager_real_data.test_system_manager_with_real_data()`

8.61 test_system_manager_real_data.py

[Go to the documentation of this file.](#)

```

00001 #!/usr/bin/env python3
00002 """
00003 Test SystemManager with real training data
00004 """
00005
00006 import sys
00007 import os
00008 sys.path.insert(0, os.path.abspath(os.path.join(os.path.dirname(__file__), 'sourcecode')))
00009
00010 import torch
00011 import numpy as np
00012 from system_manager import SystemManager
00013
00014 def test_system_manager_with_real_data():
00015     """Test SystemManager with actual training data"""
00016     print("Testing SystemManager with real training data...")
00017
00018     # Use the same input data as corrected_inference.py
00019     input_path = "traindata/11_5/inputcube/235101017859661465075472232303048949736_0.npy"
00020     ground_truth_path = "traindata/11_5/outputcube/235101017859661465075472232303048949736_0.npy"
00021
00022     if not os.path.exists(input_path):
00023         print(f"Input file not found: {input_path}")
00024         print("Available files:")
00025         if os.path.exists("traindata/11_5/inputcube/"):
00026             files = os.listdir("traindata/11_5/inputcube/")[:5]
00027             for f in files:
00028                 print(f"  {f}")
00029         return
00030
00031     if not os.path.exists(ground_truth_path):
00032         print(f"Ground truth file not found: {ground_truth_path}")
00033         return
00034
00035     # Load the real data
00036     input_data = np.load(input_path)
00037     ground_truth = np.load(ground_truth_path)
00038
00039     print(f"Input shape: {input_data.shape}")
00040     print(f"Ground truth shape: {ground_truth.shape}")
00041     print(f"Ground truth range: {ground_truth.min():.6f} to {ground_truth.max():.6f}")
00042
00043     # Create SystemManager
00044     system_manager = SystemManager(
00045         root_dir=".",
00046         transforms=None,
00047         resolutions=[(64, 64, 64)],
00048         energies=[11.5],
00049         quad_energies=[11.5],
00050         quad_weights=[1.0],
00051         batch_size=1,
00052         device=torch.device('cpu'),
00053         num_epochs=1,
00054         learning_rate=0.001,
00055         patience=10,
00056         cube_size=64
00057     )
00058
00059     # Test inference
00060     try:
00061         output_path = system_manager.run_inference(
00062             input_path, # Use the real input file
00063             model_checkpoint="unified_energy_conditioned_model_res16.0_energies3.ckpt"
00064         )
00065
00066         print(f"Inference completed successfully!")
00067         print(f"Output saved to: {output_path}")
00068

```



```

00069         # Load and check the output
00070         import nibabel as nib
00071         img = nib.load(output_path)
00072         data = img.get_fdata()
00073
00074         print(f"Output shape: {data.shape}")
00075         print(f"Output range: {data.min():.6f} to {data.max():.6f}")
00076
00077         # Calculate correlation with ground truth
00078         if data.shape == ground_truth.shape:
00079             correlation = np.corrcoef(data.flatten(), ground_truth.flatten())[0, 1]
00080             print(f"Correlation with ground truth: {correlation:.6f}")
00081         else:
00082             print(f"Shape mismatch: output {data.shape} vs ground truth {ground_truth.shape}")
00083
00084     except Exception as e:
00085         print(f"Error during inference: {e}")
00086         import traceback
00087         traceback.print_exc()
00088
00089 if __name__ == "__main__":
00090     test_system_manager_with_real_data()

```

8.62 tests/test_systemmanager.py File Reference

Namespaces

- namespace [test_systemmanager](#)

Functions

- [test_systemmanager.temp_root](#) (tmp_path)
- [test_systemmanager.manager](#) (temp_root)
- [test_systemmanager.test_run_inference_no_models_and_no_checkpoint](#) (manager)
- [test_systemmanager.test_run_inference_invalid_checkpoint_type](#) (manager)
- [test_systemmanager.test_run_inference_unsupported_ct_extension](#) (manager)

8.63 test_systemmanager.py

[Go to the documentation of this file.](#)

```

00001 import sys
00002 import os
00003 import os
00004 import pytest
00005 import torch
00006 from system_manager import SystemManager
00007
00008 @pytest.fixture
00009 def temp_root(tmp_path):
00010     """Create a temporary root directory."""
00011     return str(tmp_path)
00012
00013 @pytest.fixture
00014 def manager(temp_root):
00015     """Initialize a SystemManager with minimal settings."""
00016     return SystemManager(
00017         root_dir=temp_root,
00018         transforms=None,
00019         resolutions=[],
00020         energies=[],
00021         quad_energies=[1],
00022         quad_weights=[1.0],
00023         batch_size=1,
00024         device=torch.device('cpu'),
00025         num_epochs=1,
00026         learning_rate=0.1,
00027         patience=1,
00028         cube_size=(1, 1, 1),
00029         seed=42
00030     )
00031
00032 def test_run_inference_no_models_and_no_checkpoint(manager):
00033     """
00034     If no models_by_energy are loaded and no checkpoint is provided,
00035     run_inference should raise RuntimeError.
00036     """

```

```

00037     manager.models_by_energy = {}
00038     with pytest.raises(RuntimeError) as exc:
00039         manager.run_inference("dummy.nii")
00040     assert "No trained models found" in str(exc.value)
00041
00042 def test_run_inference_invalid_checkpoint_type(manager):
00043     """
00044     Providing a checkpoint of unsupported type should raise ValueError.
00045     """
00046     manager.models_by_energy = {1: (None, None, None)}
00047     with pytest.raises(ValueError) as exc:
00048         manager.run_inference("dummy.nii", model_checkpoint=123)
00049     assert "Invalid model checkpoint" in str(exc.value)
00050
00051 def test_run_inference_unsupported_ct_extension(manager):
00052     """
00053     Providing an unsupported CT file extension should raise ValueError.
00054     """
00055     manager.models_by_energy = {1: (None, None, None)}
00056     # Create a dummy .txt file
00057     dummy_txt = os.path.join(manager.root_dir, "scan.txt")
00058     with open(dummy_txt, "w") as f:
00059         f.write("not a ct")
00060     with pytest.raises(ValueError) as exc:
00061         manager.run_inference(dummy_txt)
00062     assert "Unsupported CT file format" in str(exc.value)

```

8.64 tests/test_training.py File Reference

Namespaces

- namespace [test_training](#)

Variables

- [test_training.level](#)
- [test_training.device](#) = torch.device("cuda" if torch.cuda.is_available() else "cpu")
- dict [test_training.config](#)
- [test_training.system_manager](#)
- [test_training.result](#) = system_manager.run_energy_conditioned_training()

8.65 test_training.py

[Go to the documentation of this file.](#)

```

00001 #!/usr/bin/env python3
00002 """
00003 Simple test script for energy-conditioned training
00004 """
00005
00006 import os
00007 import sys
00008 import torch
00009
00010 # Add paths
00011 sys.path.append('sourcecode')
00012 os.environ['PYTHONPATH'] = '/Users/giannigagliardi/Documents/Git/RadioTherapy'
00013
00014 try:
00015     from system_manager import SystemManager
00016     from monai.transforms import Compose
00017     import logging
00018
00019     print(" Starting Energy-Conditioned Training Test")
00020     print("=" * 50)
00021
00022     # Setup basic logging
00023     logging.basicConfig(level=logging.INFO)
00024
00025     # Set device
00026     device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
00027     print(f"Using device: {device}")
00028
00029     # Test configuration - keep it small for testing
00030     config = {
00031         'root_dir': 'traindata',
00032         'resolutions': [16.0], # Just one resolution

```

```

00033         'energies': [11.5, 15.75], # Just two energies for faster testing
00034         'quad_energies': [11.5, 15.75],
00035         'quad_weights': [1.0, 1.0],
00036         'batch_size': 1, # Small batch
00037         'num_epochs': 2, # Just 2 epochs for testing
00038         'learning_rate': 1e-4,
00039         'patience': 2,
00040         'cube_size': (32, 32, 32), # Small cube for speed
00041         'seed': 42
00042     }
00043
00044     print(f"Configuration:")
00045     for key, value in config.items():
00046         print(f"    {key}: {value}")
00047     print()
00048
00049     # Initialize SystemManager
00050     print("Initializing SystemManager...")
00051     system_manager = SystemManager(
00052         root_dir=config['root_dir'],
00053         transforms=Compose([]), # Will be set in method
00054         resolutions=config['resolutions'],
00055         energies=config['energies'],
00056         quad_energies=config['quad_energies'],
00057         quad_weights=config['quad_weights'],
00058         batch_size=config['batch_size'],
00059         device=device,
00060         num_epochs=config['num_epochs'],
00061         learning_rate=config['learning_rate'],
00062         patience=config['patience'],
00063         cube_size=config['cube_size'],
00064         seed=config['seed']
00065     )
00066
00067     print(" SystemManager initialized successfully")
00068
00069     # Start training
00070     print("\nStarting energy-conditioned training...")
00071     result = system_manager.run_energy_conditioned_training()
00072
00073     if result:
00074         print("\n\nTraining completed successfully!")
00075         print(f"Model saved to: {result['model_path']}")
00076         print("\n\nFinal losses:")
00077         print(f"    Autoencoder train: {result['losses']['ae_train'][-1]:.4f}")
00078         print(f"    Autoencoder val: {result['losses']['ae_val'][-1]:.4f}")
00079         if result['losses']['diff']:
00080             print(f"    Diffusion: {result['losses']['diff'][-1]:.4f}")
00081     else:
00082         print(" Training failed")
00083
00084     except Exception as e:
00085         print(f" Error: {e}")
00086         import traceback
00087         traceback.print_exc()

```

8.66 tests/test_visualization.py File Reference

Namespaces

- namespace [test_visualization](#)

Functions

- [test_visualization.test_interactive_slice_viewer_slicing\(\)](#)
- [test_visualization.test_invalid_volume_raises\(\)](#)

8.67 test_visualization.py

[Go to the documentation of this file.](#)

```

00001 import numpy as np
00002 import pytest
00003 from visualization import interactive_slice_viewer
00004
00005 def test_interactive_slice_viewer_slicing():
00006     """
00007     Test that interactive_slice_viewer correctly updates the image data
00008     when the slider value changes.

```

```

00009     """
00010     # Create a simple volume: depth=4, height=2, width=2
00011     cube = np.zeros((2, 2, 4))
00012     # Fill each slice with its index
00013     for z in range(4):
00014         cube[:, :, z] = z
00015
00016     # Call viewer in test mode (no GUI)
00017     fig, slider, im = interactive_slice_viewer(cube, show=False)
00018
00019     # Initial image should show middle slice: z=2
00020     initial = im.get_array().copy()
00021     assert np.all(initial == 2), f"Expected initial slice 2, got {initial}"
00022
00023     # Move slider to slice 0
00024     slider.set_val(0)
00025     arr0 = im.get_array()
00026     assert np.all(arr0 == 0), f"Expected slice 0 after slider=0, got {arr0}"
00027
00028     # Move slider to slice 3
00029     slider.set_val(3)
00030     arr3 = im.get_array()
00031     assert np.all(arr3 == 3), f"Expected slice 3 after slider=3, got {arr3}"
00032
00033 def test_invalid_volume_raises():
00034     """
00035     interactive_slice_viewer should reject non-3D arrays.
00036     """
00037     with pytest.raises(ValueError):
00038         interactive_slice_viewer(np.zeros((2, 2)), show=False)

```

8.68 tests/validate_energy_conditioning.py File Reference

Namespaces

- namespace [validate_energy_conditioning](#)

Variables

- [validate_energy_conditioning.level](#)
- [validate_energy_conditioning.device](#) = torch.device("cuda" if torch.cuda.is_available() else "cpu")
- dict [validate_energy_conditioning.config](#)
- [validate_energy_conditioning.checkpoint](#) = torch.load([config](#)['model_path'], map_location=[device](#))
- [validate_energy_conditioning.training_config](#) = [checkpoint](#)['training_config']
- [validate_energy_conditioning.autoencoder](#)
- str [validate_energy_conditioning.sample_input_path](#) = 'traindata/11_5/inputcube/235101017859661465075472232303_0.npy'
- str [validate_energy_conditioning.sample_output_path](#) = 'traindata/11_5/outputcube/235101017859661465075472232303_0.npy'
- [validate_energy_conditioning.physical_input](#) = np.load([sample_input_path](#))
- [validate_energy_conditioning.ground_truth](#) = np.load([sample_output_path](#))
- dict [validate_energy_conditioning.results](#) = {}
- dict [validate_energy_conditioning.predictions](#) = {}
- float [validate_energy_conditioning.energy_normalized](#) = energy / 40.0
- [validate_energy_conditioning.input_2channel](#) = np.zeros((2, *[physical_input](#).shape))
- [validate_energy_conditioning.input_tensor](#) = torch.tensor([input_2channel](#), dtype=torch.float32).unsqueeze(0).to([device](#))
- [validate_energy_conditioning.prediction](#) = [autoencoder](#)([input_tensor](#))
- [validate_energy_conditioning.prediction_np](#) = [prediction](#).cpu().numpy().squeeze()
- [validate_energy_conditioning.pred_mean](#) = np.mean([prediction_np](#))
- [validate_energy_conditioning.pred_std](#) = np.std([prediction_np](#))
- [validate_energy_conditioning.pred_max](#) = np.max([prediction_np](#))
- [validate_energy_conditioning.pred_min](#) = np.min([prediction_np](#))
- [validate_energy_conditioning.energy_list](#) = list([config](#)['energies'])
- [validate_energy_conditioning.energy1](#)
- [validate_energy_conditioning.energy2](#)
- [validate_energy_conditioning.pred1](#)

- `validate_energy_conditioning.pred2`
- `validate_energy_conditioning.mse = np.mean((pred1 - pred2) ** 2)`
- `validate_energy_conditioning.mae = np.mean(np.abs(pred1 - pred2))`
- `validate_energy_conditioning.max_diff = np.max(np.abs(pred1 - pred2))`
- `validate_energy_conditioning.correlation = np.corrcoef(pred1.flatten(), pred2.flatten())[0, 1]`
- list `validate_energy_conditioning.means = [results[e]['mean'] for e in energy_list]`
- list `validate_energy_conditioning.stds = [results[e]['std'] for e in energy_list]`
- int `validate_energy_conditioning.mean_variation = 0` else 0
- int `validate_energy_conditioning.std_variation = 0` else 0
- int `validate_energy_conditioning.max_mse = 0`
- float `validate_energy_conditioning.min_correlation = 1.0`

8.69 validate_energy_conditioning.py

[Go to the documentation of this file.](#)

```
00001 #!/usr/bin/env python3
00002 """
00003 Validation script to test if energy conditioning actually works
00004 This script tests the trained unified model to see if it produces different outputs for different energies
00005 """
00006
00007 import os
00008 import sys
00009 import torch
00010 import numpy as np
00011 import matplotlib.pyplot as plt
00012
00013 # Add paths
00014 sys.path.append('sourcecode')
00015 os.environ['PYTHONPATH'] = '/Users/giannigagliardi/Documents/Git/RadioTherapy'
00016
00017 try:
00018     import logging
00019
00020     print(" Energy Conditioning Validation Test")
00021     print("=" * 50)
00022
00023     # Setup basic logging
00024     logging.basicConfig(level=logging.INFO)
00025
00026     # Set device
00027     device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
00028     print(f"Using device: {device}")
00029
00030     # Test configuration
00031     config = {
00032         'root_dir': 'traindata',
00033         'resolutions': [16.0],
00034         'energies': [11.5, 15.75, 34.25], # Test all three energies
00035         'batch_size': 1,
00036         'cube_size': (32, 32, 32),
00037         'model_path':
00038     }
00039
00040     print(f"Model path: {config['model_path']}")
00041     print(f"Testing energies: {config['energies']}")
00042     print()
00043
00044     # Check if model exists
00045     if not os.path.exists(config['model_path']):
00046         print(f" Model file not found: {config['model_path']}")
00047         exit(1)
00048
00049     # Load the model checkpoint directly
00050     print("Loading model checkpoint...")
00051     checkpoint = torch.load(config['model_path'], map_location=device)
00052     print(" Model checkpoint loaded successfully")
00053
00054     print(f"Checkpoint keys: {list(checkpoint.keys())}")
00055
00056     # Get the actual training configuration from the checkpoint
00057     if 'training_config' in checkpoint:
00058         training_config = checkpoint['training_config']
00059         print(f"Training config: {training_config}")
00060
00061     # Load the autoencoder architecture from our training pipeline
00062     from generative.networks.nets import AutoencoderKL
```

```

00063
00064 # Initialize autoencoder with the ACTUAL parameters from training
00065 autoencoder = AutoencoderKL(
00066     spatial_dims=3,
00067     in_channels=2, # CT + energy conditioning
00068     out_channels=1, # Output dose
00069     num_channels=(32, 32, 32), # Correct configuration from training
00070     latent_channels=2, # Correct configuration from training
00071     num_res_blocks=1, # Correct configuration from training
00072     norm_num_groups=8, # Correct configuration from training
00073     attention_levels=(False, False, True), # Correct configuration from training
00074 ).to(device)
00075
00076 # Load the trained weights
00077 if 'autoencoder' in checkpoint:
00078     autoencoder.load_state_dict(checkpoint['autoencoder'])
00079     print(" Autoencoder state dict loaded from checkpoint")
00080 else:
00081     print(" Autoencoder not found in checkpoint")
00082     exit(1)
00083
00084 autoencoder.eval()
00085
00086 print(" Autoencoder model loaded and ready for testing")
00087
00088 # Load the same input data but test with different energy values
00089 print("\nLoading test data...")
00090
00091 # Load a sample input (we'll use the same physical input but different energy channels)
00092 sample_input_path = 'traindata/11_5/inputcube/235101017859661465075472232303048949736_0.npy'
00093 sample_output_path = 'traindata/11_5/outputcube/235101017859661465075472232303048949736_0.npy'
00094
00095 if not os.path.exists(sample_input_path):
00096     print(f" Sample input not found: {sample_input_path}")
00097     exit(1)
00098
00099 # Load the physical dose distribution (first channel)
00100 physical_input = np.load(sample_input_path)
00101 ground_truth = np.load(sample_output_path)
00102
00103 print(f"Loaded input shape: {physical_input.shape}")
00104 print(f"Loaded ground truth shape: {ground_truth.shape}")
00105
00106 # Test energy conditioning
00107 print("\n\nTesting Energy Conditioning...")
00108 print("-" * 30)
00109
00110 results = {}
00111 predictions = {}
00112
00113 for energy in config['energies']:
00114     print(f"\n\nTesting energy: {energy} MeV")
00115
00116     # Create energy-conditioned input
00117     # Channel 0: physical dose distribution
00118     # Channel 1: energy value (normalized)
00119     energy_normalized = energy / 40.0 # Normalize energy to [0,1] range approximately
00120
00121     # Create 2-channel input
00122     input_2channel = np.zeros((2, *physical_input.shape))
00123     input_2channel[0] = physical_input # Physical dose
00124     input_2channel[1] = energy_normalized # Energy channel
00125
00126     # Convert to tensor and add batch dimension
00127     input_tensor = torch.tensor(input_2channel, dtype=torch.float32).unsqueeze(0).to(device)
00128
00129     print(f" Input tensor shape: {input_tensor.shape}")
00130     print(f" Energy channel value: {energy_normalized:.4f}")
00131
00132     # Run inference
00133     with torch.no_grad():
00134         prediction = autoencoder(input_tensor)
00135         # AutoencoderKL returns a tuple (reconstruction, latent), we want the reconstruction
00136         if isinstance(prediction, tuple):
00137             prediction = prediction[0] # Get the reconstruction
00138         prediction_np = prediction.cpu().numpy().squeeze()
00139
00140     predictions[energy] = prediction_np
00141
00142     # Calculate some statistics
00143     pred_mean = np.mean(prediction_np)
00144     pred_std = np.std(prediction_np)
00145     pred_max = np.max(prediction_np)
00146     pred_min = np.min(prediction_np)
00147
00148     results[energy] = {
00149         'mean': pred_mean,

```

```

00150         'std': pred_std,
00151         'max': pred_max,
00152         'min': pred_min
00153     }
00154
00155     print(f" Prediction statistics:")
00156     print(f" Mean: {pred_mean:.6f}")
00157     print(f" Std: {pred_std:.6f}")
00158     print(f" Max: {pred_max:.6f}")
00159     print(f" Min: {pred_min:.6f}")
00160
00161     # Compare predictions between energies
00162     print("\n\n Energy Conditioning Analysis")
00163     print("-" * 30)
00164
00165     energy_list = list(config['energies'])
00166
00167     # Calculate differences between energy predictions
00168     for i in range(len(energy_list)):
00169         for j in range(i + 1, len(energy_list)):
00170             energy1, energy2 = energy_list[i], energy_list[j]
00171             pred1, pred2 = predictions[energy1], predictions[energy2]
00172
00173             # Calculate various difference metrics
00174             mse = np.mean((pred1 - pred2) ** 2)
00175             mae = np.mean(np.abs(pred1 - pred2))
00176             max_diff = np.max(np.abs(pred1 - pred2))
00177             correlation = np.corrcoef(pred1.flatten(), pred2.flatten())[0, 1]
00178
00179             print(f"\n{energy1} MeV vs {energy2} MeV:")
00180             print(f" MSE: {mse:.8f}")
00181             print(f" MAE: {mae:.8f}")
00182             print(f" Max Diff: {max_diff:.8f}")
00183             print(f" Correlation: {correlation:.6f}")
00184
00185             # Energy conditioning is working if:
00186             # 1. MSE > 0 (predictions are different)
00187             # 2. Correlation < 1 (not identical)
00188             # 3. Differences are meaningful relative to the prediction scale
00189
00190             if mse > 1e-8 and correlation < 0.99:
00191                 print(f" ENERGY CONDITIONING DETECTED - Model responds to energy!")
00192             elif mse < 1e-10:
00193                 print(f" NO ENERGY CONDITIONING - Predictions are identical")
00194             else:
00195                 print(f" WEAK ENERGY CONDITIONING - Small differences detected")
00196
00197     # Statistical summary
00198     print("\n\n Summary Statistics")
00199     print("-" * 20)
00200
00201     means = [results[e]['mean'] for e in energy_list]
00202     stds = [results[e]['std'] for e in energy_list]
00203
00204     mean_variation = np.std(means) / np.mean(means) if np.mean(means) != 0 else 0
00205     std_variation = np.std(stds) / np.mean(stds) if np.mean(stds) != 0 else 0
00206
00207     print(f"Mean variation across energies: {mean_variation:.6f}")
00208     print(f"Std variation across energies: {std_variation:.6f}")
00209
00210     # Overall assessment
00211     print("\n\n Overall Assessment")
00212     print("-" * 20)
00213
00214     # Check if any meaningful differences exist
00215     max_mse = 0
00216     min_correlation = 1.0
00217
00218     for i in range(len(energy_list)):
00219         for j in range(i + 1, len(energy_list)):
00220             pred1, pred2 = predictions[energy_list[i]], predictions[energy_list[j]]
00221             mse = np.mean((pred1 - pred2) ** 2)
00222             correlation = np.corrcoef(pred1.flatten(), pred2.flatten())[0, 1]
00223             max_mse = max(max_mse, mse)
00224             min_correlation = min(min_correlation, correlation)
00225
00226     if max_mse > 1e-6 and min_correlation < 0.95:
00227         print(" SUCCESS: Energy conditioning is working!")
00228         print(" The model produces meaningfully different outputs for different energies.")
00229         print(" The original problem has been SOLVED! ")
00230     elif max_mse > 1e-8:
00231         print(" PARTIAL SUCCESS: Weak energy conditioning detected.")
00232         print(" The model shows some response to energy, but may need more training.")
00233     else:
00234         print(" FAILURE: No energy conditioning detected.")
00235         print(" The model ignores the energy input. Problem NOT solved.")
00236

```

```

00237     print(f"\nMax MSE between energies: {max_mse:.2e}")
00238     print(f"Min correlation between energies: {min_correlation:.6f}")
00239
00240 except Exception as e:
00241     print(f" Error during validation: {e}")
00242     import traceback
00243     traceback.print_exc()
00244
00245 print("\n\n" + "=" * 50)
00246 print("Energy conditioning validation complete!")

```

8.70 tests/visualizeTEST.py File Reference

Namespaces

- namespace [visualizeTEST](#)

Functions

- [visualizeTEST.update](#) (val)

Variables

- [visualizeTEST.cube](#) = `np.load("/Users/giannigagliardi/Documents/Git/RadioTherapy/traindata/11_5/outputcube/235101017859661465075472232303048949736_10.npy")`
- [visualizeTEST.fig](#)
- [visualizeTEST.ax](#)
- `int visualizeTEST.slice_idx = cube.shape[2] // 2`
- `visualizeTEST.im = ax.imshow(cube[:, :, slice_idx], cmap="gray")`
- `visualizeTEST.slider_ax = plt.axes([0.2, 0.05, 0.6, 0.03])`
- [visualizeTEST.slider](#)
- `tuple visualizeTEST.cube_norm = (cube - cube.min()) / (cube.max() - cube.min())`
- [visualizeTEST.p_low](#)
- [visualizeTEST.p_high](#)
- [visualizeTEST.x](#)
- [visualizeTEST.y](#)
- [visualizeTEST.z](#)
- `visualizeTEST.cube_smoothed = ndimage.gaussian_filter(cube_norm, sigma=0.5)`
- [visualizeTEST.fig_vol](#)
- [visualizeTEST.scene](#)

8.71 visualizeTEST.py

Go to the documentation of this file.

```

00001 import matplotlib.pyplot as plt
00002 import numpy as np
00003 import plotly.graph_objects as go
00004
00005 # Load the cube
00006 cube =
    np.load("/Users/giannigagliardi/Documents/Git/RadioTherapy/traindata/11_5/outputcube/235101017859661465075472232303048949736_10.npy")
00007 # If the loaded cube has an extra energy dimension, pick the first energy
00008 if cube.ndim == 4:
00009     cube = cube[0]
00010 elif cube.ndim != 3:
00011     raise ValueError(f"Unexpected cube shape {cube.shape}; expected 3D or 4D (energy x D x H x W).")
00012 # ----- Interactive slice viewer -----
00013 import matplotlib.widgets as widgets
00014
00015 fig, ax = plt.subplots()
00016 slice_idx = cube.shape[2] // 2
00017 im = ax.imshow(cube[:, :, slice_idx], cmap="gray")
00018 ax.set_title(f"Slice {slice_idx}")
00019 ax.axis("off")
00020
00021 slider_ax = plt.axes([0.2, 0.05, 0.6, 0.03])
00022 slider = widgets.Slider(
00023     slider_ax, "z-Slice", 0, cube.shape[2] - 1,

```



```

00024     valinit=slice_idx, valfmt="%0.0f"
00025 )
00026
00027 def update(val):
00028     z = int(slider.val)
00029     im.set_data(cube[:, :, z])
00030     ax.set_title(f"Slice {z}")
00031     fig.canvas.draw_idle()
00032
00033 slider.on_changed(update)
00034 plt.show()
00035 # ----- end slice viewer -----
00036
00037 cube_norm = (cube - cube.min()) / (cube.max() - cube.min())
00038 # --- choose sensible isosurface thresholds -----
00039 # This prints the 1st and 99th percentiles so we know where most intensities lie.
00040 p_low, p_high = np.percentile(cube_norm, [1, 99])
00041 print(f"Suggested thresholds  p_low={p_low:.4f}  p_high={p_high:.4f}")
00042
00043 # create 3-D coordinate grid that matches the cube dimensions
00044 x, y, z = np.mgrid[0:cube.shape[0], 0:cube.shape[1], 0:cube.shape[2]]
00045
00046 # ----- 3-D volume rendering -----
00047 # Apply smoothing to reduce artifacts
00048 from scipy import ndimage
00049 cube_smoothed = ndimage.gaussian_filter(cube_norm, sigma=0.5)
00050
00051 # If the printed p_low/p_high look too small or too large,
00052 # feel free to tweak them below.
00053 fig_vol = go.Figure(
00054     data=go.Volume(
00055         x=x.flatten(),
00056         y=y.flatten(),
00057         z=z.flatten(),
00058         value=cube_smoothed.flatten(),
00059         opacity=0.1,          # mehr transparent für bessere Sicht
00060         surface_count=15,     # weniger Isoflächen für glatteres Aussehen
00061         isomin=p_low * 1.2,   # etwas höhere Schwelle
00062         isomax=p_high * 0.8,  # etwas niedrigere obere Schwelle
00063         colorscale="Hot",     # bessere Farbskala für Dosisverteilung
00064         caps=dict(x_show=False, y_show=False, z_show=False) # keine Kappen
00065     )
00066 )
00067 # realistische Seitenverhältnisse beibehalten
00068 fig_vol.update_layout(scene=dict(aspectmode="data"))
00069 # enforce axis ranges to match cube shape
00070 fig_vol.update_layout(
00071     scene=dict(
00072         xaxis=dict(range=[0, cube.shape[0]]),
00073         yaxis=dict(range=[0, cube.shape[1]]),
00074         zaxis=dict(range=[0, cube.shape[2]])
00075     )
00076 )
00077 fig_vol.show()
00078 # ----- end volume rendering -----

```

